
ГОСТ 19.005-85
ГОСТ 19.501-78
ГОСТ 19.201-78
ГОСТ 19.101-77
MIL-STD-498
ISO 9126:1991
IEEE 1074-1995
ГОСТ 19
В.А.БЛАГОДАТСКИХ, В.А.ВОЛНИН, К.Ф.ПОСКАКАЛОВ
СТАНДАРТИЗАЦИЯ РАЗРАБОТКИ ПРОГРАММНЫХ СРЕДСТВ
УЧЕБНОЕ ПОСОБИЕ

УДК 004.41.057.2 ББК
32.973.26-018.2ц.я73 Б68

РЕЦЕНЗЕНТЫ.

Кафедра автоматизированной обработки экономической информации
Всероссийского заочного финансово-экономического института;

СВ. Дробин,
кандидат экономических наук, заместитель директора
Главного центра информатизации Банка России

Благодатских В.А. др.

Б68 Стандартизация разработки программных средств: Учеб. пособие / В.А. Благодатских, В.А. Волнин, К.Ф. Поскакалов; Под ред. О.С. Разумова. - М.: Финансы и статистика, 2005. -288 с: ил.

ISBN 5-279-02657-3

Создание конкурентоспособной программной продукции невозможно без использования соответствующих стандартов на всех этапах ее разработки В пособии описываются жизненный цикл программных средств, его процессы, подробно рассматриваются содержание и применение действующих российских и международных стандартов в области создания программных средств Излагаются вопросы адаптации стандартов к конкретным проектам Подробно рассмотрены надежность и качество программных средств, принципы организации и методики тестирования при испытании надежности сложных программных средств.

Для студентов вузов, обучающихся по специальности 351400 «Прикладная информатика в экономике» и другим междисциплинарным специальностям.

Б-2404600000-260 „„ .__.
010(01) - 2005 ^ ~ *7ам*

УДК 004.41.057.2
ББК 32.973.26-018.2ц.я73

топит - _____
ISBN 5-279-02657-3 -

© Благодатских В.А., Волнин В.А..
Поскакалов К.Ф., 2003

ОГЛАВЛЕНИЕ

ПРЕДИСЛОВИЕ **7**

Глава 1. ОБЩИЕ ПОЛОЖЕНИЯ О СТАНДАРТАХ 9

1.1. Нормативные документы по стандартизации и виды стандартов 10
1.2. Стандарты в области программного обеспечения 15
1.3. Международные организации, разрабатывающие стандарты 23
Международная организация по стандартизации (ИСО) 23
Международная электротехническая комиссия (М.ЭК) 24
Объединенный технический комитет (JTC1) 25
1.4. Национальные организации, разрабатывающие стандарты ... 26
Государственный комитет РФ по стандартизации 26
Американский национальный институт стандартов и технологий 30
1.5. Внутрифирменные (внутрикорпоративные) стандарты 32
Назначение и классификация внутрикорпоративных стандартов 32
Организация разработки внутрифирменных стандартов 39
Пример стандарта организации хранения аналитической информации 46
Вопросы для самопроверки 55

Г л а в а 2. ЖИЗНЕННЫЙ ЦИКЛ ПРОГРАММНЫХ СРЕДСТВ . 56

2.1. Основные процессы жизненного цикла программного средства 63
2.2. Вспомогательные процессы жизненного цикла программного средства 71
2.3. Организационные процессы жизненного цикла программного средства 78
2.4. Стандарты комплекса ГОСТ 34 80
2.5. Стандарт IEEE 1074-1995. Процессы жизненного цикла для развития программных средств 85
2.6. Адаптация стандарта к конкретному проекту 86
2.7. Модели жизненного цикла программных средств 90
Вопросы для самопроверки 94

Глава 3. СТАНДАРТЫ ДОКУМЕНТИРОВАНИЯ ПРОГРАММНЫХ СРЕДСТВ..... 95

3.1. Общая характеристика состояния в области документиро вания программных средств 96
3.2. Единая система программной документации 99
ГОСТ 19.101-77 ЕСПД. Виды программ и программных документов 101
ГОСТ 19.102-77. ЕСПД. Стадии разработки..... 104
ГОСТ 19.105-78 ЕСПД. Общие требования к программным документам..... 106
ГОСТ 19.201-78 ЕСПД. Техническое задание. Требования к содержанию и оформлению..... 107
ГОСТ 19.402-78 ЕСПД. Описание программы 108
ГОСТ 19.404-79 ЕСПД. Пояснительная записка. Требования к содержанию и оформлению..... 109
ГОСТ 19.503-79 ЕСПД. Руководство системного програм миста. Требования к содержанию и оформлению ..... ПО
ГОСТ 19.504-79 ЕСПД. Руководство программиста. Требования к содержанию и оформлению..... 111
ГОСТ 19.505-79 ЕСПД. Руководство оператора. Требования К содержанию и оформлению 111
ГОСТ 19.506-79 ЕСПД. Описание языка. Требования к содержанию и оформлению..... 112
3.3. Государственные стандарты Российской Федерации (ГОСТР)..... 113

Вопросы для самопроверки..... 124

Г л а в а 4. НАДЕЖНОСТЬ И КАЧЕСТВО ПРОГРАММНЫХ СРЕДСТВ..... 125

4.1. Основные понятия и показатели надежности программных средств..... 129
4.2. Дестабилизирующие факторы и методы обеспечения надежности функционирования программных средств 141
Предупреждение ошибок..... 147
Обнаружение ошибок 148
Исправление ошибок..... 150
Устойчивость к ошибкам..... 151
Обработка сбоев аппаратуры..... 154

4.3. Модели надежности программного обеспечения 156
Аналитические модели надежности 159
Эмпирические модели надежности 171
4.4. Обеспечение качества и надежности в процессе разработки сложных программных средств 182
Сложность 182
Отношения с пользователем 183
Решение задачи 185
Поймите задачу 186
Составьте план 187
Выполните план 188
Проанализируйте решение 188
4.5. Требования к технологии и средствам автоматизации разработки сложных программных средств 189
4.6. Качество программного обеспечения 194
Вопросы для самопроверки 198

Глава 5. ТЕСТИРОВАНИЕ ПРОГРАММНОГО СРВДСГВ А . . . 200

5.1. Основные определения 201
5.2. Экономика тестирования 203
Тестирование программы как «черного ящика» 203
Тестирование программы как «белого ящика» 204
5.3. Аксиомы (принципы) тестирования 205
5.4. Философия тестирования 210
5.5. Тестирование модулей 212
Пошаговое тестирование 213
Восходящее тестирование 215
Нисходящее тестирование 216
Метод «большого скачка» 218
Метод сандвича 219
Модифицированный метод сандвича 219
5.6. Комплексное тестирование 220
Проектирование комплексного теста 221
Выполнение комплексного теста 228
5.7. ГОСТ Р ИСО/МЭК 12119-2000 228
Работы по тестированию 229
Протоколы тестирования 230
Отчет о тестировании 230
Дополнительное тестирование 231

5.8. Требования к средствам обеспечения тестирования 231
5.9. Организация и этапы тестирования при испытаниях надежности сложных программных средств 244
5.10. Методика тестирования при испытаниях надежности сложных программных средств 253
Тестирование и отладка программных компонентов в реальном времени 253
Тестирование и испытания комплекса программ по данным имитаторов внешней среды 255
Тестирование и испытания надежности комплекса программ при воздействиях операторов-пользователей 256
Испытания комплекса программ в реальной внешней среде 258
5.11. Тестирование программного обеспечения 262
Цель тестирования 264
Тестирование и качество 265
Виды тестирования 266
Место тестирования в процессе разработки ПО 266
Специалист отдела тестирования — квалификационные требования 271
Инструментарий специалиста по тестированию 272
Передовые технологии в тестировании (автоматизация тестирования) 273
Вопросы для самопроверки 276

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ 277

Предметный указатель 282

L

ПРЕДИСЛОВИЕ

Учебное пособие «Стандартизация разработки программных средств» разработано в соответствии с требованиями учебного плана и государственным образовательным стандартом одноименной дисциплины с целью изучения методики применения стандартов (международных и национальных) и получения навыков при разработке программных средств.

Отметим, что термин «стандарт» (англ. standard —норма) в широком смысле понимается как образец, эталон или модель, принимаемые за исходные для сопоставления с ними других подобных объектов.

Стандарты как нормативно-технические документы устанавливают комплекс норм, правил, требований к объекту стандартизации. Применение стандартов способствует улучшению качества программных средств, повышению развития информатизации процессов, росту эффективности внедрения и эксплуатации программных средств и устраняет разнобой при создании их различными разработчиками.

Создание или приобретение у соответствующих производителей новых программных средств вызвано необходимостью информатизации реальных функциональных процессов, модернизации существующих информационных процессов и необходимостью реализации деятельности исследуемых процессов.

Эта необходимость предполагает получить ответы на следующие вопросы.

1. Для каких целей создается программное средство?
2. Когда целесообразно закончить создание программного сред ства? (
3. Какие затраты рациональны при проектировании (приобретении) программного средства?

Естественно, создание программного средства—динамически длительный и трудоемкий процесс. Современные технологии проектирования основаны на последовательной (поэтапной) разработке. По общности целей последовательности работ (этапы) обычно объединяются в стадии.

Совокупность работ по созданию (приобретению) программного средства от момента принятия решения до внедрения и окончания функционирования называется *жизненным циклом программного средства.*

При разработке (приобретении) программного средства следует учитывать следующие требования:

- использование современных информационных технологий для полу чения запланированных эффектов при любой скорости подключения;
- стандартизацию основных этапов жизненного цикла программных средств;
- автоматическое использование комбинаций ведения протоколов с другими программными средствами;
- вариантность протоколов поддержки;
- интегрированную запись и монтаж;
- интеграцию web-сайтов;
- обеспечение надежности и качества функционирования програм много средства;
- тестирование нового программного средства.

Учебное пособие хорошо структурировано: глава 1 содержит общие положения о стандартах с характеристикой нормативных документов и видов стандартов, в том числе и для программного обеспечения. Здесь указаны международные и национальные организации по разработке стандартов. Глава 2 посвящена разработке жизненного цикла программных средств. В ней рассмотрены составляющие процессы жизненного цикла с примерами необходимых стандартов. Глава 3 знакомит со стандартами документирования программных средств. Это очень важно для разработки (приобретения) программного средства, что по значению можно поставить вровень с методикой разработки. В главе 4 приводятся основные понятия, показатели, модели надежности программных средств и обеспечения качества и надежности при разработке сложных программных средств. В главе 5 последовательно излагаются принципы, определения, аксиомы, организация и техника тестирования с иллюстрацией его практики.

В современных условиях особое значение имеет информатизация образования, система которого служит источником создания интеллектуального потенциала любой страны, а данное учебное пособие способствует информатизации образования, ведь без стандартизации невозможно создание новых программных средств для современной информатизации.

профессор *О. С. Разумов*

ГЛАВА 1 ОБЩИЕ ПОЛОЖЕНИЯ О СТАНДАРТАХ

Стандартизация — это деятельность, направленная на разработку и установление требований, норм, правил, характеристик, как обязательных для выполнения, так и рекомендуемых, обеспечивающая право потребителя на приобретение товаров надлежащего качества, а также право на безопасность и комфортность труда. *Цель стандартизации* — достижение оптимальной степени упорядочения в той или иной области посредством широкого и многократного использования установленных положений, требований, норм для решения реально существующих, планируемых или потенциальных задач. *Основными результатами деятельности по стандартизации* должны быть повышение степени соответствия продукта (услуги), процессов их функциональному назначению, устранение технических барьеров в международном товарообмене, содействие научно-техническому прогрессу и сотрудничеству в различных областях.

Стандартизация связана с такими понятиями, как объект стандартизации и область стандартизации. Объектом стандартизации обычно называют продукцию, процесс, услугу, для которых разрабатывают те или иные требования, характеристики, параметры, правила и т.п. Стандартизация может касаться либо объекта в целом, либо его отдельных составляющих (характеристик). Областью стандартизации называют совокупность взаимосвязанных объектов стандартизации.

Стандартизация осуществляется на разных уровнях. *Уровень стандартизации* зависит от того, участники какого географического, экономического, политического региона мира принимают стандарт. Если участие в стандартизации открыто для соответствующих органов любой страны, то это международная стандартизация. Региональная стандартизация — деятельность, открытая только для соответствующих органов государств одного географического, политического или экономического региона. Региональная и международная стандартизация осуществляется

специалистами стран, представленных в соответствующих региональных и международных организациях (рис. 1.1).

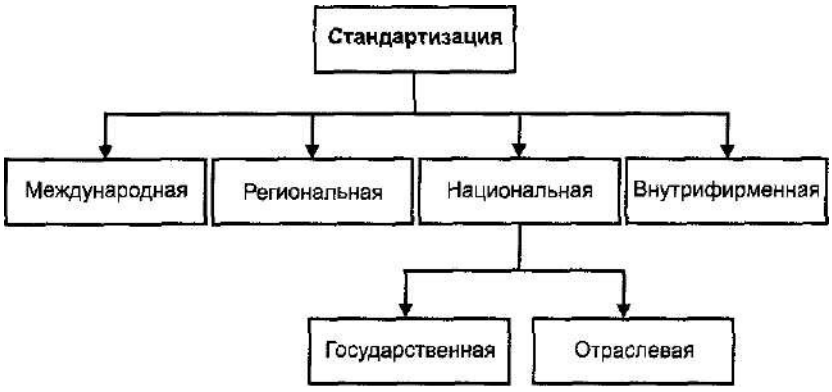


Рис. 1.1. Схема уровней стандартизации

Национальная стандартизация — стандартизация в одном конкретном государстве. При этом национальная стандартизация также может осуществляться на разных уровнях: на государственном, отраслевом, в том или ином секторе экономики (например, на уровне министерств), на уровне ассоциаций, производственных фирм, предприятий (фабрик, заводов) и учреждений.

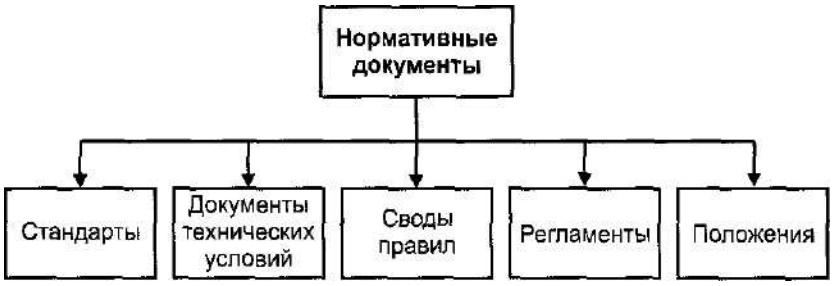
Стандартизацию, которая проводится в административно-территориальной единице (провинции, крае и т.п), принято называть *административно-территориальной стандартизацией.*

1.1. Нормативные документы по стандартизации и виды стандартов

В процессе стандартизации вырабатываются нормы, правила, требования, характеристики, касающиеся объекта стандартизации, которые оформляются в виде нормативного документа.

Рассмотрим разновидности нормативных документов, которые рекомендуются руководством 2-й Международной организации по стандартизации и Международной электротехнической комиссии (ИСО/МЭК), а также принятые в государственной си-

стеме стандартизации Российской Федерации (РФ). Руководство ИСО/МЭК рекомендует: стандарты, документы технических условий, своды правил, регламенты (технические регламенты), положения (рис. 1.2).



Рис,1.2. Схема разновидностей нормативных документов

Стандарт (от англ. standard — норма, образец) — в широком смысле слова образец, эталон, модель, принимаемые за исходные для сопоставления с ними других подобных объектов. Стандарт как нормативно-технический документ устанавливает комплекс норм, правил, требований к объекту стандартизации. Стандарт может быть разработан как на материальные предметы (продукцию, эталоны, образцы веществ), так и на нормы, правила, требования в различных областях. В переносном смысле — шаблон, трафарет, не содержащий ничего оригинального.

Стандарт — это нормативный документ, разработанный на основе консенсуса, утвержденный признанным органом, направленный на достижение оптимальной степени упорядочения в определенной области. В стандарте устанавливаются для всеобщего и многократного использования общие принципы, правила, характеристики, касающиеся различных видов деятельности или их результатов. Стандарт должен быть основан на обобщенных результатах научных исследований, технических достижений и практического опыта., тогда его использование принесет оптимальную выгоду для общества.

Предварительный стандарт — это временный документ, который принимается органом по стандартизации и доводится до широкого круга потенциальных потребителей, а также до тех, кто может его применить. Информация, полученная в процессе использования предварительного стандарта, и отзывы об этом

документе служат базой для решения вопроса о целесообразности принятия стандарта.

Стандарты бывают международными, региональными, национальными, административно-территориальными. Они принимаются соответственно международными, региональными, национальными, территориальными органами по стандартизации. Все эти категории стандартов предназначены для широкого круга потребителей. По существующим нормам стандартизации стандарты периодически пересматриваются для внесения изменений, чтобы их требования соответствовали уровню научно-технического прогресса, или, согласно терминологии ИСО/МЭК, стандарты должны представлять собой «признанные технические правила». Нормативный документ, в том числе и стандарт, считается признанным техническим правилом, если он разработан в сотрудничестве с заинтересованными сторонами путем консультаций и на основе консенсуса.

Указанные выше категории стандартов называют *общедоступными.* Другие же категории стандартов, такие, как фирменные или отраслевые, не являясь таковыми, могут, однако, использоваться и в нескольких странах согласно существующим там правовым нормам.

В практике термин «стандарт» может употребляться и по отношению к эталону, образцу или описанию продукта, процесса (услуги). По существу это не является принципиальной ошибкой, хотя эталон правильнее относить к области метрологии, а термин «стандарт» использовать применительно к нормативному документу.

Документ технических условий (technical specification) устанавливает технические требования к продукции, услуге, процессу. Обычно в документе технических условий должны быть указаны методы или процедуры, которые следует использовать для проверки соблюдения требований данного нормативного документа в таких ситуациях, когда это необходимо.

Свод правил, как и предыдущий нормативный документ, может быть самостоятельным стандартом либо самостоятельным документом, а также частью стандарта. Свод правил обычно разрабатывается для процессов проектирования, монтажа оборудования и конструкций, технического обслуживания или эксплуатации объектов, конструкций, изделий. Технические правила, содержащиеся в документе, носят рекомендательный характер.

Все указанные выше нормативные документы являются рекомендательными. В отличие от них регламент носит обязательный

характер. *Регламент* — это документ, в котором содержатся обязательные правовые нормы.

Кроме стандартов нормативными документами являются также ПР — правила по стандартизации, Р — рекомендации по стандартизации и ТУ — технические условия. Особое требование предъявляется к нормативным документам на продукцию, которая согласно российскому законодательству подлежит обязательной сертификации. В них должны быть указаны те требования к продукции (услуге), которые подтверждаются посредством сертификации, а также методы контроля (испытаний), которые следует применять для установления соответствия, правила маркировки такой продукции и виды сопроводительной документации.

Государственные стандарты разрабатывают на продукцию, работы и услуги, потребности в которых носят межотраслевой характер.

Отраслевые стандарты разрабатываются применительно к продукции определенной отрасли. Их требования не должны противоречить обязательным требованиям государственных стандартов, а также правилам и нормам безопасности, установленным для отрасли. Принимают такие стандарты государственные органы управления (например, министерства), которые несут ответственность за соответствие требований отраслевых стандартов обязательным требованиям ГОСТ Р.

Объектами отраслевой стандартизации могут быть:

- продукция, процессы и услуги, применяемые в отрасли;
- правила, касающиеся организации работ по отраслевой стан дартизации;
- типовые конструкции изделий отраслевого применения (инст рументы, крепежные детали и т.п.);
- правила метрологического обеспечения в отрасли.

Диапазон применяемости отраслевых стандартов ограничивается предприятиями, подведомственными государственному органу управления, принявшему данный стандарт. На добровольной основе возможно использование этих стандартов субъектами хозяйственной деятельности иного подчинения. Степень обязательности соблюдения требований стандарта отрасли определяется тем предприятием, которое применяет его, или по договору между изготовителем и потребителем. Контроль за выполнением обязательных требований организует ведомство, принявшее данный стандарт.

Стандарты предприятий разрабатываются и принимаются самими предприятиями. Объектами стандартизации в этом случае обычно являются составляющие подсистем организации и управления производством, совершенствование которых — главная цель стандартизации на данном уровне. Кроме того, стандартизация на предприятии может затрагивать и продукцию, производимую этим предприятием. Тогда объектами стандарта предприятия будут составные части продукции, технологическая оснастка и инструменты, общие технологические нормы процесса производства этой продукции. Стандарты предприятий могут содержать требования к различного рода услугам внутреннего характера.

Закон РФ «О стандартизации» рекомендует использовать стандартизацию на предприятии для освоения данным конкретным предприятием государственных, международных, региональных стандартов, а также для регламентирования требований к сырью, полуфабрикатам и т.п., закупаемым у других организаций. Эта категория стандартов обязательна для предприятия, принявшего этот стандарт. Но если в договоре на разработку, производство, поставку продукта или предоставление услуг имеется ссылка на стандарт предприятия, он становится обязательным для всех субъектов хозяйственной деятельности — участников такого договора.

Правила и рекомендации по стандартизации по своему характеру соответствуют нормативным документам методического содержания. Они могут касаться порядка согласования нормативных документов, предоставления информации о принятых стандартах отраслей, обществ и других организаций в Госстандарт РФ, создания службы по стандартизации на предприятии, правил проведения государственного контроля за соблюдением обязательных требований государственных стандартов и многих других вопросов организационного характера. Правила и рекомендации по стандартизации разрабатываются, как правило, организациями и подразделениями, подведомственными Госстандарту РФ. Проект этих документов обсуждается с заинтересованными сторонами, утверждается и издается этими комитетами.

Технические условия разрабатывают предприятия и другие субъекты хозяйственной деятельности в том случае, когда стандарт создавать нецелесообразно. Объектом ТУ может быть продукция разовой поставки, выпускаемая малыми партиями и т.п. [48].

1.2. Стандарты в области программного обеспечения

В Толковом словаре по информатике В.И. Першикова и В.М. Савинкова [52] понятие *стандартизация* определяется как принятие соглашения по спецификации, производству и использованию аппаратных и программных средств вычислительной техники; установление и применение стандартов, норм, правил и т.п.

Стандарты имеют большое значение — они обеспечивают возможность разработчикам программного обеспечения использовать данные и программы других разработчиков, осуществлять экспорт/импорт данных.

Такие стандарты регламентируют взаимодействие между различными программами. Для этого предназначены стандарты межпрограммного интерфейса, например OLE (Object Linking and Embedding — связывание и встраивание объектов). Без таких стандартов программные продукты были бы «закрытыми» друг для друга.

Стандарты занимают все более значительное место в направлении развития индустрии информационных технологий. Более 250 подкомитетов в официальных организациях по стандартизации работают над стандартами в области информационных технологий. Более 1000 стандартов или уже приняты этими организациями, или находятся в процессе разработки. Процесс стандартизации информационных технологий далеко не закончен (да, по нашему мнению, вряд ли когда-либо будет закончен, так как область информационных технологий постоянно динамично развивается).

Все компании-разработчики должны обеспечить приемлемый уровень качества выпускаемого программного обеспечения (ПО). Для этих целей предназначены стандарты качества программного обеспечения или отдельные разделы в стандартах разработки программного обеспечения, посвященные требованиям к качеству программного обеспечения.

С точки зрения пользователя, все многообразие ПО должно управляться единообразно. Должна быть единообразная навигация — перемещение по программе, единообразные органы управления ПО и единая реакция программного обеспечения на

действия пользователя. Для этого разработаны стандарты на пользовательский интерфейс — GUI (Graphical User Interface). Все это регламентируется стандартами, действующими в сфере информационных технологий.

Необходимость стандартизации разработки программного обеспечения наиболее удачно описана во введении в стандарт ISO/ IEC 12207: «Программное обеспечение является неотъемлемой частью информационных технологий и традиционных систем. таких, как транспортные, военные, медицинские и финансовые Имеется множество разнообразных стандартов, процедур, методов, инструментальных средств и типов операционной среды для разработки и управления программным обеспечением. Это разнообразие создает трудности при проектировании и управлении программным обеспечением, особенно при объединении программных продуктов и сервисных программ. Стратегия разработки программного обеспечения требует перехода от этого множества к общему порядку, который позволит специалистам, практикующимся в программном обеспечении, «говорить на одном языке» при разработке и управлении программным обеспечением. Этот международный стандарт обеспечивает такой общий порядок».

Попробуем внести порядок в многообразие стандартов, действующих в сфере ИТ, и классифицировать их (рис. 1.3).

Как видно, верхняя часть классификации напоминает указанные выше виды стандартов. Однако здесь появляются и свои особенности. Это относится прежде всего к стандартам «де-юре» и «де-факто». Давайте сразу определим эти понятия.

Стандарт «де-факто» — термин, обозначающий продукт какого-либо поставщика, который захватил большую долю рынка и который другие поставщики стремятся эмулировать, копировать или использовать для того, чтобы захватить свою часть рынка.

Одна из главных причин значимости современной программы стандартизации -— осознание опасности злоупотребления стандартами *«де-факто»,* В 60-е и 70-е годы XX века создание стандартов *«де-факто»* ставило пользователей в зависимое от производителей положение при использовании основных средств обработки данных и телекоммуникаций. Важный аспект сегодняшней работы по стандартизации — преодоление этой зависимости через продвижение стандартных интерфейсов. Долгое время та-

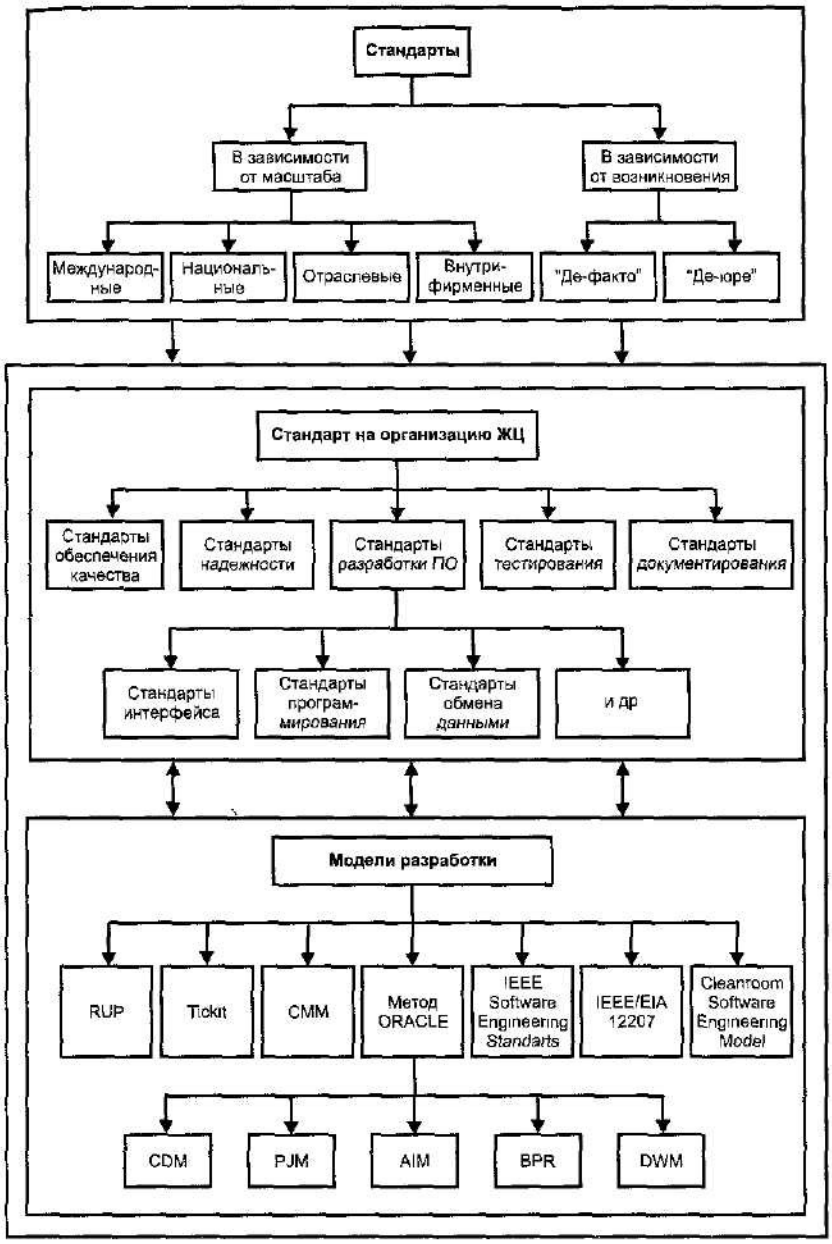


Рис. 1.3. Схема классификации стандартов в области информационных технологий

кими стандартами были SQL (Structured Query Language) и язык диаграмм Д. Росса SADT (Structured Analysis and Design Technique).

Стандарт «де-юре» создается формально признанной стандартизующей организацией. Он разрабатывается при соблюдении правил консенсуса в процессе открытой дискуссии, в которой каждый имеет шанс принять участие. Ни одна группа не может действовать независимо, создавая стандарты для промышленности. Если какая-либо группа поставщиков создаст стандарт, не учитывающий требования пользователей, она потерпит неудачу. То же самое происходит, если пользователи создают стандарт, с которым не могут или не будут соглашаться поставщики, —- этот стандарт также не будет успешным. Стандарты «де-юре» не могут быть изменены, не пройдя процесс согласования под контролем организации, разрабатывающей стандарты. Стандарты OSI (Open Systems Interconnection reference model), Ethernet, POSIX, SQL и большинство стандартов языков — примеры такого рода стандартов.

В качестве примера перехода стандарта «де-факто» в стандарт «де-юре» рассмотрим историю развития и стандартизации языка SQL.

Работы по созданию языка SQL были начаты в 70-х годах прошлого столетия в исследовательских лабораториях компании IBM. В настоящее время он стал одним из главных стандартов в области информационных систем и обеспечил технологию базового языка для целого поколения СУБД, основанных на реляционной модели. Несмотря на то, что он был коммерчески реализован в начале 80-х *годов лишь для* небольшой группы программных продуктов, SQL, бесспорно, получил признание с принятием ANSI и ISO стандарта SQL-86. Позднее, при подготовке стандарта SQL-89, в язык был включен ряд дополнительных возможностей.

Истоки SQL следует отнести к периоду рождения реляционной модели данных (описанной в статье Э.Ф. Кодда) [65]. Поскольку в течение нескольких последующих лет не появилось никаких языков, подобных SQL, в исследовательских проектах, инициированных компанией IBM *после* публикации статьи Э.Ф. Кодда, придавалось особое значение необходимости создания языков интерфейса создаваемых СУБД для проверки возможностей реляционной модели. Историю разработки языка SQL иллюстрирует рис. 1.4.

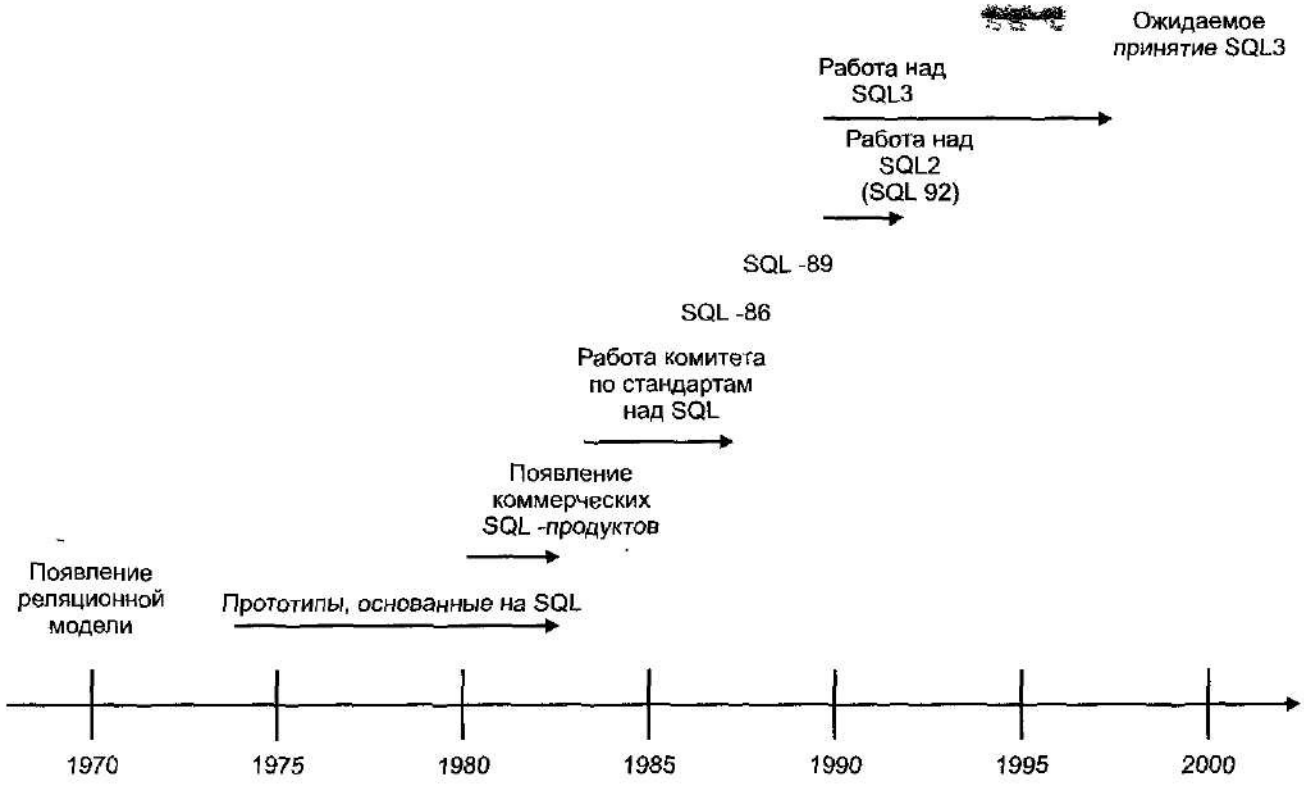


Рис. 1.4. Схема истории и истоков SQL

В исследовательских лабораториях IBM в начале 70-х годов 20 века одновременно с работой над будущим языком SQL разрабатывались и другие проекты по созданию и экспериментальной реализации реляционных языков. Вероятно, наиболее известным из них является созданный М. Злуфом из лаборатории IBM в Йорктаун-Хейтс примерно в одно и то же время с SEQUEL (ранней версией SQL) реляционный язык Query-By-Example (QBE). Этот язык используется в настоящее время во многих коммерческих программных продуктах наряду с SQL.

В 1974 г. Дональд Д. Чамберлин из Исследовательской лаборатории IBM в Сан-Хосе предложил спецификации реляционного языка, названного SEQUEL (Structured English QUEry Language). Пересмотренная версия этого языка (SEQUEL/2) была разработана в 1976—1977 годах, и он приобрел свое окончательное название — SQL (Structured Query Language).

Еще перед тем, как коммерческие продукты IBM в начале 80-х годов 20 века вышли на рынок программного обеспечения, компания Relation Software Inc. (называющаяся теперь Oracle Corporation) объявила о выпуске реляционной СУБД, основанной на языке SQL. Эта система в результате ее эволюции стала одной из доминирующих коммерческих систем и носит название ORACLE.

Продукт ORACLE с его языком SQL столкнулся с конкуренцией в сфере средних ЭВМ и мини-ЭВМ со стороны продуктов ряда других разработчиков, в частности СУБД Ingres с языком QUEL компании Relation Technology Inc., а также продукта Rdb/VMS с языком RDML компании Digital Equipment Corporation.

Одной из причин преуспевания SQL послужило формирование Американским национальным институтом стандартов (American National Standards Institute, ANSI) комитета X3H2, учрежденного для разработки стандартов языков баз данных. Представитель IBM предложил использовать в качестве предварительных спецификаций реляционного языка результаты ранее проведенной IBM работы над SEQUEL/2, и разработчики стандарта приступили к работе. Документ, озаглавленный «SQL», представлял собой по большей части трактат о различных формах SQL, используемых в коммерческих программных продуктах.

Международная организация по стандартизации (International Standards Organization, ISO) в рамках технического комитета TC97 (называемого теперь как ISO/IEC JTC1) также вела работу по

созданию стандарта языков реляционных баз данных. В середине 80-х годов как ANSI, так и ISO одобрили стандарты SQL (ANSI — в 1986 г., а ISO — в начале 1987 г.).

Первый стандарт SQL в связи со способом его разработки был весьма неполным в части функциональных возможностей систем баз данных, и многие из поставщиков продолжали вносить в свои программные продукты большой ряд расширений к стандарту. В 1989 г. была принята пересмотренная версия стандарта SQL, которая отличалась от стандарта 1986 г. главным образом именно возможностями поддержки целостности по ссылкам.

Однако еще до 1989 г. как в ANSI, так и в ISO началась работа по радикальным расширениям SQL. Эта работа, первоначально идентифицированная как «SQL2», началась в 1987 г., и ее результаты были спустя пять лет приняты в качестве стандарта SQL-92.

Добавляет путаницы еще и то обстоятельство, что работа над SQL2 перекрывалась работой над SQL3, новой версией стандарта SQL, которая еще до сих пор находится в стадии разработки. Работа над SQL3 началась еще в 1990 г. параллельно с SQL2, главным образом как над «запасным резервуаром» для возможностей, которые предполагалось не включать по тем или иным причинам в SQL2. SQL3 включает объектно-ориентированные расширения языка, а также дополнительные реляционные возможности, которым не нашлось места в SQL2 [53].

Проиллюстрируем на рис. 1.5 с привязкой к оси времени процесс разработки различных стандартов SQL.

Следует отметить, что в области информационных технологий существуют два основных исторически сложившихся подхода к

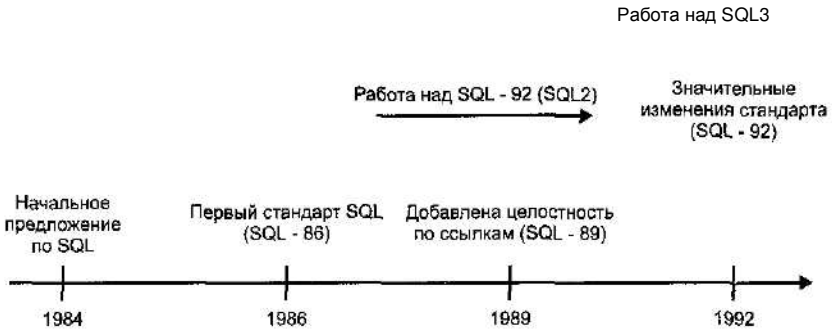


Рис. 1.5. Схема истории стандартизации SQL

разработке стандартов. Первый — когда назревает проблема, — необходимость в стандарте. В этом случае собирается группа экспертов в каком-то разделе информационных технологий и обсуждает локальные решения, придуманные отдельными компаниями — производителями программного обеспечения и научными организациями, проводит анализ этих решений и разрабатывается единый интегральный стандарт, который включает в себя лучшие идеи и наработки.

Но рынок живет по несколько иным законам. Первый подход обладает инертностью, проблема уже назрела, ее надо решать, и неизвестно, когда соберутся эксперты и разработают необходимый стандарт. Во втором случае компании — разработчики ПО разрабатывают каждая свое решение, и самое популярное, массовое с точки зрения частоты использования решение обретает статус стандарта (необязательно юридически). Так, SQL стал стандартом языка обращения к базам данных, что называется «де-факто», хотя потом статус стандарта был закреплен юридически.

Недостаток этого подхода состоит в том, что стандартом становится не самое сильное, а самое массовое коммерческое решение.

В качестве еще одного примера появления стандарта можно привести появление ныне популярного UML — Unified Modeling Language. Основные разработки по методам объектно-ориентированного анализа и проектирования появились между 1988 и 1992 гг. К 1994 г. было большое количество неформальных лидеров разработчиков-практиков (около полутора десятков), которые продвигали свои методологии. Все их методы были схожи, при этом зачастую отличия между ними заключались во второстепенных деталях. Назревал разговор о стандартизации. Команда из OMG пыталась рассмотреть проблему стандартизации, но в ответ получила открытое письмо с протестом от всех авторов. В 1996 г. три ведущих специалиста в области объектно-ориентированного анализа и проектирования Джеймс Рамбо (James Rumbaugh), Гради Буч (Grady Booch), Ивар Якобсон (Tvar Jacobson) объединились, и появился на свет Унифицированный метод версии 0.8, в 1996 г. «трое друзей» работали над своим методом, который получил название Unified Modeling Language. В январе 1997 г. различные организации представили свои предложения по стандартизации методов, предусматривающие в первую очередь возможность обмена информацией между различными моделями. В результате сейчас имеется единственное предложение — стандарт UML.

1.3.
Международные организации, разрабатывающие стандарты

Международная организация по стандартизации (ИСО)

Международная организация по стандартизации создана в 1946 г. двадцатью пятью национальными организациями по стандартизации.

При создании организации и выборе ее названия учитывалась необходимость того, чтобы аббревиатура наименования звучала одинаково на всех языках. Для этого было решено использовать греческое слово «isos» — равный. Вот почему на всех языках мира Международная организация по стандартизации имеет краткое название ISO (ИСО).

Сфера деятельности ИСО касается стандартизации во всех областях, кроме электротехники и электроники, относящихся к компетенции Международной электротехнической комиссии (МЭК). Некоторые виды работ выполняются совместными усилиями этих организаций. Кроме стандартизации ИСО занимается и проблемами сертификации.

ИСО определяет свои задачи следующим образом: содействие развитию стандартизации и смежных видов деятельности в мире с целью обеспечения международного обмена товарами и услугами, а также развития сотрудничества в интеллектуальной, научно-технической и экономической областях.

Вопросы информационной технологии, микропроцессорной техники и т.п. входят в область совместных разработок ИСО/МЭК. В последние годы ИСО уделяет много внимания стандартизации систем обеспечения качества. Практическим результатом усилий в этих направлениях являются разработка и издание международных стандартов. При их разработке ИСО учитывает ожидания всех заинтересованных сторон — производителей продукции (услуг), потребителей, правительственных кругов, научно-технических и общественных организаций.

На сегодняшний день в состав ИСО входят 120 стран своими национальными организациями по стандартизации. Россию представляет Госстандарт РФ в качестве комитета — члена ИСО. Всего в составе ИСО более 80 комитетов-членов. Кроме комитетов-

членов членство в ИСО может иметь статус членов-корреспондентов, которыми являются организации по стандартизации развивающихся государств.

Довольно широки деловые контакты ИСО: с ней поддерживают связь около 500 международных организаций, в том числе все специализированные агентства ООН, работающие в смежных направлениях.

ИСО поддерживает постоянные рабочие отношения с региональными организациями по стандартизации. Практически члены таких организаций одновременно являются членами ИСО. Поэтому при разработке региональных стандартов за основу принимается стандарт ИСО нередко еще на стадии проекта. Наиболее тесное сотрудничество поддерживается между ИСО и Европейским комитетом по стандартизации (СЕН).

Крупнейший партнер ИСО — Международная электротехническая комиссия (МЭК). В целом эти три организации охватывают международной стандартизацией все области техники. Кроме того, они стабильно взаимодействуют в области информационных технологий и телекоммуникации.

Международные стандарты ИСО не имеют статуса обязательных для всех стран-участниц. Любая страна мира вправе применять или не применять их. Решение вопроса о применении международного стандарта ИСО связано в основном со степенью участия страны в международном разделении труда и состоянием ее внешней торговли.

Международная электротехническая комиссия (МЭК)

Международная электротехническая комиссия создана на международной конференции, в работе которой участвовали 13 стран, в наибольшей степени заинтересованных в такой организации. Датой начала международного сотрудничества по электротехнике считается 1881 г., когда состоялся первый Международный конгресс по электричеству. Позже, в 1904 г., правительственные делегаты конгресса решили, что необходима специальная организация, которая бы занималась стандартизацией параметров электрических машин и терминологией в этой области.

После второй мировой войны, когда была создана ИСО, МЭК стала автономной организацией в ее составе. МЭК занимается стандартизацией в области электротехники, электроники, радио-

связи, приборостроения. Эти области не входят в сферу деятельности ИСО [48].

Объединенный технический комитет (JTC1)

В 1987 г. ИСО и МЭК объединили свою деятельность в области стандартизации информационных технологий (ИТ), создав единый орган JTC1 (Joint Technical Committee 1 — Объединенный технический комитет 1), предназначенный для формирования всеобъемлющей системы базовых стандартов в области ИТ и их расширений для конкретных сфер деятельности.

JTC1 имеет 17 подкомиссий, чья работа покрывает все: от техники программного обеспечения до языков программирования, компьютерной графики и обработки изображения, соединения оборудования, методов защиты и т.д. Работа над стандартами ИТ в JTC1 тематически распределена по подкомитетам (Subcommittees — SC).

В дополнение создана специальная группа по функциональным стандартам (Special Group on Functional Standards — SGFS) для обработки предложений по международным стандартизованным профилям (International Standardized Profiles — ISPs), представляющим определения профилей ИТ.

Ниже перечислены подкомитеты и группы JTC1, связанные с разработкой стандартов ИТ, относящихся к окружению открытых систем (Open Systems Environment — OSE):

C2 — Символьные наборы и кодирование информации;

SC6 — Телекоммуникация и информационный обмен между системами;

SC7 — Разработка программного обеспечения и системная документация;

SC18 — Текстовые и офисные системы;

SC21 — Открытая распределенная обработка (Open Distributed Processing — ODP), управление данными (Data Management — DM) и взаимосвязь открытых систем (OSI);

SC22 — Языки программирования, их окружение и интерфейсы системного программного обеспечения;

SC24 — Компьютерная графика;

SC27 — Общие методы безопасности для ИТ-приложений;

SGFS — Специальная группа по функциональным стандартам,

1.4.
Национальные организации, разрабатывающие стандарты

Среди национальных организаций, разрабатывающих стандарты, мы рассмотрим только две организации, которые интересуют нас в наибольшей степени. Это Государственный комитет РФ по стандартизации и Американский национальный институт стандартов и технологии.

Государственный комитет РФ по стандартизации

Согласно Руководству 2 ИСО/МЭК деятельность по стандартизации осуществляют соответствующие органы и организации. Орган рассматривается как юридическая или административная единица, имеющая конкретные задачи и структуру. Это могут быть органы власти, фирмы, учреждения.

Под органом, занимающимся стандартизацией, подразумевается орган, деятельность которого в области стандартизации является общепризнанной на национальном, региональном или международном уровне. Основные функции такого органа — разработка и утверждение нормативных документов, доступных широкому кругу потребителей. Однако он может выполнять немало других функций, что особенно характерно для национального органа по стандартизации.

Национальным органом по стандартизации в России является Государственный комитет Российской Федерации по стандартизации и метрологии (Госстандарт России). Это федеральный орган исполнительной власти, осуществляющий межотраслевую координацию, а также функциональное регулирование в области стандартизации, метрологии и сертификации.

Государственный комитет Российской Федерации по стандартизации и метрологии — правопреемник упраздненного Министерства промышленности и торговли Российской Федерации в отношении функций по реализации государственной политики в сфере стандартизации, метрологии и сертификации.

Государственный комитет Российской Федерации по стандартизации и метрологии — специально уполномоченный федеральный орган исполнительной власти в области сертификации. Пред-

седатель Государственного комитета Российской Федерации по стандартизации и метрологии является главным государственным инспектором Российской Федерации по надзору за государственными стандартами и обеспечением единства измерений.

В ведении Государственного комитета Российской Федерации по стандартизации и метрологии находятся службы по надзору за государственными стандартами и обеспечением единства измерений, а также центры стандартизации, метрологии и сертификации, предприятия, учреждения, учебные заведения и иные организации.

Госстандарт России выполняет следующие функции:

- координирует деятельность государственных органов управления, касающуюся вопросов стандартизации, сертификации, метрологии;
- взаимодействует с органами власти республик в составе РФ и других субъектов Федерации в области стандартизации, сертификации, метрологии;
- направляет деятельность технических, комитетов и субъектов хозяйственной деятельности по разработке, применению стандартов, другим проблемам сообразно своей компетенции;
- подготавливает проекты законов и других правовых актов в пределах своей компетенции;
- устанавливает порядок и правила проведения работ по стандартизации, метрологии, сертификации;
- принимает большую часть государственных стандартов, общероссийских классификаторов технико-экономической информации;
- осуществляет государственную регистрацию нормативных документов, а также стандартных образцов веществ и материалов;
- руководит деятельностью по аккредитации испытательных лабораторий и органов по сертификации;
- осуществляет государственный надзор за соблюдением обязательных требований стандартов, правил метрологии и обязательной сертификации;
- представляет Россию в международных организациях, занимающихся вопросами стандартизации, сертификации, метрологии, и в межгосударственном совете СНГ;
- сотрудничает с соответствующими национальными органами зарубежных стран;

- руководит работой научно-исследовательских институтов и территориальных органов, выполняющих функции Госстандар та в регионах;
- осуществляет контроль и надзор за соблюдением обязательных требований государственных стандартов, правил обязательной сертификации;
- участвует в работах по международной, региональной и меж государственной (в рамках СНГ) стандартизации;
- устанавливает правила применения в России международных, региональных и межгосударственных стандартов, норм и ре комендаций;
- при разработке государственных стандартов определяет орга низационно-технические правила, формы и методы взаимодей ствия субъектов хозяйственной деятельности как между собой, так и с государственными органами управления, которые бу дут включены в нормативный документ;
- организует подготовку и повышение квалификации специалис тов в области стандартизации.

В организационной структуре Госстандарта предусмотрены подразделения для реализации значительного объема работ: г9 научно-исследовательских институтов, 13 опытных заводов, издательство, 2 типографии, 3 учебных заведения, более 100 территориальных центров стандартизации, метрологии и сертификации (ЦСМ). На базе территориальных органов Госстандарта создаются органы по сертификации и испытательные лаборатории. По данным на 1996 г., было аккредитовано более 500 органов по сертификации различных видов услуг и около 2000 испытательных лабораторий.

Работы по государственной стандартизации планируются. Составление планов находится в ведении Госстандарта РФ, который является основным заказчиком по государственным основополагающим стандартам, стандартам общих технических условий и технических условий в части их обязательных требований, по исследованиям в области международных и региональных стандартов относительно принятия и применения их в качестве государственных. Данная норма прописана в п.4 статьи 15 Конституции Российской Федерации: «Общепризнанные принципы и нормы международного права и международные договоры Российской Федерации являются составной частью ее правовой системы. Если международным договором Российской Федерации

установлены иные правила, чем предусмотрено законом, то применяются правила международного договора».

Госстандарт определяет стратегические направления по государственной стандартизации, анализирует все заказы, планы работы технических комитетов, предложения от субъектов хозяйственной деятельности и разрабатывает планы по государственной стандартизации, как правило годовые. Приоритетными считаются задания по гармонизации отечественных нормативных документов с международными (региональными), национальными зарубежными стандартами, а также по разработке требований безопасности к объектам стандартизации и защите прав потребителей. Выполнение планов государственной стандартизации финансируется из государственного бюджета и контролируется Госстандартом **РФ.**

Технические комитеты по стандартизации. Постоянными рабочими органами по стандартизации являются технические комитеты (ТК), но это не исключает разработку нормативных документов предприятиями, общественными объединениями, другими субъектами хозяйственной деятельности. ТК могут заниматься стандартизацией как в инициативном порядке, так и по договорам на выполнение такого задания в соответствии с программами ТК и планами государственной стандартизации.

Технические комитеты специализируются в зависимости от объекта стандартизации. В рамках этой специализации в ТК проводится также работа и по международной (региональной) стандартизации.

По линии международной стандартизации ТК занимаются вопросами гармонизации отечественных стандартов с международными, готовят обоснование позиции России для голосования по проектам стандартов в международных организациях, участвуют в работе ТК международных (региональных) организаций по стандартизации, способствуя принятию государственных стандартов РФ в качестве международных, участвуют в организации проведения в России заседаний международных организаций по стандартизации и др.

Научно-технической базой для создания ТК обычно служат предприятия или организации, профиль деятельности которых соответствует специализации технического комитета. В их число включаются и научно-исследовательские институты Госстандарта РФ. Правовой основой для создания ТК служит решение этих

государственных органов. Заинтересованные предприятия, организации могут проявлять инициативу по участию их специалистов в работе технического комитета. Госстандарт РФ привлекает к работе в ТК ведущих ученых и специалистов, представителей организаций — разработчиков продукции, производственных предприятий (фирм), предприятий — основных потребителей продукции (услуг), научных и инженерных обществ и обществ по защите прав потребителей. Последнему придается особое значение, поскольку через представителей этих обществ осуществляется обратная связь с потребителем, что дает возможность получать актуальную информацию, необходимую для выполнения одной из основных целей стандартизации: обеспечить соответствие продукта ожиданиям и предпочтениям потребителя. Общества потребителей имеют право участвовать в работе технических комитетов по определению требований к качеству объекта стандартизации и выбору методов его оценки, в разработке новых и обновлении действующих стандартов.

Участие в деятельности технических комитетов всех заинтересованных сторон добровольное.

Другие службы по стандартизации. Другие субъекты хозяйственной деятельности, разрабатывающие нормативные документы (стандарты отраслей и предприятий), создают в своей организационной структуре специальные службы, которые координируют работу по созданию стандартов других участвующих в этом подразделений. Например, на предприятии научно-исследовательские, конструкторские и технологические отделы, лаборатории выполняют исследования, связанные со стандартизацией, а участие других подразделений определяется их компетенцией. Руководит работой отдел стандартизации.

Американский национальный институт стандартов и технологий

Национальным органом по стандартизации в США является Американский национальный институт стандартов и технологии (NIST). Его предшественники: • Американский комитет технической стандартизации, который

в 1928 г. был реорганизован в Американскую ассоциацию по стандартизации (ASA);

• Организация по стандартизации США (USASI), просуществовавшая менее трех лет и преобразованная в ANSI, а теперь — NIST.

NIST — неправительственная некоммерческая организация, координирующая работы по добровольной стандартизации в частном секторе экономики, руководящая деятельностью организаций — разработчиков стандартов, принимающая решения о придании стандарту статуса национального (если в нем заинтересованы различные фирмы и стандарт приобретает межотраслевой характер). NIST не разрабатывает стандарты, но является единственной организацией в США, принимающей (утверждающей) национальные стандарты. Это отвечает основной задаче NIST — содействию решения проблем, имеющих общегосударственное значение (экономия энергоресурсов, защита окружающей среды, обеспечение безопасности жизни людей и условий производства).

Институт разрабатывает целевые программы. Программно-целевое планирование охватывает производство и транспортировку топлива, снабжение электроэнергией, применение ядерной, солнечной и других видов энергии. Значительно меньше внимания уделяется разработке стандартов на готовую продукцию, поскольку в этой области действуют фирменные нормативные документы.

Национальные (федеральные) стандарты содержат обязательные к выполнению требования, касающиеся в основном аспектов безопасности. Наряду с обязательными федеральными стандартами в США действуют технические регламенты, утверждаемые органами государственного управления — Министерством торговли, Министерством обороны, Управлением служб общего назначения, Федеральным агентством по охране окружающей среды, Федеральным агентством по охране труда и здоровья на производстве, Федеральным управлением по безопасности пищевых продуктов и медикаментов, Комиссией по безопасности потребительских товаров и некоторыми другими. NIST поддерживает тесные деловые контакты с этими организациями, в частности, по информационному обеспечению фирм, частных организаций, разрабатывающих стандарты. Сами указанные выше органы управления нередко участвуют в разработке фирменных стандартов и учитывают наличие таковых при планировании создания федерального стандарта. Нередки случаи, когда фирменный стандарт, удовлетворяя их требованиям, принимается в качестве федерального.

Разрабатывают федеральные стандарты авторитетные организации, аккредитованные Американским национальным институтом стандартов. Наиболее известные из них:

- Американское общество по контролю качества (ASQC);
- Американское общество инженеров-механиков (ASME);
- Институт инженеров по электротехнике и электронике (IEEE)

и др.

Эти организации разрабатывают не только федеральные стандарты, но и стандарты, носящие добровольный характер. Всего в США разработкой добровольных стандартов занимаются более 400 различных организаций и фирм, а добровольных стандартов насчитывается более 35 тыс. [48].

На сегодняшний день членами NIST состоят более 1200 фирм, свыше 250 производственных и торговых компаний, научно-технических и инженерных обществ.

1.5. Внутрифирменные (внутрикорпоративные) стандарты

/ Внутрифирменные стандарты действуют внутри организации — разработчика программного обеспечения или любой другой компании, связанной с информационными технологиями. Такие стандарты, как правило, регламентируют порядок оформления документации, приказов и технической литературы внутри компании, пользовательский интерфейс разрабатываемых приложений (например, запрет на использование некоторых элементов интерфейса), стиль программирования, спецификацию модулей, имена используемых переменных, таблиц баз данных (БД). Внутрикорпоративные (внутрифирменные) стандарты имеют узкую сферу полномочий (одна или несколько фирм), но играют большую роль, так как они абсолютно конкретны.

Назначение и классификация внутрикорпоративных стандартов

Как уже отмечалось выше, внутрифирменные (внутрикорпоративные) стандарты занимают особое место в классификации стандартов. Это связано с тем, что они регламентируют технологические процессы, происходящие внутри фирмы (например,

процессы анализа, кодирования, тестирования), они максимально конкретны и детализируют уровень мероприятий, если пользоваться управленческой терминологией.

Внутрифирменные стандарты, как правило, базируются на применении методик и технологий, которые:

- зарекомендовали себя лучшим образом в аналогичных проектах;
- получили наибольшее распространение в области разработки программного обеспечения;
- получили наибольшее распространение в области, для которой программное обеспечение создается;
- являются передовыми и многообещающими.

Вместе с тем внутрифирменные стандарты учитывают особенности предприятия — разработчика программного обеспечения. Его конкретные особенности связаны со средством разработки, на котором кодируется программное средство, квалификацией персонала, финансовым положением фирмы.

Можно ли разработать универсальный стандарт и тиражировать его на различных предприятиях? Увы, нет. Существуют общие подходы, известны технологии разработки внутрикорпоративных стандартов, но всякий раз этот процесс уникален, поскольку не существует двух совершенно одинаковых предприятий — они различаются отраслевой спецификой, размерами, стратегией, организационной структурой и многими другими факторами. Кроме того, документы, особенно относящиеся к внутреннему документообороту, различаются в силу устоявшихся бизнес-правил, традиций, корпоративной культуры, отношений между подразделениями. Внутрикорпоративные стандарты, разработанные для одного предприятия, не подойдут для другого. Поэтому типового внутрикорпоративного стандарта просто не может быть. При этом следует различать структуру бизнес-процессов, которая действительно может быть типовой, и внутрикорпоративный стандарт, согласующий структуру бизнес-процессов и организационную структуру конкретного предприятия.

Любой внутрикорпоративный стандарт должен иметь юридическую силу внутри предприятия, т.е. быть оформлен в виде документа и быть введен в действие приказом или распоряжением. В приказе ввода в действие внутрикорпоративного стандарта, как правило, должны содержаться следующие пункты:

- срок действия стандарта (например, «со дня подписания», «с 1 мая 2002 г.»);

- область действия (распространяется на процесс кодирования и, тестирования);
- способ доведения до исполнителей (например, «Руководителям подразделений зачитать приказ в вверенных им подразделе ниях»);
- ответственные лица за контролем исполнения (например, «Кон троль за исполнением стандарта»);
- ответственность (например, «За невыполнение пунктов стан дарта сотрудник лишается премии»).

Если вышеперечисленные пункты отсутствуют, то сложнее разбирать конфликтные ситуации, которые могут произойти. Если стандарт вообще не оформлен в виде документа, то фактически это обозначает, что его не существует вовсе, в этом случае конфликтные ситуации неизбежны. На практике на вопрос о правомерности применения того или иного проектного решения (например, использования элемента интерфейса) можно услышать; «Так было всегда». Такая практика вредна, стандарт должен быть оформлен, а не передаваться старожилами из уст в уста.

Выявляется и ряд отрицательных моментов, связанных с внутрифирменными стандартами. Первый момент — стандарты должны тщательно разрабатываться, продумываться, и, создавая их, фирма должна учесть большое количество нюансов, чтобы не переделывать стандарт через месяц. Стандарт — это то, что дает стабильность. Второй момент находится в некотором противоречии с первым — стандарты могут тормозить использование современных технологий, средств. Это особенно важно в сфере информационных технологий, где развитие технологий и их смена идут очень быстро. Этого можно избежать, если разработать внутри фирмы механизм регулярного пересмотра стандарта для включения в него современных и передовых элементов. В комиссию по пересмотру стандартов должны входить специалисты высокой квалификации из всех заинтересованных подразделений, мнение конечного потребителя также должно быть учтено (например, если вопрос касается пользовательского интерфейса или совместимости с другими программными средствами).

Классификация внутрифирменных стандартов. Внутрифирменные стандарты можно разделить по отношению к процессам производства (рис. 1.6).



Рис. 1.6. Схема классификации внутрифирменных стандартов

Производственные стандарты — те стандарты, которые регламентируют процессы производства программного обеспечения по этапам и стадиям жизненного цикла.

Управленческие стандарты регламентируют *порядок* управления производственными процессами.

Для чего нужны внутрифирменные стандарты. Какую пользу дают внутрифирменные стандарты? С их помощью:

- достигаются лучшие показатели обучения персонала. Соответственно проще заменить человека в случае его увольнения. Отсюда следует, что можно брать на работу специалистов более низкой квалификации и доучивать их на месте без серьезных затрат для фирмы;
- повышаются надежность и качество программного обеспечения;
- повышается дружественность программного продукта, сокращаются сроки обучения конечного пользователя;
- улучшается обслуживание, сокращаются сроки внедрения программного продукта.

Внутрифирменные стандарты обычно создаются самыми квалифицированными людьми в своей области, которые хорошо знают разрабатываемый программный продукт, владеют методологией и богатой практикой создания программных средств, а также людьми, как правило, руководителями проекта или направлений, которые видят картину в целом, управляют процессом производства программного средства непосредственно и имеют связь с конечным пользователем.

Стандарт содержит структуру процессов, таблицы, матрицы, диаграммы, пояснительный текст. Очень важно определить компоненты, подлежащие включению во внутрифирменный стандарт, и те, которые включать в него не следует. Необходимо помнить,

что процесс документооборота представляет собой набор взаимосвязанных задач, каждая из которых имеет исполнителя, срок и ресурсы для подготовки одного документа. В последнем случае подразумеваются как календарные, так и информационные ресурсы, т.е. экономические показатели, характеризующие состояние и поведение объектов.

Нужно ли включать в стандарт характеристику показателей и алгоритм расчета? С одной стороны, да, поскольку на предприятиях могут существовать особенные формы и использоваться нетрадиционные показатели. С другой стороны, сами показатели и алгоритмы их расчета регламентированы совершенно иными документами: инструкциями, указами, приказами и распоряжениями, типовыми методиками и пр. Алгоритмы универсальны и не зависят от конкретного предприятия.

Процесс обработки показателей является многоуровневым. Самый нижний уровень — физический, характеризуется способом хранения данных. Следующий уровень обработки — логический, он характеризуется моделью данных и правилами сущностей-отношений. Функциональный уровень задает алгоритм расчета показателей, типовые методики расчетов и инструкции. Прикладной уровень — уровень интерфейса. Организационный уровень определяет отношения между подразделениями. Поскольку внутрифирменный стандарт — это стандарт организационный, именно этот уровень отношений должен быть определяющим. При этом данные для каждого документа подразделяются на входящие и исходящие. В части организации расчетов нас будут интересовать источники получения входящих показателей, т.е. формы, которые должны быть подготовлены к моменту расчета исходящих показателей текущего документа. Таким образом, для внутрифирменного стандарта имеет значение не столько алгоритм расчета, сколько перечень показателей и их источники.

Подведем некоторые итоги. Внутрифирменный стандарт представляет собой структурированное формализованное описание бизнес-процессов. Цель его разработки — перепроектирование бизнес-процессов, обеспечивающих повышение качества работы предприятия и рост его конкурентоспособности, устранение ненужных функций, дублирование процессов, уточнение организационной структуры, улучшение организации документооборота, пересмотр содержания и количества документов, определение календарного графика подготовки документов, постановка зада-

чи для автоматизации деятельности предприятия. Как показывает опыт, имеет смысл разрабатывать стандарт в том случае, когда требуется согласовать деятельность по разработке документов нескольких подразделений.

При определении структуры внутрифирменного стандарта возьмем за основу требования ГОСТа к оформлению конструкторско-технологической документации. В соответствии с этими требованиями стандарт должен иметь следующие компоненты:

- назначение;
- область применения;
- термины и сокращения;
- ответственность;
- срок действия;
- описание методики;
- указания и примечания;
- порядок разработки и предоставления пользователям;
- порядок внесения изменений;
- приложения.

Применительно к стандартам организации документооборота внутрифирменный стандарт может иметь следующую структуру.

1. Назначение стандарта.
2. Дерево задач.
3. Описание задач:
   - исполнитель;
   - срок;
   - наименование документа;
   - предшествующие документы;
   - входящие показатели (идентификатор, наименование, ис точник);
   - исходящие показатели.
4. Структура документов.
5. Матрица согласования.
6. Сетевой график.
7. Глоссарий показателей.

В целом внутрифирменный стандарт представляет собой текстовый документ с приложениями в виде диаграмм и таблиц. Для разработки дерева задач удобно использовать инструментальные средства, такие, как Design/IDEF или BPwin. Сетевой график удобно проектировать с использованием Microsoft Project или Time Line. Если внедрением стандарта документооборота занимаются сотруд-

ники предприятия, а не внешние консультанты, у них могут возникнуть проблемы чтения и трактовки диаграмм. В этом случае в самом документе лучше использовать упрощенные средства изображения. Дерево задач может быть приведено без использования методологии IDEF0, в виде нумерованного списка. Сетевой график может отсутствовать, а вместо него следует предусмотреть календарный график подготовки документов. Так удобно поступать в том случае, когда известна контрольная дата и она неизменна (например, разработка планов на следующий год).

Во внедрении стандартов в наибольшей степени заинтересованы менеджеры. Формализация процессов позволяет регламентировать отношения с подчиненными, устранить дублирование функций. Присутствуют здесь и отрицательные моменты, в основном касающиеся осведомленности и личных взаимоотношений. В наибольшей степени пострадают владельцы бизнес-процессов, что обусловлено изменением должностных обязанностей, обострением межличностных отношений, усилением ощущения нестабильности. Управленцев среднего звена могут коснуться как проблемы подчиненных, так и вышестоящих начальников.

Форма изменений может различаться; это могут быть постепенные улучшения процессов или коренная реструктуризация. Сторонниками коренной перестройки, как правило, выступают молодые руководители, зачастую не обремененные личными отношениями с подчиненными в такой степени, как если бы они постепенно продвигались по служебной лестнице, обретая сторонников и единомышленников. Поэтому слом коммуникативных связей не затрагивает их личных отношений.

Несмотря на то, что реинжиниринг в большей степени требуется крупным предприятиям, их руководители неохотно идут на изменения. Во-первых, в силу собственного консерватизма, во-вторых, в силу наличия внешних собственников и иных лиц, заинтересованных в результатах деятельности предприятия. Руководитель единолично не всегда может являться инициатором реинжиниринга из-за ограниченных полномочий. Еще один фактор: на крупных предприятиях сложилась громоздкая иерархическая структура со своими устойчивыми отношениями. Любые изменения могут вызвать негативную реакцию подчиненных. Так, расширение должностных обязанностей сотрудника без очевидной мотивации может привести к формальному отношению его к вновь появившимся задачам. Недовольство может проявляться в

ухудшении качества, отсутствии инициативности, невнимательности и т.п. Все эти издержки проявляются в потере клиентов, ухудшении имиджа организации.

Как правило, исполнители опасаются изменения должностных обязанностей, нарушений существующих взаимоотношений с другими сотрудниками, снижения заработной платы, увольнений. Что касается руководителей нижнего и среднего звена, то здесь в большей степени присутствуют опасения сужения сферы влияния, ведущей к разрушению межличностных контактов, формализации отношений, ограничения владения информацией.

От внедрения внутрифирменных стандартов руководители ожидают оптимизации бизнес-процессов, в результате чего сотрудникам будут делегированы четкие полномочия при упорядочении документооборота, определении актуальности данных, для улучшения работы с поставщиками и заказчиками. Самое же главное, они ожидают изменения корпоративной культуры: роста инициативности подчиненных; переориентации на результат деятельности, а не на процесс; командной работы. К сожалению, именно ожидания по части организационной культуры не оправдываются. Если сотрудники слабо мотивированы (как морально, так и материально), если они не представляют цели и результата бизнес-процесса, то реструктуризация приводит к формальному (в лучшем случае) выполнению должностных обязанностей.

Причиной разочарования в реинжиниринге в первую очередь может быть отсутствие быстрых видимых результатов при большом объеме капитальных вложений. Реинжиниринг требует значительных единовременных затрат, а результаты в виде улучшения обслуживания заказчиков, снижения непроизводственных затрат, расширения рынка сбыта и т.д. могут проявиться спустя некоторое время. Кроме того, затраты измеряются количественными показателями, а результат чаще всего выражается в виде экономического эффекта, который трудно выразить в денежном эквиваленте.

Организация разработки внутрифирменных стандартов

Разработка внутрифирменных стандартов должна проводиться с привлечением владельцев бизнес-процессов (персонала). Необходим аналитик, постановщик задачи. Общее руководство осуществляется директором предприятия, который инициирует

работы по проведению реинжиниринга и оказывает содействие в их реализации.

Приведем последовательносгь разработки внутрифирменного стандарта.

1. Определение дерева задач (оглавления стандарта).
2. Определение типовых форм для каждой задачи.
3. Назначение исполнителей.
4. Разработка матрицы, распределение ответственности.
5. Разработка календарного графика.
6. Описание входящих и исходящих показателей.
7. Составление глоссария терминов.

Группировка задач по разделам осуществляется логически, причем в соответствии с рекомендациями функциональной декомпозиции IDEF0 рекомендуется на одном уровне располагать от 2 до 8 задач.

Очень важно выбрать подходящий критерий декомпозиции. Группировать задачи можно различными способами, например: по функциональным областям; по последовательности создания документов; в соответствии со сложившимися правилами подготовки документа и др.

В том случае, когда не планируются кардинальные изменения бизнес-процессов, могут использоваться традиционные документы и неизменные исполнители; стоит лишь исключать дублирование функций. Если речь идет о перепроектировании, то на этом этапе следует очень тщательно подойти к отбору релевантных задач и выбрать показатели, характеризующие состояние и поведение экономических объектов. Рекомендуется подумать о целесообразности включения в стандарт тех или иных показателей.

После того как определен набор документов для включения в стандарт, рекомендуется разработать матрицу ответственности, где указываются исполнители, а также должности лиц, согласующих и утверждающих документы.

На следующем этапе следует определить структуру документов, если это не было сделано ранее.

Затем необходимо определить показатели. Здесь существует определенная сложность: не следует путать экономические объекты, экономические показатели и их значения. Для исходящих показателей необходимо определить источники информации.

Определив источники данных для показателей всех форм, разработчик внутрифирменного стандарта может разработать матрицу вхождения показателей, которая является исходным мате-

риалом для определения предшествующих задач. А это, в свою очередь, — необходимая информация для построения сетевого (календарного) графика.

После того как определены предшествующие показатели, представляется возможность разработать сетевой график. В самом документе внутрифирменных стандартов вид сетевого графика не очень удобен, поэтому можно отразить последовательность подготовки задач условными номерами периодов. Так, задачи со сроком исполнения, равным двум, выполняются обязательно после задач с единичным сроком исполнения. Под номерами можно подразумевать периоды, равные одному дню, неделе, месяцу. Периоды могут быть неравномерными.

Заключительные работы по разработке внутрифирменных стандартов проводятся по составлению глоссария терминов, где должны быть представлены экономические показатели, хотя бы один раз встретившиеся в формах. Обязательно приводятся определение и краткое наименование показателя. При необходимости дается алгоритм расчета или более детализированная характеристика.

В каждом конкретном случае разработки внутрифирменных стандартов потребуется решить не только описанные задачи, но и многие другие. Но самая большая проблема, с которой столкнется разработчик, — их внедрение. Как бы хороши ни были стандарты, как бы тщательно ни прорабатывались, их реализация неизбежно вызовет сопротивление: исполнители, являющиеся владельцами бизнес-процессов, не заинтересованы в изменении должностных обязанностей. В процессе реструктуризации исполнители играют пассивную роль, хотя иногда степень сопротивления изменениям настолько высока, что пассивной эту позицию назвать сложно. В нормальных условиях на предприятии создается команда, отвечающая за результат проведения реинжиниринга. Заинтересованность участников команды прямая, поскольку существует проект, распределены обязанности, обеспечивается контроль за проведением работ, распределяется финансирование [63].

Разработка и внедрение внутрифирменных стандартов — трудоемкие процессы, но в результате их оптимизируются бизнес-процессы, уточняется организационная структура, осуществляется постановка задачи формирования единого информационного пространства. Все эти факторы повышают качество деятельности предприятия, что непосредственно влияет на его конкурентоспособность.

Рассмотрим внутрифирменные стандарты в разрезе наиболее распространенных процессов разработки программного обеспечения: анализ и проектирование; кодирование; тестирование; документирование; внедрение; поддержка.

Общие стандарты, как правило, регламентируют общие моменты, связанные со вспомогательными процессами, касающимися деятельности, осуществляемой на фирме. Общие стандарты обычно регламентируют правила общения с клиентами компании, а также сотрудников внутри компании. Такой стандарт может содержать, например, правило формирования подписи электронного письма, состав реквизитов и порядок их следования.

Стандарт на подпись может быть таким:

Фамилия, Имя, Отчество (на русском) сотрудника, должность (на русском)
Отдел, Название фирмы (на русском)

Телефон (7-095) (код страны, код города), XXX-XX-XX (телефон)

Подпись, выполненная по такому стандарту, должна выглядеть так:

Иванов Иван Иванович, системный аналитик,
отдел системного анализа, ЗАО «Б&С»

Телефон (7-095) 964-16-44

Общие стандарты также содержат перечень программного обеспечения, предназначенного для регулярного использования и не связанного с особенностями работы в подразделении, например операционная система, электронная таблица, текстовый процессор, архиватор, программа работы с электронной почтой и др.

Очень важный момент, который обычно регламентируется общими стандартами, — это рабочее пространство для каждого подразделения. Под рабочим пространством понимается набор носителей для хранения различного рода информации со структурой каталогов и правами на них, а также правила работы с хранимой информацией.

На практике удобно выделять рабочее пространство на пространства:

- аналитиков;
- программистов;
- тестеров;
- специалистов отдела внедрения;
- технической поддержки.

Анализ и проектирование. Обычно для целей функционирования аналитического отдела разрабатывается ряд стандартов, которые регламентируют, как правило:

- применение методик структурного анализа или методов объектно-ориентированного анализа;
- описание бизнесс-процессов предметной области одним или несколькими программными средствами (Rational Rose, ERwin, BPwin и др.);
- ограничение или расширение использования отдельных элементов для выбранной методологии анализа или выбранного программного средства, поддерживающего эту методологию. Например, для объектно-ориентированного анализа, выполняемого с помощью Rational Rose, использование диаграмм состояний (State Diagram), диаграмм последовательности (Sequence Diagram);
- правила хранения проектно-аналитической документации (ПАД), правила кодирования имен файлов.

Например, всю проектно-аналитическую документацию стандарта на хранение одна из фирм — разработчиков банковского программного обеспечения разделила на следующие типы документов:

- Постановка задачи;
- Техническое задание;
- Спецификация;
- Аналитическая записка;
- Описание технологий;
- Настройки;
- Консалтинговый документ;
- Маркетинговый документ;
- Нормативный документ;
- Внутренний регламент банка;
- Внешний документ;
- Организационный документ;
- Рабочий документ.

По основным видам документов разрабатываются стандартные шаблоны, документ должен иметь обязательные части, например для постановки задачи может использоваться следующий стандартный шаблон:

Шапка.
Наименование постановки, код постановки
Автор, дата создания
Модифицировавший сотрудник, дата модификации

Тело постановки

Первичные данные для постановки:
Описание бизнес-процессов:
Постановка задачи:

Ограничения, допущения
Изменение в структуре данных
Изменение в структуре классов
Основная часть постановки

Приложения

Пример одного из стандартов для хранения проектно-аналитической документации приведен ниже.

Разработка. Стандарты разработки помогают разобраться в исходном коде программы, повышают читаемость исходного кода; использование стандартных шаблонов сокращает время разработки программного документа.

Разработка программного обеспечения включает в себя стандарты, которые регламентируют следующее.

- Формирование наименований. Может включать в себя язык об разования наименований, использование больших букв, прави ла формирования сложных наименований, правила формирова ния сокращенных наименований, формирование наименований процедур, формирование наименования состояний и переходов,
- Правила именования основных элементов модели системы (на пример, стереотип, класс, метод, форма, переопределение ме тодов и пр.).
- Структуру директорий разработки. Регламентирует располо жение директорий сборки, директорий исходных текстов, ди ректорий документации, директории базы данных.

- Документирование исходного кода.
- Регламент отладки программы. Использование заглушек, драйверов, отладочного протокола.
- Регламент использования конструкций языка программирования. Правила использования основных структур языка — циклов, условных операторов, операторов присваивания, операторов выбора. Например, может содержать запрет некоторых синтаксических особенностей: выход из цикла по оператору безусловного перехода; запрет на использование имен глобальных переменных в подпрограммах. Как правило, данный подстандарт описывает «правила хорошего тона» — то, что сложилось исторически, накоплено с опытом, связано с конкретным языком программирования.
- Визуальный интерфейс. Регламентирует использование элементов интерфейса, их взаимное расположение, выравнивание на экране.
- Сообщения, выдаваемые программой. Регламентирует использование видов сообщений, формирование текста сообщений, использование знаков препинания. Например, данным стандартом может быть запрещено использование сообщений в исходном тексте программы, для этого используется специальный файл сообщений, такой подход облегчает национальную локализацию (перевод интерфейса программы с одного национального языка на другой).
- Регламент проектирования базы данных.
- Регламент работы с программным обеспечением, используемым при разработке (среда разработки, компиляторы и пр.).
- Регламент программирования отдельных частей программного средства (механизмы настроек, программирования бизнес-транзакций, конверторов данных, многопользовательская работа и методы блокировки пользователей).
- Ведение версий разрабатываемого программного обеспечения.

Тестирование. Стандарты, связанные с тестированием и оценкой надежности программных средств, могут включать в себя:

- стандарт на разработку методики тестирования;
- стандарт на разработку и создание карт тестирования;
- регламент проведения нагрузочных испытаний.

В процессе тестирования (особенно при применении методов белого ящика) широко используются внутрифирменные стандарты разработки программного обеспечения.

Все вышеперечисленные стандарты, а также пункты, входящие в них, не являются догмой, т.е. могут быть расширены или сужены, все зависит от конкретной необходимости для предприятия — разработчика программного обеспечения.

Пример стандарта организации хранения аналитической информации

1. Назначение документа и область действия

Настоящий документ является частью корпоративного стандарта фирмы «Б&С». В документе описаны правила создания, хранения и удаления проектной аналитической документации.

Документ предназначен для специалистов Отдела Системного Анализа (ОСА) и других специалистов фирмы, осуществляющих подготовку и использование проектной аналитической документации.

Под *архивом* понимается совокупность проектной аналитической документации, разработанной или находящейся в ведении ОСА.

Под *проектно-апалитической документацией (НАД)* понимаются следующие типы документов.

Постановка задачи — документ, содержащий детальное описание задачи и *прикладных* требований к ней. Документ должен содержать описание варианта (вариантов) реализации данной задачи на концептуальном уровне.

Техническое задание — документ, содержащий, помимо описания прикладных требований, конкретные пути реализации задачи. Если это необходимо, техническое задание должно включать в себя логическую схему данных.

Спецификация — документ, содержащий *краткое* описание задачи и способа ее решения.

Аналитическая записка — документ, содержащий аналитическое исследование проблемы и не являющийся заданием на разработку.

Описание технологий — документ, описывающий технологическую реализацию различных задач в системе.

Настройки — файл, экспортированный из продукта фирмы и содержащий его настройки.

Консалтинговый документ — документ, содержащий материалы предпроектного исследования, информационно-технологического консалтинга и пр.

Маркетинговый документ — документ, выпускаемый сотрудниками отдела для публикации во внешнем мире. В документы данного типа входят статьи, доклады, рекламные материалы и пр.

Нормативный документ — документ, содержащий законодательный акт или инструкцию уполномоченного государственного органа, на основе которого выполняется подготовка проектной документации.

Внутренний регламент банка —■ документ, содержащий утвержденные банком для внутреннего использования регламент или инструкцию.

Внешний документ — документ, поступивший на фирму извне и содержащий описания технологий, материалы конкурентов, предметные статьи и пр.

Организационный документ — документ, регламентирующий работу отдела и фирмы.

Рабочий документ — другой документ, не попавший под вышеперечисленную классификацию.

При необходимости список типов документов может расширяться с одновременным внесением изменений в настоящие правила.

2. Правила ведения

архива 2.1. ***Общие***

положения

2.1.1. Разработанная специалистами отдела ПАД хранится в единичном экземпляре в разделяемом каталоге. Хранимая ПАД является первоисточником при разработке программного продукта, его тестировании и внедрении.

2.1.2. Доступ к ПАД осуществляется с помощью программно го средства Visual Source Safe. Создание, редактирование содер жания ПАД осуществляются только сотрудниками отдела систем ного анализа. Сотрудники других подразделений фирмы могут осуществлять только чтение ПАД.

2.1.3. Сотрудники других подразделений фирмы могут разра батывать проектные документы, необходимые для дальнейшего осуществления производственной деятельности. Например: тех нические задания, описание модели данных, методики тестирова ния, описание технологий внедрения и пр. Данные документы располагаются в соответствии со стандартами, принятыми в этих

подразделениях. При согласовании документов, подготовленных сотрудниками других подразделений и относящихся к ПАД, документ переносится в архив ПАД согласовавшим его сотрудником.

2.1.4. Предоставление прав доступа к архиву ПАД (заведение, удаление, изменение каталогов и пр.) осуществляется по запросу руководителем аналитического отдела или назначенным им со трудником.

2.1.5. Ссылки на наименование ПАД в других документах фирмБ1 осуществляются сотрудниками фирмы посредством уникаль ной идентификации документа в VSS.

2.2. *Идентификация ПАД*

Идентификация ПАД есть способ построения обозначений документов, используемых для ссылок на эти документы.

Документ идентифицируется посредством указания уникального пути в VSS или уникального кода, включаемого в наименование документа в виде префикса.

2.3. *Структура каталогов архива:*

№	Раздел / подраздел						Примечание
	1	2	3	4	5		
1	Автоматизированная система						
1.1		Продуктовый ряд					В разделе хранятся постановки задач, ТЗ и спецификации. Подразделы могут вестись в разрезе версий (в том числе индивидуальных для банков) и содержать настройки
1.1.1			Учетное ядро				
1.1.1.1				План счетов			
1.1.1.2				Клиенты			
1.1.1.3				Счета			

Продолжение

№	Раздел / подраздел						Примечание
	1	2	3	4	5		
1.1.1.4				Документы			
1.1.2			Расчетно-кассовое обслуживание юридических лиц				
1.1.2.1				Расчетные операции			
1.1.2.2				Кассовые операции			
1.1.2.3				Конверсионные операции			
1.1.3			Кредиты юридических лиц				
1.1.4			Векселя				
1.1.5			Розничное обслуживание				
1.1.5.1				Частные вклады			
1.1.5.2				Пластиковые карточки			
1.1.5.3				Пункт обмена валют			
1.1.5.4				Выносная операционная касса			
1.1.5 5				Коммунальные платежи			
1.1.6			Дилинг				
1.1.6.1				Валютный рынок			
1.1.6.2				Маржинальная торговля			
1.1.6.3				Денежный рынок			
1.1.6.4				Фондовый рынок			
1.1.7			Корреспондентские отношения				
1.1.8			Обмен электронными документами				
1.1.8.1				Расчеты через учреждения ЦБ			
1.1.8.1.1					МЦИ		
1.1.8.1.2					Региональные системы электронных расчетов		Ведется в разрезе форматов обмена региональных расчетно-кассовых центров
1.1.8.1.3					Системы расчетов национальных банков		Ведется в разрезе форматов обмена национальных банков

Продолжение

№	Раздел / подраздел						Примечание
	1	2	3	4	5		
1.1.8.2				Межбанковские расчеты			
1.1.8.2 1					SWIFT		
1.1.8.2.2					Расчеты с филиалами (TELEX)		
1.1.8.3				Расчеты по системе Клиент — Банк			
1.1.8.3.1					TELEX-BSS		
1.1 8.3.2					ARPO		
1.1.8.3.3					Dmsoft Client		
1.1.8.4				Обмен с внешними системами			Ведется в разрезе внешних систем
1.1.9			Депозитарный учет				
1.1.10			Доверительное управление				
1.1.11			Внутренняя бухгалтерия				
1.1.11.1				Учет материальных ценностей			
1.1.11.3.1					Основные средства		
1.1.11.1.2					Нематериальные активы		
1.1.11.1.3					Малоценные предметы		
1.1.11.1.4					Складской учет		
1.1.11.2				Расчет заработной платы			
1.1 11.3				Кадровый учет			
1.1.12			Финансовая отчетность и анализ				
1.1.12.3				Отчеты ЦБ			Ведется в разрезе отчетных форм ЦБ
3.1.12.2				Управленческая отчетность			Ведется в разрезе продуктов
1.1.13			Бюджетирование				
1.1.14			Хранилище данных филиалов				
1.1.15			Администрирование				
1.1.15.1			Настройки				

Продолжение

№	Раздел / подраздел						Примечание
	1	2	3	4	5		
1.1.15.1.1					Настроечные параметры		Ведется в разрезе продуктов
1.1.15.1.2					Классификаторы		Ведется в разрезе продуктов
1.1.15.1.3					Стандартные транзакции		Ведется в разрезе продуктов
1.1.15.1.4					Метасхема		Ведется в разрезе классов данных
1.1.15.2				Администрация			
1.1.15.2.1					Пользователи		
					Оргструктура банка		
					История изменений		
1.1.16			Ассамблея				
1.1.17			Анализ XL				
1.2		Общая архитектура системы					
1.3		Планы на версии системы					
2	Переписка с банками — клиентами						В разделе хранятся документы, полученные от банков или переданные им. Раздел ведется в разрезе банков и продуктов, по которым ведется переписка
3	Нормативные акты						В разделе хранятся документы, регулирующие банковскую деятельность. Раздел ведется в разрезе регулирующих органов
4	Исследовательские материалы						

Продолжение

№	Раздел / подраздел						Примечание
	1	2	3	4	5		
4.1		Обследование банков					В разделе хранятся консалтинговые документы обследования банков. Раздел ведется в разрезе банков и продуктов
4.2		Исследование технологий					В разделе хранятся внешние документы с описанием технологий (продуктов) сторонних фирм и аналитические записки по их исследованию. Раздел ведется в разрезе технологий (продуктов) и фирм
5	Справочные материалы						В разделе хранится справочная информация. Раздел ведется в разрезе справочников и их версий
6	Маркетинговые материалы						В разделе хранятся маркетинговые документы
6.1		Рекламные материалы и доклады					Раздел ведется в разрезе продуктов и маркетинговых мероприятий
6.2		Публикации					Раздел ведется в разрезе авторов — сотрудников фирмы БИС и печатных изданий

Продолжение

№	Раздел / подраздел						Примечание
	1	2	3	4	5		
6.3		Сравнительный анализ					В разделе хранятся аналитические записки, содержащие сравнительный анализ технологий (продуктов) сторонних фирм. Раздел ведется в разрезе технологий (продуктов) и фирм
7	Организационные материалы						В разделе хранятся организационные документы
7.1		Приказы, положения					Раздел ведется в разрезе подразделений фирмы БИС
7.2		Планы работ					Раздел ведется в разрезе периодов планирования и подразделений
8	Личные материалы						В разделе хранятся личные материалы сотрудников, которые необходимо архивировать. Раздел ведется в разрезе сотрудников. Структуру подразделов каждый сотрудник определяет самостоятельно

Продолжение

№	Раздел / подраздел						Примечание
	1	2	3	4	5		
9	Прочее						В разделе хранятся документы, не вошедшие ни в один из вышеперечисленных разделов. Структура подразделов уточняется по мере необходимости

Данная структура каталогов является исходной **и** может изменяться в процессе эксплуатации архива.

2.4. *Резервное **копирование архива***

Архив ПАД должен находиться на дисковом пространстве, подвергающемся регулярному резервному копированию.

Допускается уничтожать на диске устаревшие каталоги при условии, что данная информация не подвергается текущему редактированию, и при обязательной регистрации переноса каталога в резервное копирование с указанием в спецификации состава, даты переноса и конкретного носителя резервного копирования, содержащего удаленный каталог.

3. Правила проведения операций в архиве

3.1. *Занесение документов в архив*

Документ размещается аналитиком в архиве с помощью программного средства Source Safe. При первичном занесении требуется указать реквизиты документа:

- ФИО сотрудника, вводящего документ;
- основание разработки документа (для разработанных в ОСА — ссылка на заявку и т.п.) либо, если требуется, краткие данные о происхождении документа.

Если требуется, документ можно увидеть в разных каталогах.

3.2. *Внесение изменений в документы архива*

В случае необходимости доработки и изменения документа требуется указать реквизиты:

- ФИО сотрудника, внесшего изменения;
- основание для внесения изменений (для разработанных в ОСА — ссылка на заявку и т.п.), а также, если требуется, крат кие сведения об изменениях.

3.3. *Удаление документов и модификация каталогов*

Удаление документа из архива осуществляет исполнитель, создавший документ, по согласованию с руководителем ОСА.

Модификацию каталогов осуществляет руководитель ОСА или назначенный им сотрудник.

ВОПРОСЫ РЯ САМОПРОВЕРКИ

1. Дайте определение понятию «стандартизация».
2. Охарактеризуйте основные уровни стандартизации.
3. Назовите основные виды нормативных документов.
4. Дайте определение понятию «стандарт».
5. Как определяется понятие «стандарт» в области программного обес печения?
6. В чем различие между понятиями стандарта «де-факто» и «де-юре»?
7. Назовите известные вам международные организации, разрабаты вающие стандарты.
8. Объясните, почему нужны внутрифирменные стандарты.

ГЛАВА 2 ЖИЗНЕННЫЙ ЦИКЛ ПРОГРАММНЫХ СРЕДСТВ

При возникновении потребностей в заказе, приобретении, разработке, эксплуатации и сопровождении программ перед всеми сторонами, вовлеченными в жизненный цикл программного средства (ПС), возникает целый ряд вопросов, связанных с определением и детальным структурированием *жизненного цикла* (ЖЦ) ПС, с организационными и техническими правами и обязанностями сторон, с управлением ЖЦ и контролем за его реализацией. Одним из действенных инструментов для решения данных вопросов является использование унифицированных подходов, закрепленных в современных международных и российских стандартах.

Слова «жизненный цикл системы» или «жизненный цикл программного средства» часто появляются в статьях и звучат в разговорах разработчиков, по крайней мере руководителей проектов и подразделений. Всем понятно, что относятся они к тому, что и в какой последовательности должно делаться при создании и эксплуатации систем. Но прежде чем две организации или два специалиста договорятся о том, что конкретно входит или не входит в ЖЦ, проходит значительное время. А позже вполне может обнаружиться, что эти двое (две «стороны») все-таки поразному понимают, какие работы будут входить в ЖЦ, а какие — нет, какие проверки будут планироваться, когда и т. д. Естественно, общие принципы организации работ описаны давно, но что делать сторонам в конкретном проекте -— это каждый раз приходится решать заново.

В стандартах, регламентирующих жизненный цикл программных средств, обобщаются опыт и результаты исследований множества специалистов и рекомендуются наиболее эффективные современные методы и процессы создания и развития комплексов программ. В результате таких обобщений оттачиваются технологические процессы и приемы разработки, а также методическая база для их автоматизации.

ЖЦ ПС в стандартах представляет собой набор этапов, частных работ и операций в последовательности их выполнения и взаимосвязи, регламентирующих ведение работ от подготовки технического задания до завершения испытаний ряда версий и окончания эксплуатации ПС или информационной системы (ИС). Стандарты включают правила описания исходной информации, способов и методов выполнения операций, устанавливают правила контроля технологических процессов, требования к оформлению их результатов, а также регламентируют содержание технологических и эксплуатационных документов на комплексы программ. Они определяют организационную структуру коллектива, обеспечивают распределение и планирование заданий, а также контроль за ходом создания ПС.

Кроме вопросов выбора типа общего устройства ЖЦ есть проблемы с решением частных вопросов о включении или невключении в ЖЦ отдельных работ, очень важных для качества ПС и системы: что документировать при создании системы и ПС, какие работы должны будут гарантировать качество продукта, с какой степенью организационной независимости должны выполняться проверочные процедуры разных типов, чем будет обеспечиваться соответствие разрабатываемого ПС требованиям ко всей системе и соответствие ПС потребностям в системе.

Для того чтобы привнести порядок и понимание, общие для любых сторон, участвующих в ЖЦ систем и ПС, давно разрабатывались стандарты различных уровней утверждения — национальные и международные.

Существующее многообразие номенклатуры и функциональных возможностей эксплуатируемых, разрабатываемых и перспективных ПС затрудняет использование для них традиционных методов стандартизации групп (видов) однородной продукции. В то же время обязательная реализация в ходе проекта типовых процессов ЖЦ (заказ, поставка, разработка, эксплуатация, сопровождение и т.д.) дает возможность использовать принципы и методы функциональной стандартизации, основанные на применении *базовых стандартов* и разработанных на их основе *профилей стандартов* для конкретного типа объекта (в нашем случае — проекта и системы).

Под *базовым стандартом* понимается принятый нормативный документ, регламентирующий типовые (возможно, много-

вариантные) требования, нормы и правила применительно к данному объекту стандартизации.

Под *профилем стандарта* понимается принятый нормативный документ, регламентирующий требования, нормы и правила, выбранные из базовых стандартов и при необходимости дополненные и/или уточненные (ограниченные) применительно к конкретной классификационной группе данного объекта стандартизации [58].

Основные современные зарубежные стандарты ориентированы на описание ЖЦ сложных ПС обработки информации и управления в реальном времени. К таким ПС предъявляются наиболее высокие требования по качеству функционирования, они создаются большими коллективами специалистов в течение длительного времени.

Впервые формализованный и утвержденный стандарт жизненного цикла был утвержден в 1985 г. (уточнен в 1988 г.) для проектирования ПС систем военного назначения по заказам Министерства обороны США — стандарт ***DOD-STD-2167 A.*** Этим документом регламентированы 8 фаз (этапов) при создании сложных, критических ПС и около 250 типовых обязательных требований к процессам и объектам проектирования на этих этапах. ПС рассматриваются как часть специализированных информационных систем военного назначения. Поэтому начальные этапы проектирования и заключительные этапы испытаний и сдачи заказчику объединены в совместный анализ программных и аппаратных средств цельной системы вооружения, полностью решающей поставленные функциональные задачи.

В стандарте **DOD** представлена часть ЖЦ ПС, отражающая только непосредственно создание программ. Отсутствуют этапы эксплуатации и сопровождения, а также не полностью раскрыты процессы управления разработкой и интегральные процессы технологической поддержки ЖЦ ПС. В начале стандарта DOD-STD-2167 A определены область его действия и общие условия применения. Приведены базовый перечень ссылочных документов и определения понятий, терминов и аббревиатур. Основная совокупность требований изложена в двух крупных разделах: наиболее общие требования ко всему процессу создания ПС и детальные требования к каждому его этапу. Общие требования касаются планирования и управления разработкой ПС, правил взаимодействия с субподрядчиками и испытателями, а также доку-

ментирования результатов. Изложены общие требования к технологии и средствам автоматизации создания программ, к структуре и организации комплекса программ и поддерживающей его БД. Специальный раздел посвящен требованиям к квалификационным испытаниям, средствам и организации тестирования программ на всех этапах. Изложены требования к организации, выполнению и документированию оценок качества программной продукции, а также требования к конфигурационному управлению ПС. Завершаются общие требования правилами перехода к сопровождению ПС, к организации и документированию этого процесса. Детальные требования распределены по восьми этапам разработки. В этом стандарте после того, как сформулированы концепция и общие требования к системе (этап 1), выделяются и детализируются требования к ПС (этап 2). Далее начинается собственно процесс создания программ (этапы 3-6). Названия, последовательность и содержание этапов предварительного (этап 3) и детального (этап 4) проектирования, а также разработки компонентов (этап 5) и их интеграции (комплексирования) и тестирования (этап 6) достаточно близки к соответствующим процессам в стандартах ISO (см. ниже). По окончании этапов 3-6 проводятся тестирование ПС на реализующей (объектной) ЭВМ (этап 7), интегрирование и испытание ПС в составе системы (этап 8). Для всех этапов детальные требования имеют одинаковую структуру разделов. В них для каждого этапа конкретизируются разделы общих требований и отражены требования к управлению проектом, технологии, официальным квалификационным испытаниям, оценке качества программной продукции и к конфигурационному управлению. Весь процесс создания ПС поддерживается комплектом из 30 частных документов и 7 сводных отчетов (обзоров) по этапам. Наиболее полно раскрыты этапы предварительного (эскизного) и детального (технического) проектирования, каждый из которых состоит из 6-7 процессов. В результате представленную схему можно использовать как базу для планирования, организации и автоматизации процессов разработки ПС. Для замены стандартов DOD-STD-2167 A, 7935 A, 1703 Министерством обороны США в декабре 1994 г. утвержден военный стандарт ***MIL^STD-498 Разработка и*** *документирование* ***программного*** *обеспечения.* Он предназначен для применения всеми организациями и предприятиями, получающими заказы Министерства обороны США. Этот стандарт базируется на про-

цессах и документах, представленных в стандарте ISO 12207 и предшествующих военных стандартах. Общая структура стандарта близка к структуре DOD-STD-2167 A, однако детальные требования пятого раздела изложены значительно глубже и шире в 19 подразделах. Кроме того, число приложений увеличено до девяти. В 1996 г. утверждено очень подробное (407 страниц) руководство «Применение и рекомендации к стандарту MIL-STD-498». Основную часть пятого раздела составляют 75 подразделов — рекомендаций по обеспечению и реализации процессов ЖЦ сложных критических ПС высокого качества и надежности, функционирующих в реальном времени [61].

В России первые основы построения и использования профилей стандартов ЖЦ ПС заложены принятием в качестве базового стандарта ***ГОСТ Р ИСО/МЭК12207.*** Данный документ введен в действие с 1 июля 2000 г., тесно взаимоувязан с рядом стандартов, принятых ранее, и с некоторыми стандартами, разрабатываемыми в данное время на основе прямого применения стандартов ИСО.

Актуальность стандарта ГОСТ Р ИСО/МЭК 12207 для современных условий настолько высока, что принятие в ISO его исходного, международного варианта вскоре вызвало самую положительную оценку российских экспертов. Был дан ряд рекомендаций по его использованию в реальных условиях.

В данном стандарте программное обеспечение ПО *(или программный продукт)* определяется как набор компьютерных программ, процедур и, возможно, связанной с ними документации и данных. *Процесс* определяется как совокупность взаимосвязанных действий, преобразующих некоторые входные данные в выходные. Каждый процесс характеризуется определенными задачами и методами их решения, исходными данными, полученными от других процессов, и результатами [58].

Следует отметить, что в России создание ПО первоначально, в 70-е годы 20 века, регламентировалось стандартами ГОСТ ЕСПД (Единой системы программной документации — серия ГОСТ 19.XXX), которые были ориентированы на класс относительно простых программ небольшого объема, создаваемых отдельными программистами. В настоящее время эти стандарты устарели концептуально и по форме, сроки их действия закончились и использование нецелесообразно. Процессы создания автоматизированных систем (АС), в состав которых входит и ПО,

регламентированы стандартами ГОСТ 34.601-90 «Информационная технология. Комплекс стандартов на автоматизированные системы. Автоматизированные системы. Стадии создания», ГОСТ 34.602-89 «Информационная технология. Комплекс стандартов на автоматизированные системы. Техническое задание на создание автоматизированной системы» и ГОСТ 34.603-92 «Информационная технология. Виды испытаний автоматизированных систем». Однако процессы создания ПО для современных распределенных ЭИС, функционирующих в неоднородной среде, в этих стандартах отражены недостаточно, а отдельные их положения явно устарели. В результате для каждого серьезного проекта ЭИС приходится создавать комплекты нормативных и методических документов, регламентирующих процессы создания конкретного прикладного ПО, поэтому в отечественных разработках целесообразно использовать современные международные стандарты.

В стандарте ГОСТ Р ИСО/МЭК 12207 впервые реализован принцип структурной стандартизации ЖЦ ПС на основе регламентации требований к процессам, работам и задачам, входящим в полную типовую структуру ЖЦ ПС.

Процессы ЖЦ ПС выделены по принципу ответственности субъекта (заказчика, поставщика, разработчика и т. д.), реализующего конкретный процесс. В свою очередь, каждый из процессов состоит из ряда работ и решаемых при выполнении соответствующей работы задач. С точки зрения соподчиненности и важности данных процессов они разбиты на три группы: основные; вспомогательные; организационные (рис. 2.1).

Группа *основных* процессов включает в себя процессы: *приобретение; поставка; разработка; эксплуатация; сопровождение.*

Группа *вспомогательных процессов* включает в себя процессы, обеспечивающие выполнение основных процессов: *документирование; управление конфигурацией; обеспечение качества; верификация; аттестация; оценка; аудит; решение проблем.*

Группа *организационных* процессов включает в себя процессы: *управление проектами; создание инфраструктуры проекта; определение, оценка и улучшение самого ЖЦ; обучение.*

Очень важное отличие ISO: каждый процесс, действие или задача инициируется и выполняется другим процессом по мере необходимости, причем нет заранее определенных последовательностей (естественно, при сохранении логики связей по исходным сведениям задач и т. п.).

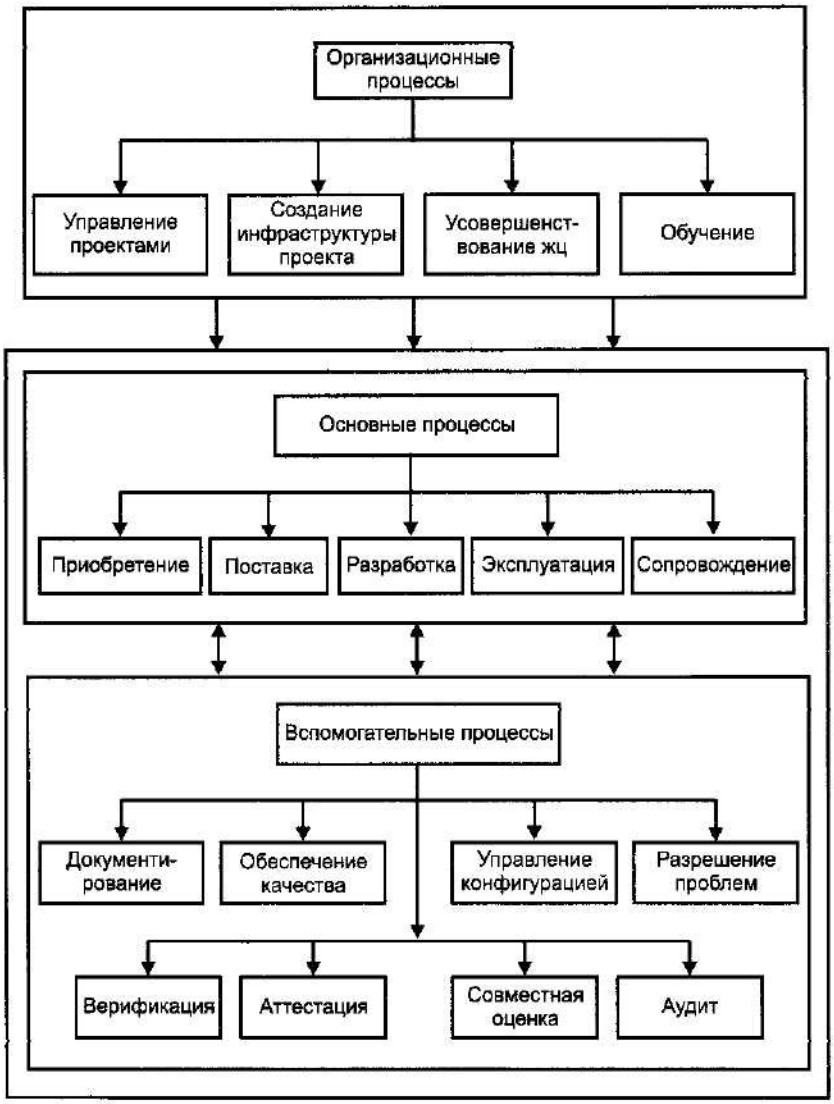


Рис. 2.1. Схема процессов жизненного цикла

2.1.
Основные процессы жизненного цикла программного средства

Процесс приобретения (acquisition process) состоит из действий и задач заказчика, приобретающего программное средство (рис. 2.2). *Инициирование приобретения* включает следующие задачи:

- определение заказчиком своих потребностей в приобретении, разработке или усовершенствовании системы, программных продуктов или услуг;
- анализ требований к системе;
- принятие решения относительно приобретения, разработки или усовершенствования существующего ПС;
- проверку наличия необходимой документации, гарантий, сертификатов, лицензий и поддержки в случае приобретения программного продукта;
- подготовку и утверждение плана приобретения, включающего требования к системе, тип договора, ответственность сторон и т. д.

Заявочные предложения должны содержать:

- требования к системе;
- перечень программных продуктов;
- условия и соглашения;
- технические ограничения (например, среда функционирования системы).

Заявочные предложения направляются выбранному поставщику (или нескольким поставщикам в случае проведения тендера). *Поставщик* — это организация, которая заключает договор с заказчиком на поставку системы, ПС или программной услуги на условиях, оговоренных в договоре.

Подготовка и корректировка договора включают следующие задачи:

- определение заказчиком процедуры выбора поставщика, включающей критерии оценки предложений возможных поставщиков;
- выбор конкретного поставщика на основе анализа предложений;
- подготовку и заключение договора с поставщиком;
- внесение изменений (при необходимости) в договор в процессе его выполнения.

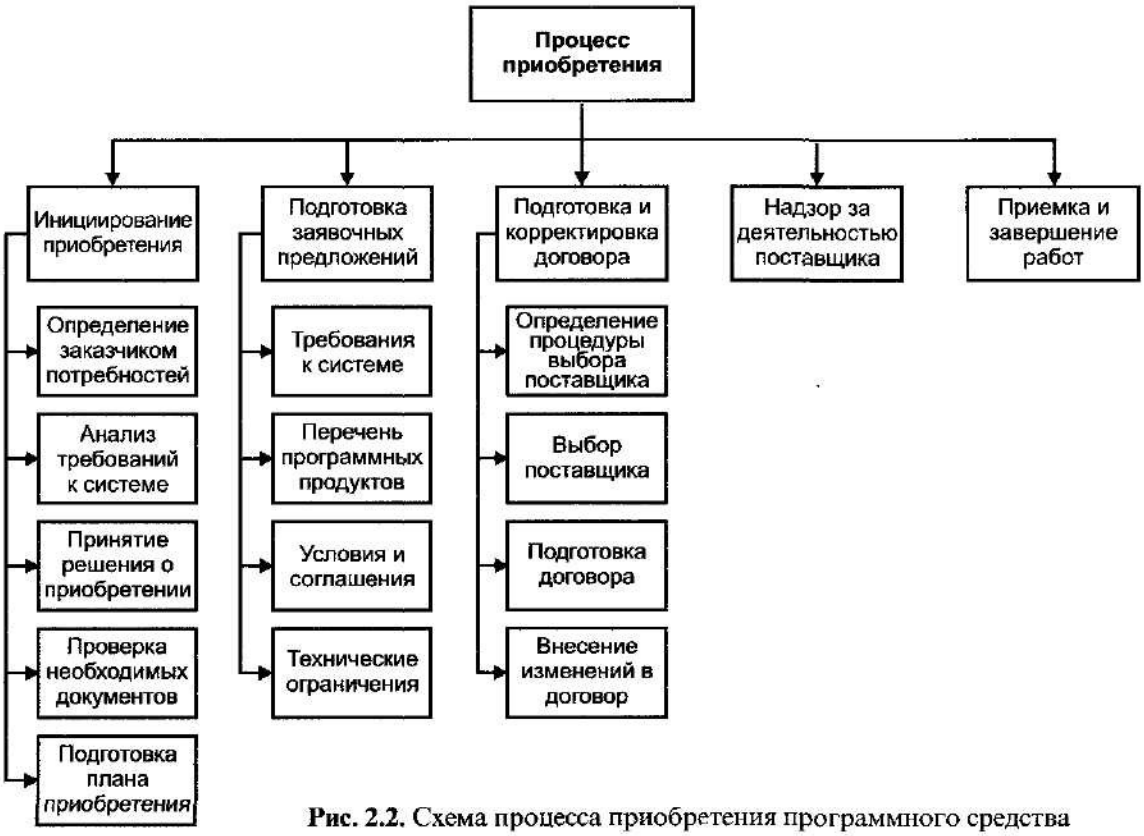


Рис. 2.2. Схема процесса приобретения программного средства

Надзор за деятельностью поставщика осуществляется в соответствии с действиями, предусмотренными в процессах *совместной оценки* и *аудита.*

В процессе *приемки* подготавливаются и выполняются необходимые тесты. *Завершение работ* по договору осуществляется в случае удовлетворения всех условий приемки.

Процесс поставки (supply process) охватывает действия и задачи, выполняемые поставщиком, который снабжает заказчика программным продуктом или услугой (рис. 2.3).

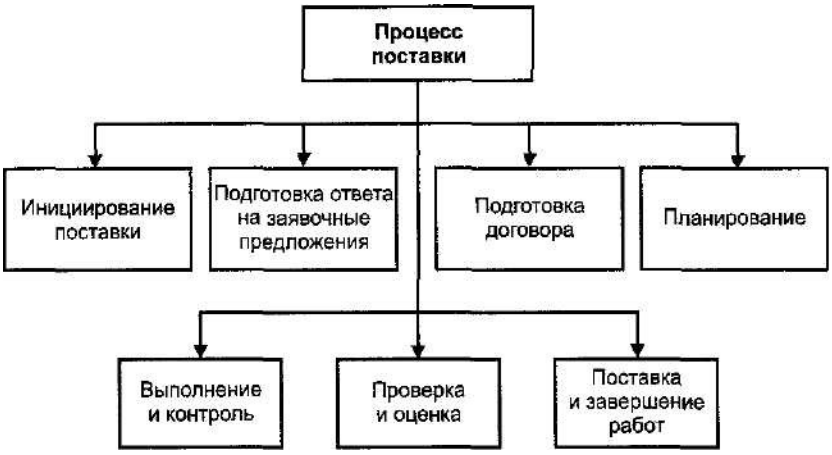


Рис. 2.3. Схема процесса поставки

Инициирование поставки заключается в рассмотрении поставщиком заявочных предложений и принятии решения о согласии с выставленными требованиями и условиями или предложение своих.

Планирование включает следующие задачи:

- принятие решения поставщиком относительно выполнения работ своими силами или с привлечением субподрядчика;
- разработку поставщиком плана управления проектом, содер жащего организационную структуру проекта, разграничение ответственности, технические требования к среде разработки и ресурсам, управление субподрядчиками и др.

Процесс разработки (development process) предусматривает действия и задачи, выполняемые разработчиком, и охватывает работы по созданию ПС и его компонентов в соответствии с за-

данными требованиями, включая оформление проектной и эксплуатационной документации; подготовку материалов, необходимых для проверки работоспособности и соответствующего качества программных продуктов, материалов, необходимых для организации обучения персонала, и т. д. (рис. 2.4).

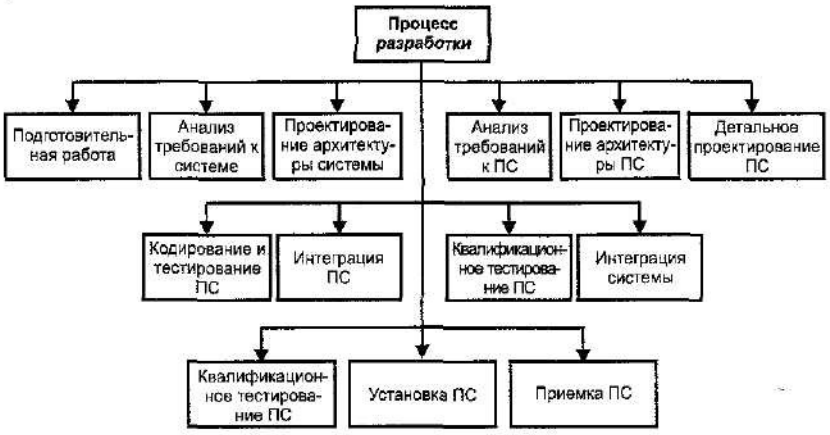


Рис. 2.4. Схема процесса разработки

Подготовительная работа начинается с выбора модели ЖЦ ПС, соответствующей масштабу, значимости и сложности проекта. Действия и задачи процесса разработки должны соответствовать выбранной модели. Разработчик должен выбрать, адаптировать к условиям проекта и использовать согласованные с заказчиком стандарты, методы и средства разработки, а также составить план выполнения работ.

Анализ требований к системе подразумевает определение ее функциональных возможностей, пользовательских требований, требований к надежности и безопасности, требований к внешним интерфейсам и т. д. Требования к системе оцениваются исходя из критериев реализуемости и возможности проверки при тестировании.

Проектирование архитектуры системы на высоком уровне заключается в определении компонентов ее оборудования, ПС и операций, выполняемых эксплуатирующим систему персоналом. Архитектура системы должна соответствовать требованиям, предъявляемым к системе, а также принятым проектным стандартам и методам.

Анализ требований к ПС предполагает определение следующих характеристик для каждого компонента ПС:

- функциональных возможностей, включая характеристики производительности и среды функционирования компонента;
- внешних интерфейсов;
- спецификаций надежности и безопасности;
- эргономических требований;
- требований к используемым данным;
- требований к установке и приемке;
- требований к пользовательской документации;
- требований к эксплуатации и сопровождению.

Требования к ПС оцениваются исходя из критериев соответствия требованиям к системе, реализуемости и возможности проверки при тестировании.

Проектирование архитектуры ПС включает следующие задачи (для каждого компонента ПС):

- трансформацию требований к ПС в архитектуру, определяющую на высоком уровне структуру ПС и состав его компонентов;
- разработку и документирование программных интерфейсов ПС и баз данных;
- разработку предварительной версии пользовательской документации;
- разработку и документирование предварительных требований к тестам и плана интеграции ПС.

Архитектура компонентов ПС должна соответствовать требованиям, предъявляемым к ним, а также принятым проектным стандартам и методам.

Детальное проектирование ПС включает следующие задачи:

- описание компонентов ПС и интерфейсов между ними на более низком уровне, достаточном для их последующего самостоятельного кодирования и тестирования;
- разработку и документирование детального проекта базы данных;
- обновление (при необходимости) пользовательской документации;
- разработку и документирование требований к тестам и плана тестирования компонентов ПС;
- обновление плана интеграции ПС.

Кодирование и тестирование ПС охватывают следующие задачи:

- разработку (кодирование) и документирование каждого компонента ПС и базы данных, а также совокупности тестовых процедур и данных для их тестирования;
- тестирование каждого компонента ПС и базы данных на соответствие предъявляемым к ним требованиям. Результаты тестирования компонентов должны быть документированы;
- обновление (при необходимости) пользовательской документации;
- обновление плана интеграции ПС.

Интеграция ПС предусматривает сборку разработанных компонентов ПС в соответствии с планом интеграции и тестирование агрегированных компонентов. Для каждого из агрегированных компонентов разрабатываются наборы тестов и тестовые процедуры, предназначенные для проверки каждого из квалификационных требований при последующем квалификационном тестировании. *Квалификационное требование* — это набор критериев или условий, которые необходимо выполнить, чтобы квалифицировать программный продукт как соответствующий своим спецификациям и готовый к использованию в условиях эксплуатации.

Квалификационное тестирование ПС проводится разработчиком в присутствии заказчика (по возможности) для демонстрации того, что ПС удовлетворяет своим спецификациям и готово к использованию в условиях эксплуатации. Квалификационное тестирование выполняется для каждого компонента ПС по всем разделам требований при широком варьировании тестов. При этом также проверяются полнота технической и пользовательской документации и ее адекватность самим компонентам ПС.

Интеграция системы заключается в сборке всех ее компонентов, включая ПС и оборудование. После интеграции система, в свою очередь, подвергается *квалификационному тестированию* на соответствие совокупности требований к ней. При этом также производятся оформление и проверка полного комплекта документации на систему.

Установка ПС осуществляется разработчиком в соответствии с планом в той среде и на том оборудовании, которые предусмотрены договором. В процессе установки проверяется работоспособность ПС и баз данных. Если устанавливаемое ПС заменя-

ет существующую систему, разработчик должен обеспечить их параллельное функционирование в соответствии с договором.

Приемка ПС предусматривает оценку результатов квалификационного тестирования ПС и системы и документирование результатов оценки, которые проводятся заказчиком с помощью разработчика. Разработчик выполняет окончательную передачу ПС заказчику в соответствии с договором, обеспечивая при этом необходимое обучение и поддержку.

Процесс эксплуатации (operation process) охватывает действия и задачи оператора — организации, эксплуатирующей систему (рис. 2.5).

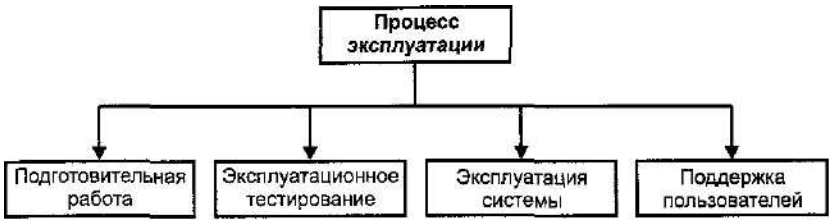


Рис. 2.5. Схема процесса эксплуатации

Подготовительная работа включает проведение оператором следующих задач:

- планирование действий и работ, выполняемых в процессе эксплуатации, и установку эксплуатационных стандартов;
- определение процедур локализации и разрешения проблем, возникающих в процессе эксплуатации.

Эксплуатационное тестирование осуществляется для каждой очередной редакции программного продукта, после чего она передается в эксплуатацию.

Эксплуатация системы выполняется в предназначенной для этого среде в соответствии с пользовательской документацией.

Поддержка пользователей заключается в оказании помощи и консультаций при обнаружении ошибок в процессе эксплуатации ПС.

Процесс сопровождения (maintenance process) предусматривает действия и задачи, выполняемые сопровождающей организацией (службой сопровождения). Данный процесс активизируется при изменениях (модификациях) программного продукта и соот-

ветствующей документации, вызванных возникшими проблемами или потребностями в модернизации либо адаптации ПС. В соответствии со стандартом IEEE-90 под *сопровождением* понимается внесение изменений в ПС в целях исправления ошибок, повышения производительности или адаптации к изменившимся условиям работы или требованиям.

Изменения, вносимые в существующее ПС, не должны нарушать его целостность. Процесс сопровождения включает перенос ПС в другую среду (миграцию) и заканчивается снятием ПС с эксплуатации (рис. 2.6).



Рис. 2.6. Схема процесса сопровождения

Подготовительная работа службы сопровождения включает следующие задачи:

- планирование действий и работ, выполняемых в процессе со провождения;
- определение процедур локализации и разрешения проблем, воз никающих в процессе сопровождения.

Анализ проблем и запросов на модификацию ПС, выполняемый службой сопровождения, включает следующие задачи:

- анализ сообщения о возникшей проблеме или запроса на моди фикацию ПС относительно его влияния на организацию, суще ствующую систему и интерфейсы с другими системами. При этом определяются следующие характеристики возможной модифи кации: тип (корректирующая, улучшающая, профилактическая или адаптирующая к новой среде); масштаб (размеры модифи кации, стоимость и время ее реализации); критичность (воздей ствие на производительность, надежность или безопасность);
- оценку целесообразности проведения модификации и возмож ных вариантов ее проведения;
- утверждение выбранного варианта модификации.

Модификация ПС предусматривает определение компонентов ПС, их версий и документации, подлежащих модификации, и внесение необходимых изменений в соответствии с правилами *процесса разработки.* Подготовленные изменения тестируются и проверяются по критериям, определенным в документации. При подтверждении корректности изменений в программах проводится корректировка документации.

Проверка и приемка заключаются в проверке целостности модифицированной системы и утверждении внесенных изменений.

При *переносе ПС в другую среду* используются имеющиеся или разрабатываются новые средства переноса, затем выполняется конвертирование программ и данных в новую среду. С целью облегчить переход предусматривается параллельная эксплуатация ПС в старой и новой среде в течение некоторого периода, когда проводится необходимое обучение пользователей работе в новой среде.

Снятие ПС с эксплуатации осуществляется по решению заказчика при участии эксплуатирующей организации, службы сопровождения и пользователей. При этом программные продукты и соответствующая документация подлежат архивированию в соответствии с договором. Аналогично переносу ПС в другую среду с целью облегчить переход к новой системе предусматривается параллельная эксплуатация старого и нового ПС в течение некоторого периода, когда выполняется необходимое обучение пользователей работе с новой системой.

2.2. Вспомогательные процессы жизненного цикла программного средства

Процесс документирования (documentation process) предусматривает формализованное описание информации, созданной в течение ЖЦ ПС. Данный процесс состоит из набора действий, с помощью которых планируют, проектируют, разрабатывают, выпускают, редактируют, распространяют и сопровождают документы, необходимые для всех заинтересованных лиц, таких, как руководители, технические специалисты и пользователи системы (рис. 2.7).

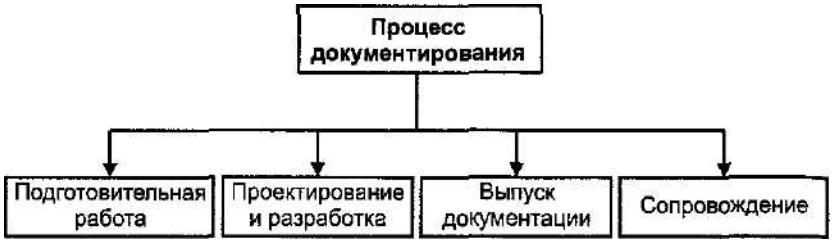


Рис. 2.7. Схема процесса документирования

Процесс управления конфигурацией (configuration management process) предполагает применение административных и технических процедур на всем протяжении ЖЦ ПС для определения состояния компонентов ПС в системе, управления модификациями ПС, описания и подготовки отчетов о состоянии компонентов ПС и запросов на модификацию, обеспечения полноты, совместимости и корректности компонентов ПС, управления хранением и поставкой ПС. Согласно стандарту IEEE-90 под *конфигурацией ПС* понимается совокупность его функциональных и физических характеристик, установленных в технической документации и реализованных в ПС.

Управление конфигурацией позволяет организовать, систематически учитывать и контролировать внесение изменений в ПС на всех стадиях ЖЦ (рис. 2.8).



Рис. 2.8. Схема процесса управления конфигурацией

Подготовительная работа заключается в планировании управления конфигурацией.

Идентификация конфигурации устанавливает правила, с помощью которых можно однозначно идентифицировать и различать компоненты ПС и их версии. Кроме того, каждому компо-

центу и его версиям соответствует однозначно обозначаемый комплект документации. В результате создается база для однозначного выбора и манипулирования версиями компонентов ПС, использующая ограниченную и упорядоченную систему символов, идентифицирующих различные версии ПС.

Контроль конфигурации предназначен для систематической оценки предполагаемых модификаций ПС и координированной их реализации с учетом эффективности каждой модификации и затрат на выполнение. Он обеспечивает контроль состояния и развития компонентов ПС и их версий, а также адекватность реально изменяющихся компонентов их комплектной документации.

Учет состояния конфигурации представляет собой регистрацию состояния компонентов ПС, подготовку отчетов обо всех реализованных и отвергнутых модификациях версий компонентов ПС. Совокупность отчетов обеспечивает однозначное отражение текущего состояния системы и ее компонентов, а также ведение истории модификаций.

Оценка конфигурации заключается в оценке функциональной полноты компонентов ПС, а также соответствия их физического состояния текущему техническому описанию.

Управление выпуском и поставка охватывают изготовление эталонных копий программ и документации, их хранение и поставку пользователям в соответствии с порядком, принятым в организации.

Процесс обеспечения качества (quality assurance process) обеспечивает соответствующие гарантии того, что ПС и процессы его ЖЦ соответствуют заданным требованиям и утвержденным планам. Под *качеством ПС* понимается совокупность свойств, которые характеризуют способность ПС удовлетворять заданным требованиям (рис. 2.9).

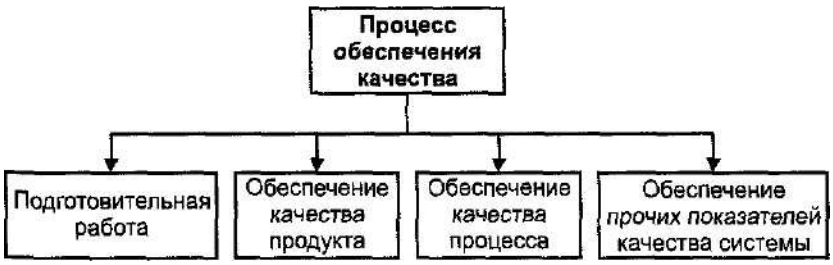


Рис. 2.9. Схема процесса обеспечения качества

Для получения достоверных оценок создаваемого ПС процесс обеспечения его качества должен происходить независимо от субъектов, непосредственно связанных с разработкой ПС. При этом могут использоваться результаты других вспомогательных процессов, таких, как верификация, аттестация, совместная оценка, аудит и разрешение проблем.

Подготовительная работа заключается в координации с другими вспомогательными процессами и планировании самого процесса обеспечения качества с учетом используемых стандартов, методов, процедур и средств.

Обеспечение качества продукта подразумевает гарантирование полного соответствия программных продуктов и документации на них требованиям заказчика, предусмотренным в договоре.

Обеспечение качества процесса предполагает гарантирование соответствия процессов ЖЦ ПС, методов разработки, среды разработки и квалификации персонала условиям договора, установленным стандартам и процедурам.

Обеспечение прочих показателей качества системы осуществляется в соответствии с условиями договора и стандартом качества ISO 9001.

Процесс верификации (verification process) состоит в определении того, что программные продукты, являющиеся результатами некоторого действия, полностью удовлетворяют требованиям или условиям, обусловленным предшествующими действиями *(верификация* в «узком» смысле означает формальное доказательство правильности ПС). Для повышения эффективности верификация должна как можно раньше интегрироваться с использующими ее процессами (такими, как поставка, разработка, эксплуатация или сопровождение). Данный процесс может включать анализ, оценку и тестирование (рис. 2.10).

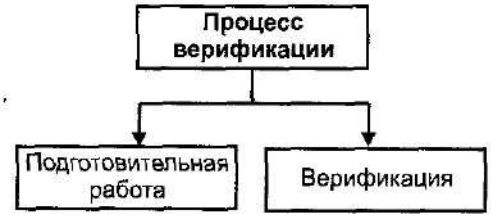


Рис. 2,10. Схема процесса верификации

Верификация может проводиться с различными степенями независимости. Степень независимости может варьироваться от выполнения верификации самим исполнителем или другим специалистом данной организации до ее выполнения специалистом другой организации с различными вариациями. Если процесс верификации осуществляется организацией, не зависящей от поставщика, разработчика, оператора или службы сопровождения, то он называется *процессом независимой верификации.*

В процессе верификации проверяются следующие условия:

- непротиворечивость требований к системе и степень учета по требностей пользователей;
- возможности поставщика выполнить заданные требования;
- соответствие выбранных процессов ЖЦ ПС условиям договора;
- адекватность стандартов, процедур и среды разработки про цессам ЖЦ ПС;
- соответствие проектных спецификаций ПС заданным требова ниям;
- корректность описания в проектных спецификациях входных и выходных данных, последовательности событий, интерфей сов, логики и т.д.;
- соответствие кода проектным спецификациям и требованиям;
- тестируемость и корректность кода, его соответствие приня тым стандартам кодирования;
- корректность интеграции компонентов ПС в систему;
- адекватность, полнота и непротиворечивость документации.

Процесс аттестации (validation process) предусматривает определение полноты соответствия заданных требований и созданной системы или программного продукта их конкретному функциональному назначению. Под *аттестацией* обычно понимаются подтверждение и оценка достоверности проведенного тестирования ПС. Аттестация должна гарантировать полное соответствие ПС спецификациям, требованиям и документации, а также возможность его безопасного и надежного применения пользователем. Аттестацию рекомендуется выполнять путем тестирования во всех возможных ситуациях и использовать при этом независимых специалистов. Аттестация может проводиться на начальных стадиях ЖЦ ПС или как часть работы по приемке ПС (рис. 2.11).



Рис. 2.11. Схема процесса аттестации

Аттестация, так же как и верификация, может осуществляться с различными степенями независимости. Если процесс аттестации выполняется организацией, не зависящей от поставщика, разработчика, оператора или службы сопровождения, то он называется *процессом независимой аттестации.*

Процесс совместной оценки (joint review process) предназначен для оценки состояния работ по проекту и ПС, создаваемому при выполнении данных работ (действий). Он сосредоточен в основном на контроле планирования и управления ресурсами, персоналом, аппаратурой и инструментальными средствами проекта (рис. 2.12).

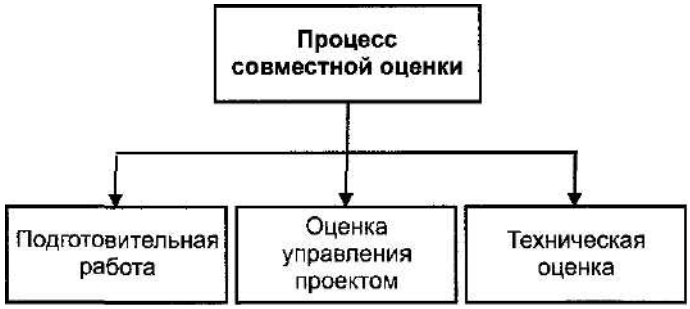


Рис. 2.12. Схема процесса оценки

Оценка применяется как на уровне управления проектом, так и на уровне технической реализации проекта и проводится в течение всего срока действия договора. Данный процесс может выполняться двумя любыми сторонами, участвующими в договоре, при этом одна сторона проверяет другую.

Процесс аудита (audit process) представляет собой определение соответствия требованиям, планам и условиям договора.

Аудит может выполняться двумя любыми сторонами, участвующими в договоре, когда одна сторона проверяет другую.

Аудит — это ревизия (проверка), проводимая компетентным органом (лицом) в целях обеспечения независимой оценки степени соответствия ПС или процессов установленным требованиям. Аудит служит для установления соответствия реальных работ и отчетов требованиям, планам и контракту. Аудиторы (ревизоры) не должны иметь прямой зависимости от разработчиков ПС. Они определяют состояние работ, использование ресурсов, соответствие документации спецификациям и стандартам, корректность тестирования (рис. 2.13).



Рис. 2.13. Схема процесса аудита

Процесс разрешения проблем (problem resolution process) предусматривает анализ и решение проблем (включая обнаруженные несоответствия) независимо от их происхождения или источника, которые обнаружены в ходе разработки, эксплуатации, сопровождения или других процессов. Каждая обнаруженная проблема должна быть идентифицирована, описана, проанализирована и разрешена (рис. 2.14).

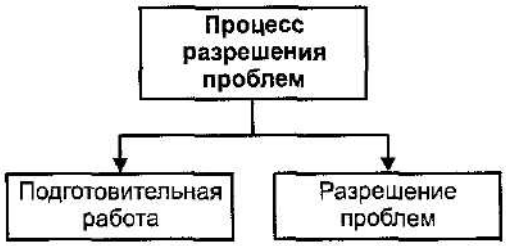


Рис. 2.14. Схема процесса разрешения проблем

2.3.

Организационные процессы жизненного цикла программного средства

Процесс управления (management process) состоит из действий и задач, которые могут выполняться любой стороной, управляющей своими процессами. Данная сторона (менеджер) отвечает за управление выпуском продукта, управление проектом и управление задачами соответствующих процессов, таких, как приобретение, поставка, разработка, эксплуатация, сопровождение и др. (рис. 2.15).

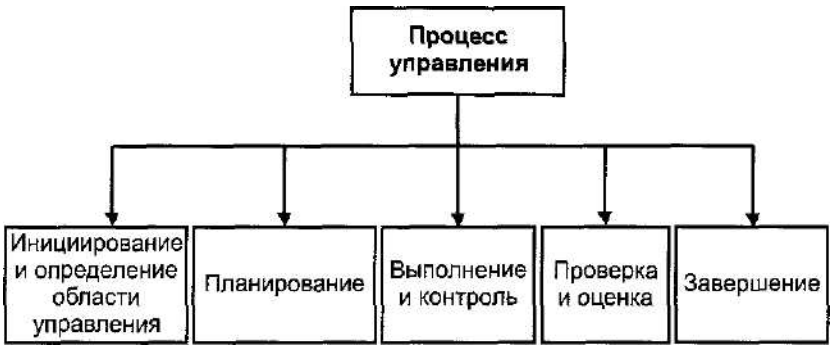


Рис. 2.15. Схема процесса управления

При *инициировании* менеджер должен убедиться, что необходимые'для управления ресурсы (персонал, оборудование и технология) имеются в его распоряжении в достаточном количестве.

Планирование подразумевает выполнение как минимум следующих задач:

- составление графиков выполнения работ;
- оценку затрат;
- выделение требуемых ресурсов;
- распределение ответственности;
- оценку рисков, связанных с конкретными задачами;
- создание инфраструктуры управления.

Процесс создания инфраструктуры (infrastructure process) охватывает выбор и поддержку (сопровождение) технологии, стан-

дартов и инструментальных средств, выбор и установку аппаратных и программных средств, используемых для разработки, эксплуатации или сопровождения ПС. Инфраструктура должна модифицироваться и сопровождаться в соответствии с изменениями требований к соответствующим процессам. Инфраструктура, в свою очередь, является одним из объектов управления конфигурацией (рис. 2.16).

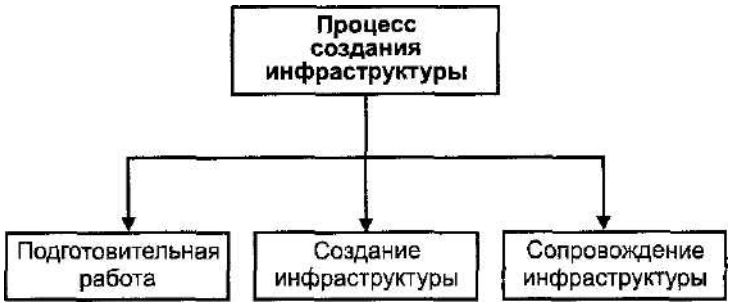


Рис. 2.16. Схема процесса создания инфраструктуры

Процесс усовершенствования (improvement process) предусматривает оценку, измерение, контроль и усовершенствование процессов ЖЦ ПС (рис. 2.17).

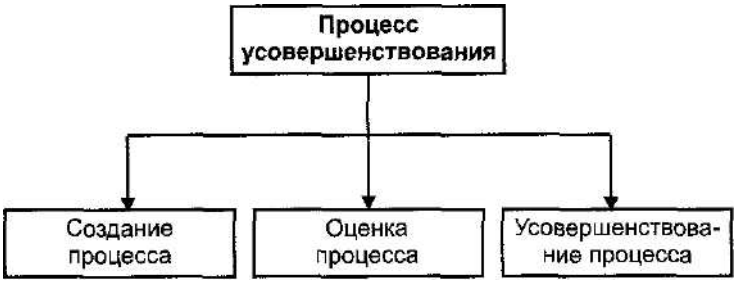


Рис. 2.17. Схема процесса усовершенствования

Усовершенствование процессов ЖЦ ПС направлено на повышение производительности труда всех участвующих в них специалистов за счет совершенствования используемой технологии, методов управления, выбора инструментальных средств и обучения пер-

сонала. Усовершенствование основано на анализе достоинств и недостатков каждого процесса. Такому анализу в большой степени способствует накопление в организации исторической, технической, экономической и иной информации по реализованным проектам.

Процесс обучения (training process) охватывает первоначальное обучение и последующее постоянное повышение квалификации персонала. Приобретение, поставка, разработка, эксплуатация и сопровождение ПС в значительной степени зависят от уровня знаний и квалификации персонала. Например, разработчики ПС должны пройти необходимое обучение методам и средствам программной инженерии. Содержание процесса обучения определяется требованиями к проекту. Оно должно учитывать необходимые ресурсы и технические средства обучения. Должны быть разработаны и представлены методические материалы, необходимые для обучения пользователей в соответствии с учебным планом (рис. 2.18) [45].

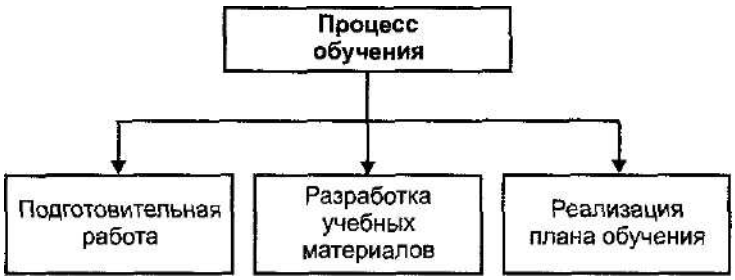


Рис. 2.18. Схема процесса обучения

2.4. Стандарты комплекса ГОСТ 34

Стандарты комплекса ГОСТ 34 на создание и развитие АС — обобщенные, но воспринимаемые как весьма жесткие по структуре ЖЦ и проектной документации. Кроме того, эти стандарты многими считаются бюрократическими до вредности и консервативными до устарелости.

ГОСТ 34 задумывался в конце 80-х годов 20 века как всеобъемлющий комплекс взаимоувязанных межотраслевых документов.

Объектами стандартизации являются АС различных видов и все виды их компонентов, а не только ПС и БД.

Комплекс рассчитан на взаимодействие заказчика и разработчика. Аналогично ISO 12207 предусмотрено, что заказчик может разрабатывать АС для себя сам (если создаст для этого специализированное подразделение). Однако формулировки ГОСТ 34 не ориентированы на столь явное и, в известном смысле, симметричное отражение действий обеих сторон, как ISO 12207. Поскольку ГОСТ 34 в основном уделяет внимание содержанию проектных документов, распределение действий между сторонами обычно делается, отталкиваясь от этого содержания.

Наиболее популярными можно считать стандарты ГОСТ 34.601-90 (Стадии создания АС), ГОСТ 34.602-89 (ТЗ на создание АС) и методические указания РД 50-34.698-90 (Требования к содержанию документов). Стандарты предусматривают стадии и этапы выполнения работ по созданию АС, но не предусматривают сквозных процессов в явном виде.

Стадии и этапы создания АС в общем случае приведены в табл. 2.1.

В приложении 1 к ГОСТ 34 приведено содержание работ на каждом этапе. Это определяет потенциальные возможности выделения работ, выполняемых параллельно или последовательно (т. е. по сути — процессов), и составляющих их задач. Такой прием может использоваться при построении профиля стандартов ЖЦ проекта, включающего согласованные подмножества стандартов ГОСТ 34 и ISO 12207.

В 80-х годах 20 века сложилось положение, при котором в различных отраслях и областях деятельности использовалась плохо согласованная или несогласованная нормативно-техническая документация. Это затрудняло интеграцию систем, обеспечение их эффективного совместного функционирования. Действовали следующие комплексы и системы стандартов, устанавливающие требования к различным видам АС;

- единая система стандартов автоматизированных систем управления (24-я система) для АСУ, ОАСУ, АСУП, АСУ ТП и других организационно-экономических систем;
- комплекс стандартов системы 23501, распространявшихся на САПР — системы автоматизированного проектирования;
- четвертая группа 14-й системы стандартов, распространяющаяся на АС технологической подготовки производства.

Таблица 2.1

Стадии	Этапы работ
1. Формирование требований к АС	1.1. Обследование объекта и обоснование необходимости создания АС. *1.2.* Формирование требований пользователя к АС. 1.3. Оформление отчета о выполненной работе и заявки на разработку АС (тактико-технического задания)
2. Разработка концепции АС	2.1. Изучение объекта. 2.2. Проведение необходимых научно-исследовательских работ. 2.3. Разработка вариантов концепция АС, удовлетворяющих требованиям пользователя. 2.4 Оформление отчета о выполненной работе
3. Техническое задание	3.1. Разработка и утверждение технического задания на создание АС
4. Эскизный проект	4.1. Разработка предварительных проектных решений по системе и ее частям 4.2. Разработка документации на АС и ее части
5. Технический проект	5.1. Разработка проектных решений по системе и ее частям 5.2. Разработка документации на АС и ее части. 5.3. Разработка и оформление документации на поставку изделий для комплектования АС и (или) технических требований (технических заданий) на их разработку 5.4. Разработка заданий на проектирование в смежных частях проекта объекта автоматизации
6 Рабочая документация	6.1. Разработка рабочей документации на систему и ее части 6.2. Разработка или адаптация программ
7. Ввод в действие	7.1. Подготовка объекта автоматизации к вводу АС в действие. 7.2. Подготовка персонала. 7.3. Комплектация АС поставляемыми изделиями (программными и техническими средствами, программно-техническими комплексами, информационными изделиями). 7.4. Строительно-монтажные работы. 7.5. Пусконал ад очные работы. 7 6. Проведение предварительных испытаний. 7.7 Проведение опытной эксплуатации. 7.8. Проведение приемочных испытаний
8. Сопровождение АС	8.1. Выполнение работ в соответствии с гарантийными обязательствами. 8.2. Послегарантийное обслуживание

Практика применения стандартов на АСУ, АСУП, АСУ ТП, САПР, *АСТТШ* показала, что в них применяется по существу единый понятийный аппарат, есть много общих объектов стандартизации, однако требования стандартов не согласованы между собой, имеются различия по составу и содержанию работ, обозначению, составу, содержанию, оформлению документов и пр.

Конечно, эта ситуация отчасти отражала и естественное многообразие условий разработки АС, целей разработчиков, применяемых подходов и методик.

В этих условиях можно было провести анализ такого многообразия и далее выбрать один из двух во многом противоположных способов:

1. Выработать одну обобщенную понятийную и терминоло гическую систему, общую схему разработки, общий набор доку ментов с их содержанием и определить их как обязательные для всех АС.

2. Определить одну общую понятийную и терминологичес кую систему, обобщенный комплекс системных требований, на бор критериев качества, но предоставить максимальную свободу в выборе схемы разработки, состава документов и других аспек тов, предусмотрев только минимум обязательных требований, ко торые позволяли бы:

- определять уровень качества результата;
- выбирать те конкретные методики (с их моделями ЖЦ, набо ром документов и др.), которые наиболее подходят к условиям разработки и соответствуют используемым информационным технологиям;
- работать, таким образом, с минимальными ограничениями эффективных действий проектировщика АС.

Разработчики комплекса стандартов 34 выбрали способ, близкий к первому из указанных выше, т. е. пошли по пути, более близкому к схемам конкретных методик, чем к стандартам типа ISO 12207. Тем не менее благодаря общности понятийной базы стандарты остаются применимыми в весьма широком диапазоне случаев.

Степень адаптивности формально определяется возможностями;

- опускать стадию эскизного проектирования и объединять ста дии «Технический проект» и «Рабочая документация»;

- опускать этапы, объединять и опускать большинство документов и их разделов;
- вводить дополнительные документы, разделы документов и работы;
- динамически создавая так называемые ЧТЗ — частные технические задания, достаточно гибко формировать ЖЦ АС. Как правило, этот прием используется на уровне крупных единиц (подсистем, комплексов), ради которых считается оправданным создавать ЧТЗ, однако нет никаких существенных оснований сильно ограничивать этот способ управления ЖЦ.

Стадии и этапы, выполняемые организациями — участниками работ по созданию АС, устанавливаются в договорах и техническом задании, что близко к подходу ISO.

Введение единой, качественно определенной терминологии, наличие разумной классификации работ, документов, видов обеспечения и других, безусловно, полезно. ГОСТ 34 способствует более полной и качественной стыковке действительно разных систем, что особенно важно в условиях, когда разрабатывается все больше сложных комплексных АС, которые включают в свой состав АСУ ТП, АСУП, САПР-конструктора, САПР-технолога и другие системы.

Определено несколько важных положений, отражающих особенности АС как объекта стандартизации, например: «в общем случае АС состоит из программно-технических комплексов (ПТК), программно-методических комплексов (ПМК) и отдельных компонентов организационного, технического, программного и информационного обеспечения». Разделение понятий ПТК и АС закрепляло принцип, по которому АС не «ИС с БД», но:

- «организационно-техническая система, обеспечивающая выработку решений на основе автоматизации информационных процессов в различных сферах деятельности (управление, проектирование, производство и т. д.) или их сочетаниях», что особенно актуально в аспектах бизнес-реинжиниринга;
- «система, состоящая из персонала и комплекса средств автоматизации его деятельности, реализующая информационную технологию выполнения установленных функций» (по ГОСТ 34.003-90).

Эти определения указывают на то, что АС — это в первую очередь персонал, принимающий решения и выполняющий другие управляющие действия, поддержанный организационно-техническими средствами.

Эти определения и определение системы в ISO12207 вполне совместимы для их совместного использования.

Гарантирование качества определяется в ТЗ — техническом задании на АС и производится на любых последующих этапах и с любой степенью независимости экспертизы. В каскадной траектории ЖЦ эти экспертизы располагаются несколько позже, чем в TSO12207. Тем не менее имеются очень большие потенциальные возможности, которые часто слабо эксплуатируются на практике.

Степень обязательности: прежняя полная обязательность отсутствует, материалы ГОСТ 34 по сути стали методической поддержкой, причем чаще для заказчиков, имеющих в стандарте набор требований к содержанию ТЗ и проведению испытаний АС. При этом польза ГОСТ 34 может многократно возрасти в случае их более гибкого использования при формировании профиля ЖЦ АС.

Ключевым документом взаимодействия сторон является ТЗ — техническое задание на создание АС. ТЗ является основным исходным документом для создания АС и его приемки, ТЗ определяет важнейшие точки взаимодействия заказчика и разработчика. При этом ТЗ разрабатывает организация-разработчик (по ГОСТ 34.602-89), но формально выдает ТЗ разработчику заказчик [60].

2.5. Стандарт IEEE 1074-1995. Процессы жизненного цикла для развития программных средств

Стандарт IEEE 1074—1995 охватывает полный жизненный цикл ПС, в котором выделяются шесть крупных базовых процессов. Эти процессы детализируются 16 частными процессами. В последних имеется еще более мелкая детализация в совокупности на 65 процессов-работ. Содержание каждого частного процесса начинается с описания общих его функций и задач и перечня действий — работ при последующей детализации. Для каждого процесса в стандарте представлены входная и результирующая информация о его выполнении и краткое описание сущности процесса. Внимание сосредоточено преимущественно на непосредственном создании ПС и на процессах предварительного про-

ектирования. В приложении представлены четыре варианта адаптации максимального состава компонентов ЖЦ ПС к конкретным особенностям типовых проектов.

Хотя основные процессы близки к описанным в стандарте ISO 12207, общая архитектура и детализация частных процессов и работ в данном стандарте значительно отличаются. Процессы непосредственного создания ПС и его поддержка в стандарте представлены наибольшим числом частных процессов (около 70%), начинающихся с разработки требований к ПС и завершающихся приемосдаточными испытаниями, проводимыми заказчиком или пользователем.

В разделе 2 приведена информация, необходимая для понимания взаимодействия между ЖЦ системы и ЖЦ ПС.

Раздел 3 является иллюстративным описанием ЖЦ ПС. Разделы 4 и 5 посвящены процессам планирования и разработки ПС. Интегральные процессы — верификация, управление конфигурацией, обеспечение качества и сертификационное сопровождение — представлены в разделах 6-9. Раздел 11 содержит рекомендации по структуре документов, создаваемых главным образом для обеспечения сертификации ПС. В разделе 12 обсуждаются дополнительные соглашения, включая руководство по использованию ранее разработанных ПС, сертификации инструментальных средств и применению альтернативных методов для тех задач, которые обсуждаются в разделах 2-11 [62].

2.6. Адаптация стандарта к конкретному проекту

Не бывает двух одинаковых проектов. Вариации в организационных службах и процедурах, методах и стратегиях приобретения, размере и сложности проекта, требованиях системы и методах разработки среди прочего влияют на способ создания, применения и сопровождения ПС. Используемые реально в фирмах жизненные циклы ПС в последнее время часто отличаются от приведенных в стандартах в связи с развитием и внедрением объектно-ориентированного анализа и проектирования, а также методов быстрой разработки прикладных программ, CASE-систем и языков четвертого поколения. В новых технологиях сокращаются стадии непосредственного создания программных и инфор-

мационных компонентов и детализируются процессы системного анализа и проектирования ПС в целом. Кроме того, возрастают роль и конкретизация работ по технологической поддержке и графической визуализации проектирования, а также по стандартизации интерфейсов компонентов в создаваемых приложениях.

Особое внимание уделяется детализации процессов ЖЦ, обеспечивающих высокое качество создаваемых ПС, и возможности их эффективного итерационного развития длительное время в многочисленных версиях. Отечественные разработчики и пользователи современных инструментальных средств создания программ, как правило, не знают и не учитывают опыт, формализованный и отраженный в зарубежных стандартах на ЖЦ ПС. Технологические комплексы собираются из отдельных, слабо связанных инструментальных пакетов прикладных программ, решающих частные задачи автоматизации без анализа и учета всего ЖЦ ПС. В результате технология и процессы разработки формируются не системно — с позиции достижения наивысших показателей эффективности и качества всего жизненного цикла конкретного ПС, а с позиции скорейшего достижения видимых для заказчика результатов проекта. В случае критических ПС это сказывается впоследствии на их низкой надежности функционирования и безопасности применения, а также затрудняет модернизацию и развитие версий.

Альтернативой являются выбор и формирование комплекса инструментальных средств под технологию, формализованную на базе одного из адаптированных стандартов ЖЦ ПС.

Для снижения затрат и обеспечения качества выбранный стандарт ЖЦ следует адаптировать к индивидуальному проекту ПС. Должны быть определены характеристики окружения проекта, которые могут воздействовать на адаптацию. Этими характеристиками могут быть: функции ЖЦ информационной системы; требования системы и ПО; организационные основы коллективов специалистов, процедуры и стратегии их работы; размер, критичность и типы системы; число задействованного персонала и сторон-участников.

Применение требований ГОСТ Р ИСО/МЭК 12207 к конкретному проекту (его адаптация) состоит из работ следующих видов:

- определение условий выполнения проекта;
- запрос исходных данных для адаптации;

• выбор процессов, работ и задач;
• документирование решений по адаптации и их обоснований.

При определении условий выполнения проекта должны быть определены характеристики условий выполнения проекта, влияющие на адаптацию (например, модель ЖЦ; влияние ЖЦ цикла существующей системы; требования к системе и ПС; организационные подходы, процедуры и цели; размер, критичность и типы системы, ПС продукта или услуги; количество задействованного персонала и участвующих в проекте сторон).

При запросе исходных данных для адаптации от участвующих в проекте субъектов должны быть запрошены и получены исходные данные, которые могут повлиять на решения по адаптации. В работы по адаптации должны быть вовлечены пользователи, персонал сопровождения, заказчик и потенциальные поставщики.

При выборе процессов, работ и задач должны быть определены необходимые для построения модели ЖЦ ПС процессы, работы и задачи. При этом должны быть охвачены разрабатываемая документация и обязанности исполнителей. Дополнительные процессы, работы и задачи, необходимые для реализации проекта, но не описанные в ГОСТ Р ИСО/МЭК 12207, следует установить в договорной документации проекта.

Все решения по адаптации и их обоснования должны быть документально оформлены.

При проведении работ по адаптации следует руководствоваться также рекомендациями в части классификации ПС и в части выбора и построения модели ЖЦ ПС.

Так, построение модели ЖЦ ПС должно базироваться на концептуальной идее ПС (системы), охватывать разработку (создание), эксплуатацию и сопровождение и оканчиваться снятием (утилизацией). Модель ЖЦ обычно разбивается на периоды реализации, например стадии или этапы. Каждое такое разбиение должно охватывать отдельные работы и задачи, реализуемые в данном периоде (стадии, этапе), и при их завершении может потребоваться разрешение сторон на переход к следующему периоду модели.

Вопросы адаптации общей структуры ЖЦ ПС, описанной в ГОСТ Р ИСО/МЭК 12207, являются ключевыми при выборе (построении) модели ЖЦ ПС (автономной или входящей в состав общей модели ЖЦ создаваемой системы) в условиях реализации конкретного проекта.

Процессы общей структуры ЖЦ ПС по ГОСТ Р ИСО/МЭК 12207 основаны на двух исходных принципах: модульности и ответственности.

Принцип *модульности* основан на следующих положениях.

- Каждый процесс сильно связан, т. е. организован таким обра зом, что все части процесса (работы, задачи) строго взаимо связаны.
- Процессы свободно соединены между собой. Количество ин терфейсов между процессами сведено к минимуму.
- В принципе каждый процесс предназначен для реализации уни кальной функции в ЖЦ и может привлекать другой процесс для выполнения специализированной функции. При определе нии области применения и структурирования процессов долж ны использоваться следующие правила.
- Процесс должен быть своего рода модулем ЖЦ, т. е. каждый процесс должен выполнять только собственную функцию в ЖЦ, а интерфейсы между двумя любыми процессами должны быть минимальны.
- Каждый процесс должен быть привязан к архитектуре системы.
- Если процесс А вызван процессом В и только процессом В, тогда А принадлежит к В.
- Если работа или задача вызвана более чем одним процессом, тогда она сама становится процессом.
- Должна быть возможность для проверки любого процесса, ра боты и задачи в модели ЖЦ.
- Каждый процесс должен иметь внутреннюю структуру, уста новленную в соответствии с тем, что должно выполняться.

Принцип ответственности базируется на определенных обязанностях каждого субъекта, вовлеченного в ЖЦ. Субъект может выполнять один или несколько процессов. Процесс может быть выполнен одним или несколькими субъектами, при этом один из субъектов должен быть определен ответственным за процесс. Субъект, выполняющий процесс, несет ответственность за весь данный процесс, даже если выполнение отдельных работ (задач) поручено другим субъектам.

Ответственность является особенностью структуры ЖЦ применительно к условиям проекта, в который закономерно может быть вовлечено множество субъектов.

Безусловно, применение ГОСТ Р ИСО/МЭК 12207 требует от соответствующих субъектов определенных усилий по его адапта-

ции к условиям реализации конкретных проектов. Кроме того, требуются усилия по его взаимоувязке с конкретными методиками разработки систем и другими стандартами. Тем не менее можно полагать, что внедрение данного стандарта в практическую деятельность должно облегчить упорядочение взаимоотношений между субъектами, вовлеченными в ЖЦ ПС [58].

В стандартах на ЖЦ ПС отражено содержание этапов работ и результирующих документов на методологическом и концептуальном уровне. Методы и средства реализации каждой работы в этих стандартах не раскрываются и адресуются к специальным, детализирующим нормативным документам различного уровня. Однако ряд характерных особенностей этапов принципиально не позволяет создать полную гамму международных стандартов, поддерживающих все этапы и процессы ЖЦ ПС. Например, быстро оснащающиеся различными методами и инструментальными средствами этапы системного анализа, моделирования и предварительного проектирования не позволяют стабилизировать основу этих процессов, достаточную для формализации на уровне международных стандартов, для подготовки которых требуется несколько лет. Поэтому для этих этапов создаются нормативные документы на уровне стандартов «де-факто», руководств фирм или сопровождающей документации на конкретные инструментальные средства.

2.7. Модели жизненного цикла программных средств

Модель жизненного цикла: структура, состоящая из процессов, работ и задач, включающих в себя разработку, эксплуатацию и сопровождение программного продукта, охватывающая жизнь системы от установления требований к ней до прекращения ее использования.

К настоящему времени наибольшее распространение получили следующие основные модели ЖЦ:

- каскадная модель (70-80-е годы 20 века);
- спиральная модель (80-90-е годы 20 века).

В изначально существовавших однородных ИС каждое приложение представляло собой единое целое. Для разработки такого типа приложений применялся каскадный способ. Его основ-

ной характеристикой является разбиение всей разработки на этапы, причем переход с одного этапа на следующий происходит только после того, как будет полностью завершена работа на текущем (рис. 2.19). Каждый этап завершается выпуском полного комплекта документации, достаточной для того, чтобы разработка могла быть продолжена другой командой разработчиков.

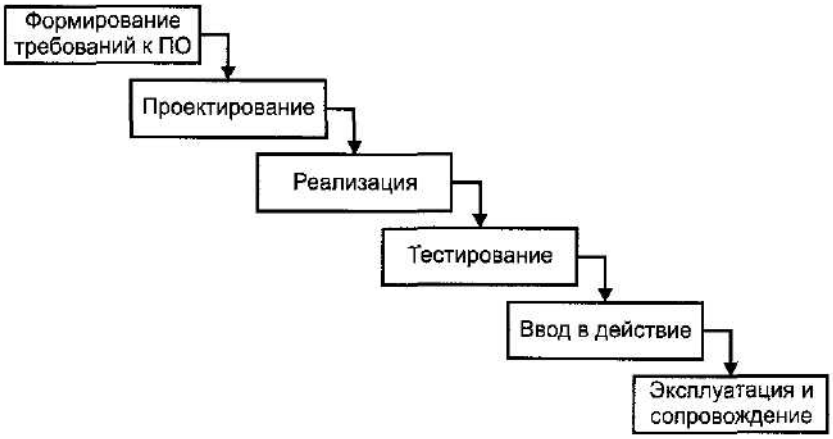


Рис. 2.19. Каскадная схема разработки программного средства

Положительные стороны применения каскадного подхода заключаются в следующем:

- на каждом этапе формируется законченный набор проектной документации, отвечающий критериям полноты и согласован ности;
- выполняемые в логичной последовательности этапы работ по зволяют планировать сроки завершения всех работ и соответ ствующие затраты.

Каскадный подход хорошо зарекомендовал себя при построении ИС, для которых в самом начале разработки можно достаточно точно и полно сформулировать все требования, с тем чтобы предоставить разработчикам свободу реализовать их как можно лучше с технической точки зрения. В эту категорию попадают сложные расчетные системы, системы реального времени и другие подобные задачи. Однако в процессе использования этого

подхода обнаружился ряд его недостатков, вызванных прежде всего тем, что реальный процесс создания ПС никогда полностью не укладывался в такую жесткую схему. В процессе создания ПС постоянно возникала потребность в возврате к предыдущим этапам и уточнении или пересмотре ранее принятых решений. В результате реальный процесс создания ПС принимал следующий вид (рис 2.20).

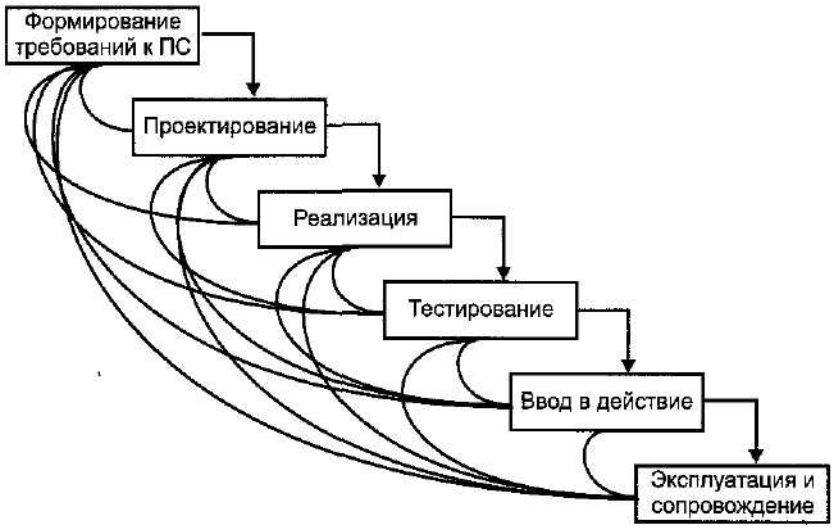


Рис. 2.20. Схема реального процесса разработки ПС по каскадной схеме

Основным недостатком каскадного подхода является существенное запаздывание с получением результатов. Согласование результатов с пользователями производится только в точках, планируемых после завершения каждого этапа работ, требования к ИС «заморожены» в виде технического задания на все время ее создания. Таким образом, пользователи могут внести свои замечания только после того, как работа над системой будет полностью завершена. В случае неточного изложения требований или их изменения в течение длительного периода создания ПС пользователи получают систему, не удовлетворяющую их потребностям. Модели (как функциональные, так и информационные) автоматизируемого объекта могут устареть одновременно с их утверждением.

Для преодоления перечисленных проблем была предложена спиральная модель ЖЦ (рис. 2.21), делающая упор на начальные этапы ЖЦ: анализ и проектирование. На этих этапах реализуемость технических решений проверяется путем создания прототипов. Каждый виток спирали соответствует созданию фрагмента или версии ПС, на нем уточняются цели и характеристики проекта, определяется его качество и планируются работы следующего витка спирали. Таким образом, углубляются и последовательно конкретизируются детали проекта, и в результате выбирается обоснованный вариант, который доводится до реализации.



Рис. 2.21. Схема спиральной модели жизненного цикла

Разработка итерациями отражает объективно существующий спиральный цикл создания системы. Неполное завершение работ на каждом этапе позволяет переходить на следующий этап, не дожидаясь полного завершения работы на текущем. При итеративном способе разработки недостающую работу можно будет выполнить на следующей итерации. Главная же задача — как можно быстрее показать пользователям системы работоспособный продукт, тем самым активизируя процесс уточнения и дополнения требований.

Основная проблема спирального цикла — определение момента перехода на следующий этап. Для ее решения необходимо ввести временные ограничения на каждый из этапов жизненного

цикла. Переход осуществляется в соответствии с планом, даже если не вся запланированная работа закончена. План составляется на основе статистических данных, полученных в предыдущих проектах, и личного опыта разработчиков [45].

ВОПРОСЫ РЯ САМОПРОВЕРКИ

1. Что понимается под профилем стандарта?
2. Объясните понятие жизненного цикла программного сред ства.
3. Назовите основные стандарты, характеризующие жизненный цикл программного средства.
4. Назовите и кратко охарактеризуйте процессы жизненного цикла программного средства, описанные в стандарте ГОСТ Р ИСО/МЭК 12207.
5. Определите основные положения, на которых основаны прин ципы модульности и ответственности.
6. Дайте определение модели жизненного цикла программного средства.
7. Объясните смысл каскадной и спиральной модели жизненно го цикла программного средства.
8. В чем заключаются главные положительные свойства каскад ной модели?
9. Охарактеризуйте недостатки каскадной модели.
10. В чем заключается основная проблема спиральной модели?

ГЛАВА 3 СТАНДАРТЫ ДОКУМЕНТИРОВАНИЯ ПРОГРАММНЫХ СРЕДСТВ

Создание программной документации — важный этап, так как

пользователь начинает свое знакомство с программным продуктом именно с документации. Для чего предназначен программный продукт, как установить программный продукт, как начать с ним работать — вот одни из первых вопросов, на которые должна отвечать программная документация (Installation Guide, Getting Started). Составлением программной документации обычно занимаются специальные люди — технические писатели (иногда программную документацию пишут сами программисты или аналитики). Этот этап является самым неприятным и тяжелым в программистской работе. К сожалению, обычно этому либо не учат совсем, либо в лучшем случае не обращают на качество получаемых документов должного внимания. Тем не менее владение этим искусством является одним из важнейших факторов, определяющих качество программиста.

Умение создавать программную документацию определяет профессиональный уровень программиста. Заказчик не будет вникать в тонкости и особенности даже самой замечательной программы. Заказчик будет сначала читать документацию. Большую роль играет в этом и психологический фактор.

Грамотно составленный (точнее, созданный) пакет программной документации избавит вас от многих неприятностей. В частности, избавиться от назойливых вопросов и необоснованных претензий можно, просто отослав пользователя к документации. Это касается прежде всего важнейшего документа — Технического задания. Можно напомнить о многомиллионном иске к компании IBM, который предъявило одно крупное издательство, не удовлетворенное качеством вычислительной техники и программного обеспечения. IBM суд выиграла только благодаря тому, что предъявила подписанное обеими сторонами Техническое задание. Было это давно, еще в 70-х годах 20 века, однако сути дела это не меняет. На Западе важность программной документации

поняли давно, вместе с программным обеспечением поставляется целый пакет документации.

Вообще программную документацию можно разделить по отношению к пользователю на внутреннюю и внешнюю. Внешняя — всевозможные руководства для пользователей, техническое задание, справочники; внутренняя документация — та, которая используется в процессе разработки программного обеспечения и недоступна конечному пользователю (различные внутренние стандарты, комментарии исходного текста, технологии программирования и т.д.) [61].

Когда программист-разработчик получает в той или иной форме задание на программирование, перед ним, перед руководителем проекта и перед всей проектной группой встают вопросы:

- Что должно быть сделано, кроме собственно программы?
- Что и как должно быть оформлено в виде документации?
- Что передавать пользователям, а что — службе сопровождения?
- Как управлять всем этим процессом?
- Что должно входить в само задание на программирование?

На эти и другие вопросы когда-то отвечали государственные стандарты на программную документацию — комплекс стандартов 19-й серии ГОСТ ЕСПД. Но уже тогда у программистов была масса претензий к этим стандартам. Что-то требовалось дублировать в документации много раз (как оказалось — неоправданно), а многое не было предусмотрено, как, например, отражение специфики документирования программ, работающих с интегрированной базой данных.

Прошло много лет, программирование происходит в среде совершенно новых технологий, многие программисты, работая в стиле drag-and-drop, могут годами не видеть текстов своих программ. Это не значит, что исчезла необходимость в их документировании. Вопросы о наличии хоть какой-то системы, регламентирующей эту сторону создания программных средств, продолжают задавать постоянно.

3.1. Общая характеристика состояния в области документирования программных средств

Основу отечественной нормативной базы в области документирования ПС составляет комплекс стандартов Единой системы программной документации (ЕСПД). Основная и большая часть

комплекса ЕСПД была разработана в 70-е и 80-е годы 20 века. Сейчас этот комплекс представляет собой систему межгосударственных стандартов стран СНГ (ГОСТ), действующих на территории Российской Федерации на основе межгосударственного соглашения по стандартизации.

Единая система программной документации — это комплекс государственных стандартов, устанавливающих взаимоувязанные правила разработки, оформления и обращения программ и программной документации.

Стандарты ЕСПД в основном охватывают ту часть документации, которая создается в процессе разработки ПС, и связаны, по большей части, с документированием функциональных характеристик ПС. Следует отметить, что стандарты ЕСПД (ГОСТ 19) носят рекомендательный характер. Впрочем, это относится и ко всем другим стандартам в области ПС (ГОСТ *34,* международному стандарту ISO/IEC и др.). Дело в том, что в соответствии с Законом РФ «О стандартизации» эти стандарты становятся обязательными на контрактной основе, т е. при ссылке на них в договоре на разработку (поставку) ПС.

В состав ЕСПД входят:

- основополагающие и организационно-методические стандарты;
- стандарты, определяющие формы и содержание программных документов, применяемых при обработке данных;
- стандарты, обеспечивающие автоматизацию разработки про граммных документов.

Говоря о состоянии ЕСПД в целом, можно констатировать, что большая часть стандартов ЕСПД морально устарела. К числу основных недостатков ЕСПД можно отнести:

- ориентацию на единственную «каскадную» модель жизненного цикла ПС;
- отсутствие четких рекомендаций по документированию харак теристик качества ПС;
- отсутствие системной увязки с другими действующими отече ственными системами стандартов по ЖЦ и документированию продукции в целом, например ЕСКД;
- нечетко выраженный подход к документированию ПС как то варной продукции;
- отсутствие рекомендаций по самодокументированию ПС, на пример, в виде экранных меню и средств оперативной помощи пользователю (хелпов);

- отсутствие рекомендаций по составу, содержанию и оформле нию перспективных документов на ПС, согласованных с реко мендациями международных и региональных стандартов.

ЕСПД нуждается в полном пересмотре на основе стандарта ИСО/МЭК 12207-95 на процессы жизненного цикла ПС.

Тем не менее до пересмотра всего комплекса многие стандарты могут с пользой применяться в практике документирования ПС. Эта позиция основана на следующем:

- стандарты ЕСПД вносят элемент упорядочения в процесс до кументирования ПС;
- предусмотренный стандартами ЕСПД состав программных до кументов вовсе не такой «жесткий», как некоторым кажется: стандарты позволяют вносить в комплект документации на ПС дополнительные виды программных документов (ПД). необхо димых в конкретных проектах, и исключать многие ПД;
- стандарты ЕСПД позволяют вдобавок мобильно изменять] структуры и содержание установленных видов ПД исходя из требований заказчика и пользователя.

При этом стиль применения стандартов может соответствовать современному общему стилю адаптации стандартов к специфике проекта: заказчик и руководитель проекта выбирают уместное в проекте подмножество стандартов и ПД, дополняют выбранные ПД нужными разделами и исключают ненужные, привязывают создание этих документов к той схеме ЖЦ, которая используется в проекте.

Надо сказать, что наряду с комплексом ЕСПД официальная нормативная база РФ в области документирования ПС и в смежных областях включает ряд перспективных стандартов (отечественного, межгосударственного и международного уровней).

Международный стандарт ISO/IEC 12207:1995 на организацию ЖЦ продуктов программного обеспечения (ПО), казалось бы, весьма неконкретный, но вполне новый и отчасти «модный» стандарт.

Стандарты комплекса ГОСТ 34 на создание и развитие автоматизированных систем — обобщенные, но воспринимаемые как весьма жесткие по структуре ЖЦ и проектной документации. Но эти стандарты многими считаются бюрократическими до вредности и консервативными до устарелости [59].

3.2. Единая система программной документации

Стандарты ЕСГТД (как и другие ГОСТы) подразделяют на группы, приведенные в табл. 3.1.

Таблица 3.1

Код группы	Наименование группы
0	Общие положения
1	Основополагающие стандарты
2	Правила выполнения документации разработки
3	Правила выполнения документации изготовления
4	Правила выполнения документации сопровождения
5	Правила выполнения эксплуатационной документации
6	Правила обращения программной документации
7	Резервные группы
8	
9	Прочие стандарты

Обозначение стандарта ЕСПД должно состоять из:

- числа 19 (присвоенных классу стандартов ЕСПД);
- одной цифры (после точки), обозначающей код классификаци онной группы стандартов, указанной в таблице;
- двузначного числа (после тире), указывающего год регистра ции стандарта.

Вообще перечень документов ЕСПД очень обширен. В него, в частности, входят следующие ГОСТы:

ГОСТ 19.001-77 ЕСПД. Общие положения.

ГОСТ 19.005-85 ЕСПД. Р-схемы алгоритмов и программ. Обозначения условные графические и правила выполнения.

ГОСТ 19.101-77 ЕСПД. Виды программ и программных документов.

ГОСТ 19.102-77 ЕСПД. Стадии разработки.

ГОСТ 19.103-77 ЕСПД. Обозначение программ и программных документов.

ГОСТ 19.10^78 ЕСПД. Основные надписи.

ГОСТ 19.105-78 ЕСПД. Общие требования к программным документам.

ГОСТ 19.106-78 ЕСПД. Требования к программным документам, выполненным печатным способом.
ГОСТ 19.201-78 ЕСПД. Техническое задание. Требования к содержанию и оформлению.
ГОСТ 19.202-78 ЕСПД. Спецификация. Требования к содержанию и оформлению.
ГОСТ 19.301-79 ЕСПД. Порядок и методика испытаний.
ГОСТ 19.401-78 ЕСПД. Текст программы. Требования к содержанию и оформлению.
ГОСТ 19.402-78 ЕСПД. Описание программы.
ГОСТ 19.403-79 ЕСПД. Ведомость держателей подлинников.
ГОСТ 19.404-79 ЕСПД. Пояснительная записка. Требования к содержанию и оформлению.
ГОСТ 19.501-78 ЕСПД. Формуляр. Требования к содержанию и оформлению.
ГОСТ 19.502-78 ЕСПД. Описание применения. Требования к содержанию и оформлению.
ГОСТ 19.503-79 ЕСПД. Руководство системного программиста. Требования к содержанию и оформлению.
ГОСТ 19.504-79 ЕСПД. Руководство программиста. Требования к содержанию и оформлению.
ГОСТ 19.505-79 ЕСПД. Руководство оператора. Требования к содержанию и оформлению.
ГОСТ 19.506-79 ЕСПД. Описание языка. Требования к содержанию и оформлению.
ГОСТ 19.507-79 ЕСПД. Ведомость эксплуатационных документов.
ГОСТ 19.508-79 ЕСПД. Руководство по техническому обслуживанию. Требования к содержанию и оформлению.
ГОСТ **19.601—78** ЕСПД. Общие правила дублирования, учета и хранения.
ГОСТ 19.602-78 ЕСПД. Правила дублирования, учета и хранения программных документов, выполненных печатным образом.
ГОСТ 19.603-78 ЕСПД. Общие правила внесения изменений.
ГОСТ 19.604-78 ЕСПД. Правила внесения изменений в программные документы, выполняемые печатным способом.
ГОСТ 19.701-90 ЕСПД. Схемы алгоритмов, программ, данных и систем. Условные обозначения и правила выполнения.
ГОСТ 19781-90. Обеспечение систем обработки информации программное. Термины и определения.

Прежде чем приступить к рассмотрению правил составления программной документации, необходимо сделать следующее замечание. Каждый документ желательно предварять некоторым введением. Во введении говорится об актуальности, о необходимости и т.п. Цель исполнителя здесь — показать значимость и необходимость выполнения этой работы.

Из всех стандартов ЕСПД остановимся только на тех, которые могут чаще использоваться на практике.

Первым укажем стандарт, который устанавливает виды программ и программных документов для вычислительных машин, комплексов и систем независимо от их назначения и области применения.

ГОСТ 19.101-77 ЕСПД. Виды программ и программных документов

ГОСТ подразделяет программы на следующие виды:

Компонент — программа, рассматриваемая как единое целое, выполняющая законченную функцию и применяемая самостоятельно или в составе комплекса.

Комплекс — программа, состоящая из двух или более компонентов, выполняющих взаимосвязанные функции, и применяемая самостоятельно или в составе другого комплекса.

Документация, разработанная на программу, может использоваться для реализации и передачи программы на носителях данных, а также для изготовления программного изделия.

К числу программных данный ГОСТ относит документы, содержащие сведения, необходимые для разработки, изготовления, сопровождения и эксплуатации программ. Рассмотрим виды программных документов и их содержание:

Спецификация — содержит состав программы и документацию на нее.

Ведомость держателей подлинников — содержит перечень предприятий, на которых хранят подлинники программных документов.

Текст программы — представляет запись программы с необходимыми комментариями.

Описание программы — содержит сведения о логической структуре и функционировании программы.

Программа и методика испытаний — содержит требования, подлежащие проверке при испытании программы, а также порядок и методы их контроля.

Техническое задание — описывает назначение и область применения программы, технические, технико-экономические и специальные требования, предъявляемые к программе, необходимые стадии и сроки разработки, виды испытаний.

Пояснительная записка — содержит схему алгоритма, общее описание алгоритма и (или) функционирования программы, а также обоснование принятых технических и технико-экономических решений.

Эксплуатационные документы — содержат сведения для обеспечения функционирования и эксплуатации программы.

Рассмотрим виды и содержание этих документов (табл. 3.2).

Таблица 3.2

Вид эксплуатационного документа	Содержание эксплуатационного документа
Ведомость эксплуатационных документов	Перечень эксплуатационных документов на программу
Формуляр	Основные характеристики программы, комплектность и сведения об эксплуатации программы
Описание применения	Сведения о назначении программы, области применения, применяемых методах, классе решаемых задач, ограничениях для применения, минимальной конфигурации технических средств
Руководство системного программиста	Сведения для проверки, обеспечения функционирования и настройки программы на условия конкретного применения
Руководство программиста	Сведения для эксплуатации программы
Руководство оператора	Сведения для обеспечения процедуры общения оператора с вычислительной системой в процессе выполнения программы
Описание языка	Описание синтаксиса и семантики языка
Руководство по обслуживанию	Сведения для применения тестовых и диагностических программ при обслуживании технических средств

В зависимости от способа выполнения и характера применения программные документы подразделяются на подлинник, дубликат и копию (ГОСТ 2.102-68), предназначенные для разработки, сопровождения и эксплуатации программы.

Рассмотрим виды программных документов, разрабатываемых на разных стадиях, и их коды (табл. 3.3).

Таблица 3.3

Код вида документа	Вид документа	Стадии разработки			
		Эскизный проект	Технический проект	Рабочий проект	
				компонент	комплекс
_	Спецификация	-	-		•
05	Ведомость держателей подлинников	-	-	-	О
12	Текст программы	-	_	•	О
13	Описание программы	-	-	О	О
20	Ведомость эксплуатационных документов	-	-	О	О
30	Формуляр	-	-	О	О
31	Описание применения	-	-	**О**	О
32	Руководство системного программиста	-	-	**О**	О
33	Руководство программиста	-	-	**О**	О
34	Руководство оператора	-	-	**О**	О
35	Описание языка	-	-	**О**	О
46	Руководство по техническому обслуживанию			**О**	О
51	Программа и методика испытаний	-	-	**О**	О
81	Пояснительная записка	О	О	**-**	-
90-99	Прочие документы	О	О	**О**	О

Условные обозначения: • — документ обязательный; *Ь* — документ обязательный для компонентов, имеющих самостоятельное применение; О - — необходимость составления документа определяется на этапе разработки и утверждения технического задания; - — документ не составляют.

Допускается объединять отдельные виды эксплуатационных документов (за исключением ведомости эксплуатационных документов и формуляра). Необходимость объединения этих документов указывается в техническом задании. Объединенному документу присваивают наименование и обозначение одного из объединяемых документов. В объединенных документах должны быть приведены сведения, которые необходимо включать в каждый объединяемый документ.

ГОСТ 19.102-77 ЕСПД. Стадии разработки

Устанавливает стадии разработки программ и программной документации для вычислительных машин, комплексов и систем независимо от их назначения и области применения (табл. 3.4).

Таблица 3.4

Стадии разработки, этапы и содержание работ

Стадия разработки	Этап работы	Содержание работ
Техническое задание	Обоснование необходимости разработки программы	Постановка задачи. Сбор исходных материалов. Выбор и обоснование критериев эффективности и качества разрабатываемой программы. Обоснование необходимости проведения научно-исследовательских работ
	Научно-исследовательские работы	Определение структуры входных и выходных данных. Предварительный выбор методов решения задач. Обоснование целесообразности применения ранее разработанных программ. Определение требований к техническим средствам. Обоснование принципиальной возможности решения поставленной задачи

Продолжение

Стадия разработки	Этап работы	Содержание работ
	Разработка и утверждение технического задания	Определение требований к программе. Разработка технико-экономического обоснования разработки программы. Определение стадий, этапов и сроков разработки программы и документации на нее. Выбор языков программирования. Определение необходимости проведения научно-исследовательских работ на последующих стадиях. Согласование и утверждение технического задания
Эскизный проект	Разработка эскизного проекта	Предварительная разработка структуры входных и выходных данных. Уточнение методов решения задачи. Разработка общего описания алгоритма решения задачи. Разработка тетшико- экономического обоснования
	Утверждение эскизного проекта	Разработка пояснительной записки. Согласование и утверждение эскизного проекта
Технический проект	Разработка технического проекта	Уточнение структуры входных и выходных данных. Разработка алгоритма решения задачи Определение формы представления входных и выходных данных. Определение семантики и синтаксиса языка. Разработка структуры программы. Окончательное определение конфигурации технических средств
	Утверждение технического проекта	Разработка плана мероприятий по разработке и внедрению программ. Разработка пояснительной записки. Согласование и утверждение технического проекта

Продолжение

Стадия разработки	Этап работы	Содержание работ
Рабочий проект	Разработка программы	Программирование и отладка программы
	Разработка программной документации	Разработка программных документов в соответствии с требованиями ГОСТ 19.101-77
	Испытания программы	Разработка, согласование и утверждение программы и методики испытаний Проведение предварительных государственных, межведомственных, приемосдаточных и других видов испытаний. Корректировка программы и программной документации по результатам испытаний
Внедрение	Подготовка и передача программы	Подготовка и передача программы и программной документации для сопровождения и (или) изготовления. Оформление и утверждение акта о передаче программы на сопровождение и (или) изготовление. Передача программы в фонд алгоритмов и программ

Допускается исключать вторую стадию разработки, а в технически обоснованных случаях — вторую и третью стадии. Необходимость проведения этих стадий указывается в техническом задании.

Допускается объединять, исключать этапы работ и (или) их содержание, а также вводить другие этапы работ по согласованию с заказчиком.

ГОСТ 19.105-78 ЕСПД. Общие требования к программным документам

Данный стандарт устанавливает общие требования к оформлению программных документов для вычислительных машин, комплексов и систем независимо от их назначения и области

применения и предусмотренных стандартами Единой системы программной документации (ЕСПД) для любого способа выполнения документов на различных носителях данных.

Программный документ может быть представлен на различных типах носителей данных и состоит из следующих условных частей:

- титульной;
- информационной;
- основной.

Правила оформления документа и его частей на каждом носителе данных устанавливаются стандартами ЕСПД на правила оформления документов на соответствующих носителях данных.

Титульная часть оформляется согласно ГОСТ 19.104—78.

Информационная часть должна состоять из аннотации и содержания. В аннотации приводят сведения о назначении документа и краткое изложение основной части. Содержание включает перечень записей о структурных элементах основной части документа.

Состав и структура основной части программного документа устанавливаются стандартами ЕСПД на соответствующие документы.

ГОСТ 19.201-78 ЕСПД. Техническое задание. Требования к содержанию и оформлению

Техническое задание (ТЗ) содержит совокупность требований к ПС и может использоваться как критерий проверки и приемки разработанной программы. Поэтому достаточно полно составленное (с учетом возможности внесения дополнительных разделов) *и* принятое заказчиком и разработчиком ТЗ является одним из основополагающих документов проекта ПС.

Техническое задание должно содержать следующие разделы:

- введение;
- основания для разработки;
- назначение разработки;
- требования к программе или программному изделию;
- требования к программной документации;
- технико-экономические показатели;
- стадии и этапы разработки;
- порядок контроля и приемки;
- в техническое задание допускается включать приложения.

В зависимости от особенностей программы или программного изделия допускается уточнять содержание разделов, вводить новые разделы или объединять отдельные из них.

Содержание разделов ТЗ мы рассмотрим позднее, а пока рассмотрим только **требования к программной документации. В** данном разделе должны быть указаны предварительный состав программной документации и при необходимости специальные требования к ней.

ГОСТ 19.402-78 ЕСПД. Описание программы

Данный стандарт определяет состав и требования к содержанию программного документа «Описание программы». Описание программы включает:

1. Общие сведения.
2. Функциональное назначение.
3. Описание логической структуры.
4. Используемые технические средства.
5. Вызов и загрузка.
6. Входные данные.
7. Выходные данные.

В разделе *Общие сведения* указывают:

- обозначение и наименование программы;
- программное обеспечение, необходимое для функционирования программы;
- языки программирования, на которых написана программа.

Раздел *Функциональное* ***назначение*** должен отражать классы решаемых задач и/или назначение программы, сведения о функциональных ограничениях на применение.

При описании *логической структуры* должны быть отражены:

- алгоритм программы;
- используемые методы;
- структура программы с описанием функций составных частей и связей между ними;
- связи программы с другими программами.

В разделе *Используемые технические средства* указывают типы ЭВМ и устройств, которые используются при работе программы.

При описании раздела *Вызов и загрузка* указывают способ вызова программы с соответствующего носителя данных и входные точки в программу.

Раздел *Входные данные* отражает:

- характер, организацию и предварительную подготовку вход ных данных;
- формат, описание и способ кодирования входных данных.

Раздел *Выходные данные* отражает:

- характер и организацию выходных данных;
- формат, описание и способ кодирования выходных данных.

ГОСТ 19.404-79 ЕСПД. Пояснительная записка. Требования к содержанию и оформлению

Согласно данному стандарту пояснительная записка должна включать следующие разделы:

1. Введение.
2. Назначение и область применения.
3. Технические характеристики.
4. Ожидаемые технико-экономические показатели.
5. Источники, использованные при разработке.

Введение должно содержать наименование программы и/или обозначение темы разработки, а также документы, на основе которых ведется разработка.

При описании *назначения и области применения* указывают назначение программы, краткую характеристику области применения программы.

В разделе *Технические характеристики* содержатся:

- постановка задачи на разработку программы, описание при меняемых математических методов и различных ограничений, связанных с выбранным математическим аппаратом;
- описание алгоритма и/или функционирования программы с обоснованием выбора схемы алгоритма решения задачи, воз можного взаимодействия программы с другими программами;
- описание и обоснование выбора метода организации входных и выходных данных;
- описание и обоснование выбора состава технических и про граммных средств на основе проведенных расчетов и анализов, распределение носителей данных, которые использует прог рамма.

В качестве *ожидаемых технико-экономических показателей* указывают показатели, обосновывающие преимущество выбранного варианта технического решения, а также при необходимости ожидаемые оперативные показатели.

При описании источников, использованных при разработке, необходимо привести перечень научно-технических публикаций, нормативно-технических документов и других научно-технических материалов, на которые есть ссылки в основном тексте.

ГОСТ 19.503-79 ЕСПД. Руководство системного программиста. Требования к содержанию и оформлению

Руководство системного программиста должно содержать следующие разделы:

1. Общие сведения о программе.
2. Структура программы.
3. Настройка программы.
4. Проверка программы.
5. Дополнительные возможности.
6. Сообщения системному программисту.

При необходимости допускается опускать раздел, описывающий дополнительные возможности.

При описании *общих сведений о программе* необходимо указать назначение и функции программы и сведения о технических и программных средствах, обеспечивающих выполнение данной программы.

В разделе *Структура программы* приводятся сведения о структуре программы, ее составных частях и связях с другими программами.

Раздел *Настройка программы* должен содержать описание действий по настройке программы на условия конкретного применения.

При описании *проверки программы* необходимо привести и описать способы проверки, позволяющие дать общее заключение о работоспособности программы (контрольные примеры, методы прогона, результаты).

Раздел *Дополнительные возможности* должен содержать описание дополнительных разделов функциональных возможностей программы и способов их выбора.

В разделе *Сообщения системному программисту* необходимо указать тексты сообщений, выдаваемых в ходе выполнения программы, описание содержания и действий, которые необходимо предпринять по этим сообщениям.

ПО

ГОСТ 19.504-79 ЕСПД. Руководство программиста. Требования к содержанию и оформлению

Руководство программиста должно содержать разделы:

1. Назначение и условия применения программы.
2. Характеристики программы.
3. Обращение к программе.
4. Входные и выходные данные.
5. Сообщения.

При описании *назначения и условий применения программы* необходимо указать назначение и функции, выполняемые программой; условия, необходимые для выполнения программы; объем оперативной памяти, требования к составу и параметрам периферийных устройств; требования к ПО и т.д.

В разделе *Характеристики программы* необходимо привести описание основных характеристик и особенностей программы: временных характеристик, режима работы, средств контроля правильности выполнения и самовосстанавливаемости программы и т.д.

Раздел *Обращение к программе* представляет собой описание процедур вызова программы (способов передачи управления и параметров данных и др.).

Раздел *Входные и выходные данные* должен содержать описание организации используемой входной и выходной информации и при необходимости ее кодирования.

При описании *сообщений* необходимо привести тексты сообщений, выдаваемых программисту или оператору в ходе выполнения программы, описание их содержания и действия, которые необходимо предпринять по этим сообщениям.

ГОСТ 19.505-79 ЕСПД. Руководство оператора. Требования к содержанию и оформлению

Руководство оператора должно включать:

1. Назначение программы.
2. Условия выполнения программы.
3. Выполнение программы.
4. Сообщения оператору.

При описании *назначений программы* необходимо указать сведения о назначении программы и информацию, достаточную для понимания функций программы и ее эксплуатации.

Условия выполнения программы должны содержать условия, необходимые для выполнения программы: минимальный и/или максимальный состав аппаратурных и программных средств.

В разделе *Выполнение программы* необходимо указать последовательность действий оператора, обеспечивающих загрузку, запуск, выполнение и завершение программы; привести описание функций, формата и возможных вариантов команд, с помощью которых оператор осуществляет загрузку и управляет выполнением программы, а также ответы программы на эти команды.

При описании *сообщений оператору* приводят тексты сообщений, выдаваемых в ходе выполнения программы, описание их содержания и соответствующие действия оператора: действия в случае сбоя, возможности повторного запуска программы и т.д.

ГОСТ 19.506-79 ЕСПД. Описание языка. Требования к содержанию и оформлению

При описании языка необходимо указать:

1. Общие сведения.
2. Элементы языка.

Кроме того, допускается вводить дополнительные разделы.

3. Способы структурирования программы.
4. Средства обмена данными.
5. Встроенные элементы.
6. Средства отладки программы.

Общие сведения должны содержать назначение и описание общих характеристик языка, его возможностей, основных областей применения и др.

В разделе *Элементы языка* приводят описание синтаксиса и семантики базовых и составных элементов языка.

Раздел *Способы структурирования программы* должен описывать способы вызова процедур передачи управления и другие элементы структурирования программы.

Раздел *Средства обмена данными* должен содержать описание языковых средств обмена данными (например, средств ввода-вывода, средств внутреннего обмена данными и т.д.).

В разделе *Встроенные элементы* описываются встроенные в язык элементы: функции, классы и т.д. и правила их использования.

При описании *средств отладки* необходимо привести описание имеющихся в языке средств отладки программ, семантики этих средств, дать рекомендации по их применению.

3.3. Государственные стандарты Российской Федерации (ГОСТ Р)

В Российской Федерации действует ряд стандартов в части документирования ПС, разработанных на основе прямого применения международных стандартов ИСО. Это самые «свежие» по времени принятия стандарты. Некоторые из них напрямую адресованы руководителям проекта или директорам информационных служб. Вместе с тем они неоправданно мало известны в среде профессионалов. Вот их представление.

ГОСТ Р ИСО/МЭК 9294-93. Информационная технология. Руководство по управлению документированием программного обеспечения. Стандарт полностью соответствует международному стандарту ИСО/МЭК 9294:1990 и устанавливает рекомендации по эффективному управлению документированием ПС для руководителей, отвечающих за их создание. Целью стандарта является оказание помощи в определении стратегии документирования ПС; выборе стандартов по документированию; выборе процедур документирования, определении необходимых ресурсов; составлении планов документирования.

ГОСТ Р ИСО/МЭК 9126-93. Информационная технология. Оценка программной продукции. Характеристики качества и руководства по их применению. Стандарт полностью соответствует международному стандарту ИСО/МЭК 9126:1991. В его контексте под характеристикой качества понимается «набор свойств (атрибутов) программной продукции, по которым ее качество описывается и оценивается». Стандарт определяет шесть комплексных характеристик, которые с минимальным дублированием описывают качество ПС (ПО, программной продукции):

- функциональные возможности;
- надежность;
- практичность;

- эффективность;
- сопровождаемость;
- мобильность.

Эти характеристики образуют основу для дальнейшего уточнения и описания качества ПС.

ГОСТ Р ИСО 9127-94. Системы обработки информации. Документация *пользователя* ***и информация на упаковке*** *для* ***потребительских программных*** *пакетов.* Стандарт полностью соответствует международному стандарту ИСО 9127:1989. В контексте настоящего стандарта под потребительским программным пакетом (ПП) понимается «программная продукция, спроектированная и продаваемая для выполнения определенных функций; программа и соответствующая ей документация, упакованные для продажи как единое целое». Под документацией пользователя понимается документация, которая обеспечивает конечного пользователя информацией по установке и эксплуатации ПП. Под информацией на упаковке понимают информацию, воспроизводимую на внешней упаковке ПП. Ее целью является предоставление потенциальным покупателям первичных сведений о ПП.

ГОСТ Р ИСО/МЭК 8631-94. Информационная технология. Программные конструктивы и условные обозначения для их представления. Описывает представление процедурных алгоритмов.

Как уже говорилось, пока нет лучшего, можно извлекать пользу и из тех стандартов ЕСПД, которые приняты еще около 20 лет назад. Но всем ясно, что ориентироваться надо на современные стандарты.

Практики используют еще один путь: сами переводят и используют в своих проектах современные стандарты на организацию ЖЦ ПС и их документирование. Но этот путь страдает как минимум тем недостатком, что разные переводы и адаптации стандартов, сделанные разными разработчиками и заказчиками, будут отличаться массой деталей. Эти отличия неизбежно касаются не только наименований, но и их содержательных определений, вводимых и используемых в стандартах. Таким образом, на этом пути неизбежно постоянное возникновение путаницы, а это прямо противоположно целям не только стандартов, но и любых грамотных методических документов [59].

ГОСТ Р ИСО/МЭК 12119:1994. Информационная технология. Пакеты программных средств. Требования к качеству и испытания. В этом стандарте установлены требования к качеству пакетов про-

грамм и инструкции по их испытаниям на соответствие заданным требованиям. Понятие «пакет программных средств» фактически отождествляется с более общим понятием «программный продукт», рассматриваемым как совокупность программ, процедур и правил, поставляемых нескольким пользователям для общего применения или функционирования. Каждый пакет программ должен иметь описание продукта и пользовательскую документацию.

Рассмотрим более подробно содержание данного стандарта.

Стандарт определяет требования к ***описанию продукта, к*** *пользовательской документации, программам* ***и*** *данным,* входящим **в** пакет программ, и испытаниям пакетов программ.

Предполагается, что документ «Описание продукта» должен помочь пользователю или потенциальному покупателю в оценке того, подходит ли для них данный продукт, а пользовательская документация должна содержать всю информацию, необходимую для применения продукта.

В контексте данного стандарта требования к качеству продукта рассматриваются с точки зрения описания реальных свойств продукта в «Описании продукта» и пользовательской документации. Требования к программам и данным в основном сводятся к утверждению необходимости соответствия реальных свойств продукта свойствам, объявленным в документации. В связи с этим документ формально не может рассматриваться как стандарт требований. Несмотря на эту ограниченность, стандарт может оказаться весьма полезным при определении исходных требований к продукту:

- требования, согласно которому каждый пакет программ должен содержать описание продукта и документацию пользователя;
- требования к описанию продукта. В частности, требования, со гласно которому описание продукта должно содержать конкрет ную информацию, а все приводимые в нем формулировки долж ны быть проверяемыми (контролируемыми) и корректными;
- требования к документации пользователя;
- требования к любым программам и данным, входящим в со став пакета программ.

Описание продукта. Описание продукта (product description): документ, определяющий свойства пакета программ, основным назначением которого является оказание помощи потенциальным покупателям в оценке пригодности для них данного продукта до его приобретения.

Каждый пакет программ должен содержать описание продукта. Оно должно являться частью документации пакета для данного продукта и содержать информацию по документации пользователя, программам и соответствующим данным.

Основным назначением описания продукта является помощь пользователю и потенциальному покупателю при оценке ими пригодности продукта для их нужд. Для обеспечения этого описание продукта также должно содержать соответствующую торговую информацию.

Описывая любой программный продукт, необходимо придерживаться установленных требований к содержанию. В связи с этим можно выстроить определенную иерархию материала, подлежащего описанию:

1. Общие требования к содержанию.
2. Обозначения и указания.
3. Функциональные возможности.
4. Надежность.
5. Практичность.
6. Эффективность.
7. Сопровождаемость и мобильность.

Описание продукта должно быть доступным для человека, заинтересованного в данном продукте, и удовлетворять **общим требованиям к содержанию:**

- быть достаточно понятным, полным и простым при изучении, чтобы обеспечить помощь потенциальным покупателям при оценке ими пригодности данного продукта для их нужд до его покупки;
- быть внутренне непротиворечивым. Каждый термин должен иметь один и тот же смысл по всему документу;
- формулировки должны быть проверяемыми и корректными.

При описании продукта необходимо приводить следующие ***указания*** *и* ***обозначения:***

1. При обозначении одного или нескольких продуктов в рам ках одного пакета необходимо хотя бы включать наименова ние продукта и обозначение его версии или даты выпуска.
2. Должны быть включены наименование и адрес поставщика.
3. Должны быть определены целевые рабочие задачи, которые могут быть выполнены данным продуктом.
4. Из описания продукта могут быть даны ссылки на норма тивные документы, которым удовлетворяет данный продукт,

в этом случае должны быть указаны соответствующие редакции данных документов.

5. Должна быть определена система (технические и программные средства и их конфигурация), необходимая для ввода продукта в эксплуатацию, включая наименования изготовителей и обозначения типов всех ее частей, например:

- процессоры, включая сопроцессоры;
- объем основной (оперативной) памяти;
- типы и объемы (памяти) периферийных запоминающих устройств;
- расширяющие платы;
- оборудование ввода и вывода;
- сетевое оборудование;
- системные и прочие программные средства.

6. Должны быть определены соответствующие интерфейсы или продукты, если в описании продукта имеются ссылки на интерфейсы с другими продуктами.
7. Должен быть определен каждый физический компонент поставляемого продукта, в частности, все печатные документы и все носители данных.
8. Должен быть установлен вид поставляемых программ, например исходные программы, объектные (рабочие) модули или загрузочные модули.
9. Должно быть указано, будет ли инсталляция продукта проводиться пользователем или нет.
10. Должно быть указано, будет или не будет предлагаться поддержка при эксплуатации продукта.
11. Должно быть указано, будет или не будет предлагаться сопровождение продукта. Если сопровождение предусматривается, то должно быть установлено, что оно подразумевает.

При описании *функциональных возможностей* необходимо отразить:

1. *Обзор функций.*

В описании продукта должен быть приведен обзор функций продукта, вызываемых пользователем, необходимых для них данных и предоставляемых средств. Для каждой функции (особенно для ее опции или варианта) должно быть четко установлено, является ли она частью:

- продукта;

- расширения продукта, полностью приведенного в описании продукта;
- расширения продукта, на которое дана ссылка в описании продукта;
- негарантируемого (необязательного) приложения.

2. Граничные значения.

Если использование продукта ограничено конкретными граничными значениями для продукта, они должны быть указаны в описании продукта. Например:

- минимальные или максимальные значения;
- длины ключей;
- максимальное число записей в файле;
- максимальное число критериев поиска;
- минимальный объем выборки.

В случае, когда невозможно определить фиксированные граничные значения (например, когда они зависят от типа приложения или от исходных данных), на них должны быть наложены соответствующие ограничения. Могут быть приведены допустимые комбинации значений и даны ссылки на более конкретную информацию из документации пользователя.

3. *Защита.*

При необходимости в описание продукта должна быть включена информация по средствам предотвращения случайного или преднамеренного несанкционированного доступа к программам и данным.

Описывая *надежность продукта,* необходимо провести информацию по процедурам сохранение данных. Здесь также могут быть описаны дополнительные характеристики продукта, которые обеспечивают его функциональные возможности, например:

- проверки достоверности исходных данных;
- защиту против серьезных последствий ошибки пользователя;
- восстановление при ошибках.

При описании *практичности* необходимо затронуть следующие направления:

1. *Интерфейс пользователя.* Должен быть назван тип интерфейса пользователя, например:

- строка команд;
- меню;
- окна;

- функциональная клавиша;
- функция подсказки и др.

2. *Требуемые знания.* Должны быть определены конкретные знания, которые необходимо усвоить пользователю для приме нения соответствующего продукта, например:

- знание соответствующей технической области;
- знание операционной системы;
- знания, получаемые в результате специального обучения;
- знание языков, отличных от языка, на котором написано опи сание продукта. Должны быть указаны все естественные язы ки, на которых написана документация пользователя и описан интерфейс пользователя (включая сообщения об ошибках и ви димую информацию) как для самого пакета программ, так и для всех других продуктов, упомянутых в описании данного продукта.

3. *Адаптация к потребностям пользователя.* Если продукт может настраиваться (адаптироваться) пользователем, то долж ны быть указаны инструментальные средства для проведения та кой настройки и условия их применения, например:

- изменение параметров:
- изменение алгоритмов вычислений;
- назначение функциональных клавиш.

4. *Защита от нарушения авторских прав.* Если техническая защита от нарушения авторских прав может ухудшить практич ность описываемого продукта, то в описании продукта должны быть указаны виды и средства такой защиты. Например:

- техническая защита от копирования;
- запрограммированные даты окончания использования продукта;
- интерактивные напоминания об оплате за копии.

5. *Эффективность применения и удовлетворение потребностей пользователя.* В описание продукта может быть внесена инфор мация по эффективности применения продукта и удовлетворе нию им потребностей пользователя.

Описывая *эффективность,* необходимо отразить информацию о характере поведения продукта во времени, например указать время ответа и время оценки производительности для заданных функций при установленных условиях (например, для заданных конфигураций системы и профилей загрузки).

В описание продукта могут быть внесены формулировки тре-бований (правил) по *сопровождению и мобильности* продукта.

Документация пользователя

Документация пользователя (user documentation): полный комплект документов, поставляемых в печатном или другом виде, который обеспечивает применение продукта, а также является его неотъемлемой частью.

Документация пользователя должна отвечать следующим характеристикам.

Полнота (completeness). Документация пользователя должна содержать информацию, необходимую для использования продукта. В ней должны быть полностью описаны все функции, установленные в описании продукта, и все вызываемые пользователем функции из программы. Кроме того, граничные значения, заданные в описании продукта, должны быть продублированы в документации пользователя. Если установка (инсталляция) продукта может быть проведена пользователем, то в документацию пользователя должно быть включено руководство по установке продукта, содержащее всю необходимую информацию. Если сопровождение продукта может проводиться пользователем, то в документацию пользователя должно быть включено руководство по сопровождению программы, содержащее всю информацию, которая необходима для обеспечения данного вида сопровождения.

Правильность (correctness). Вся информация в документации пользователя должна быть правильной. Кроме того, представление данной информации не должно содержать неоднозначных толкований и ошибок.

Непротиворечивость (consistency). Документы, входящие в комплект документации пользователя, не должны противоречить сами себе, друг другу и описанию продукта. Каждый термин должен иметь один и тот же смысл во всех документах.

Понятность (understandability). Документация пользователя должна быть понятной для сообщества пользователей, выполняющих указанную рабочую задачу, например, посредством использования в ней соответствующим образом подобранных терминов, графических вставок, уточняющих пояснений и путем ссылок на полезные источники информации.

Простота обозрения (ease of overview). Документация пользователя должна быть достаточно проста для изучения пользовате-

лем, чтобы он мог выявить все описываемые в ней взаимосвязи компонентов продукта. В каждый документ могут быть включены оглавление и предметный указатель.

Программы и данные

Функциональные возможности

1. *Установка (инсталляция).*

Если установка пакета может быть выполнена пользователем, то при ее проведении должна быть обеспечена возможность успешной установки программ в соответствии с информацией, содержащейся в руководстве по установке. Каждая из необходимых систем, указанных в описании продукта, должна быть пригодной для установки программ. В процессе установки должно быть определено, могут ли установленные программы функционировать, например, путем использования поставленных с программами контрольных примеров или самотестирования с выдачей соответствующих сообщений.

2. *Реализация функций.*

Все функции, указанные в документации пользователя, должны выполняться в виде, заданном в документации пользователя, на соответствующих средствах, с соответствующими характеристиками и данными, в рамках граничных значений, заданных там же.

3. *Правильность.*

Программы и данные должны соответствовать всем обязательным формулировкам, приведенным в описании продукта и документации пользователя. Функции должны выполняться методом, соответствующим рабочей задаче. В частности, программы и данные должны удовлетворять всем требованиям из любого нормативного документа, на который дана ссылка в описании продукта.

4. *Непротиворечивость.*

Программы и данные не должны противоречить сами себе, а также описанию продукта и документации пользователя. Каждый термин везде должен иметь один и тот же смысл. Управление работой программы со стороны пользователя и соответствующая реакция программы (например, сообщения, выходные экранные форматы и печатные отчеты) должны быть единообразно структурированы.

Надежность

Система, включая технические средства, необходимые программные средства и те программы, которые входят в продукт, не должна приходить в такое состояние, чтобы пользователь не мог их контролировать, а данные не должны ни повреждаться, ни теряться.

Это требование должно одинаково удовлетворяться в случаях, когда:

- возможность реализуется при конкретных ограничениях;
- имеют место попытки реализации возможности вне заданных ограничений;
- неправильные исходные данные вводятся пользователем или от других программ, перечисленных в описании продукта;
- нарушаются инструкции, заданные в документации пользователя.

Исключаются только те возможности прерывания технических средств и операционной системы, которые не могут быть распознаны любой программой (например, клавиша или комбинация клавиш для сброса системы).

Программы должны обнаруживать нарушения синтаксических правил для исходных данных В случае, когда программа определяет исходные данные как ошибочные или неопределенные, она не должна их обрабатывать как допустимые исходные данные.

Практичность

1. *Понятность.*

Запросы, сообщения и результаты выполнения программ должны быть понятными, например:

- путем соответствующего выбора терминов;
- путем графических представлений;
- путем представления исходной информации;
- путем пояснений из функции подсказки.

В сообщениях об ошибках должна содержаться подробная информация, разъясняющая причину или способ исправления соответствующих ошибок из-за неправильного применения продукта (например, путем ссылки на элемент документации пользователя).

2. *Простота обозрения*

На каждый носитель данных должно быть нанесено обозначение продукта, а если носителей несколько — различающий номер или текст.

Пользователю, работающему с программами, всегда должна быть предоставлена возможность выяснения, какая из функций выполняется.

Программы должны предоставлять пользователю информацию в таком виде, чтобы данная информация им легко воспринималась и читалась. Пользователь может руководствоваться соответствующими кодификаторами и классификаторами информации. При необходимости программы должны выдавать пользователю соответствующие уведомления.

Сообщения от программ следует проектировать таким образом, чтобы пользователь мог легко различать их типы, например:

- подтверждение приема;
- запросы от программ;
- предупреждения;
- сообщения об ошибках.

Форматы входных экранов, отчеты, а также другие исходные и выходные данные следует проектировать так, чтобы их можно было легко просматривать. Для этого могут быть использованы следующие возможности:

- алфавитно-цифровые поля и левостороннее выравнивание;
- числовые поля и правостороннее выравнивание;
- расположение в таблицах десятичных точек или запятых на одной вертикальной линии;
- распознаваемые ограничители полей;
- соответствующее распознавание полей, использование которых обязательно;
- указание ошибок ввода непосредственной засветкой на вход ном экране;
- привлечение внимания пользователя к изменению содержания экрана путем подачи визуального или звукового сигнала.

3. *Простота использования.*

Исполнение функций, приводящих к серьезным последствиям при эксплуатации системы, должно быть обратимым или программы должны выдавать четкие предупреждения о последствиях выполнения данных функций и запрашивать разрешающее подтверждение перед выполнением соответствующей команды. В частности, к серьезным последствиям могут привести стирание или перезапись данных, а также прерывания режима продолжительной обработки.

Если текст документа предоставляется в диалоговом режиме, то пользователю следует обеспечить возможность непосредственного доступа к отдельным структурным элементам текста (разделам, пунктам, абзацам и т.д.), например, путем выбора данных элементов из отображенного на экране содержания документа или с помощью функции поиска по ключевым словам.

Эффективность, сопровождаемость, мобильность (переносимость)

В описании продукта должны присутствовать соответствующие формулировки эффективности, сопровождаемое™, мобильности.

Резюмируя, скажем, что возникла настоятельная потребность во введении в отечественные стандарты на документирование ПС тех норм, правил, требований и рекомендаций, которые установлены на международном и передовом зарубежном уровнях. Но при проведении этих работ нельзя ограничиваться прямым переводом отдельных международных стандартов. Нужно, чтобы новые стандарты правильно стыковались со всем имеющимся и будущим множеством нормативных документов.

ВОПРОСЫ РЯ САМОПРОВЕРКИ

1. Как можно охарактеризовать понятие «программная документа ция»?
2. Что представляет собой внешняя и внутренняя программная доку ментация?
3. Дайте определение понятию «единая система программной доку ментации».
4. В чем заключаются основные недостатки единой системы програм мной документации?
5. Дайте определение понятию «техническое задание».
6. Объясните смысл понятия «документация пользователя».
7. Какими свойствами должна обладать документация пользовате ля? Дайте краткую характеристику.

ГЛАВА 4 НАДЕЖНОСТЬ И КАЧЕСТВО ПРОГРАММНЫХ СРЕДСТВ

Опыт создания и применения сложных информационных систем (ИС) в последние десятилетия выявил множество ситуаций, при которых сбои и отказы их функционирования были обусловлены дефектами комплексов программ, что приводило к большому экономическому ущербу. Вследствие ошибок в программах автоматического управления погибло несколько отечественных, американских и французских спутников, происходили отказы и катастрофы в сложных административных, банковских и технологических информационных системах.

В результате около двадцати лет назад появились первые обобщающие работы, в которых были сформулированы концепция и основные положения теории надежности программных средств для информационных систем. В это время были заложены основы методологии и технологии создания высоконадежных сложных комплексов программ. Стало ясно, что для обеспечения высокой надежности функционирования и безопасности применения создаваемых сложных комплексов программ необходимы четкая организация и высокая квалификация всего коллектива специалистов, участвующих в таком проекте. В коллективе разработчиков целесообразно выделять специалистов, ответственных за соблюдение технологии создания и развития программ, за обеспечение и контроль качества, а также за надежность и безопасность проекта ПС и его компонентов.

Обеспечение надежности должно реализовываться специалистами в жизненном цикле программных средств на основе использования современной методологии, технологического инструментария, стандартов и нормативных документов.

Для обеспечения надежности программных средств необходимы разработка и применение эффективных методов и средств, предупреждающих и выявляющих дефекты, а также удостоверяющих надежность программ и оперативно защищающих функционирование ПС при их проявлениях. Для систематической,

координированной борьбы с угрозами надежности должны проводиться исследования конкретных факторов, влияющих на качество функционирования и безопасность применения программ со стороны реально существующих и потенциально возможных дефектов в создаваемых комплексах программ. В каждом проекте должен целенаправленно разрабатываться скоординированный комплекс методов и средств обеспечения заданной надежности функционирования ПС при реально достижимом снижении уровня дефектов и ошибок разработки. Учет факторов, влияющих на затраты ресурсов при создании конкретного ПС, должен позволять рационализировать их использование и добиваться заданной надежности функционирования ПС при минимальных или допустимых затратах.

Для обеспечения надежности программных средств в конкретных проектах должны быть организованы и стимулированы разработка, освоение и применение современных автоматизированных технологий и инструментальных средств, обеспечивающих *предупреждение или исключение* большинства видов дефектов и ошибок при создании и модификации ПС и их компонентов. Ограниченные ресурсы на разработку приводят к необходимости упорядоченного применения методов и рационального использования средств автоматизации проектирования. Поэтому процесс разработки должен планироваться и последовательно проходить этапы, охватывающие все компоненты ПС. *Контроль надежности и безопасности* создаваемых и модифицируемых программ должен сопровождать весь жизненный цикл ПС посредством специальной, достаточно эффективной технологической системы обеспечения их качества. Разработка и сопровождение сложных ПС на базе CASE-технологий позволяют предупреждать и устранять наиболее опасные системные и алгоритмические ошибки на ранних стадиях проектирования, а также использовать неоднократно проверенные в других проектах программные и информационные компоненты высокого качества. Предупреждение ошибок должно поддерживаться высококачественной документацией в процессе создания ПС в целом и их компонентов.

Одним из эффективных путей повышения надежности ПС является *стандартизация технологических процессов и объектов* проектирования, разработки и сопровождения программ. В стандартах жизненного цикла ПС обобщаются опыт и результаты исследований множества специалистов и рекомендуются наибо-

лее эффективные современные методы и процессы. В результате таких обобщений отрабатываются технологические процессы и приемы разработки, а также методическая база для их автоматизации. Стандарты ЖЦ ПС могут использоваться как непосредственно директивные, руководящие или как рекомендательные документы, а также как организационная база при создании средств автоматизации соответствующих технологических этапов или процессов. Подобная стандартизация процессов отражается не только на их технико-экономических показателях, но и, что особенно важно, на качестве создаваемых ПС. Надежность программ тесно связана с методами и технологией их разработки, поэтому важной группой стандартов в этой области являются стандарты по обеспечению качества ПС.

Поддержка этапов и работ ЖЦ ПС международными стандартами весьма неравномерная. Наиболее полно стандартизированы этапы ЖЦ ПС, прошедшие длительное историческое развитие и требующие наименее квалифицированных специалистов. При создании сложных проектов ПС и обеспечении их ЖЦ целесообразно применять выборку из всей совокупности существующих стандартов, а имеющиеся весьма обширные пробелы в стандартизации заполнять утвержденными технологическими документами, регламентирующими применение выбранных средств автоматизации разработки ПС. В результате на начальном этапе проектирования следует формировать весь комплект документов — *профиль,* обеспечивающий регламентирование всех этапов и работ при создании надежных ПС. Для реализации положений этих документов должны быть выбраны инструментальные средства, совместно образующие взаимосвязанный комплекс технологической поддержки и автоматизации ЖЦ и не противоречащие предварительно скомпонованному набору нормативных документов профиля. Применение профилей при проектировании ПС позволяет ориентироваться на построение систем из крупных функциональных узлов, отвечающих требованиям стандартов профиля, применять достаточно отработанные и проверенные проектные методы и решения.

Для обнаружения и устранения ошибок проектирования все этапы разработки и сопровождения ПС должны быть поддержаны методами и средствами систематического, автоматизированного *тестирования и испытаний.* При разработке ПС целесообразно применять различные методы, эталоны и виды тестирова-

ния, каждый из которых ориентирован на обнаружение, локализацию или диагностику определенных типов дефектов. Удостоверению достигнутого качества и надежности функционирования сложных критических ПС должна сопутствовать *обязательная сертификация* аттестованными проблемно-ориентированными сертификационными лабораториями.

В сложных комплексах программ при любой технологии разработки невозможно гарантировать абсолютное отсутствие дефектов и ошибок. Непредсказуемость вида, места и времени проявления дефектов ПС в процессе эксплуатации приводит к необходимости создания специальных, дополнительных *систем автоматической оперативной защиты* от непредумышленных, случайных искажений вычислительного процесса, программ и данных.

В отечественных ИС все больше применяются программные компоненты зарубежных фирм, которые также не могут быть абсолютно гарантированы от проявления дефектов проектирования, программирования и документации. Для обеспечения надежности функционирования комплексов программ с использованием импортных компонентов следует закупать только *лицензионно-чистые продукты,* поддерживаемые гарантированным сопровождением конкретных фирм-поставщиков. Эти компоненты должны сопровождаться полной эксплуатационной и технической документацией, сертификатом соответствия и комплектами тестов. В контрактах на закупку должны специально фиксироваться обязательства поставщиков по длительному сопровождению и замене версий ПС при выявлении дефектов или совершенствовании функций. Все версии зарубежных ПС следует проверять на надежность функционирования в конкретном окружении проекта ИС путем повторных испытаний или отдельными проверками, подтверждающими зарубежный сертификат.

Экспериментальное определение реальной надежности функционирования сложных комплексов программ — весьма трудоемкая, трудноавтоматизируемая и не всегда безопасная часть жизненного цикла ПС. Накоплен значительный опыт определения надежности ПС, применяемых в авиационной, ракетно-космической и других областях современной высокоинтеллектуальной техники. В этих областях недопустимо для тестирования и определения надежности ПС использовать функционирование реальных объектов. В результате особое значение приобрели методы и средства моделирования внешней среды для автоматизи-

рованной генерации тестов при испытаниях надежности таких ПС. В этих случаях на базе программных моделей и компонентов реальных систем должны создаваться моделирующие испытательные стенды, обеспечивающие возможность определения надежности функционирования конкретных ПС в условиях штатных и критических внешних воздействий, соответствующих подлинным характеристикам внешней среды.

Быстрый рост числа сфер использования, сложности и ответственности функций, выполняемых комплексами программ в информационных системах, резко повысил в последнее время требования к надежности их функционирования и безопасности применения. Для удовлетворения таких требований в жизненном цикле ПС необходимы выделение задач и работ по обеспечению надежности программ и концентрация усилий разработчиков на теоретическом и практическом их решении. Для каждого проекта ПС, выполняющего ответственные функции, должны разрабатываться и применяться специальные план и программа, методология и инструментальные средства, обеспечивающие требуемые надежность и безопасность. Только скоординированное, комплексное применение в проектах ПС современных методов и средств обеспечения надежности функционирования и безопасности применения комплексов программ путем автоматизации их разработки и испытаний позволяет достигать высокого качества ПС, необходимого для их применения в критических и сложных системах управления и обработки информации [49].

4.1. Основные понятия и показатели надежности программных средств

Основные понятия надежности систем. По определению, установленному в ГОСТ 13377-75, *надежность* — свойство объекта выполнять заданные функции, сохраняя во времени значения установленных эксплуатационных показателей в заданных пределах, соответствующих заданным режимам и условиям использования, технического обслуживания, ремонта, хранения и транспортирования. Таким образом, надежность является внутренним свойством системы, заложенным при ее создании и проявляющимся *во времени* при функционировании и эксплуатации.

Р. Гласе *надежность* определяет как уровень, при котором система программ удовлетворяет поставленным требованиям и пригодна для эксплуатации. При этом следует отличать надежность от *корректности,* которая определяется как степень удовлетворения требованиям. Надежность является составной частью более общего понятия — *качества.* Качественная программа не только надежна, но и компактна, совместима с другими программами, эффективна, удобна в сопровождении, портативна и вполне понятна [46].

Свойства надежности изделий изучаются *теорией надежности,* которая является системой определенных идей, математических моделей и методов, направленных на решение проблем предсказания, оценки и оптимизации различных показателей надежности. Надежность технических систем определяется в основном двумя факторами: *надежностью компонентов и дефектами в конструкции,* допущенными при проектировании или изготовлении. Относительно невысокая физическая надежность компонентов, их способность к разрушению, старению или снижению надежности в процессе эксплуатации привели к тому, что этот фактор оказался доминирующим для большинства комплексов аппаратуры. Этому способствовала также невысокая сложность многих технических систем, вследствие чего дефекты проектирования проявлялись относительно редко.

Надежность сложных программных средств определяется этими же факторами, однако доминирующими являются дефекты и ошибки проектирования, так как физическое хранение программ на магнитных носителях характеризуется очень высокой надежностью. Программа любой сложности и назначения при строго фиксированных исходных данных и абсолютно надежной аппаратуре исполняется по однозначно определенному маршруту и дает на выходе строго определенный результат. Однако случайное изменение исходных данных и накопленной при обработке информации, а также множество условных переходов в программе создают огромное число различных маршрутов исполнения каждого сложного ПС. Источниками ненадежности являются непроверенные сочетания исходных данных, при которых функционирующее ПС дает неверные результаты или отказы. В результате комплекс программ не соответствует требованиям функциональной пригодности и работоспособности.

Приведем пример, который опубликован в статье Юрия Батурина в журнале «Новое время» [57]. Автор приводит несколько факторов, которые описывают проблемы функционирования сложных программных средств. Так, в качестве одного из факторов выступает фактор сложности. «Существуют фундаментальные причины, почему программное обеспечение невозможно сделать достаточно надежным, чтобы можно было не сомневаться в том, что система «звездных войн» действительно сработает», — считает Д. Парнас, крупнейший авторитет по крупномасштабному программированию. Он был назначен Организацией по осуществлению Стратегической Оборонной Инициативы (СОИ) членом консультативного комитета по программированию управления боевыми операциями. «Мне обещали 1000 долларов в день плюс накладные расходы». Но, ознакомившись подробнее с тем, чего от него ждут, Д. Парнас отклонил сделанное ему предложение, одновременно представив восемь технических документов, которые объясняли, почему программа не сможет работать так, как требуется. В качестве примера приведем еще один из факторов — фактор надежности. О том, насколько уязвимо математическое обеспечение, можно судить по следующему примеру. Когда в 1979 г. американский космический зонд, запущенный на Венеру, не достиг своей цели, в космос вылетело почти полмиллиарда долларов. Причина в том, что в программе коррекции курса зонда запятая была спутана с двоеточием.

При применении понятий надежности к программным средствам следует учитывать *особенности и отличия этих объектов от традиционных технических систем,* для которых первоначально разрабатывалась теория надежности:

- не для всех видов программ применимы понятия и методы тео рии надежности — их можно использовать только к програм мным средствам, функционирующим в реальном времени и не посредственно взаимодействующим с внешней средой;
- при оценке качества программных компонентов к ним неприме нимы понятия надежности функционирования, если при обра ботке информации они не используют значения реального вре мени и не взаимодействуют непосредственно с внешней средой;
- доминирующими факторами, определяющими надежность про грамм, являются дефекты и ошибки проектирования и разра ботки, и второстепенное значение имеет физическое разруше ние программных компонентов при внешних воздействиях;

- относительно редкое разрушение программных компонентов и необходимость их физической замены приводят к принципиальному изменению понятий сбоя и отказа программ и к разделению их по длительности восстановления относительно некоторого допустимого времени простоя для функционирования информационной системы;
- для повышения надежности комплексов программ особое значение имеют методы автоматического сокращения длительности восстановления и преобразования отказов в кратковременные сбои путем введения в программные средства временной, программной и информационной избыточности;
- непредсказуемость места, времени и вероятности проявления дефектов и ошибок, а также их редкое обнаружение при реальной эксплуатации достаточно надежных программных средств, не позволяют эффективно использовать традиционные методы априорного расчета показателей надежности сложных систем, ориентированные на стабильные, измеряемые значения надежности составляющих компонентов;
- традиционные методы форсированных испытаний надежности систем путем физического воздействия на их компоненты не применимы для программных средств, и их следует заменять на методы форсированного воздействия информационных потоков внешней среды.

С учетом перечисленных особенностей применение основных понятий теории надежности сложных систем к жизненному циклу и оценке качества комплексов программ позволяет адаптировать и развивать эту теорию в особом направлении — *надежности программных средств.* Предметом изучения теории надежности комплексов программ (Software Reliability) является работоспособность сложных программ обработки информации в реальном времени. К задачам теории и анализа надежности сложных программных средств можно отнести следующие:

- формулирование основных понятий, используемых при исследовании и применении показателей надежности программных средств;
- выявление и исследование основных факторов, определяющих характеристики надежности сложных программных комплексов;
- выбор и обоснование критериев надежности для комплексов программ различного,типа и назначения;

- исследование дефектов и ошибок, динамики их изменения при отладке и сопровождении, а также влияния на показатели на дежности программных средств;
- исследование и разработка методов структурного построения сложных ПС, обеспечивающих их необходимую надежность;
- исследование методов и средств контроля и защиты от искаже ний программ, вычислительного процесса и данных путем ис пользования различных видов избыточности и помехозащиты;
- разработка методов и средств определения и прогнозирования характеристик надежности в жизненном цикле комплексов про грамм с учетом их функционального назначения, сложности, структурного построения и технологии разработки.

Результаты решения этих задач являются основой для создания современных сложных программных средств с заданными показателями надежности. Использование и объединение результатов экспериментальных и теоретических исследований надежности ПС позволили заложить основы теории и методов в этой области. В жизненном цикле ПС значения показателей качества и надежности компонентов и комплексов программ в целом рекомендуется непрерывно анализировать и прогнозировать с целью гарантированного обеспечения заданных показателей надежности. В реальных проектах работы по исследованию и обеспечению надежности программ целесообразно выделять в отдельную группу под единым руководством со специальным планом.

В основе теории надежности лежат понятия о двух возможных состояниях объекта или системы: работоспособном и неработоспособном. *Работоспособным* называется такое состояние объекта, при котором он способен выполнять заданные функции с параметрами, установленными технической документацией. В процессе функционирования возможен переход объекта из работоспособного состояния в неработоспособное и обратно. С этими переходами связаны события отказа и восстановления.

Определение степени работоспособности системы предполагает наличие в ней средств, способных установить соответствие ее характеристик требованиям технической документации. Для этого должны использоваться *методы и средства контроля и диагностики функционирования системы.* Глубина и полнота проверок, степень автоматизации контрольных операций, длительность и порядок их выполнения влияют на работоспособность системы и достоверность ее оценки. Методы и средства диагностичес-

кого контроля предназначены для установления степени работоспособности системы, локализации отказов, определения их характеристик и причин, скорейшего восстановления работоспособности, для накопления, обобщения и анализа данных, характеризующих работоспособность системы. Диагноз состояния системы принято делить на тестовый и функциональный. При тестовом диагнозе используются специально подготовленные исходные данные и эталонные результаты, которые позволяют проверять работоспособность определенных компонентов системы. Функциональный диагноз организуется на базе реальных исходных данных, поступающих в систему при ее использовании по прямому назначению. Основные задачи технической диагностики включают в себя:

- контроль исправности системы и полного соответствия ее со стояния и функций технической документации;
- проверку работоспособности системы и возможности выполнения всех функций в заданном режиме работы в любой момент времени с характеристиками, заданными технической документацией;
- поиск, выявление и локализацию источников и результатов сбо ев, отказов и неисправностей в системе.

Показатели качества и надежности программных средств. Формализации показателей качества программных средств посвящена группа нормативных документов. В международном стандарте ***ISO 9126:1991*** при отборе минимума стандартизируемых показателей выдвигались и учитывались следующие принципы: ясность и измеряемость значений, отсутствие перекрытия между используемыми показателями, соответствие установившимся понятиям и терминологии, возможность последующего уточнения и детализации. Выделены характеристики, которые позволяют оценивать ПС с позиции пользователя, разработчика и управляющего проектом. Рекомендуются б основных характеристик качества ПС, каждая из которых детализируется несколькими (всего 21) субхарактеристиками (рис. **4.1).**

Функциональная пригодность детализируется пригодностью для применения, точностью, защищенностью, способностью к взаимодействию и согласованностью со стандартами и правилами проектирования.

Надежность рекомендуется характеризовать уровнем завершенности (отсутствия ошибок), устойчивостью к ошибкам и перезапускаемостью.

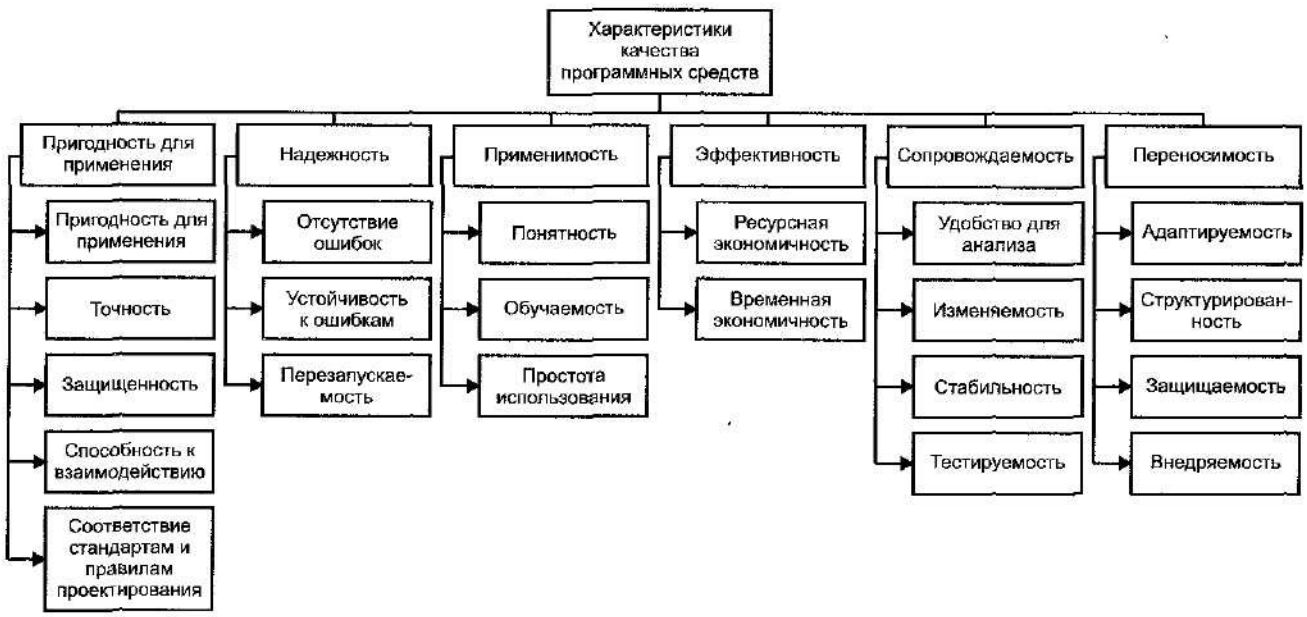


Рис. 4.1. Схема характеристик качества программных средств по стандарту ISO 9126:1991

Применимость предлагается описывать понятностью, обучаемостью и простотой использования.

Эффективность рекомендуется характеризовать ресурсной и временной экономичностью.

Сопровождаемость характеризуется удобством для анализа, изменяемостью, стабильностью и тестируемостью.

Переносимость предлагается отражать адаптируемостью, структурированностью, замещаемостью и внедряемостью.

Характеристики и субхарактеристики в стандарте определены очень кратко, без комментариев и рекомендаций по их применению к конкретным системам и проектам.

Близким к описанном}' стандарту по идеологии, структуре и содержанию является **ГОСТ 28195-89.** На верхнем, первом, уровне выделено 6 показателей — факторов качества: надежность, корректность, удобство применения, эффективность, универсальность и сопровождаемость. Эти факторы детализируются в совокупности 19 критериями качества на втором уровне. Дальнейшая детализация показателей качества представлена метриками и оценочными элементами, которых насчитывается около 240. Каждый из них рекомендуется экспертно оценивать в пределах от 0 до 1. Состав используемых факторов, критериев и метрик предлагается выбирать в зависимости от назначения, функций и этапов жизненного цикла ПС.

В стандарте **ГОСТ 28806-90** формализуются общие понятия программы, программного средства, программного продукта и их качества. Даются определения 18 наиболее употребляемых терминов, связанных с оценкой характеристик программ. Уточнены понятия базовых показателей качества, приведенных в ГОСТ 28195-89.

Функциональная пригодность — это набор атрибутов, определяющий назначение, номенклатуру, основные необходимые и достаточные функции ПС. заданные техническим заданием заказчика или потенциального пользователя. В процессе проектирования ПС атрибуты функциональной пригодности конкретизируются в спецификации на компоненты. Эти атрибуты можно численно представить точностью вычислений, относительным числом поэтапно изменяемых функций, числом спецификаций требований заказчиков и т.д. Кроме них функциональную пригодность отражает множество различных специализированных критериев, которые тесно связаны с конкретными функциями

программ. Их можно рассматривать как частные критерии или как факторы, влияющие на основные показатели. В наиболее общем виде функциональная пригодность проявляется в *корректности и надежности* ПС.

Понятие *корректной (правильной) программы* может рассматриваться статически вне ее исполнения во времени. Корректность программы не определена вне области изменения исходных данных, заданных требованиями спецификации, и не зависит от динамики функционирования программы в реальном времени. Степень некорректности программ определяется вероятностью попадания реальных исходных данных в область, которая задана требованиями спецификации и технического задания (ТЗ). однако не была проверена при тестировании и испытаниях. Значения этого показателя зависят от функциональной корректности применяемых компонентов и могут рассматриваться в зависимости от методов их достижения и оценивания: детерминированно, стохастически и в реальном времени. При анализе видов корректности и способов их измерения, естественно, они связываются с видами и методами процесса тестирования и испытания программ

Надежная программа прежде всего должна обеспечивать достаточно низкую вероятность отказа в процессе функционирования в реальном времени. Быстрое реагирование на искажения программ, данных или вычислительного процесса и восстановление работоспособности за время, меньшее, чем порог между сбоем и отказом, обеспечивают высокую надежность программ. При этом некорректная программа может функционировать абсолютно надежно. В реальных условиях по различным причинам исходные данные могут попадать в области значений, вызывающих сбои, не проверенные при испытаниях, а также не заданные требованиями спецификации и технического задания. Если в этих ситуациях происходит достаточно быстрое восстановление, такое, что не фиксируется отказ, то такие события не влияют на *основные* показатели надежности — наработку на отказ и коэффициент готовности. Следовательно, надежность функционирования программ является понятием динамическим, проявляющимся во времени, и существенно отличается от понятия корректности программ.

При оценке качества программ по показателям надежности регистрируются только такие искажения в процессе динамического тестирования с исполнением программ, которые приводят

к потере работоспособности ПС или их крупных компонентов. Первопричиной нарушения работоспособности программ при безотказности аппаратуры всегда является конфликт между реальными исходными данными, подлежащими обработке, и программой, осуществляющей эту обработку. Работоспособность ПС можно гарантировать при конкретных исходных данных, которые использовались при отладке и испытаниях. Реальные исходные данные могут иметь значения, отличающиеся от заданных техническим заданием и от использованных при применении программ. При таких исходных данных функционирование программ трудно предсказать заранее и весьма вероятны различные аномалии, завершающиеся отказами.

Непредсказуемость вида, места и времени проявления дефектов ПС в процессе эксплуатации приводит к необходимости создания специальных, дополнительных систем оперативной защиты от непредумышленных, случайных искажений вычислительного процесса, программ и данных. Системы оперативной защиты предназначены для выявления и блокирования распространения негативных последствий проявления дефектов и уменьшения их влияния на надежность функционирования ПС до устранения их первичных источников. Для этого в ПС должна вводиться временная, программная и информационная избыточность, осуществляющая оперативное обнаружение дефектов функционирования, их идентификацию и автоматическое восстановление (рестарт) нормального функционирования ПС. Надежность ПС должна повышаться за счет средств обеспечения помехоустойчивости, оперативного контроля и восстановления функционирования программ и баз данных. Эффективность такой защиты зависит от используемых методов, координированности их применения и выделяемых вычислительных ресурсов на их реализацию.

Основным *принципом классификации сбоев и отказов* в программах при отсутствии их физического разрушения является разделение по временному показателю длительности восстановления после любого искажения программ, данных или вычислительного процесса, регистрируемого как нарушение работоспособности. При длительности восстановления, меньшей заданного порога, дефекты и аномалии при функционировании программ следует относить к *сбоям,* а при восстановлении, превышающем по длительности пороговое значение, происходящее искажение соответствует *отказу.* Классификация программных сбоев и отказов по длитель-

ности восстановления приводит к необходимости анализа динамических характеристик абонентов, являющихся потребителями данных, обработанных исследуемым ПС, а также временных характеристик функционирования программ. Временная зона перерыва нормальной выдачи информации и потери работоспособности, которую следует рассматривать как зону сбоя, тем шире, чем более инертный объект находится под воздействием сообщений, подготовленных данным ПС. Пороговое время восстановления работоспособного состояния системы, при превышении которого следует фиксировать отказ, близко к периоду решения задач для подготовки информации соответствующему абоненту.

При нормальном темпе решения задач и выдаче их результатов потребителю отклонения его характеристик от траектории, рассчитываемой ПС, находятся в допустимых пределах. Для любого потребителя информации существует допустимое время отсутствия данных от ПС, при котором его характеристики, изменяясь по инерции, достигают предельного отклонения от значения, которое должно быть рассчитано программами. Соответствующая этому отклонению временная зона перерыва выдачи информации потребителю позволяет установить границу допустимой длительности нарушения работоспособности, которая разделяет зоны сбоев и отказов.

Чем более инерционным является потребитель информации, тем больше может быть время отсутствия у него результатов функционирования и воздействий от ПС без катастрофических последствий нарушения работоспособности, соответствующего отказ}'. Это допустимое отклонение результатов после перерыва функционирования ПС зависит в основном от динамических характеристик источников и потребителей информации. Таким образом, установив в результате системного анализа динамических характеристик объектов информационной системы величину порогового значения, можно определить интервал времени функционирования ПС при отсутствии выдачи потребителю данных, которые разделяют события сбоя и отказа без физического разрушения программ.

Надежность функционирования ПС наиболее широко характеризуется *устойчивостью,* или способностью к безотказному функционированию, и *восстанавливаемостью* работоспособного состояния после произошедших сбоев или отказов. В свою очередь, устойчивость зависит от уровня неустраненных дефектов и ошибок и способности ПС реагировать на их проявления так, чтобы

это не отражалось на показателях надежности. Последнее определяется эффективностью контроля данных, поступающих из внешней среды, и средств обнаружения аномалий функционирования ПС.

Восстанавливаемость характеризуется полнотой и длительностью восстановления функционирования программ в процессе перезапуска — рестарта. Перезапуск должен обеспечивать возобновление нормального функционирования ПС, на что требуются ресурсы ЭВМ и время. Поэтому полнота и длительность восстановления функционирования после сбоев отражают качество, надежность ПС и возможность его использования по прямому назначению.

Показатели надежности ПС в значительной степени адекватны аналогичным характеристикам, принятым для других технических систем. Наиболее широко используется *критерий длительности наработки на отказ.* Для определения этой величины измеряется время работоспособного состояния системы между двумя последовательными отказами или началом нормального функционирования системы после них. Вероятностные характеристики этой величины в нескольких формах используются как разновидности критериев надежности. Критерий надежности восстанавливаемых систем учитывает возможность многократных отказов и восстановлений. Для оценки надежности таких систем, которыми чаще всего являются сложные ПС, кроме вероятностных характеристик наработки на отказ, важную роль играют характеристики функционирования после отказа в процессе восстановления. Основным показателем процесса восстановления являются *длительность восстановления* и ее вероятностные характеристики. Этот критерий учитывает возможность многократных отказов и восстановлений. Обобщение характеристик отказов и восстановлений производится в критерии *коэффициент готовности.* Этот показатель отражает вероятность иметь восстанавливаемую систему в работоспособном состоянии в произвольный момент времени. Значение коэффициента готовности соответствует доле времени полезной работы системы на достаточно большом интервале, содержащем отказы и восстановления.

Наработка на отказ учитывает ситуации потери работоспособности, когда длительность восстановления достаточно велика и превышает пороговое значение времени, разделяющее события сбоя и отказа. При этом большое значение имеют средства оперативного контроля и восстановления. Качество проведенной отладки программ более полно отражает значение длитель-

ности между потерями работоспособности программ — ***наработка на отказовую ситуацию, или устойчивость,*** независимо от того,
насколько быстро произошло восстановление. Средства оперативного контроля и восстановления не влияют на наработку на отказовую ситуацию, однако могут значительно улучшать показатели надежности программ. Поэтому при оценке необходимой отладки целесообразно измерять и контролировать наработку на отказовую ситуацию, а объем и длительность тестирования в ряде случаев устанавливать по наработке на отказ с учетом эффективности средств рестарта.

В реальных системах достигаемая при отладке и испытаниях наработка на отказ ПС обычно должна быть соизмерима с наработкой на отказ аппаратуры, на которой исполняется программа. Для систем обработки информации и управления в реальном времени наработка на отказ программ измеряется десятками и сотнями часов, а для особо важных или широко тиражируемых систем может достигать десятков тысяч часов. При достаточно развитом программном оперативном контроле и восстановлении наработка на отказовую ситуацию может быть на один—два порядка меньше, чем наработка на отказ. Реально очень трудно достичь наработки на отказовую ситуацию на уровне сотен часов, и поэтому для получения высокой надежности программ невозможно ограничиваться тестированием и отладкой без использования средств рестарта. Невозможно обеспечить абсолютное отсутствие дефектов проектирования в сложных ПС, вследствие чего надежность их функционирования имеет всегда конечное, ограниченное значение.

4.2.
Дестабилизирующие факторы и методы обеспечения надежности функционирования программных средств

Модель факторов, определяющих надежность программных средств. При любом виде деятельности людям свойственно непредумышленно ошибаться, результаты чего проявляются в процессе создания или применения изделий или систем. В общем случае под ошибкой подразумевается дефект, погрешность или не-

умышленное искажение объекта или процесса. При этом предполагается, что известно правильное, эталонное состояние объекта, по отношению к которому может быть определено наличие отклонения — *дефекта или ошибки** Для систематической, координированной борьбы с ними необходимы исследования факторов, влияющих на надежность ПС со стороны случайных, существующих и потенциально возможных дефектов в конкретных программах. Это позволит целенаправленно разрабатывать комплексы методов и средств обеспечения надежности сложных ПС различного назначения при реально достижимом снижении уровня дефектов проектирования.

При строго фиксированных исходных данных программы исполняются по определенным маршрутам и выдают совершенно определенные результаты. Многочисленные варианты исполнения программ при разнообразных исходных данных представляются для внешнего наблюдателя как случайные. В связи с этим дефекты функционирования программных средств, не имеющие злоумышленных источников, проявляются внешне как случайные, имеют разную природу и последствия. В частности, они могут приводить к последствиям, соответствующим нарушениям работоспособности, и к отказам при использовании ПС [49].

Последующий анализ надежности ПС базируется на модели взаимодействия основных компонентов, представленных на рис. 4.2. *Объектами уязвимости,* влияющими на надежность ПС. являются:

- динамический вычислительный процесс обработки данных, ав томатизированной подготовки решений и выработки управ ляющих воздействий;
- информация, накопленная в базах данных, отражающая объек ты внешней среды, и процессы ее обработки;
- объектный код программ, исполняемых вычислительными сред ствами в процессе функционирования ПС;
- информация, выдаваемая потребителям и на исполнительные механизмы, являющаяся результатом обработки исходных дан ных и информации, накопленной в базе данных.

На эти объекты воздействуют различные *дестабилизирующие факторы,* которые можно разделить на внутренние, присущие самим объектам уязвимости, и внешние, обусловленные средой, в которой эти объекты функционируют. *Внутренними источники*

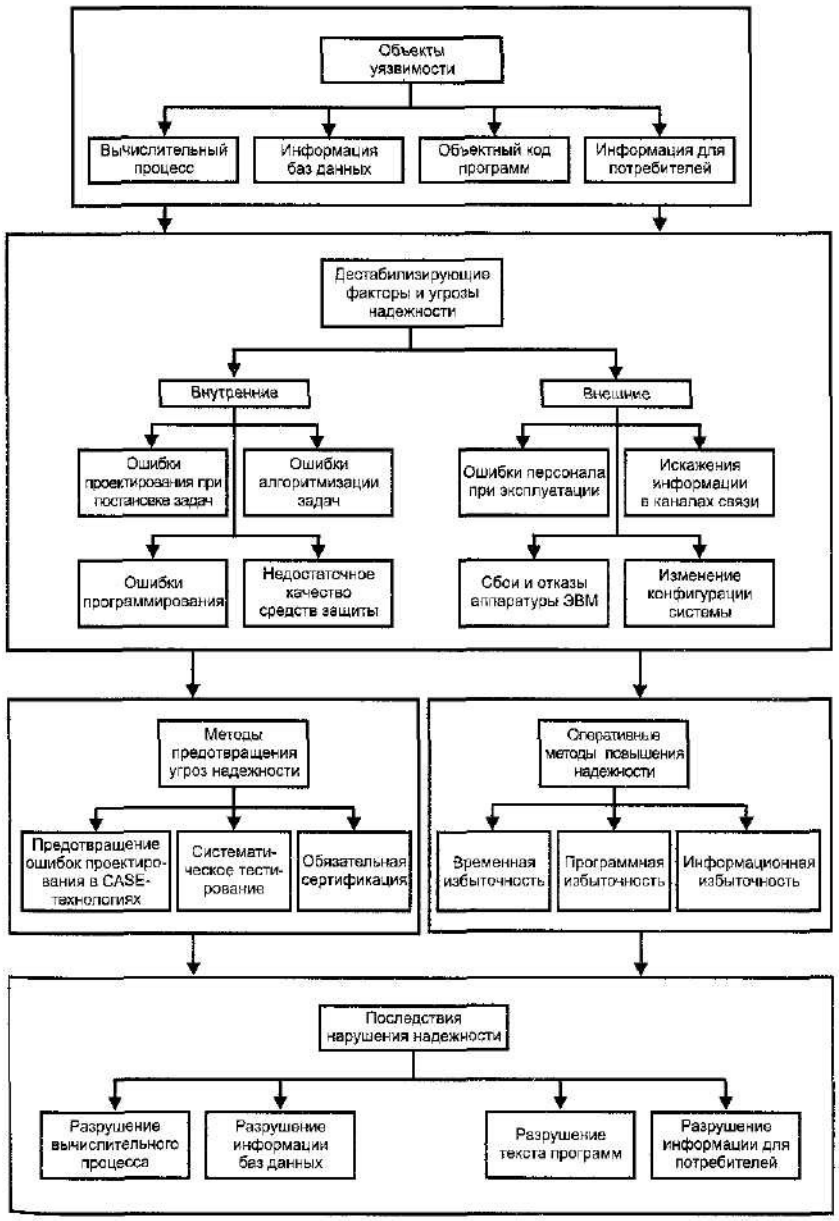


Рис. 4.2. Схема модели анализа надежности программных средств

коми угроз надежности функционирования сложных ПС можно считать следующие дефекты программ:

- системные ошибки при постановке целей и задач создания ПС. при формулировке требований к функциям и характеристикам решения задач, определении условий и параметров внешней среды, в которой предстоит применять ПС;
- алгоритмические ошибки разработки при непосредственной спецификации функций программных средств, при определе нии структуры и взаимодействия компонентов комплексов про грамм, а также при использовании информации баз данных;
- ошибки программирования в текстах программ и описаниях данных, а также в исходной и результирующей документации на компоненты и ПС в целом;
- недостаточную эффективное гь используемых методов и средств оперативной защиты программ и данных о'т сбоев и отказов и обеспечения надежности функционирования ПС в условиях слу чайных негативных воздействий.

Внешними дестабилизирующими факторами, отражающимися на надежности функционирования перечисленных объектов уязвимости в ПС, являются:

- ошибки оперативного и обслуживающего персонала в процес се эксплуатации ПС;
- искажения в каналах телекоммуникации информации, посту пающей от внешних источников и передаваемой потребителям, а также недопустимые для конкретной информационной систе мы характеристики потоков внешней информации;
- сбои и отказы в аппаратуре вычислительных средств;
- изменения состава и конфигурации комплекса взаимодейству ющей аппаратуры информационной системы за пределы, про веренные при испытаниях или сертификации и отраженные в эксплуатационной документации.

Полное устранение перечисленных негативных воздействий и дефектов, отражающихся на надежности функционирования сложных ПС, принципиально невозможно. Проблема состоит в выявлении факторов, от которых они зависят, создании методов и средств уменьшения их влияния на надежность ПС, а также в эффективном распределении ресурсов на защиту для обеспечения необходимой надежности комплекса программ, равноправного при их реальных воздействиях.

Современные достижения микроэлектроники значительно снизили влияние сбоев и отказов вычислительных средств на надежность функционирования ПС. Однако ошибки персонала, искажения данных в каналах телекоммуникации, а также случайные (при отказах части аппаратуры) и необходимые изменения конфигурации вычислительных средств остаются существенными внешними угрозами надежности ПС. Негативное влияние этих факторов может быть значительно снижено соответствующими методами и средствами защиты и восстановления программ и данных. Внешние дестабилизирующие факторы имеют различную природу и широкий спектр характеристик, которые представлены во многих публикациях Поэтому ниже внимание акцентируется на внутренних дестабилизирующих факторах, различного рода дефектах и ошибках проектирования и эксплуатации, которые оказывают наибольшее влияние на надежность функционирования ПС.

Различия между ожидаемыми и полученными результатами функционирования программ могут быть следствием ошибок не только в созданных программах и данных, но и системных ошибок в первичных требованиях спецификаций, явившихся исходной базой при создании ПС. Тем самым проявляется объективная реальность, заключающаяся в невозможности абсолютной корректности и полноты исходных данных для проектирования спецификаций сложных ПС. На практике в процессе разработки ПС исходные требования уточняются и детализируются по согласованию между заказчиком и разработчиком. Базой таких уточнений являются неформализованные представления и знания специалистов, а также результаты промежуточных этапов проектирования. Однако установить *ошибочность исходных данных и спецификаций* еще труднее, чем обнаружить ошибки в созданных программах и данных, так как принципиально отсутствуют формализованные данные, которые можно использовать как эталонные, и их заменяют неформализованные представления заказчиков и разработчиков.

Степень влияния всех внутренних дестабилизирующих факторов, а также некоторых внешних угроз на надежность ПС определяется в наибольшей степени *качеством технологий проектирования, разработки, сопровождения и документирования ПС* и их основных компонентов. При ограниченных ресурсах на разработку ПС для достижения заданных требований по надежнос-

ти необходимо управление обеспечением качества в течение все го цикла создания программ и данных. Такое управление пред полагает высокую дисциплину и проектировочную культуру все го коллектива специалистов, использование им методик, типо вых нормативных документов и средств автоматизацш разработки. Кроме того, обеспечение качества ПС предполагает формализацию и сертификацию технологии их разработки, а так-же выделение в специальный процесс поэтапного измерения и анализа текущего качества создаваемых и применяемых компонентов. Попытки создания сложных, распределенных ПС на базе мультипроцессорных ЭВМ и концепции клиент-сервер без использования эффективных технологий и средств автоматизации проектирования связаны с высоким риском полного провала проектов вследствие трудностей обеспечения необходимой надежности функционирования таких систем.

Методы обеспечения надежности программных средств. В современных автоматизированных технологиях создания и развития сложных ПС с позиции обеспечения их необходимой и заданной надежности можно выделить методы и средства, позволяющие:

- *создавать программные модули и функциональные компоненты* высокого, гарантированного качества;
- *предотвращать дефекты проектирования* за счет эффективных технологий и средств автоматизации обеспечения всего жиз ненного цикла комплексов программ и баз данных;
- *обнаруживать и устранять различные дефекты и ошибки* про ектирования, разработки и сопровождения программ путем систематического тестирования на всех этапах жизненного цик ла ПС;
- *удостоверять достигнутое качество и надежность функциони рования* ПС в процессе их испытаний и сертификации перед передачей в регулярную эксплуатацию;
- *оперативно выявлять последствия дефектов программ и данных* и восстанавливать нормальное, надежное функционирование комплексов программ.

Комплексное, скоординированное применение этих методов и средств в процессе создания, развития и применения ПС позволяет исключать некоторые виды угроз или значительно ослаблять их влияние. Тем самым уровень достигаемой надежности ПС становится предсказуемым и управляемым, непосредственно за-

висящим от ресурсов, выделяемых на его достижение, а главное от качества и эффективности технологии, используемой на всех этапах жизненного цикла ПС.

Все принципы и методы обеспечения надежности в соответствии с их целью можно разбить на четыре группы: *предупреждение ошибок, обнаружение ошибок, исправление ошибок* и *обеспечение устойчивости к ошибкам.* К первой группе относятся принципы и методы, позволяющие минимизировать или вообще исключить ошибки. Методы второй группы сосредоточивают внимание на функциях самого программного обеспечения, помогающих выявлять ошибки. К третьей группе относятся функции программного обеспечения, предназначенные для исправления ошибок или их последствий. Устойчивость к ошибкам (четвертая группа) — это мера способности системы программного обеспечения продолжать функционирование при наличии ошибок.

Предупреждение ошибок

К этой группе относятся принципы и методы, цель которых — не допустить появления ошибок в готовой программе. Большинство методов концентрируется на отдельных процессах перевода и направлено на предупреждение ошибок в этих процессах. Их можно разбить на следующие категории:

1) методы, позволяющие справиться со сложностью, свести ее к минимуму, так как это — главная причина ошибок перевода;

2) методы достижения большей точности при переводе;

3) методы улучшения обмена информацией,

4) методы немедленного обнаружения и устранения ошибок. Эти методы направлены на обнаружение ошибок на каждом шаге перевода, не откладывая до тестирования программы после ее написания.

Должно быть очевидно, что предупреждение ошибок — оптимальный путь к достижению надежности программного обеспечения.

Лучший способ обеспечить надежность — прежде всего не допустить возникновения ошибок. Гарантировать отсутствие ошибок, однако, невозможно никогда. Другие три группы методов опираются на предположение, что ошибки все-таки будут.

Обнаружение ошибок

Если предполагать, что в программном обеспечении какие-то ошибки все же будут, то лучшая (после предупреждения ошибок) стратегия — включить средства обнаружения ошибок в само программное обеспечение.

Большинство методов направлено по возможности на незамедлительное обнаружение сбоев. Немедленное обнаружение имеет два преимущества: можно минимизировать влияние ошибки и последующие затруднения для человека, которому придется извлекать информацию о ней, находить ее и исправлять.

Меры по обнаружению ошибок можно разбить на две подгруппы: *пассивные* попытки обнаружить симптомы ошибки в процессе «обычной» работы программного обеспечения и *активные* попытки программной системы периодически обследовать свое состояние в поисках признаков ошибок.

Пассивное обнаружение. Меры по обнаружению ошибок могут быть приняты на нескольких структурных уровнях программной системы. Здесь мы будем рассматривать уровень подсистем, или компонентов, т.е. нас будут интересовать меры по обнаружению симптомов ошибок, предпринимаемые при переходе от одного компонента к другому, а также внутри компонента. Все это, конечно, применимо также к отдельным модулям внутри компонента.

Разрабатывая эти меры, мы будем опираться на следующее.

1. *Взаимное недоверие.* Каждый из компонентов должен пред полагать, что все другие содержат ошибки. Когда он получает какие-нибудь данные от другого компонента или из источника вне системы, он должен предполагать, что данные могут быть неправильными, и пытаться найти в них ошибки.

2. *Немедленное обнаружение.* Ошибки необходимо обнаружить как можно раньше. Это не только ограничивает наносимый ими ущерб, но и значительно упрощает задачу отладки.

3. *Избыточность.* Все средства обнаружения ошибок основа ны на некоторой форме избыточности (явной или неявной).

Когда разрабатываются меры по обнаружению ошибок, важно принять согласованную стратегию для всей системы. Действия, предпринимаемые после обнаружения ошибки в программном обеспечении, должны быть единообразными для всех компонентов системы. Это ставит вопрос о том, какие именно действия следует предпринять, когда ошибка обнаружена. Наилучшее ре-

щение — немедленно завершить выполнение программы или (в случае операционной системы) перевести центральный процессор в состояние ожидания. С точки зрения предоставления человеку, отлаживающему программу, например системному программисту, самых благоприятных условий для диагностики ошибок немедленное завершение представляется наилучшей стратегией. Конечно, во многих системах подобная стратегия бывает нецелесообразной (например, может оказаться, что приостанавливать работу системы нельзя). В таком случае используется метод *регистрации ошибок.* Описание симптомов ошибки и «моментальный снимок» состояния системы сохраняются во внешнем файле, после чего система может продолжать работу. Этот файл позднее будет изучен обслуживающим персоналом.

Всегда, когда это возможно, лучше приостановить выполнение программы, чем регистрировать ошибки (либо обеспечить как дополнительную возможность работу системы в любом из этих режимов). Различие между этими методами проиллюстрируем на способах выявления причин возникающего иногда скрежета вашего автомобиля. Если автомеханик находится на заднем сиденье, то он может обследовать состояние машины в тот момент, когда скрежет возникает. Если вы выбираете метод регистрации ошибок, задача диагностики станет сложнее.

Активное обнаружение ошибок. Не все ошибки можно выявить пассивными методами, поскольку эти методы обнаруживают ошибку лишь тогда, когда ее симптомы подвергаются соответствующей проверке. Можно делать и дополнительные проверки, если спроектировать специальные программные средства для активного поиска признаков ошибок в системе. Такие средства называются *средствами активного обнаружения ошибок.*

Активные средства обнаружения ошибок обычно объединяются в *диагностический монитор:* параллельный процесс, который периодически анализирует состояние системы с целью обнаружить ошибку. Большие программные системы, управляющие ресурсами, часто содержат ошибки, приводящие к потере ресурсов на длительное время. Например, управление памятью операционной системы сдает блоки памяти «в аренду» программам пользователей и другим частям операционной системы. Ошибка в этих самых «других частях» системы может иногда вести к неправильной работе блока управления памятью, занимающегося возвратом сданной ранее в аренду памяти, что вызывает медленное вырождение системы.

Диагностический монитор можно реализовать как периодически выполняемую задачу (например, она планируется на каждый час) либо как задачу с низким приоритетом, которая планируется для выполнения в то время, когда система переходит в состояние ожидания. Как и прежде, выполняемые монитором конкретные проверки зависят от специфики системы, но некоторые идеи будут понятны из примеров. Монитор может обследовать основную память, чтобы обнаружить блоки памяти, не выделенные ни одной из выполняемых задач и не включенные в системный список свободной памяти. Он может проверять также необычные ситуации: например, процесс не планировался для выполнения в течение некоторого разумного интервала времени. Монитор может осуществлять поиск «затерявшихся» внутри системы сообщений или операций ввода-вывода, которые необычно долгое время остаются незавершенными, участков памяти на диске, которые не помечены как выделенные и не включены в список свободной памяти, а также различного рода странностей в файлах данных.

Иногда желательно, чтобы в чрезвычайных обстоятельствах монитор выполнял диагностические тесты системы. Он может вызывать определенные системные функции, сравнивая их результат с заранее определенным и проверяя, насколько разумно время выполнения. Монитор может также периодически предъявлять системе «пустые» или «легкие» задания, чтобы убедиться, что система функционирует хотя бы самым примитивным образом.

Исправление ошибок

Следующий шаг — методы исправления ошибок; после того как ошибка обнаружена, либо она сама, либо ее последствия должны быть исправлены программным обеспечением. Исправление ошибок самой системой — плодотворный метод проектирования надежных систем аппаратного обеспечения. Некоторые устройства способны обнаружить неисправные компоненты и перейти к использованию идентичных запасных. Аналогичные методы неприменимы к программному обеспечению вследствие глубоких внутренних различий между сбоями аппаратуры и ошибками в программах. Если некоторый программный модуль содержит ошибку, идентичные «запасные» модули также будут содержать ту же ошибку.

Другой подход к исправлению связан с попытками восстановить разрушения, вызванные ошибками, например искажения записей в базе данных или управляющих таблицах системы. Польза от методов борьбы с искажениями ограничена, поскольку предполагается, что разработчик заранее предугадает несколько возможных типов искажений и предусмотрит программно реализуемые функции для их устранения. Это похоже на парадокс, поскольку, если знать заранее, какие ошибки возникнут, можно было бы принять дополнительные меры по их предупреждению. Если методы ликвидации последствий сбоев не могут быть обобщены для работы со многими типами искажений, лучше всего направлять силы и средства на предупреждение ошибок. Вместо того, чтобы, разрабатывая операционную систему, оснащать ее средствами обнаружения и восстановления цепочки искаженных таблиц или управляющих блоков, следовало бы лучше спроектировать систему так, чтобы только один модуль имел доступ к этой цепочке, а затем настойчиво пытаться убедиться в правильности этого модуля.

Устойчивость к ошибкам

Методы этой группы ставят своей целью обеспечить функционирование программной системы при наличии в ней ошибок. Они разбиваются на три подгруппы: динамическая избыточность, методы отступления и методы изоляции ошибок.

1. Истоки концепции *динамической избыточности* лежат в проектировании аппаратного обеспечения. Один из подходов к динамической избыточности — *метод голосования.* Данные обрабатываются независимо несколькими идентичными устройствами, и результаты сравниваются. Если большинство устройств выработало одинаковый результат, этот результат и считается правильным. И опять, вследствие особой природы ошибок в программном обеспечении ошибка, имеющаяся в копии программного модуля, будет также присутствовать во всех других его копиях, поэтому идея голосования здесь, видимо, неприемлема. Предлагаемый иногда подход к решению этой проблемы состоит в том, чтобы иметь несколько неидентичных копий модуля. Это значит, что все копии выполняют одну и ту же функцию, но либо реализуют различные алгоритмы, либо созданы разными разработчиками. Этот подход бесперспективен по следующим причи-

нам. Часто трудно получить существенно разные версии модуля, выполняющие одинаковые функции. Кроме того, возникает необходимость в дополнительном программном обеспечении для организации выполнения этих версий параллельно или последовательно и сравнения результатов. Это дополнительное программное обеспечение повышает уровень сложности системы, что, конечно, противоречит основной идее предупреждения ошибок — стремиться в первую очередь минимизировать сложность.

Второй подход к динамической избыточности — выполнять эти запасные копии только тогда, когда результаты, полученные с помощью основной копии, признаны неправильными. Если это происходит, система автоматически вызывает запасную копию. Если и ее результаты неправильны, вызывается другая запасная копия и т. д.

2. Вторая подгруппа методов обеспечения устойчивости к ошибкам называется *методами отступления* или сокращенного обслуживания. Эти методы приемлемы обычно лишь тогда, когда для системы программного обеспечения существенно важно корректно закончить работу. Например, если ошибка оказывается в системе, управляющей технологическими процессами, и в результате эта система выходит из строя, то может быть загружен и выполнен особый фрагмент программы, призванный подстраховать систему и обеспечить безаварийное завершение всех управляемых системой процессов. Аналогичные средства часто необходимы в операционных системах. Если операционная система обнаруживает, что вот-вот выйдет из строя, она может загрузить аварийный фрагмент, ответственный за оповещение пользователей у терминалов о предстоящем сбое и за сохранение всех критических для системы данных.

3. Последняя подгруппа — *методы изоляции ошибок.* Основная их идея — не дать последствиям ошибки выйти за пределы как можно меньшей части системы программного обеспечения, так чтобы, если ошибка возникнет, то не вся система оказалась неработоспособной; отключаются лишь отдельные функции в системе либо некоторые ее пользователи. Например, во многих операционных системах изолируются ошибки отдельных пользователей, так что сбой влияет лишь на некоторое подмножество пользователей, а система в целом продолжает функционировать. В телефонных переключательных системах для восстановления после ошибки, чтобы не рисковать выходом из строя всей систе-

мы, просто разрывают телефонную связь. Другие методы изоляции ошибок связаны с защитой каждой из программ в системе от ошибок других программ. Ошибка в прикладной программе, выполняемой под управлением операционной системы, должна оказывать влияние только на эту программу. Она не должна сказываться на операционной системе или других программах, функционирующих в этой системе.

В большой вычислительной системе изоляция программ является ключевым фактором, гарантирующим, что отказы в программе одного пользователя не приведут к отказам в программах других пользователей или к полному выводу системы из строя. Основные правила изоляции ошибок перечислены ниже. Хотя в формулировке многих из них употребляются слова «операционная система», они применимы к любой программе (будь то операционная система, монитор телеобработки или подсистема управления файлами), которая занята обслуживанием других программ.

1. Прикладная программа не должна иметь возможности не посредственно ссылаться на другую прикладную программу или данные в другой программе и изменять их.

2. Прикладная программа не должна иметь возможности не посредственно ссылаться на программы или данные операцион ной системы и изменять их. Связь между двумя программами (или программой и операционной системой) может быть разрешена только при условии использования четко определенных сопря жений и только в случае, когда обе программы дают согласие на эту связь.

3. Прикладные программы и их данные должны быть защи щены от операционной системы до такой степени, чтобы ошиб ки в операционной системе не могли привести к случайному из менению прикладных программ или их данных.

4. Операционная система должна защищать все прикладные программы и данные от случайного их изменения операторами системы или обслуживающим персоналом.

5. Прикладные программы не должны иметь возможности ни остановить систему, ни вынудить ее изменить другую приклад ную программу или ее данные.

6. Когда прикладная программа обращается к операционной системе, должна проверяться допустимость всех параметров. Прикладная программа не должна иметь возможности изменить

эти параметры между моментами проверки и реального их использования операционной системой.

7. Никакие системные данные, непосредственно доступные прикладным программам, не должны влиять на функционирование операционной системы. Ошибка в прикладной программе, вследствие которой содержимое этой памяти может быть случайно изменено, приводит в конце концов к сбою системы.

8. Прикладные программы не должны иметь возможности в обход операционной системы прямо использовать управляемые ею аппаратные ресурсы. Прикладные программы не должны прямо вызывать компоненты операционной системы, предназначенные для использования только ее подсистемами.

9. Компоненты операционной системы должны быть изолированы друг от друга так, чтобы ошибка в одной из них не привела к изменению других компонентов или их данных.

10. Если операционная система обнаруживает ошибку в себе самой, она должна попытаться ограничить влияние этой ошибки одной прикладной программой и в крайнем случае прекратить выполнение только этой программы.

11. Операционная система должна давать прикладным программам возможность по требованию исправлять обнаруженные в них ошибки, а не безоговорочно прекращать их выполнение.

Реализация многих из этих принципов влияет на архитектуру лежащего в основе системы аппаратного обеспечения.

Обработка сбоев аппаратуры

Улучшая общую надежность системы, следует заботиться не только об ошибках в программном обеспечении (хотя надежность программного обеспечения требует наибольшего внимания). Другая сторона, о которой необходимо подумать, — это ошибки во входных данных системы (ошибки пользователя).

Наконец, еще один интересующий нас класс ошибок —- сбои аппаратуры. Во многих случаях они обрабатываются самой аппаратурой за счет использования кодов, исправляющих ошибки, исправления последствий сбоев (например, переключением на запасные компоненты) и средств, обеспечивающих устойчивость к ошибкам (например, голосование). Некоторые сбои, однако, нельзя обработать только аппаратными средствами, они требуют помощи со стороны программного обеспечения. Ниже при-

водится список возможностей, которые часто бывают необходимы в программных системах для борьбы со сбоями аппаратуры.

1. *Повторное выполнение операций.* Многие сбои аппаратуры не постоянны (например, скачки напряжения, шум в телекоммуникационных линиях, колебания при механическом движении). Всегда имеет смысл попытаться выполнить операцию, искаженную сбоем (например, команду машины или операцию ввода-вывода), несколько раз, прежде чем принимать другие меры.

2. *Восстановление памяти.* Если обнаруженный случайный сбой аппаратуры вызывает искажение области основной памяти и эта область содержит статические данные (например, команды объектной программы), то последствия сбоя можно ликвидировать, повторно загрузив эту область памяти.

3. *Динамическое изменение конфигурации.* Если аппаратная подсистема, такая, как процессор, канал ввода-вывода, блок основной памяти или устройство ввода-вывода, выходит из строя, работоспособность системы можно сохранить, динамически исключая неисправное устройство из набора ресурсов системы.

4. *Восстановление файлов.* Системы управления базами данных обычно обеспечивают избыточность данных, сохраняя копию текущего состояния базы данных на выделенных устройствах ввода-вывода, регистрируя все изменения базы данных или периодически автономно копируя всю базу данных. Поэтому программы восстановления могут воссоздать базу данных в случае катастрофического сбоя ввода-вывода.

5. *Контрольная точка/рестарт* Контрольная точка — это периодически обновляемая копия состояния прикладной программы или всей системы. Если происходит отказ аппаратуры, такой, как ошибка ввода-вывода, сбой памяти или питания, программа может быть запущена повторно с последней контрольной точки.

6. *Предупреждение отказов питания.* Некоторые вычислительные системы, особенно те, в которых используется энергозависимая память, предусматривают прерывание, предупреждающее программу о предстоящем отказе питания. Это дает возможность организовать контрольную точку или перенести жизненно важные данные во вторичную память.

7. *Регистрация ошибок* Все сбои аппаратуры, с которыми удалось справиться, должны регистрироваться во внешнем файле, чтобы обслуживающий персонал мог получать сведения о постепенном износе устройств.

Из рассмотренных выше трех подгрупп методов обеспечения устойчивости к ошибкам только третья, изоляция ошибок, применима для большинства систем программного обеспечения.

Важное обстоятельство, касающееся всех *четырех подходов,* состоит в том, что обнаружение, исправление ошибок и устойчивость к ошибкам в некотором отношении противоположны методам предупреждения ошибок. В частности, обнаружение, исправление и устойчивость требуют дополнительных функций от самого программного обеспечения. Тем самым не только увеличивается сложность готовой системы, но и появляется возможность внести новые ошибки при реализации этих функций. Как правило, все рассматриваемые методы предупреждения и многие методы обнаружения ошибок применимы к любому программному проекту. Методы исправления ошибок и обеспечения устойчивости применяются не очень широко. Это, однако, зависит от области приложения. Если рассматривается, скажем, система реального времени, то ясно, что она должна сохранить работоспособность и при наличии ошибок, а тогда могут оказаться желательными и методы исправления и обеспечения устойчивости. К системам такого типа относятся телефонные переключательные системы, системы управления технологическими процессами, аэрокосмические и авиационные диспетчерские системы и операционные системы широкого назначения [51].

4.3. Модели надежности программного обеспечения

Термин *модель надежности программного обеспечения,* как правило, относится к математической модели, построенной для оценки зависимости надежности программного обеспечения от некоторых определенных параметров. Значения таких параметров либо предполагаются известными, либо могут быть измерены в ходе наблюдений или экспериментального исследования процесса функционирования программного обеспечения. Данный термин может быть использован также применительно к математической зависимости между определенными параметрами, которые хотя и имеют отношение к оценке надежности программного обеспечения, но тем не менее не содержат ее характеристик в явном виде. Например, поведение некоторой ветви программы

на подмножестве наборов входных данных, с помощью которых эта ветвь контролируется, существенным образом связано с надежностью программы, однако характеристики этого поведения могут быть оценены независимо от оценки самой надежности. Другим таким параметром является частота ошибок, которая позволяет оценить именно качество систем реального времени, функционирующих в непрерывном режиме, и в то же время получать только косвенную информацию относительно надежности программного обеспечения (например, в предположении экспоненциального распределения времени между отказами).

Одним из видов модели надежности программного обеспечения, которая заслуживает особого внимания, является так называемая *феноменологическая,* или *эмпирическая,* модель. При разработке моделей такого типа предполагается, что связь между надежностью и другими параметрами является статической. С помощью подобного подхода пытаются количественно оценить те характеристики программного обеспечения, которые свидетельствуют либо о высокой, либо о низкой его надежности. Так, например, параметр *сложность программы* характеризует степень уменьшения уровня ее надежности, поскольку усложнение программы всегда приводит к нежелательным последствиям, в том числе к неизбежным ошибкам программистов при составлении программ и трудности их обнаружения и устранения. Иначе говоря, при разработке феноменологической модели надежности программного обеспечения стремятся иметь дело с такими параметрами, соответствующее изменение значений которых должно приводить к повышению надежности программного обеспечения [54].

Рассмотрим классификацию моделей надежности ПС, приведенную на рис. 4.3. Модели надежности программных средств (МНПС) подразделяются на аналитические и эмпирические. Аналитические модели дают возможность рассчитать количественные показатели надежности, основываясь на данных о поведении программы в процессе тестирования (измеряющие и оценивающие модели). Эмпирические модели базируются на анализе структурных особенностей программ. Они рассматривают зависимость показателей надежности от числа межмодульных связей, количества циклов в модулях, отношения количества прямолинейных участков программы к количеству точек ветвления и *т.д.* Часто

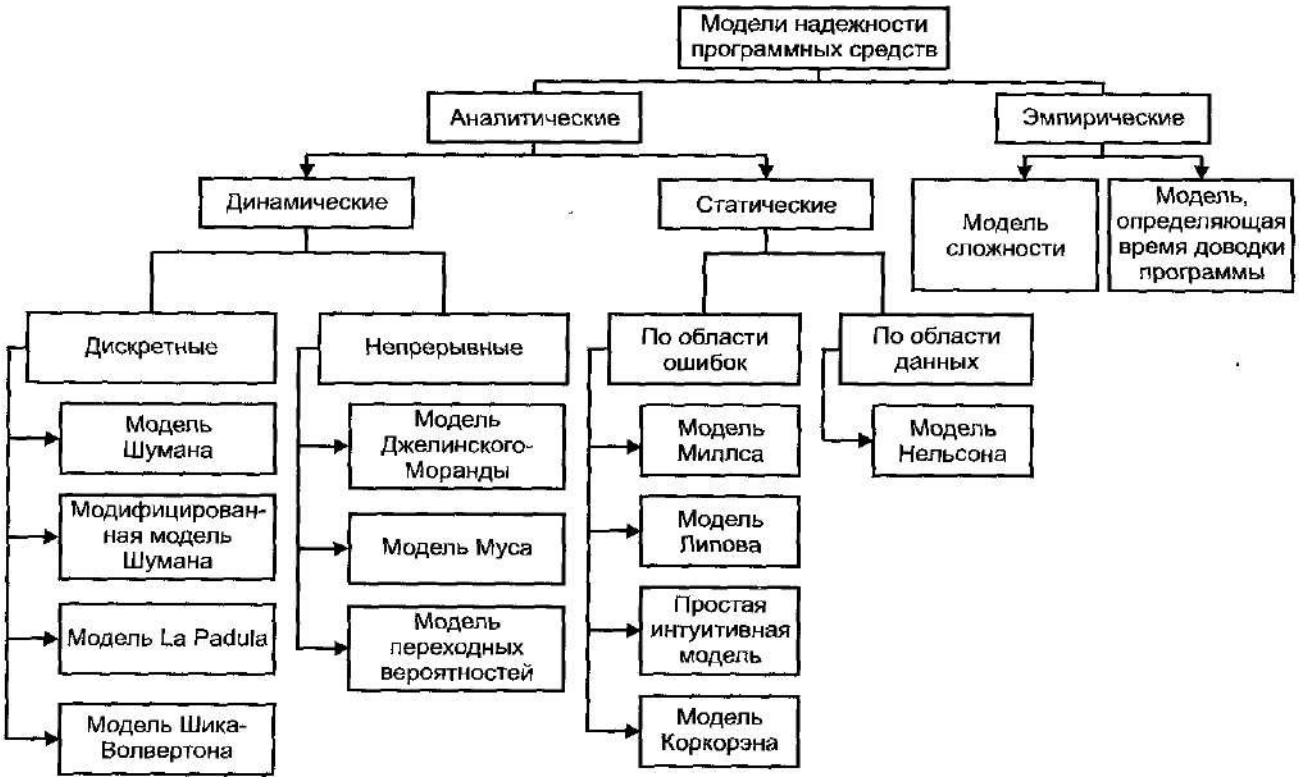


Рис. 4.3. Классификационная схема моделей надежности *ПС*

эмпирические модели не дают конечных результатов показателей надежности, однако они включены в классификационную схему, так как развитие этих моделей позволяет выявлять взаимосвязь между сложностью ПС и его надежностью. Эти модели можно использовать на этапе проектирования ПС, когда осуществлена разбивка на модули и известна его структура.

Аналитические модели представлены двумя группами: *динамические модели* и *статические.* В динамических МНПС поведение ПС (появление отказов) рассматривается во времени. В статических моделях появление отказов не связывают со временем, а учитывают только зависимость количества ошибок от числа тестовых прогонов (по области ошибок) или зависимость количества ошибок от характеристики входных данных (по области данных).

Для использования динамических моделей необходимо иметь данные о появлении отказов во времени. Если фиксируются интервалы каждого отказа, то получается непрерывная картина появления отказов во времени (группа динамических моделей с непрерывным временем). Может фиксироваться только число отказов за произвольный интервал времени. В этом случае поведение ПС может быть представлено только в дискретных точках (группа динамических моделей с дискретным временем). Рассмотрим основные предпосылки, ограничения и математический аппарат моделей, представляющих каждую группу, выделенную по схеме.

Аналитические модели надежности

Аналитическое моделирование надежности ПС включает четыре шага:

1) определение предположений, связанных с процедурой тес тирования ПС;

2) разработка или выбор аналитической модели, базирующей ся на предположениях о процедуре тестирования;

3) выбор параметров моделей с использованием полученных данных;

4) применение модели — расчет количественных показателей надежности по модели.

Динамические модели надежности. ***Модель Шумана.*** Исходные данные для модели Шумана, которая относится к динамическим моделям дискретного времени, собираются в процессе те-

стирования ПС в течение фиксированных или-случайных временных интервалов. Каждый интервал —- это стадия, на которой выполняется последовательность тестов и фиксируется некоторое число ошибок.

Модель Шумана может быть использована при определенным образом организованной процедуре тестирования. Использование модели Шумана предполагает, что тестирование проводится в несколько этапов. Каждый этап представляет собой выполнение программы на полном комплексе разработанных тестовых данных. Выявленные ошибки регистрируются (собирается статистика об ошибках), но не исправляются. По завершении этапа на основе собранных данных о поведении ПС на очередном этапе тестирования может быть использована модель Шумана для расчета количественных показателей надежности. После этого исправляются ошибки, обнаруженные на предыдущем этапе, при необходимости корректируются тестовые наборы и проводится новый этап тестирования. При использовании модели Шумана предполагается, что исходное количество ошибок в программе постоянно и в процессе тестирования может уменьшаться по мере того, как ошибки выявляются и исправляются. Новые ошибки при корректировке не вносятся. Скорость обнаружения ошибок пропорциональна числу оставшихся ошибок. Общее число машинных инструкций в рамках одного этапа тестирования постоянно.

Предполагается, что до начала тестирования в ПС имеется E_m ошибок. В течение времени тестирования % обнаруживается e_c ошибок в расчете на команду в машинном языке.

Таким образом, удельное число ошибок на одну машинную команду, оставшихся в системе после т времени тестирования, равно:

$$\varepsilon_r(\tau) = \frac{E_T}{I_T}\ \varepsilon_c(\tau), \qquad \textbf{(1)}$$

где *Ij* — общее число машинных команд, которое предполагается постоянным в рамках этапа тестирования.

Автор предполагает, что значение функции частоты отказов *Z(t)* пропорционально числу ошибок, оставшихся в ПС после израсходованного на тестирование времени т:

$$Z(t) = C\varepsilon_r(\tau),$$

где C — некоторая константа;

/ — время работы ПС без отказа.

Тогда, если время работы ПС без отказа t отсчитывается от точки $t = 0$, а τ остается фиксированным; функция надежности, или вероятность безотказной работы на интервале времени от О до г, равна:

$$R(t,\tau) = \exp\{C[E_T / I_T - \varepsilon_c(\tau)]t\};$$

(2) (3)

$$t_{\text{ср}} = \frac{1}{C[E_T / I_T - \varepsilon_c(\tau)]}.$$

Из величин, входящих в формулы (2) и (3), не известны начальное значение ошибок в ПС *(E?)* и коэффициент пропорциональности *C.* Для их определения прибегают к следующим рассуждениям. В процессе тестирования собирается информация о времени и количестве ошибок на каждом прогоне, т.е. общее время тестирования τ складывается из времени каждого прогона:

$$\tau = \tau_1 + \tau_2 + \tau_3 + \ldots + \tau_n.$$

Предполагая, что интенсивность появления ошибок постоянна и равна *X,* можно вычислить ее как число ошибок в единицу времени:

$$\lambda = \frac{\sum_{i=1}^{k} A_i}{\tau}, \qquad (4$$

где A_i — количество ошибок на г-м прогоне;

$$t_{\text{ср}} = \frac{\tau}{\sum_{i=1}^{k} A_i}. \qquad)$$

Имея данные для двух различных моментов тестирования τ_A и τ_b, которые выбираются произвольно с учетом требования, чтобы $\varepsilon_c(\tau_b) > \varepsilon_c(\tau_A)$, можно сопоставить уравнения (3) и (5) при τ_A и τ_b

$$\frac{1}{\lambda_{\tau A}} = \frac{1}{C[E_T / I_T - \varepsilon_c(\tau_A)]};$$

$$\frac{1}{\lambda_{\tau b}} = \frac{1}{C[E_T / I_T - \varepsilon_c(\tau_b)]}.$$

Вычисляя отношения (6) и (7), получим:

$$E_T = \frac{E_T[\lambda\tau_b / \lambda\tau_A \varepsilon_c(\tau_A) - \varepsilon_c(\tau_b)]}{(\lambda\tau_b / \lambda\tau_A) - 1}.$$

Подставив полученную оценку параметров E_m в выражение (6), получим оценку для второго неизвестного параметра:

$$C = \frac{\lambda\tau_A}{[E_T / I_T - \varepsilon_c(\tau_A)]}.$$

Получив неизвестные *Em* и С, можно рассчитать надежность программы по формуле (2).

Модель La Padula. По этой модели выполнение последовательности тестов производится в m этапов. Каждый этап заканчивается внесением изменений (исправлений) в ПС. Возрастающая функция надежности базируется на числе ошибок, обнаруженных в ходе каждого тестового прогона.

Надежность ПС в течение z'-го этапа:

$$R(t) = R(\infty) - A/(i),\ i = 1,2\ldots,$$

где A — параметр роста:

$R(\infty) = \lim_{t\to\infty} R(t)$ — предельная надежность ПС.

Эти неизвестные величины автор предлагает вычислить, решив следующие уравнения:

$$\sum_{i=1}^{m}\left\{\frac{S_i - m_i}{S_i} - R(\infty) + \frac{A}{i}\right\} = 0;$$

$$\sum_{i=1}^{m}\left[\left(\frac{S_i - m_i}{S_i} - R(\infty) + \frac{A}{i}\left(\frac{1}{i}\right)\right)\right] = 0,$$

где S_i — число тестов;
m_i — число отказов во время г'-го этапа; m — число этапов; $i = 1, 2, \ldots, m$.

Определяемый по этой модели показатель есть надежность ПС на *г-м* этапе: $R(t) = R(\infty) - A/(i)$, $i = m + 1, m + 2\ldots$

Преимущество модели заключается в том, что она является прогнозной и, основываясь на данных, полученных в ходе тестирования, дает возможность предсказать вероятность безотказной работы программы на последующих этапах ее выполнения.

Модель Джелинского - Моранды. Модель Джелинского - Моранды ОТНОСИТСЯ к динамическим моделям непрерывного времени. Исходные данные для использования этой модели собираются в процессе тестирования ПС. При этом фиксируется время до очередного отказа. Основное положение, на котором базируется модель, заключается в том, что значение интервалов времени тестирования между обнаружением двух ошибок имеет экспоненциальное распределение с частотой ошибок (или интенсивностью отказов), пропорциональной числу еще не выявленных ошибок. Каждая обнаруженная ошибка устраняется, число оставшихся ошибок уменьшается на единицу.

Функция плотности распределения времени обнаружения 1-й ошибки, отсчитываемого от момента выявления (М)-й ошибки, имеет вид:

$$P(t_i) = \lambda_i e^{-\lambda_i t_i}, \quad (10)$$

где λ_i — частота отказов (интенсивность отказов), которая пропорциональна числу еще не выявленных ошибок в программе.

(11)

где N — число ошибок, первоначально присутствующих в программе;
C — коэффициент пропорциональности.

$$\lambda_i = C(N - i + 1),$$

Наиболее вероятные значения величин $\hat{N}$ и $\hat{C}$ (оценка максимального правдоподобия) можно определить на основе данных, полученных при тестировании. Для этого фиксируют время выполнения программы до очередного отказа $t_1, t_2, t_3, \ldots, t_k$.

Значения $\hat{N}$ и $\hat{C}$ _ предлагается получить, решив систему уравнений:

$$\sum_{i=1}^{k} (\hat{N} - i + 1)^{-1} = K/(\hat{N} + 1 - QK). \qquad (12)$$

$$\hat{C} = \frac{K/A}{\hat{N} + 1 - QK},$$

где

$$Q = B/AK; \quad A = \sum_{i=1}^{k} t_i; \quad B = \sum_{i=1}^{k} i \cdot t_i.$$

Поскольку полученные значения $\hat{N}$ и $\hat{C}$ — вероятностные и точность их зависит от количества интервалов тестирования (или количества ошибок), найденных к моменту оценки надежности, асимптотические оценки дисперсий авторы предлагают определить с помощью следующих формул:

$$Var(\hat{N}) = K/C^2 D,$$

$$Var(\hat{C}) = S/D,$$

$$D = KS/C^2 - A^2 \text{ и } S = \sum_{i=1}^{k} (N - i + 1)^2.$$

Чтобы получить числовые значения λ_i, нужно подставить вместо JV и С их возможные значения $\hat{N}$ и $\hat{C}$.Рассчитав K значений по формуле (11) и подставив их в формулу (10), можно опреде-

лить вероятность безотказной работы на различных временных интервалах. На основе полученных расчетных данных строится график зависимости вероятности безотказной работы от времени.

Модель Шика - Волвертона. Модификация модели Джелинского - Моранды для случая возникновения на рассматриваемом интервале более одной ошибки предложена Волвертоном и Шиком. При этом считается, что исправление ошибок производится лишь после истечения интервала времени, на котором они возникли. В основе модели Шика - Волвертона лежит предположение, согласно которому частота ошибок пропорциональна не только количеству ошибок в программах, но и времени тестирования, т.е. вероятность обнаружения ошибок с течением времени возрастает. Частота ошибок (интенсивность обнаружения ошибок) λ_i предполагается постоянной в течение интервала времени ?, и пропорциональна числу ошибок, оставшихся в программе по истечении (z-l)-го интервала; но она пропорциональна также и суммарному времени, уже затраченному на тестирование (включая среднее время выполнения программы в текущем интервале):

$$\lambda_i = C(N - n_{i-1})(T_{i-1} + t_i / 2). \qquad (13)$$

В данной модели наблюдаемым событием является число ошибок, обнаруживаемых в заданном временном интервале, а не время ожидания каждой ошибки, как это было для модели Джелинского - Моранды. В связи с этим модель относят к группе дискретных динамических моделей.

Модель Муса. Модель Муса относят к динамическим моделям непрерывного времени. Это значит, что в процессе тестирования фиксируется время выполнения программы (тестового прогона) до очередного отказа. Но считается, что не всякая ошибка ПС может вызвать отказ, поэтому допускается обнаружение более одной ошибки при выполнении программы до возникновения очередного отказа.

Считается, что на протяжении всего жизненного цикла ПС может произойти *M»* отказов и при этом будут выявлены все *NQ* ошибки, которые присутствовали в ПС до начала тестирования.

Общее число отказов *MQ* связано с первоначальным числом ошибок *No* соотношением

$$N_0 = BM_0, \qquad (14)$$

где *B* — коэффициент уменьшения числа ошибок.

В момент, когда проводится оценка надежности, после тестирования, на которое потрачено определенное время *t,* зафиксировано *m* отказов и выявлено *n* ошибок.

Тогда из соотношения

$$n\text{-}Bm \tag{15}$$

можно определить коэффициент уменьшения числа ошибок *B* как число, характеризующее количество устраненных ошибок, приходящихся на один отказ.

В модели Муса различают два вида времени:

1) суммарное время функционирования т, которое учитывает чистое время тестирования до контрольного момента, когда проводится оценка надежности;

2) оперативное время *t* — время выполнения программы, планируемое от контрольного момента и далее при условии, что дальнейшего устранения ошибок не будет (время безотказной работы в процессе эксплуатации).

Для суммарного времени функционирования т предполагается:

- интенсивность отказов пропорциональна числу неустраненных ошибок;
- скорость изменения числа устраненных ошибок, измеряемая относительно суммарного времени функционирования, пропорциональна интенсивности отказов.

Один из основных показателей надежности, который рассчитывается по модели Муса, — средняя наработка на отказ. Этот показатель определяется как математическое ожидание временного интервала между последовательными отказами и связан с; надежностью:

$$T = \int_0^\infty tf(t)dt = \int_0^\infty R(t)dt,$$

где *t* — время работы до отказа.

Если интенсивность отказов постоянна (т.е. когда длительность интервалов между последовательными отказами имеет экспоненциальное распределение), то средняя наработка на отказ обратно пропорциональна интенсивности отказов.

Модель переходных вероятностей. Эта модель основана на марковском процессе, протекающем в дискретной системе с непрерывным временем.

Процесс, протекающий в системе, называется *марковским* (или процессом без последствий), если для каждого момента времени вероятность любого состояния системы в будущем зависит только от состояния системы в настоящее время *(to)* и не зависит от того, каким образом система пришла в это состояние. Процесс тестирования ПС рассматривается как марковский процесс.

В начальный момент тестирования *(t* - 0) в ПС было *n* ошибок. Предполагается, что в процессе тестирования выявляется по одной ошибке. Тогда последовательность состояний системы *(n,* и-1, *n-2,* и-3) и т.д. соответствует периодам времени, когда предыдущая ошибка уже исправлена, а новая еще не обнаружена. Например, в состоянии л-5 пятая ошибка уже исправлена, а шестая еще не обнаружена.

Последовательность состояний *{m,* т-1, *m-2, m-Ъ* и т.д.} соответствует периодам времени, когда ошибки исправляются. Например, в состоянии *m*-\ вторая ошибка уже обнаружена, но еще не исправлена. Ошибки обнаруживаются с интенсивностью *X,* а исправляются с интенсивностью *i*

Статические модели надежности. Статические модели принципиально отличаются от динамических прежде всего тем, что в них не учитывается время появления ошибок в процессе тестирования и не используется никаких предположений о поведении функции рискаλ(*t*).Эти модели строятся на твердом статистическом фундаменте.

Модель Миллса. Использование этой модели предполагает необходимость перед началом тестирования искусственно вносить в программу («засорять») некоторое количество известных ошибок. Ошибки вносятся случайным образом и фиксируются в протоколе искусственных ошибок. Специалист, проводящий тестирование, не знает ни количества, ни характера внесенных ошибок до момента оценки показателей надежности по модели Миллса. Предполагается, что все ошибки (как естественные, так и искусственно внесенные) имеют равную вероятность быть найденными в процессе тестирования.

Тестируя программу в течение некоторого времени, собирают статистику об ошибках. В момент оценки надежности по про-

токолу искусственных ошибок все ошибки делятся на собственные и искусственные. Соотношение

$$N = \frac{S \; n}{V} \tag{16}$$

дает возможность оценить N — первоначальное число ошибок в программе. В данном соотношении, которое называется формулой Миллса, S — количество искусственно внесенных ошибок, n — число найденных собственных ошибок, V — число обнаруженных к моменту оценки искусственных ошибок.

Вторая часть модели связана с проверкой гипотезы от N Предположим, что в программе имеется K собственных ошибок, и внесем в нее еще S ошибок. В процессе тестирования были обнаружены все S внесенных ошибок и n собственных ошибок.

Тогда по формуле Миллса мы предполагаем, что первоначально в программе было $N - n$ ошибок. Вероятность, с которой можно высказать такое предположение, возможно рассчитать по следующему соотношению:

$$C = \begin{cases} 1, \text{если } n > K; \\ \dfrac{S}{S+K+1}, \text{если } n =< K. \end{cases} \tag{17}$$

Таким образом, величина С является мерой доверия к модели и показывает вероятность того, насколько правильно найдено значение N. Эти два связанных между собой по смыслу соотношения образуют полезную модель ошибок: первое предсказывает возможное число первоначально имевшихся в программе ошибок, а второе используется для установления доверительного уровня прогноза.

Модель Липово. Липов модифицировал модель Миллса, рассмотрев вероятность обнаружения ошибки при использовании различного числа тестов. Если сделать то же предположение, что и в модели Миллса, т.е. что собственные и искусственные ошибки имеют равную вероятность быть найденными, то вероятность обнаружения n собственных и V внесенных ошибок равна:

$$Q(n,V) = \left(\frac{m}{n+V}\right) q^{n+V} (1-q)^{m-n-V} \frac{\frac{N}{n} \cdot \frac{S}{V}}{\frac{N+S}{n+V}}$$

где m — количество тестов, используемых при тестировании;

q — вероятность обнаружения ошибки в каждом из m тестов, рассчитанная по формуле

$$q = \frac{n+V}{n};$$

S — общее количество искусственно внесенных ошибок; N — количество собственных ошибок, имеющихся в ПС до начала тестирования.

Для использования модели Липова должны выполняться следующие условия:

$$N \geq n \geq 0;\ S \geq V \geq 0;\ m \geq n + V \geq 0.$$

Оценки максимального правдоподобия (наиболее вероятное значение для *N)* задаются соотношениями

$$N = \frac{S \cdot n}{V} \text{ при } n \geq 1,\ V \geq 1;$$

$$n \cdot S \text{ при } V = 0,\ 0 \text{ при } n = 0.$$

Модель Липова дополняет модель Миллса, давая возможность оценить вероятность обнаружения определенного количества ошибок к моменту оценки.

Простая интуитивная модель. Использование этой модели предполагает проведение тестирования двумя группами программистов (или двумя программистами в зависимости от величины программы) независимо друг от друга, использующими независимые тестовые наборы. В процессе тестирования каждая из групп фиксирует все найденные ею ошибки. При оценке числа оставшихся в программе ошибок результаты тестирования обеих групп собираются и сравниваются.

Получается, что первая группа обнаружила *N* ошибок, вторая — 7V2, а *N\i* — это ошибки, обнаруженные обеими группами.

Если обозначить через N неизвестное количество ошибок, присутствовавших в программе до начала тестирования, то можно эффективность тестирования каждой из групп определить как

$$E_1 = \frac{N_1}{N};\ E_2 = \frac{N_2}{N} \tag{18}$$

Предполагая, что возможность обнаружения всех ошибок одинакова для обеих групп, можно допустить, что если первая группа обнаружила определенное количество всех ошибок, она могла бы определить то же количество любого случайным обра-зом выбранного подмножества. В частности, можно допустить:

$$E_1 = \frac{N_1}{N} = \frac{N_{12}}{N_2}. \tag{19}$$

Из формулы (18) JV2 = E_2N, подставив в (19), получим:

$$E_1 = \frac{N_{12}}{E_2 N} = \frac{N_1 N_2}{N_{12}}$$

Модель Коркорэна. Модель Коркорэна относится к статическим моделям надежности ПС, так как в ней не используются параметры времени тестирования и учитывается только результат N испытаний, в которых выявлено $N\backslash$ ошибок /-го типа. Модель использует изменяющиеся вероятности отказов для различных типов ошибок.

В отличие от двух рассмотренных выше статических моделей, по модели Коркорэна оценивается вероятность безотказного выполнения программы на момент оценки:

$$R = \frac{N_0}{N} + \sum_{i=1}^{K} Y_i (N_i - 1)/N,$$

где NQ — число безотказных выполнений программы;
N — общее число прогонов,
K — априори известное число типов

$$Y_i = \begin{cases} a_i, & \text{если } N_i > 0 \\ 0, & \text{если } N_i = 0 \end{cases}$$

a_i — вероятность выявления при тестировании ошибки *1-го* типа.

В этой модели вероятность *a,* должна оцениваться на основе априорной информации или данных предшествующего периода функционирования однотипных программных средств.

Модель Нельсона. Данная модель при расчете надежности ПС учитывает вероятность выбора определенного тестового набора для очередного выполнения программы.

Предполагается, что область данных, необходимых для выполнения тестирования программного средства, разделяется на Л*взаимоисключающих подобластей Z,. *i*= 1,2, ... , *к.* Пусть P_z — вероятность того, что набор данных Z, будет выбран для очередного выполнения программы. Предполагая, что к моменту оценки надежности было выполнено *N,* прогонов программы на Z, наборе данных и из них *n,* количество прогонов закончилось отказом, надежность ПС в этом случае равна:

$$R = 1 - \sum_{i=1}^{k} \frac{(n_i)}{N_i} P_i.$$

На практике вероятность выбора очередного набора данных для прогона (P,) определяется путем разбиения всего множества значений входных данных на подмножества и нахождения вероятностей того, что выбранный для очередного прогона набор данных будет принадлежать конкретному подмножеству. Определение этих вероятностей основано на эмпирической оценке вероятности появления тех или иных входов в реальных условиях Функционирования.

Эмпирические модели надежности

Эмпирические модели в основном базируются на анализе структурных особенностей программного средства (или программы). Как указывалось ранее, эмпирические модели часто не дают конечных результатов показателей надежности, однако их исполь-

зование на этапе проектирования ПС полезно для прогнозирования требующихся ресурсов тестирования, уточнения плановых сроков завершения проекта и т.д.

Модель сложности. В литературе неоднократно подчеркивается тесная взаимосвязь между сложностью и надежностью ПС. Если придерживаться упрощенного понимания сложности ПС, то она может быть описана такими характеристиками, как размер ПС (количество программных модулей), количество и сложность межмодульных интерфейсов.

Под программным модулем в данном случае следует понимать программную единицу, выполняющую определенную функцию (ввод, вывод, вычисление и т.д.) и взаимосвязанную с другими модулями ПС. Сложность модуля ПС может быть описана, если рассматривать структуру программы.

В качестве структурных характеристик модуля ПС используются:

1) отношение действительного числа дуг к максимально воз можному числу дуг, получаемому искусственным соединением каждого узла с любым другим узлом дугой;

2) отношение числа узлов к числу дуг;

3) отношение числа петель к общему числу дуг.

Для сложных модулей и для больших многомодульных программ составляется имитационная модель, программа которой «засоряется» ошибками и тестируется по случайным входам. Оценка надежности осуществляется по модели Миллса.

При проведении тестирования известна структура программы, имитирующей действия основной, но не известен конкретный путь, который будет выполняться при вводе определенного тестового входа. Кроме того, выбор очередного тестового набора из множества тест-входов случаен, т.е. в процессе тестирования не обосновывается выбор очередного тестового входа. Эти условия вполне соответствуют реальным условиям тестирования больших программ.

Полученные данные анализируются, проводится расчет показателей надежности по модели Миллса (или любой другой из описанных выше), и считается, что реальное ПС, выполняющее аналогичные функции, с подобными характеристиками и в реальных условиях должно вести себя аналогичным или похожим образом.

Преимущества оценки показателей надежности по имитационной модели, создаваемой на основе анализа структуры будущего реального ПС, заключаются в следующем:

- модель позволяет на этапе проектирования ПС принимать оптимальные проектные решения, опираясь на характеристики ошибок, оцениваемые с помощью имитационной модели;
- модель позволяет прогнозировать требуемые ресурсы тестирования;
- модель дает возможность определить меру сложности программ и предсказать возможное число ошибок и т.д.

К недостаткам можно отнести высокую стоимость метода, так как он требует дополнительных затрат на составление имитационной модели, и приблизительный характер получаемых показателей.

Модель, определяющая время доводки программ. Эта модель используется для ПС, которые имеют иерархическую структуру, т.е. ПС как система может содержать подсистемы, которые состоят из компонентов, а те, в свою очередь, состоят из V модулей. Таким образом, ПС может иметь К различных уровней композиции. На любом уровне иерархии возможна взаимная зависимость между любыми парами объектов системы. Все взаимозависимости рассматриваются в терминах зависимости между парами модулей.

Анализ модульных связей строится на том, что каждая пара модулей имеет конечную (возможно, нулевую) вероятность, изменения в одном модуле вызовут изменения в другом модуле.

Данная модель позволяет на этапе тестирования, а точнее при тестовой сборке системы, определять возможное число необходимых исправлений и время, необходимое для доведения ПС до рабочего состояния.

Основываясь на описанной процедуре оценки общего числа изменений, требуемых для доводки ПС, можно построить две различные стратегии корректировки ошибок:

- фиксировать все ошибки в одном выбранном модуле и устранить все побочные эффекты, вызванные изменениями этого модуля, отрабатывая таким образом последовательно все модули;
- фиксировать все ошибки нулевого порядка в каждом модуле, затем фиксировать все ошибки первого порядка и т.д.

Исследование этих стратегий доказывает, что время корректировки ошибок на каждом шаге тестирования определяется максимальным числом изменений, вносимых в ПС на этом шаге, а общее время — суммой максимальных времен на каждом шаге.

Это подтверждает известный факт, что тестирование обычно является последовательным процессом и обладает значительными возможностями для параллельного исправления ошибок, что часто приводит к превышению затрачиваемых на него ресурсов над запланированными [56].

Для *удостоверения качества, надежности и безопасности применения* сложных, критических ИС используемые в них ПС следует подвергать *обязательной сертификации* аттестованными, проблемно-ориентированными испытательными лабораториями. Такие испытания необходимо проводить, когда программы управляют сложными процессами или обрабатывают столь важную информацию, что дефекты в них или недостаточное качество могут нанести значительный ущерб. Сертификационные испытания должны устанавливать соответствие комплексов программной документации и допускать их к эксплуатации в пределах изменения параметров внешней среды, исследованных при проведенных проверках. Эти виды испытаний характеризуются наибольшей строгостью и глубиной проверок и должны проводиться специалистами, не зависимыми от разработчиков и заказчиков (пользователей). Испытания ПС должны опираться на стандарты, формализованные методики и нормативные документы разных уровней. Множество видов испытаний целесообразно упорядочивать и проводить поэтапно в процессе разработки для сокращения затрат на завершающихся сертификационных испытаниях.

Сертификация комплексов программ является их испытанием в наиболее жестких условиях тестирования особым третейским коллективом специалистов, имеющим право на официальный государственный или ведомственный контроль функций и качества ПС и гарантирующим их соответствие стандартам и другим нормативным документам, а также надежность и безопасность применения. Получение и обобщение результатов испытаний, а также принятие решения о выдаче сертификата являются прерогативой *испытательных лабораторий.* Они должны быть специализированными для проведения испытаний объектов определенных классов, целенаправленно и систематически работать по созданию и совершенствованию методик и средств автоматизации испытаний ПС конкретного функционального назначения.

Специалисты-сертификаторы имеют право на расширение условий испытаний и на создание различных критических и стрессовых ситуаций в пределах нормативной документации, при которых должны обеспечиваться заданное качество и надежность решения предписанных задач. Если все испытания проходят успешно, то на соответствующую версию ПС оформляется специальный документ — *сертификат соответствия.* Этот документ официально подтверждает соответствие стандартам, нормативным и эксплуатационным документам функций и характеристик испытанных средств, а также допустимость их применения в определенной области.

Методология принятия решений о допустимости выдачи сертификата на ПС определяется оценкой степени его соответствия действующим и/или специально разработанным документам. В исходных нормативных документах должны быть сосредоточены все функциональные и эксплуатационные характеристики ПС, обеспечивающие заказчику и пользователям возможность корректного применения сертифицированного объекта во всем многообразии его функций и показателей качества. Выбор и ранжирование показателей должны проводиться с учетом классов ПС, их функционального назначения, режимов эксплуатации, степени критичности и жесткости требований к результатам функционирования и проявлениям возможных дефектов и ошибок. При этом могут привлекаться документы предшествующих этапов испытаний и документы, подтверждающие соблюдение аттестованных технологий при разработке программ на всех этапах. Испытания ПС в конкретных проблемно-ориентированных системах проводятся по правилам и методикам, принятым для соответствующих классов критических информационных систем, например, авиационных или космических комплексов.

Работы по сертификации объединяются *в технологический процесс,* на каждом этапе которого регистрируются документы, отражающие состояние и качество результатов разработки ПС. В зависимости от характеристик объекта сертификации на ее выполнение выделяются ресурсы различных видов. В результате сложность программ, а также доступные для сертификации ресурсы становятся косвенными критериями или факторами, влияющими на выбор методов испытаний, а также на достигаемые качество и надежность ПС.

Сертификационные испытания удостоверяют качество и надежность ПС только в условиях, ограниченных конкретными стандартами и нормативными документами, с некоторой конечной вероятностью. В реальных условиях эксплуатации принципиально возможны отклонения от характеристик внешней среды функционирования ПС за пределы, ограниченные сертификатом, и ситуации, не проверенные при сертификационных испытаниях. Эти обстоятельства способны вызывать катастрофические последствия, угрожающие надежности функционирования и безопасности применения ПС. Наличие сертификата у ПС для критических систем является необходимым условием их допуска к эксплуатации. Однако любой сертификат на сложные системы не может гарантировать абсолютную их надежность применения, и всегда остается некоторый риск возникновения отказовых ситуаций.

Отсутствие гарантии достижения в процессе создания ПС абсолютной надежности их функционирования за счет использования высоких технологий, тестирования и сертификации заставляет искать дополнительные методы и средства повышения надежности ПС. Для этого разрабатываются и применяются *методы оперативного обнаружения дефектов и искажений при исполнении программ путем введения в них временной, информационной и программной избыточности.* Эти же виды избыточности используются для оперативного восстановления искаженных программ и данных и предотвращения возможности развития результатов реализации угроз до уровня, нарушающего надежность функционирования ПС. Основная задача ввода избыточности состоит в ограничении или исключении возможности аварийных последствий от возмущений, соответствующих отказу системы. Любые аномалии при исполнении программ необходимо блокировать и по возможным последствиям сводить до уровня сбоя путем быстрого восстановления.

Особенности обеспечения надежности функционирования импортных программных средств. При использовании зарубежных ПС в принципе в них возможны как злоумышленные, так и случайные, непредумышленные искажения вычислительного процесса, программ и данных, отражающиеся на надежности их функционирования. Злоумышленные вирусы и «закладки», хотя и маловероятны в серийных, широко тиражируемых в мире ПС, тем не менее требуют особых методов и средств целенаправленного их обнаружения и устранения. Зарубежным специалистам свойствен-

но ошибаться так же, как и отечественным, однако более высокое качество используемых технологий разработки и современная проектировочная культура позволяют значительно снижать уровень дефектов в изделиях, поступающих на рынок и в эксплуатацию. Тем не менее *в любых сложных импортных ПС всегда не гарантировано полное, абсолютное отсутствие случайных ошибок,* которые остаются важнейшими дестабилизирующими факторами. Их применение в критических отечественных ПС требует соответствующего дополнительного контроля качества и специальных работ по обеспечению надежности при эксплуатации.

Представленные выше *объекты уязвимости, дестабилизирующие факторы и угрозы надежности* присуши любым программам и данным независимо от фирм-разработчиков. Однако методы предотвращения и снижения влияния угроз надежности для зарубежных ПС значительно отличаются. Разрыв в пространстве и времени при проектировании конкретного ПС между первичными зарубежными создателями программных компонентов и потребителями, интеграторами, непосредственными создателями отечественных ИС затрудняет взаимодействие по предотвращению ошибок за счет применения CASE-технологий. Отечественный покупатель импортных ПС обычно не знает, какая технология была применена при их разработке и какие классы ошибок могли быть оставлены. В составе пользовательской документации, как правило, отсутствуют исходные тексты программ и номенклатура тестов, использованных при их отладке. Поэтому методы предотвращения ошибок в импортных программах и данных почти всегда остаются недоступными и неизвестными отечественным специалистам. Это отражается на хроническом недоверии к качеству и надежности применения зарубежных программных компонентов или на слепой вере в их абсолютную безупречность.

Комплексирование готовых импортных прикладных ПС в конкретной отечественной ИС создает условия для их функционирования, не всегда адекватные предусмотренным разработчиками и проверенным при испытаниях, хотя и не выходящие за пределы требований эксплуатационной документации. Это способствует проявлению ранее скрытых дефектов и ошибок проектирования и их устранению. Для этого ответственные и квалифицированные поставщики зарубежных ПС имеют службы сопровождения, регистрации и накопления претензий пользователей

и быстрого реагирования для устранения реальных дефектов функционирования. Легальная закупка и использование лицензионно чистых ПС, обеспеченных сопровождением солидной фирмы-поставщика, позволяют в значительной степени снижать влияние на надежность ПС дефектов, не предотвращенных в процессе проектирования.

Этому же может способствовать применение разработчиками ИС той же CASE-технологии, которая использовалась зарубежными решателями применяемых ПС. Для этого, в частности, наиболее популярные СУБД при продаже комплектуются средствами соответствующей CASE-технологии. Поставки прикладных программ различного назначения могут содержать рекомендации по использованию определенных CASE-технологий при комплексировании импортных компонентов в составе конкретной ИС. Применение той же CASE-технологии позволяет более полно понимать функциональные и технические возможности закупленных ПС в процессе их комплексирования в проблемно-ориентированной ИС. Это предотвращает наиболее сложные системные ошибки при использовании и интегрировании импортных ПС. Таким образом, хотя непосредственное предотвращение и исправление ошибок импортных ПС отечественными потребителями в процессе разработки ИС затруднительны, при соответствующем взаимодействии с конкретными зарубежными фирмами надежность ИС при использовании зарубежных программных продуктов можно поставить под достаточно жесткий контроль.

Систематическое тестирование импортных ПС в процессе проектирования производится самими разработчиками ИС. При отработке критических ПС целесообразны создание или закупка комплектов и генераторов тестов для тестирования конкретных ПС в составе ИС или автономно. Такое дополнительное тестирование повышает уверенность в качестве и надежности применяемых импортных продуктов в конкретном окружении, а также может приводить к обнаружению некоторых ошибок проектирования и комплексирования зарубежных программных компонентов. Их устранение в большинстве случаев целесообразно проводить силами зарубежной фирмы-разработчика с использованием организационно и юридически оформленного механизма сопровождения изделий поставщиком.

Обязательная сертификация зарубежных ПС для сложных, критических ИС предполагает сопровождение закупаемых, лицензионно чистых изделий сертификатом соответствия, выданным специализированной испытательной фирмой. Такое юридическое утверждение качества и надежности применения импортного изделия может быть недостаточным для особо важных, критических ИС, так как сертификат соответствия не всегда сопровождается протоколами испытаний и использованными при этом тестами, что не позволяет оценить полноту испытаний. В этих случаях *следует ориентироваться на дополнительную сертификацию импортных ПС отечественными проблемно-ориентированными, аттестованными сертификационными лабораториями.*

Такие испытания позволяют удостовериться в надежности применяемых зарубежных ПС, а также дополнительно выявить некоторые некорректности программ или документации. Их устранение требует взаимодействия с зарубежной фирмой-поставщиком для корректировки изделий и исключения дефектов. Самостоятельное исправление выявленных ошибок отечественными специалистами сопряжено с риском внесения дополнительных вторичных ошибок из-за недостаточной квалификации и неполной информации о детальном содержании текстов программ и описаний данных. Кроме того, любые изменения в сертифицированных изделиях помимо фирмы-поставщика приводят к автоматическому аннулированию выданного ею сертификата. Дополнительное подтверждение сертификата соответствия отечественными специалистами может значительно повысить уверенность в надежности зарубежных ПС.

Оперативные методы повышения надежности функционирования ПС предусматриваются в некоторых зарубежных изделиях и, в частности, в механизмах обеспечения целостности информации баз данных в реляционных СУБД. Однако разнообразие условий функционирования импортных ПС в сложных отечественных ИС не позволяет удовлетвориться только штатными методами оперативного обнаружения аномалий и восстановления вычислительного процесса, программ и данных. Методы и средства для этого могут быть в ряде случаев достаточно автономными и ориентированными на оперативное повышение надежности конкретной ИС в целом, а не только отдельных используемых ПС. Эти специализированные методы и средства могут разрабатываться отечественными специалистами для обеспечения комплексной на-

дежности с использованием всех импортных компонентов. Такой подход позволяет обеспечить комплексирование разнородных ПС различных зарубежных поставщиков и специализированной отечественной системы оперативной защиты в едином комплексе программ. При этом важно использовать концепцию и стандарты открытых систем при взаимодействии между как закупаемыми, так и вновь разрабатываемыми компонентами ПС, а также при их взаимодействии с внешней средой. Применение стандартизированных интерфейсов открытых систем между прикладными программами и CASE-технологией является эффективным современным методом повышения надежности информационных систем при наличии разнородных поставщиков компонентов.

Таким образом, для обеспечения надежности функционирования зарубежных ПС в составе отечественных ИС прежде всего следует полностью отказаться от применения нелегальных импортных программ и баз данных. Процессы закупки, контроля и применения импортных ПС для сложных отечественных ИС должны быть организованы и поддержаны дополнительными испытаниями. Использование лицензионно чистых ПС и тесное взаимодействие с их зарубежными фирмами-поставщиками позволяют эффективно продолжать тестирование программ при их комплексировании в отечественных ИС, оценивать и повышать надежность функционирования. При закупке зарубежных ПС целесообразно требовать сертификат соответствия и сопроводительную документацию по методам, тестам и результатам испытаний. В ряде случаев может быть необходима дополнительная сертификация импортных программ отечественными сертификационными лабораториями. Кроме того, для каждой критической ИС должна разрабатываться специализированная система обеспечения надежности ее функционирования путем оперативного контроля, выявления дефектов и восстановления вычислительного процесса, программ и данных при искажениях, угрожающих надежности и безопасности применения.

В импортных программах, кроме случайных ошибок, возможны *преднамеренные фрагменты — «закладные элементы» и вирусы* с целью реализации вредных для эксплуатации функций, которые не описаны в документации. До наступления определенного события «закладной элемент» остается неактивным, а при выполнении некоторого условия осуществляет разрушительные действия, приводящие к отказу и не предусмотренные функциональ-

ным назначением и документацией. Сертификация импортных программ для удостоверения отсутствия в них вирусов или «закладных элементов» может осуществляться в *двух ситуациях:*

- при наличии в составе поставляемой документации исходных текстов программ на языке программирования и описаний ал горитмов обработки информации;
- при наличии только эксплуатационной документации, недостаточной для анализа содержания и текстов программ.

В первом случае определение наличия в программе посторонних компонентов может производиться последовательной сверкой текста программы на языке программирования с описанием программы и спецификации. По тексту программы составляется блок-схема анализируемого алгоритма, которая сравнивается с алгоритмом, изложенным в описании программы. Если логическая структура алгоритмов различается, то следует проводить дополнительный анализ элементов блок-схем, в которых обнаружены различия. Такие различия могут быть обусловлены дефектами документации на программу, случайными или предумышленными дефектами самой программы. Дефекты программы подлежат подробному анализу, классификации и корректировке, после чего ее следует подвергнуть полному тестированию и повторной сертификации на полное соответствие всей документации и отсутствие вредных компонентов.

Во втором случае, который является наиболее массовым, задача значительно усложняется, так как исходные документы о структуре и содержании программ и алгоритмов не поставляются. Для получения текста программы и алгоритма необходимо провести дизассемблирование объектного кода программы и выразить каждую функциональную команду кода ассемблера в виде логической процедуры для представления как оператора блок-схемы алгоритма. Построенная блок-схема подлежит анализу на наличие сомнительных конструкций, тупиков и висячих вершин, которые могут оказаться «закладными элементами». Каждая сомнительная группа процедур подлежит дальнейшему анализу на возможность ее принадлежности «закладному элементу», вирусу или случайной ошибке. Выявленные участки программы, содержащие случайные и предумышленные дефекты, должны корректироваться. После их исключения программа подлежит полному тестированию на соответствие эксплуатационной документации [49].

Четкое экономическое и юридическое взаимодействие с определенными фирмами — поставщиками импортных ПС позволяет держать под контролем не только достижимую надежность ИС, но и значительно снижает вероятность злоумышленных аномалий в поставляемых ими изделиях. Обнаружение и публикация сведений о предумышленных негативных компонентах в программных продуктах способны нанести значительный ущерб репутации и бизнесу фирмы.

4.4. Обеспечение качества и надежности в процессе разработки сложных программных средств

Сложность

Сложность — это одна из главных причин ненадежности программного обеспечения Сложность почти не поддается ни точному определению, ни измерению. Однако можно сказать, что мерой сложности объекта является количество интеллектуальных усилий, необходимых для понимания этого объекта.

В общем случае сложность объекта является функцией взаимодействия между его компонентами. Например, сложность внешнего проекта программной системы в некоторой степени определяется связями между всеми ее внешними сопряжениями, например между командами пользователя и соотношениями между входной и выходной информацией системы. Сложность архитектуры системы определяется связями между подсистемами. Сложность проекта программы — функция связей между модулями. Сложность отдельного модуля — функция связей между его командами.

В борьбе со сложностью программного обеспечения можно привлечь две концепции из общей теории систем. Первая — *независимость.* В соответствии с этой концепцией для минимизации сложности необходимо максимально усилить независимость компонентов системы. По существу это означает такое разбиение системы, чтобы высокочастотная динамика ее была заключена в единых компонентах, а межкомпонентные взаимодействия представляли лишь низкочастотную динамику системы.

IITIr.

Вторая концепция — *иерархическая структура.* Каждый уровень представляет собой совокупность структурных отношений между элементами нижних уровней. Концепция уровня позволяет понять систему, скрывая несущественные уровни детализации. Например, система, которую мы называем «человек», представляется иерархией. Социолог может интересоваться взаимоотношениями людей, не заботясь об их внутреннем устройстве. Психолог работает на более низком уровне иерархии. Он может исследовать различные логические и физические процессы в мозге, не рассматривая внутреннего строения областей мозга. Еще ниже в этой иерархии находится нейролог — он имеет дело со структурой основных компонентов мозга. Однако он может изучать мозг на этом уровне, не заботясь о молекулярной структуре отдельных белков в нейроне. Химик-органик интересуется построением сложных аминокислот из таких компонентов, как атомы углерода, водорода, кислорода и хлора. Физик-ядерщик изучает систему на уровне элементарных частиц в атоме и взаимодействия между ними.

Иерархия позволяет проектировать, описывать и понимать сложные системы. Если бы нельзя было принять описанный подход к изучению человека, социологу пришлось бы рассматривать его как необъятное и сложное множество субатомных частиц. Очевидно, что такое количество деталей подавило бы его, так что невозможны были бы даже те ограниченные знания о человеке, которыми мы располагаем.

К этим двум концепциям сокращения сложности (независимость и иерархическая структура) можно добавить третью: *проявление связей* всюду, где они возникают. Основная проблема многих больших программных систем — огромное количество независимых побочных эффектов, создаваемых компонентами системы. Из-за этих побочных эффектов систему невозможно понять. И можно быть уверенным, что систему, в которой нельзя разобраться, было очень трудно спроектировать хотя бы с минимальной гарантией надежности.

Отношения с пользователем

Две самые распространенные ошибки при работе над программными проектами — это отказ от вовлечения пользователя системы в процессы принятия решений и неспособность понять

его культурный уровень и окружающую его обстановку. При работе над многими проектами имеется тенденция умышленно исключать пользователя из процесса принятия решений. Обычно причина этого в том, что разработчик программного обеспечения чувствует: если вовлечь пользователя, тот никогда не придет к окончательному решению, его требования будут постоянно меняться. Для такой тревоги есть некоторые основания, но на практике преимущества от участия пользователя значительно перевешивают эти возможные неудобства. Вторая ошибка в программных проектах — разработчик системы часто слабо знает (или не знает вовсе) обстановку, в которой находится пользователь, т. е. плохо понимает, с какими именно трудностями сталкивается пользователь и как он будет применять программную систему. Бывает, например, так, что в проектировании операционной системы участвуют люди, сами никогда не использовавшие операционные системы. Есть разработчики языков программирования, никогда не пробовавшие реализовать прикладную систему на языке высокого уровня. Есть разработчики систем управления базами данных, которые никогда не пытались использовать базу данных в прикладной программе. Это не может не вести к серьезным ошибкам в программном обеспечении.

Единственно возможный способ избежать этих ошибок — поддерживать прочный контакт с пользователем в течение всего цикла разработки. С коллективом пользователей должны быть установлены такие отношения, чтобы те серьезно участвовали в процессе принятия решений на этапах определения требований, целей и внешнего проектирования. Привлечение пользователей на последующих этапах также желательно, особенно в процессе тестирования, когда пользователь может помочь разработчику системы значительно лучше понять, как следует тестировать систему. Будьте, однако, осмотрительны, привлекая пользователя к обсуж-(дению деталей, судить о которых он некомпетентен. Например, хорошо, чтобы пользователь принимал участие в проектировании внешних характеристик системы, но привлекать его к такой работе, как анализ логики конкретного модуля, неразумно.

Имеется (уже упоминавшаяся) опасность, что пользователь может изменять свои требования к системе. Отметим, однако, что это никак не связано с непосредственным его участием в работе над проектом. Если требования к системе должны измениться, это произойдет независимо от того, привлечен ли пользователь

непосредственно к работе или нет. В действительности если сам пользователь в работе не участвует, разработчик, вероятно, не узнает об изменении требований до тех пор, пока не станет слишком поздно. Если же пользователь непосредственно привлечен к работе, он может значительно лучше представлять себе стоимость каждого изменения. Если правильно предусмотреть условия для изменения требований, участие пользователя может оказаться выгодным и с этой точки зрения.

Участие потенциальных пользователей в создании новых систем, которые разрабатываются не по заказу и сведения о которых составляют коммерческую тайну, также не является недопустимым.

Хотя такой продукт предназначен не для конкретного потребителя, разработчик и в этом случае, вероятно, хорошо представляет себе возможных покупателей. С одним или несколькими из них может быть подписано соглашение о сохранении коммерческой тайны, что позволит и возможным покупателям системы участвовать в ее разработке до того, как о ней будет публично объявлено.

Преодоление второй трудности — непонимание запросов пользователя и окружающей его обстановки — требует, чтобы проектировщики программной системы досконально представили себе особенности его работы. Обычно принимается весьма неэффективное решение — командировать основных проектировщиков для изучения положения дел. Проектировщики получают поверхностное представление о существующих системах и совсем не получают сколько-нибудь глубокого представления о том, как система используется и в чем же состоят подлинные проблемы.

Решение задачи

Большинство процессов разработки программного обеспечения — это процессы решения некоторых задач. Внешнее проектирование сводится к решению такой задачи: «Переведите множество целей системы во внешние спецификации», где цели — данные, а внешние спецификации — неизвестные. В задаче проектирования логики модуля даны внешние спецификации модуля, а неизвестное — текст его программы. Отладка — это задача на построение исправления ошибки (неизвестное) по описанию ее симптомов (данные).

Приведем один из методов решения задачи.

1. Поймите задачу
Изучите данные.
Изучите неизвестные.
Достаточно ли данных для решения? Непротиворечивы ли они?
2. Составьте план
Чего вы должны добиваться?
Какие методы проектирования будут использоваться? Встречалась ли вам уже такая задача? Не знаете ли вы близкой задачи? Можете ли вы воспользоваться ее результатом? Можете ли вы решить более специализированную или аналогичную задачу?
Можете ли вы решить часть задачи? 5.
Выполните план
Следуйте своему плану решения задачи.
Проверяйте правильность каждого шага.
4. Проанализируйте решение Все ли
данные вы использовали? Проверьте
правильность решения.
Можете ли вы воспользоваться полученным результатом или примененным методом при решении других задач?

Рассмотрим основные положения этого метода.

Поймите задачу

Худшая из ошибок, которые могут быть сделаны при решении задачи, — не вполне разобраться в ее постановке. Понять задачу — это значит понять два ее компонента: данные и неизвестное. Данные — это все элементарные факты, касающиеся задачи, и связи между фактами и неизвестным. Усвоение всех данных о сложной задаче — большая, но абсолютно неизбежная работа. При этом в первую очередь необходимо хорошо охватить «общую картину» данных без деталей, которые, однако, также запоминаются «в сторонке», чтобы их можно было легко вспомнить позже, Есть много способов добиться этого. Например, кое-кто физически разрезает спецификации на куски, которые затем расклеиваются на стене в определенном порядке. Это позволяет увидеть общую картину и при этом определить место для каждой детали.

Вторая часть задачи — неизвестное. Проектировщику следует понимать, какую форму должно иметь решение. Если, например, задача — детальное внешнее проектирование программы, то проектировщик должен ясно представлять назначение внешних спецификаций, их потенциальных читателей, формат и т. д.

Исследуя задачу, проектировщик должен также исследовать данные, чтобы убедиться, что их достаточно для решения задачи и они не противоречат друг другу.

Составьте план

Прежде чем приступить к решению, следует разработать его план. Отсутствие плана — очень распространенная ошибка. Например, проектировщики программной системы, которые потратили время на то, чтобы понять задачу, но затем немедленно приступили к ее решению, не пожелав тратить время на планирование своих усилий, в конце концов могут прийти к хорошему решению, но не раньше чем после нескольких ненужных фальстартов.

Прежде всего в плане нужно определить, чего вы хотите добиться. Десять человек могут иметь десять разных мнений относительно «правильного» ответа на задачу проектирования; проектировщик должен предусмотреть те конкретные аспекты решения, которые требуют наибольшего внимания. К сожалению, в большинстве проектов разработчики имеют слишком много свободы в этом отношении: каждый проектировщик принимает компромиссные решения, основываясь исключительно на собственном мнении, что приводит к несогласованности многих решений в системе. Идея *целей проекта* является решением этой проблемы. Суть идеи состоит в том, что на уровне всего проекта определяются общие цели, которыми следует руководствоваться во всех решениях при проектировании.

Ключевым компонентом успешного проектирования является методология. Выбор подходящей методологии для каждого конкретного процесса проектирования должен быть зафиксирован в качестве одной составляющей плана.

Накопленный опыт, образование и имеющиеся решения проблем также существенно влияют на успех дела. Обработка данных в своей эволюции достигла такой точки, когда проектировщик крайне редко сталкивается с задачей, которая уже не была бы решена Частично или полностью. Например, разработчик новой опера-

ционной системы должен понимать, что уже созданы сотни операционных систем и на эту тему написан не один учебник. Проектировщик, столкнувшись с задачей сортировки, должен знать, что уже придумано и проанализировано множество алгоритмов сортировки. При разумном подходе к решению задач начинать следует с анализа своего опыта и опыта других с тем, чтобы проверить, не была ли задача уже решена. Даже если готовое решение найти не удается, вероятно, когда-то была решена близкая задача. Проектировщик системы резервирования авиабилетов может сообразить, что у нее есть много сходства с другими системами резервирования, например с системой резервирования мест в гостинице. Тогда ему, возможно, удастся выделить и применить у себя элементы решения задачи о резервировании мест в гостинице.

Если все эти методы не приносят успеха, может оказаться эффективным решение более специализированной задачи или части задачи. Если количество деталей в постановке задачи слишком велико, разработчику следует посмотреть, нельзя ли упростить ее, отбросив часть деталей. В результате либо станет ясно, как следует изменить упрощенное решение, чтобы учесть и отброшенные детали, либо удастся лучше увидеть возможные решения, так что можно будет прекратить заниматься упрощенным вариантом и начать сначала.

Выполните план

Следующий шаг —- действительно решить задачу в соответствии с запланированным подходом. Поскольку решение обычно состоит из ряда последовательных шагов, разработчик в процессе решения должен пытаться проверить правильность каждого шага.

Проанализируйте решение

После того как результат получен, нужно еще его проверить. Разработчик должен просмотреть все данные, чтобы убедиться, что учтено все, что имеет отношение к делу. Полезно для этого еще раз перечитать буквально каждое слово постановки задачи, вычеркивая каждый использованный в решении факт, а затем проверить, насколько существенно для задачи то, что осталось незачеркнутым. Разработчик должен также проверить правильность решения задачи [51].

4.5.
Требования к технологии и средствам автоматизации разработки сложных программных средств

В стандартах и моделях жизненного цикла ПС с различной глубиной определено содержание этапов и частных работ при создании и модификации компонентов и ПС в целом. Для планирования и управления обеспечением качества и надежности ПС эти модели служат структурной базой объектов, работ и документов при детализации и реализации требований к показателям качества ожидаемых результатов. Необходимая надежность объектов формируется и обеспечивается в процессе выполнения частных работ каждого этапа и окончательно удостоверяется испытаниями и документами при их завершении. Для обеспечения качества и надежности ПС стандартами *рекомендуется формулировать требования:*

- к объекту разработки на данном этапе — к его программным и информационным компонентам, а также к интерфейсу между ними и внешней средой;
- к процессу, технологии и организации выполнения совокупности работ и документов каждого этапа;
- к методам и характеристикам средств автоматизации выполнения работ, обеспечивающим необходимую надежность функционирования и качество ПС;
- к методам и средствам контроля, измерения и документирования качества процессов и результатов выполненных работ.

Выполнение этих требований должно контролироваться путем измерения объектов и процессов разработки. Измерения объектов разработки сводятся к регулярной поэтапной регистрации показателей качества, а также к сопоставлению их с заданными требованиями. При обнаружении отклонений от требований должны приниматься меры либо для улучшения реальных показателей, либо по корректировке требований к показателям для данного компонента на контролируемом этапе.

Требования к инструментальным средствам автоматизации разработки надежных ПС наиболее полно изложены в стандарте *IEEE 1209-1992.* Стандарт содержит рекомендации по оценке и выбору инструментальных средств, поддерживающих процессы

жизненного цикла программных средств, включая процессы управления проектами, процессы разработки и процессы, следующие за разработкой, а также интегральные процессы жизненного цикла ПС. Для оценки и выбора инструментальной среды и CASE-средств стандартом рекомендуется использовать приведенные ниже наборы правил и критериев. Группы критериев в стандарте выделены и сформированы с учетом общих требований стандарта ISO 9126:1991 по оценке качества программных продуктов.

Технологическую среду и CASE-средства стандартом рекомендуется описывать и выбирать в соответствии с показателями:

- соответствие стандартам среды, указанным в списке характе ристик и функций, поддерживаемых CASE-средств ом, включая стандарты на языки, базы данных, репозиторий, коммуника ции, графический интерфейс пользователя, документацию, раз работку, управление конфигурацией, безопасность, обмен ин формацией, интеграцию данных, управление или пользователь ский интерфейс;
- совместимость с другими инструментальными средствами, вклю чая возможность взаимодействия и/или прямого обмена дан ными (например, с системами подготовки текстов и другими средствами документирования, базами данных, репозитория- ми и другими CASE-средствами);
- поддержка конкретных методологий, например, объектно-ори ентированного анализа, объектно-ориентированного проекти рования, проектирования «сверху-вниз»;
- языковая поддержка, включая языки программирования, язы ки определения данных, языки структурированных запросов, графические языки;
- ввод и редактирование спецификаций требований к разраба тываемому ПС, включая требования к функциям, данным, ин терфейсам, качеству, производительности, среде функциониро вания, стоимости и планированию;
- языки спецификаций требований — возможность CASE-средств
  импортировать, экспортировать или редактировать информа цию требований, используя формальный язык, контроль не противоречивости спецификаций и полноты;
- возможность моделировать аспекты потенциального функци онирования разрабатываемой системы на основе требований и/или проектных данных, имеющихся в распоряжении CASE-

средства, включая эффективность системы, интерфейс оператора, архитектурную производительность (время отклика, загрузку, пропускную способность);
- прототипирование — возможность проектирования и генерации предварительной версии всей системы или ее части на основе требований и/или проектных данных, имеющихся в распоряжении CASE-средства;
- формирование структуры отчетов, которые будут создаваться разрабатываемой системой.

В соответствии со стандартом должен быть обеспечен *анализ потенциальной корректности и надежности* входящих в ПС программных компонентов, включающий;
- процедуры оценки сложности программ, связанной с числом вложенных циклов, полноты покрытия тестами, оценку количества остающихся ошибок;
- обратную (реверсную) инженерию, т.е. возможность ввода действующего исходного кода в одном или нескольких языках и получения из него проектных данных с предоставлением результатов пользователю;
- реструктуризацию исходного кода: ввод исходного кода в одном или нескольких языках, модифицирование его формата и/или структуры и выдачу файла исходного кода на том же самом языке;
- анализ исходного кода и предоставления результатов пользователю: измерения размеров, вычисления метрик сложности, генерации перекрестных ссылок, обзора соответствия использованным стандартам;
- отладку: поддержку идентификации *и* изоляции ошибок в программе, включая выполнение программ с трассировкой, обеспечение обратного выполнения и ловушек, идентификацию мест, где имеются ошибки, и часто выполняемых сегментов в терминах исходного кода.

Требования стандарта к средствам *управления проектом сложного ПС* включают:
- способность CASE-средства оценивать стоимость, формировать планы и другие показатели проекта по данным, вводимым пользователем;
- управление действиями и ресурсами путем поддержки ввода пользователем данных для планирования проекта, данных о фактических действиях и анализ этих данных, включая планы,

ресурсы компьютеров, назначение персонала, бюджет проекта, а также возможность определения условий выполнения проекта;
- управление тестовыми процедурами: возможность поддержки управления действиями по тестированию и тестовыми програм мами, планирования действий по тестированию, регистрации результатов тестирования, генерации отчетов о состоянии тес тируемых программ;
- управление качеством разрабатываемого ПС — ввод и обра ботка данных о качестве, их анализ и генерация отчетов об уп равлении качеством;
- управление действиями по корректировке плана проекта, отче тов о проблемах и дефектах, возникших в ходе выполнения проекта.

Управление конфигурацией версий проекта ПС должно обеспечивать:
- возможность управлять физическим доступом к элементам дан ных и их изменением, включая возможность специфицировать с помощью идентификаторов компоненты, к которым возмо жен доступ только для чтения, запрещен доступ, а также воз можность отлаживать элементы данных для их модификации, ограничивать доступ к ним до тех пор, пока они не исправлены и не проверены, и отменять ограничения после внесения изме нений;
- трассирование модификаций — запись всех модификаций, сде ланных в системе при ее разработке или сопровождении,
- управление версиями, возможность записи и выполнения фун кций управления многократными версиями системы, которые могут иметь общие компоненты;
- учет конфигурационного статуса и предоставление пользова телю отчетов, устанавливающих историю, содержимое и ста тус различных единиц конфигурации, находящихся под управ лением;
- генерацию выпусков (релизов) ПС и его компонентов, возмож ность поддержки определения пользователем шагов, необхо димых для создания версии и автоматизированного выполне ния этих шагов; ■
- возможности автоматического архивирования элементов дан ных для последующего поиска и применения.

Поддержка разработки технологической и эксплуатационной документации на комплекс программ и его компоненты по требованиям стандарта ШЕЕ 1209 должна включать:

- редактирование текстов — возможность вводить и редактиро вать данные в текстовом формате;
- графическое редактирование — ввод и редактирование данных в графическом формате;
- редактирование на базе форм — поддержка ввода и редакти рование данных в форме, определенной пользователем;
- возможности настольного издательства для оформления доку ментации;
- контроль соответствия выходных результатов CASE-средства стандартам на документацию ПС;
- автоматическое извлечение текстовых и графических данных и генерация документов, специфицированных пользователем

Критерии удобства применения CASE-средства в процессе разработки ПС включают:

- непротиворечивость пользовательского интерфейса, включая размещение и представление экранных элементов, совместно появляющихся на экране, и методы входа пользователя в сис тему:
- легкость изучения, измеряемую количеством времени и усилий, которые требуются от пользователя, чтобы понять штатные операции CASE-средства и производительно его использовать;
- адаптируемость CASE-средства силами пользователя к его спе цифичным потребностям, включая различные наборы симво лов, разные способы представления символов и графики, раз ные форматы данных, методы ввода и вывода;
- качество документации CASE-средства, включая полноту, яс ность, читаемость, полезность;
- доступность и качество учебных материалов, включая учебные материалы, доступные в режиме on-line, руководства по обуче нию, курсы обучения и визуальные материалы;
- уровень требований к знаниям пользователя, необходимым для эффективного использования CASE-средства, и легкость рабо ты с CASE-средств ом как для новичков, так и для опытных пользователей;
- общность пользовательского интерфейса между CASE-сред-ством и другими инструментальными средствами, функциони рующими в среде проектируемой системы;

- полноту и качество функций помощи в режиме «help»;
- ясность диагностики — понимаемость и полезность диагнос тических сообщений, получаемых пользователем;
- приемлемое время отклика — время, требующееся для того, что
  бы ответить на запрос пользователя в условиях применяемой операционной среды CASE-средства;
- легкость инсталляции CASE-средства, как первоначальной, так и при последующих изменениях.

Критерии оценки эффективности CASE-средства по требованиям стандарта должны учитывать данные для выполняемых объектов и работ как типичного, так и максимального размера и сложности:

- оптимальные требования к объему внешней, общей памяти, чтобы обеспечить работу с любыми требующимися и/или гене рируемыми данными на приемлемом уровне производитель ности;
- оптимальные требования к объему оперативной памяти, адре суемой центральным процессором, для того, чтобы CASE-сред ство могло загружаться и функционировать на приемлемом уровне производительности;
- оптимальные требования к процессору для функционирования CASE-средства на приемлемом уровне производительности;
- производительность, измеряемую как время, в течение которо го CASE-средство выполняет характерные задачи, например время ответа на запрос [49].

4.6. Качество программного обеспечения

Программное обеспечение является важной составляющей многих сфер жизни, используется повсеместно в промышленности, медицине, активно начинает использоваться в образовании (дистанционное образование, открытое образование). От программного обеспечения зависит не только эффективность производственного процесса, но и жизнь людей (медицина, военная, космическая сфера). По этой причине встает вопрос о качестве программного обеспечения.

Существует множество определений качества, в основе понятия качества продукта или услуги лежит идея об удовлетворении

потребностей конечного пользователя — реального или потенциального потребителя. Вот определение этого понятия в соответствии со стандартом ISO 8402:1994.

Качество — совокупность характеристик объекта, относящихся к его способности удовлетворить установленные и предполагаемые потребности.

Можно выделить три большие группы факторов, влияющих на качество программного обеспечения:

- *функциональная* — связана с полнотой и удобством использо вания реализованных функций программного средства;
- *административная* — связана с квалификацией персонала, орга
  низационной структурой и управлением персоналом;
- *программно-архитектурная* — связана с процессом разработ ки программного обеспечения, выбранными методологиями, инструментальными средствами, использованными на различ ных этапах жизненного цикла программного обеспечения, а так же архитектурой программного средства.

Современная техника управления качеством (например, концепция Total Quality Management (TQM)) базируется именно на управлении качеством. На современном этапе уже недостаточно иметь только методы оценки качества произведенного и используемого программного средства (выходной контроль), необходимо иметь возможность планировать качество, измерять его на всех этапах жизненного цикла программного средства и корректировать процесс производства программного обеспечения для улучшения качества. Международные стандарты серии ISO 9000 регламентируют создание системы управления качеством. Однако они являются общими, лишь рекомендательными. Каждая компания, производящая программное обеспечение и желающая внедрить у себя действенную систему управления качеством на основе стандартов ISO 9000-й серии, должна учесть специфику своей отрасли и разработать систему показателей качества, которая бы отражала реальное влияние факторов качества на программный продукт.

Программное обеспечение как продукт имеет некоторые отличия от других промышленных продуктов:

- наращивание объемов выпуска какого-то вида программного продукта происходит практически мгновенно и имеет низкую стоимость, так как производство следующей единицы програм много продукта связано только с копированием информации на носитель (компактдиск, дискету или жесткий диск);

- большие ресурсы затрачиваются на стадии планирования, реа лизации и тестирования;
- сильное влияние человеческого фактора на производство про граммного продукта, так как производство программного про дукта — интеллектуальная и творческая деятельность;
- в жизненном цикле программного продукта, как правило, от сутствует этап утилизации;
- программный продукт не подвержен физическому старению, а только моральному.

Все эти, а также многие другие особенности должны быть уч-тены в программе оценки качества и управления качеством.

Сейчас остро стоит задача измерения качества программного обеспечения с целью оперативного воздействия на процесс производства программного продукта. Для измерения некоторых показателей качества могут служить тестирование, тестирование пользователем (так называемое |3-тестирование), а также информация от пользователя о найденных проблемах, получаемая от службы технической поддержки. Вышеперечисленные действия дают обильную пищу для анализа (выраженную в количественных единицах, а значит, измеряемую). Главное — найти между ними зависимости (например, зависимость количества ошибок, обнаруженных специалистом по тестированию, и количества ошибок, зафиксированных пользователем, может служить показателем надежности программного средства), тогда можно будет говорить об измерении качества программного средства.

При построении системы качества могут быть использованы математические методы: методы корреляционного анализа (для выяснения выявления зависимости и тесноты связи между отдельными свойствами программного продукта и степенью удовлетворения пользователя), методы факторного анализа (для построения функции качества), методы кластеризации.

Сегодня наступил этап планирования качества программного обеспечения, мониторинга качества и управления им в процессе производства. Заинтересованность пользователя и производителя программных средств есть; аппарат для управления качеством программного обеспечения разрабатывается зарубежными и российскими учеными.

Мероприятия, обеспечивающие приемлемый уровень качества программного средства, можно условно разделить на административные и технологические.

К административным можно отнести следующие мероприяти

1. Проведение обучения персонала, переподготовки.

2. Тщательное документирование всех изменений в структуре программного средства. Для этого используются средства под держки версионности.

3. Назначение ответственных лиц за каждую доработку про граммного средства.

4. Уделение внимания текущему контролю качества и заклю чительному контролю качества.

5. Обеспечение мониторинга качества, например, фиксирова ние ошибок, поступивших от пользователя программного сред ства. Использование систематических испытательных методов, где испытания будут разработаны параллельно с разработкой про граммы.

6. Введение внутренних стандартов. Такие стандарты обычно содержат соглашения о именовании переменных в программном коде, наименовании файлов данных, процедур и функций.

7. Организация отдела тестирования как самостоятельного подразделения.

8. Проведение совместных аттестаций с пользователем.

9. Обращение внимания на уровень и простоту обслуживае мости программного обеспечения. Здесь речь идет как о решении проблем, возникших у пользователя, так и о простоте и надеж ности внесения изменений в программное обеспечение. Даже если очень надежный кусок программного обеспечения был разрабо тан, он может вскоре стать ненадежным, если сложно сделать из менения в программе. Часто изменения должны быть выполнены вследствие новых потребностей внешней среды, например вслед ствие изменений в законодательстве или требований заказчика.

К технологическим относятся следующие мероприятия.

1. Выбор стандарта качества и четкое следование ему на всех этапах. Создание модели проекта с регулярными проверками, которые будут выполняться независимыми командами эксперти зы. Такая модель может быть построена, например, на основе стандартов качества (например, ISO 9000).

2. Единая среда разработки. Лучшие результаты дают про граммные продукты разработки, которые поддерживают несколь ко или все этапы жизненного цикла программного обеспечения. На данный момент такими комплексными решениями являются, например, продукты Oracle Designer, продукты фирмы Rational.

3. Использовать формальный язык спецификаций (например, UML, DESIGN IDEF).

4. Выбор надежной СУБД (если программное средство работает с массивами информации и использование СУБД оправдано).

5. Тщательное тестирование программного обеспечения.

6. Широкое внедрение автоматизации тестирования.

7. Использование полностью проверенной программной среды окружения (ОС) и языка программирования, которые минимизируют опасность внесения ошибки.

8. Использование статистических методов для сбора информации о качестве ПС.

9. Изучение результатов испытаний (тестов) и ошибок для использования в постоянном усовершенствовании программы. Источник в случае возникновения отказа должен быть найден и устранен. Недостаточно найти ошибку в программном обеспечении и исправить ее. Изменения должны быть сделаны в процессе разработки ПО.

10. Использование испытательной среды, которая предостерегает от передачи пользователю ненадежного программного обеспечения. Создание автоматических средств приемки.

В книге мы не будем давать всестороннюю картину контроля качества и действий по его улучшению в связи с разработкой программного обеспечения. Это было бы почти невозможно из-за широты области, кроме того, однородной картины в этой области нет.

Как видно, качество программного обеспечения тесно связано с жизненным циклом программного обеспечения, с тестированием, речь о котором пойдет в следующей главе. Качество является комплексной проблемой.

ВОПРОСЫ ДЛЯ САМОПРОВЕРКИ

1. Дайте определение понятию «надежность» согласно ГОСТ 13377—75.
2. Какими факторами характеризуется надежность программного средства?
3. Назовите основные характеристики качества программного средства по стандарту ISO 9126:1991.

4. Назовите основные факторы, влияющие на надежность програм много средства.
5. Охарактеризуйте внутренние и внешние дестабилизирующие фак торы.
6. Опишите основные методы обеспечения надежности программно го средства.
7. Что представляет собой термин «модель надежности программно го средства»?
8. В чем заключается различие между аналитическими и эмпиричес кими моделями надежности программного средства?
9. Объясните основные различия между статическими и динамичес кими аналитическими моделями.
10. Каково влияние сложности программных средств на обеспечение их качества и надежности?
11. Назовите основные группы факторов, влияющих на качество про граммного обеспечения.

ГЛАВА 5 ТЕСТИРОВАНИЕ ПРОГРАММНОГО СРЕДСТВА

Многие организации, занимающиеся созданием программного обеспечения, до 50% средств, выделенных на разработку программ, тратят на тестирование, что составляет миллиарды долларов по всему миру в целом. И все же, несмотря на громадные капиталовложения, знаний о сути тестирования явно не хватает, и большинство программных продуктов неприемлемо, ненадежно даже после «основательного тестирования».

О состоянии дел лучше всего свидетельствует тот факт, что большинство людей, работающих в области обработки данных, даже не могут правильно определить понятие «тестирование», и это на самом деле главная причина неудач. Если попросить любого профессионала определить понятие «тестирование» либо открыть (как правило, слишком краткую) главу о тестировании любого учебника программирования, то скорее всего можно встретить такое определение:

«Тестирование — процесс, подтверждающий правильность программы и демонстрирующий, что ошибок в программе нет». Основной недостаток подобного определения заключается в том, что оно совершенно неправильно; фактически это почти определение антонима слова «тестирование». Поэтому определение описывает невыполнимую задачу, а так как тестирование зачастую все же выполняется с успехом, по крайней мере с некоторым успехом, то такое определение логически некорректно. Правильное определение тестирования таково:

Тестирование — процесс выполнения программы с намерением найти ошибки.

Тестирование оказывается довольно необычным процессом (вот почему оно и считается трудным), так как это процесс разрушительный. Ведь цель проверяющего (тестовика) — заставить программу сбиться. Он доволен, если это ему удается; если же программа на его тесте не сбивается, он не удовлетворен.

Невозможно гарантировать отсутствие ошибок в программе; в лучшем случае можно попытаться показать наличие ошибок. Если программа правильно ведет себя для значительного набора тестов, нет оснований утверждать, что в ней нет ошибок; со всей определенностью можно лишь утверждать, что неизвестно, когда эта программа не работает. Конечно, если есть причины считать данный набор тестов способным с большой вероятностью обнаружить все возможные ошибки, то можно говорить о некотором уровне уверенности в правильности программы, устанавливаемой этими тестами.

О тестировании говорить довольно трудно, поскольку, хотя оно и способствует повышению надежности программного обеспечения, его значение ограничено. Надежность невозможно внести в программу в результате тестирования, она определяется правильностью этапов проектирования. Наилучшее решение проблемы надежности — с самого начала не допускать ошибок в программе. Однако вероятность того, что удастся безупречно спроектировать большую программу, бесконечно мала. Роль тестирования состоит как раз в том, чтобы определить местонахождение немногочисленных ошибок, оставшихся в хорошо спроектированной программе. Попытки с помощью тестирования достичь надежности плохо спроектированной программы совершенно бесплодны.

5.1. Основные определения

Хотя в тестировании можно выделить несколько различных процессов, такие термины, как «тестирование», «отладка», «доказательство», «контроль» и «испытание», часто используются как синонимы и, к сожалению, для разных людей имеют разный смысл. Нашу классификацию различных форм тестирования мы начнем с того, чго дадим эти определения, слегка их дополнив и расширив их список.

Тестирование (testing) —процесс выполнения программы (или части программы) с намерением (или целью) найти ошибки.

Доказательство (proof) — попытка найти ошибки в программе безотносительно к внешней для программы среде. Большинство методов доказательства предполагает формулировку утвер-

ждений о поведении программы и затем вывод и доказательство математических теорем о правильности программы. Доказательства могут рассматриваться как форма тестирования, хотя они и не предполагают прямого выполнения программы. Многие исследователи считают доказательство альтернативой тестированию — взгляд во многом ошибочный.

Контроль (verification) — попытка найти ошибки, выполняя программу в тестовой, или моделируемой, среде.

Испытание (validation) — попытка найти ошибки, выполняя программу в заданной реальной среде.

Аттестация (certification) — авторитетное подтверждение правильности программы. При тестировании с целью аттестации выполняется сравнение с некоторым заранее определенным стандартом.

Отладка (debugging) не является разновидностью тестирования. Хотя слова «отладка» и «тестирование» часто используются как синонимы, под ними подразумеваются разные виды деятельности. Тестирование — деятельность, направленная на обнаружение ошибок; отладка направлена на установление точной природы известной ошибки, а затем — на исправление этой ошибки. Эти два вида деятельности связаны — результаты тестирования являются исходными данными для отладки.

Эти определения представляют один взгляд на тестирование — со стороны среды, на которую оно опирается. Другой ряд определений, приведенный ниже, охватывает вторую сторону тестирования: типы ошибок, которые предполагается обнаружить, и стандарты, с которыми сопоставляются тестируемые программы.

Тестирование модуля, или автономное тестирование (module testing, unit testing), — контроль отдельного программного модуля, обычно в изолированной среде (т. е. изолированно от всех остальных модулей). Тестирование модуля иногда включает также математическое доказательство.

Тестирование сопряжений (integration testing) — контроль сопряжений между частями системы (модулями, компонентами; подсистемами).

Тестирование внешних функций (external function testing) — контроль внешнего поведения системы, определенного внешними спецификациями.

Комплексное тестирование (system testing) — контроль и/или испытание системы по отношению к исходным целям. Комплекс-

ное тестирование является процессом контроля, если оно выполняется в моделируемой среде, и процессом испытания, если выполняется в среде реальной, жизненной.

Тестирование приемлемости (acceptance testing) — проверка соответствия программы требованиям пользователя.

Тестирование настройки (installation testing) — проверка соответствия каждого конкретного варианта установки системы с целью выявить любые ошибки, возникшие в процессе настройки системы.

5.2. Экономика тестирования

Дав такое определение тестированию, необходимо на следующем шаге рассмотреть возможность создания теста, обнаруживающего все ошибки программы. Покажем, что ответ будет отрицательным даже для самых тривиальных программ. В общем случае невозможно обнаружить все ошибки программы. А это в свою очередь порождает экономические проблемы, задачи, связанные с функциями человека в процессе отладки, способы построения тестов.

Тестирование программы как «черного ящика»

Одним из способов изучения поставленного вопроса является исследование стратегии тестирования, называемой стратегией «черного ящика», *тестированием с управлением по данным* или *тестированием с управлением по входу-выходу.* При использовании этой стратегии программа рассматривается как «черный ящик». Иными словами, такое тестирование имеет целью выяснение обстоятельств, в которых поведение программы не соответствует спецификации. Тестовые же данные используются только в соответствии со спецификацией программы (т. е. без учета знаний о ее внутренней структуре).

При таком подходе обнаружение всех ошибок в программе является критерием *исчерпывающего входного тестирования.* Последнее может быть достигнуто, если в качестве тестовых наборов использовать все возможные наборы входных данных.

Если такое испытание представляется сложным, то еще сложнее создать исчерпывающий тест для большой программы. Образно говоря, число тестов можно оценить «числом большим, чем бесконечность».

Построение исчерпывающего входного теста невозможно. Это подтверждается двумя аргументами: во-первых, нельзя создать тест, гарантирующий отсутствие ошибок; во-вторых, разработка таких тестов противоречит экономическим требованиям. Поскольку исчерпывающее тестирование исключается, нашей целью должна стать максимизация результативности капиталовложений в тестирование (иными словами, максимизация числа ошибок, обнаруживаемых одним тестом). Для этого мы можем рассматривать внутреннюю структуру программы и делать некоторые разумные, но, конечно, не обладающие полной гарантией достоверности предположения.

Тестирование программы как «белого ящика»

Стратегия *«белого ящика»,* или стратегия тестирования, *управляемого логикой программы,* позволяет исследовать внутреннюю структуру программы. В этом случае тестирующий получает тестовые данные путем анализа логики программы (к сожалению, здесь часто не используется спецификация программы).

Сравним способ построения тестов при данной стратегии с исчерпывающим входным тестированием стратегии «черного ящика». Непосвященному может показаться, что достаточно построить такой набор тестов, в котором каждый оператор исполняется хотя бы один раз; нетрудно показать, что это неверно. Не вдаваясь в детали, укажем лишь, что исчерпывающему входному тестированию может быть поставлено в соответствие *исчерпывающее тестирование маршрутов.* Подразумевается, что программа проверена полностью, если с помощью тестов удается осуществить выполнение программы по всем возможным маршрутам ее потока (графа) передач управления.

Последнее утверждение имеет два слабых пункта. Первый из них состоит в том. что число не повторяющих друг друга маршрутов в программе — астрономическое. Второй слабый пункт утверждения заключается в том, что, хотя исчерпывающее тестирование маршрутов является полным тестом и хотя каждый маршрут программы может быть проверен, сама программа будет содержать ошибки. Это объясняется следующим образом.

Во-первых, исчерпывающее тестирование маршрутов не может дать гарантии того, что программа соответствует описанию. Например, вместо требуемой программы сортировки по возрастанию случайно была написана программа сортировки по убыванию. В этом случае ценность тестирования маршрутов невелика, поскольку после тестирования в программе окажется одна ошибка, т. е. программа неверна.

Во-вторых, программа может быть неверной в силу того, что пропущены некоторые маршруты. Исчерпывающее тестирование маршрутов не обнаружит их отсутствия.

В-третьих, исчерпывающее тестирование маршрутов не может обнаружить ошибок, *появление которых зависит от обрабатываемых данных.*

5.3. Аксиомы (принципы) тестирования

Сформулируем основные принципы тестирования, используя главную предпосылку настоящей главы о том, что наиболее важными в тестировании программ являются вопросы психологии. Эти принципы интересны тем, что в основном они интуитивно ясны, но в то же время на них часто не обращают должного внимания.

Хорош тот тест, для которого высока вероятность обнаружить ошибку. Эта аксиома является фундаментальным принципом тестирования. Поскольку невозможно показать, что программа не имеет ошибок и, значит, все такие попытки бесплодны, процесс тестирования должен представлять собой попытки обнаружить в программе прежде не найденные ошибки.

Одна из самых сложных проблем при тестировании — решить, когда нужно его закончить. Как уже говорилось, исчерпывающее тестирование (т. е. испытание всех входных значений) невозможно. Таким образом, при тестировании мы сталкиваемся с экономической проблемой: как выбрать конечное число тестов, которое дает максимальную отдачу (вероятность обнаружения ошибок) для данных затрат. Известно слишком много случаев, когда написанные тесты имели крайне малую вероятность обнаружения новых ошибок, в то время как довольно очевидные хорошие тесты оставались незамеченными.

Не нужно тестировать свою собственную программу. Ни один программист не должен пытаться тестировать свою собственную программу. Это относится ко всем формам тестирования — как к тестированию системы, так и к тестированию внешних функций и даже отдельных модулей. Тестирование должно быть в высшей степени разрушительным процессом, но имеются глубокие психологические причины, по которым программист не может относиться к своей собственной программе как разрушитель. Дополнительное давление (например, жесткий график) на отдельного программиста или весь коллектив разработчиков проекта часто мешает программисту или всему коллективу выполнить адекватное тестирование. Если модуль содержит дефекты вследствие каких-то ошибок перевода, довольно высока вероятность того, что программист допустит ту же ошибку перевода (например, неправильно интерпретирует спецификации) и при подготовке тестов. Все ошибки в его понимании других модулей и их сопряжении также отразятся на тестах.

Тестирование всегда должна выполнять внешняя группа, которая в некотором смысле стоит особняком от программиста и проекта. Вместо того чтобы выполнять автономное тестирование модулей самостоятельно, программист должен иметь набор тестов, подготовленных разработчиком одного из модулей, вызывающих тестируемый модуль (а может быть, даже выполненных на машине и проверенных этим разработчиком). Комплексное тестирование всегда должно выполняться независимой группой, например специальным отделом обеспечения качества, или группой пользователей-добровольцев.

Отсюда вовсе не следует, что программист не может тестировать свою программу. Многие программисты с этим вполне успешно справляются. Здесь лишь делается вывод о том, что тестирование является более эффективным, если оно выполняется кем-либо другим. Заметим, что все наши рассуждения не относятся к отладке, т. е. к исправлению уже известных ошибок. Эта работа эффективнее выполняется самим автором программы.

Необходимая часть всякого теста — описание ожидаемых выходных данных или результатов. Одна из самых распространенных ошибок при тестировании состоит в том, что результаты каждого теста не прогнозируются до его выполнения. Ожидаемые результаты нужно определять заранее, чтобы не возникла ситуация, когда «глаз видит то, что хочет увидеть». Чтобы со-

всем исключить такую возможность, лучше разрабатывать самопроверяющиеся тесты либо пользоваться инструментами тестирования, способными автоматически сверять ожидаемые и фактические результаты.

Хотя эта аксиома чрезвычайно важна, иногда, например, при тестировании математического программного обеспечения приходится допускать исключения. Математическое программное обеспечение обладает тем свойством, что выходные данные являются только приближением правильного результата. Это происходит из-за использования конечных вычислительных процессов вместо бесконечных математических процессов, из-за ошибок округления, связанных с конечной точностью машинной арифметики и неточностью представления чисел, а также ошибок из-за конечной точности представления входных данных и констант. Поэтому во многих случаях оказывается важной не абсолютная точность, а корреляция ошибок. Например, когда математическая программа возвращает массив чисел, может оказаться важным, чтобы вычисленное решение было точным решением для набора входных данных, аппроксимирующего реальные выходные данные. Поэтому при тестировании математического программного обеспечения прогнозирование точных выходных данных затруднительно.

Избегайте невоспроизводимых тестов, не тестируйте «слету». Использование диалоговых систем иногда мешает хорошему тестированию. Для того чтобы тестировать программу в пакетной системе, программист обычно должен оформить тест в виде специальной ведущей программы или в виде файла тестовых данных. В условиях диалога программист слишком часто выполняет тестирование «с лету», т. е. сидя за терминалом, задает конкретные значения и выполняет программу, чтобы «посмотреть, что получится». Это неряшливая и по многим причинам нежелательная форма тестирования. Основной ее недостаток в гом, что такие тесты мимолетны; они исчезают по окончании их выполнения. Всякий раз, когда программу понадобится тестировать повторно (например, после исправления ошибок или после модификации), придется придумывать тесты заново.

Тестирование обходится слишком дорого и без этих дополнительных расходов. Никогда не используйте тестов, которые тут же выбрасываются. Тесты следует документировать и хранить в такой форме, чтобы каждый мог использовать их повторно.

Готовьте тесты как для правильных, так и для неправильных входных данных. Многие программисты ориентируются в своих тестах на «разумные» условия на входе, забывая о последствиях появления непредусмотренных или ошибочных входных данных. Однако многие ошибки, которые потом неожиданно обнаруживаются в работающих программах, проявляются вследствие никак не предусмотренных действий пользователя программы. Тесты, представляющие неожиданные или неправильные входные данные, часто лучше обнаруживают ошибки, чем правильные тесты.

Детально изучите результаты каждого теста. По всей вероятности, это наиболее очевидный принцип, но и ему часто не уделяется должного внимания. Самые изощренные тесты ничего не стоят, если их результаты удостаиваются лишь беглого взгляда. Тестирование программы означает большее, нежели выполнение достаточного количества тестов; оно также предполагает изучение результатов каждого теста. «Да, я уже проверял такую ситуацию, но *такого* как-то не заметил в выдаче», — довольно распространенная реакция программиста на обнаруженную новую ошибку.

По мере того как число ошибок, обнаруженных в некотором компоненте программного обеспечения, увеличивается, растет относительная вероятность существования в нем необнаруженных ошибок. Этот противоречащий интуиции феномен, иллюстрируемый рис. 5.1, означает, что ошибки образуют кластеры, т. е. встречаются группами. С ростом числа ошибок, обнаруженных в компоненте программы (например, в модуле, подсистеме, функции пользователя), увеличивается также вероятность существования в этом компоненте еще не обнаруженных ошибок. Если при тестировании двух модулей в них обнаружены одна и восемь ошибок соответственно, кривая на рис. 5.1 показывает, что для модуля с восьмью ошибками вероятность того, что в нем еще есть ошибки, выше.

Поручайте тестирование самым способный программистам. Тестирование и в особенности проектирование тестов — этап в разработке программного обеспечения, особенно требующий творческого подхода. К сожалению, во многих организациях на тестирование смотрят совсем не так. Его часто считают рутинной, нетворческой работой, вследствие чего коллективы, занимающиеся тестированием, укомплектованы в основном неопытными или мо-

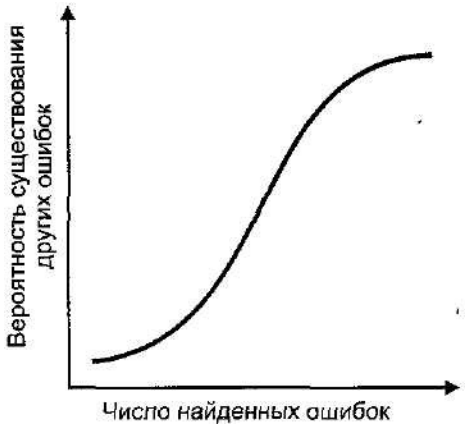


Рис. 5.1. График соотношения между обнаруженными и необнаруженными ошибками

лодыми программистами. Однако практика показывает, что более правильным было бы сделать наоборот. Проектирование тестов требует даже больше творчества, чем разработка архитектуры и проектирование программного обеспечения.

Считайте тестируемость ключевой задачей вашей разработки.

Проект системы должен быть таким, чтобы каждый модуль подключался к системе только один раз. Множество проблем во многих больших программных системах возникает из-за нарушения этой аксиомы. Ситуация, когда во время цикла тестирования большой системы некоторые модули приходится подключать больше десяти раз, — не редкость. Каждая версия такого модуля содержит еще одну маленькую дополнительную функцию, необходимую для текущего уровня системы. Если следовать правилам и проектировать небольшие модули, каждый из которых выполняет отдельную функцию, бороться с этой проблемой станет легче.

Никогда не изменяйте программу, чтобы облегчить ее тестирование. Часто возникает соблазн изменить программу, чтобы было легче ее тестировать. Например, программист, тестируя модуль, содержащий цикл, который должен повторяться 100 раз, меняет его так, чтобы цикл повторялся только 10 раз. Может быть, этот программист и занимается тестированием, но только другой программы.

Тестирование, как почти всякая другая деятельность, должно начинаться с постановки целей. Как уже говорилось, цикл тестирования подобен полному циклу разработки программного обеспечения. Тесты должны быть спроектированы, реализованы, проверены и, наконец, выполнены. Поэтому задачи тестирования должны быть сформулированы на каждом его этапе, например, для каждого конкретного типа тестирования должны быть определены ориентиры (число пройденных путей, проверенных условных переходов и т. п.) и доля ошибок, которые должны быть обнаружены на этом этапе.

Приведем еще раз три наиболее важных принципа тестирования.

1. *Тестирование — это процесс выполнения программ с целью обнаружения ошибок,*

2. *Хорошим считается тест, который имеет высокую вероятность обнаружения еще не выявленной ошибки.*

3. *Удачным считается тест, который обнаруживает еще не выявленную ошибку.*

5.4. Философия тестирования

Тестирование программного обеспечения охватывает ряд видов деятельности, аналогичный последовательности процессов разработки программного обеспечения. Сюда входят постановка задачи для теста, проектирование, написание тестов, тестирование тестов, выполнение тестов и изучение результатов тестирования. Решающую роль играет проектирование теста. Возможен целый спектр подходов к выработке философии, или стратегии проектирования. Чтобы ориентироваться в стратегиях проектирования тестов, стоит рассмотреть два крайних подхода, находящихся на границах спектра. Следует отметить также, что многие из тех, кто работает в этой области, часто бросаются в одну или другую крайность.

Сторонник подхода, соответствующего левой границе спектра (рис. 5.2), проектирует свои тесты, исследуя внешние спецификации или спецификации сопряжения программы или модуля, которые он тестирует. Программу он рассматривает как «черный ящик». Позиция его такова: «Меня не интересует, как выглядит эта программа и выполнил ли я все команды или все пути.

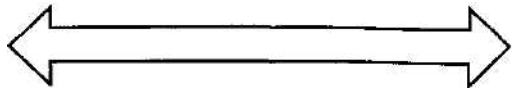


Рис. 5.2. Схема спектра подходов к проектированию тестов

Я буду удовлетворен, если программа будет вести себя так, как указано в спецификациях». Его идеал — проверить все возможные комбинации и значения на входе.

Приверженец подхода, соответствующего другому концу спектра, проектирует свои тесты, изучая логику программы. Он начинает с того, что стремится подготовить достаточное число тестов для того, чтобы каждая команда была выполнена по крайней мере один раз. Если он немного более искушен, то проектирует тесты так. чтобы каждая команда условного перехода выполнялась в каждом направлении хотя бы раз. Его идеал — проверить каждый путь, каждую ветвь алгоритма. При этом его совсем (или почти совсем) не интересуют спецификации.

Ни одна из этих крайностей не является хорошей стратегией. Первая из них. а именно та, в соответствии с которой программа рассматривается как «черный ящик», предпочтительней. К сожалению, она страдает тем недостатком, что совершенно неосуществима. Рассмотрим попытку тестирования тривиальной программы, получающей на входе три числа и вычисляющей их среднее арифметическое. Тестирование этой программы для всех значений входных данных невозможно. Даже для машины с относительно низкой точностью вычислений количество тестов исчислялось бы миллиардами. Даже имея вычислительную мощность, достаточную для выполнения всех тестов в разумное время, мы потратили бы на несколько порядков больше времени для того, чтобы эти тесты подготовить, а затем проверить. Такие программы, как системы реального времени, операционные системы и программы управления данными, которые сохраняют «память» о предыдущих входных данных, еще хуже. Нам потребовалось бы тестировать программу не только для каждого входного значения, но и для каждой последовательности, каждой комбинации входных данных. Поэтому исчерпывающее тестирование для всех входных данных любой программы неосуществимо.

Эти рассуждения приводят ко второму фундаментальному принципу тестирования: *тестирование —- проблема в значительной степени экономическая.* Поскольку исчерпывающее тестирование невозможно, мы должны ограничиться чем-то меньшим. Каждый тест должен давать максимальную отдачу по сравнению с нашими затратами. Эта отдача измеряется вероятностью того, что тест выявит не обнаруженную прежде ошибку. Затраты измеряются временем и стоимостью подготовки, выполнения и проверки результатов теста. Считая, что затраты ограничены бюджетом и графиком, можно утверждать, что искусство тестирования, по существу, представляет собой искусство отбора тестов с максимальной отдачей. Каждый тест должен быть представителем некоторого класса входных значений, так чтобы его правильное выполнение создавало у нас некоторую убежденность в том, что для определенного класса входных данных программа будет выполняться правильно. Это обычно требует некоторого знания алгоритма и структуры программы, и мы, таким образом, смещаемся к правому концу спектра.

5.5. Тестирование модулей

Вторым по важности аспектом тестирования после проектирования тестов является последовательность слияния всех модулей в систему или программу. Эта сторона вопроса обычно не получает достаточного внимания и часто рассматривается слишком поздно. Выбор этой последовательности, однако, является одним из самых жизненно важных решений, принимаемых на этапе тестирования, поскольку он определяет форму, в которой записываются тесты, типы необходимых инструментов тестирования, последовательность программирования модулей, а также тщательность и экономичность всего этапа тестирования. По этой причине такое решение должно приниматься на уровне проекта в целом и на достаточно ранней его стадии.

Тестирование модулей (или блоков) представляет собой процесс тестирования отдельных подпрограмм или процедур программы. Здесь подразумевается, что, прежде чем начинать тестирование программы в целом, следует протестировать отдельные небольшие модули, образующие эту программу. Такой подход

мотивируется тремя причинами. Во-первых, появляется возможность управлять комбинаторикой тестирования, поскольку первоначально внимание концентрируется на небольших модулях программы. Во-вторых, облегчается задача отладки программы, т.е. обнаружение места ошибки и исправление текста программы. В-третьих, допускается параллелизм, что позволяет одновременно тестировать несколько модулей.

Цель тестирования модулей — сравнение функций, реализуемых модулем, со спецификациями его функций или интерфейса.

Тестирование модулей в основном ориентировано на принцип «белого ящика». Это объясняется прежде всего тем, что принцип «белого ящика» труднее реализовать при переходе в последующем к тестированию более крупных единиц, например программ в целом. Кроме того, последующие этапы тестирования ориентированы на обнаружение ошибок различного типа, т. е. ошибок, не обязательно связанных с логикой программы, а возникающих, например, из-за несоответствия программы требованиям пользователя.

Имеется большой выбор возможных подходов, которые могут быть использованы для слияния модулей в более крупные единицы. В большинстве своем они могут рассматриваться как варианты шести основных подходов, описанных ниже.

Пошаговое тестирование

Реализация процесса тестирования модулей опирается на два ключевых положения: построение эффективного набора тестов и выбор способа, посредством которого модули комбинируются при построении из них рабочей программы. Второе положение является важным, так как оно задает форму написания тестов модуля, типы средств, используемых при тестировании, порядок кодирования и тестирования модулей, стоимость генерации тестов и стоимость отладки. Рассмотрим два подхода к комбинированию модулей: пошаговое и монолитное тестирование.

Возникает вопрос: «Что лучше — выполнить по отдельности тестирование каждого модуля, а затем, комбинируя их, сформировать рабочую программу или же каждый модуль для тестирования подключать к набору ранее оттестированных модулей?». Первый подход обычно называют *монолитным методом,* или *методом «большого удара»,* при тестировании и сборке программы;

второй подход известен как *пошаговый метод* тестирования или сборки.

Метод пошагового тестирования предполагает, что модули тестируются не изолированно друг от друга, а подключаются поочередно для выполнения теста к набору уже ранее оттестированных модулей. Пошаговый процесс продолжается до тех пор, пока к набору оттестированных модулей не будет подключен последний модуль.

Детального разбора обоих методов мы делать не будем, приведем лишь некоторые общие выводы.

1. Монолитное тестирование требует больших затрат труда. При пошаговом же тестировании «снизу-вверх» затраты труда сокращаются.

2. Расход машинного времени при монолитном тестировании меньше.

3. Использование монолитного метода предоставляет боль шие возможности для параллельной организации работы на на чальной фазе тестирования (тестирования всех модулей одновре менно). Это положение может иметь важное значение при вы полнении больших проектов, в которых много модулей и много исполнителей, поскольку численность персонала, участвующего в проекте, максимальна на начальной фазе.

4. При пошаговом тестировании раньше обнаруживаются ошибки в интерфейсах между модулями, поскольку раньше начи нается сборка программы. В противоположность этому при мо нолитном тестировании модули «не видят друг друга» до после дней фазы процесса тестирования.

5. Отладка программ при пошаговом тестировании легче. Если есть ошибки в межмодульных интерфейсах, а обычно так и быва ет, то при монолитном тестировании они могут быть обнаруже ны лишь тогда, когда собрана вся программа. В этот момент ло кализовать ошибку довольно трудно, поскольку она может на ходиться в любом месте программы. Напротив, при пошаговом тестировании ошибки такого типа в основном связаны с тем мо дулем, который подключается последним.

6. Результаты пошагового тестирования более совершенны.

В заключение отметим, что п. 1, 4, 5, 6 демонстрируют преимущества пошагового тестирования, а п. 2 и 3 — его недостатки. Поскольку для современного этапа развития вычислительной техники характерны тенденции к уменьшению стоимости аппа-

ратуры и увеличению стоимости труда, последствия ошибок в математическом обеспечении весьма серьезны, а стоимость устранения ошибки тем меньше, чем раньше она обнаружена; преимущества, указанные в п. 1, 4, 5, 6, выступают на первый план. В то же время ущерб, наносимый недостатками (п. 2 и 3), невелик. Все это позволяет нам сделать вывод, что *пошаговое тестирование является предпочтительным.*

Убедившись в преимуществах пошагового тестирования перед монолитным, исследуем две возможные стратегии тестирования: *нисходящее* и *восходящее.* Прежде всего внесем ясность в терминологию.

Во-первых, термины «нисходящее тестирование», «нисходящая разработка», «нисходящее проектирование» часто используются как синонимы. Действительно, термины «нисходящее тестирование» и «нисходящая разработка» являются синонимами (в том смысле, что они подразумевают определенную стратегию при тестировании и создании текстов модулей), но нисходящее проектирование — это совершенно иной и независимый процесс. Программа, спроектированная нисходящим методом, может тестироваться и нисходящим, и восходящим методами.

Во-вторых, восходящая разработка, или тестирование, часто отождествляется с монолитным тестированием. Это недоразумение возникает из_-за того, что начало восходящего тестирования идентично монолитному при тестировании нижних или терминальных модулей. Но выше мы показали, что восходящее тестирование на самом деле представляет собой пошаговую стратегию.

Восходящее тестирование

При восходящем подходе программа собирается и тестируется «снизу вверх». Только модули самого нижнего уровня («терминальные» модули; модули, не вызывающие других модулей) тестируются изолированно, автономно. После того как тестирование этих модулей завершено, вызов их должен быть так же надежен, как вызов встроенной функции языка или оператор присваивания. Затем тестируются модули, непосредственно вызывающие уже проверенные. Эти модули более высокого уровня тестируются не автономно, а вместе с уже проверенными модулями более низкого уровня. Процесс повторяется до тех пор, пока не будет

достигнута вершина. Здесь завершается и тестирование модулей, и тестирование сопряжений программы.

При восходящем тестировании для каждого модуля необходим *драйвер:* нужно подавать тесты в соответствии с сопряжением тестируемого модуля. Одно из возможных решений — написать для каждого модуля небольшую ведущую программу. Тестовые данные представляются как «встроенные» непосредственно в эту программу переменные и структуры данных, и она многократно вызывает тестируемый модуль, с каждым вызовом передавая ему новые тестовые данные. Имеется и лучшее решение: воспользоваться программой тестирования модулей —- это инструмент тестирования, позволяющий описывать тесты на специальном языке и избавляющий от необходимости писать драйверы.

Здесь отсутствуют проблемы, связанные с невозможностью или трудностью создания всех тестовых ситуаций, характерные для нисходящего тестирования. Драйвер как средство тестирования применяется непосредственно к тому модулю, который тестируется, где нет промежуточных модулей, которые следует принимать во внимание. Анализируя другие проблемы, возникающие при нисходящем тестировании, можно заметить, что при восходящем тестировании невозможно принять неразумное решение о совмещении тестирования с проектированием программы, поскольку нельзя начать тестирование до тех пор, пока не спроектированы модули нижнего уровня. Не существует также и трудностей с незавершенностью тестирования одного модуля при переходе к тестированию другого, потому что при восходящем тестировании с применением нескольких версий заглушки нет сложностей с представлением тестовых данных.

Нисходящее тестирование

Нисходящее тестирование (называемое также нисходящей разработкой) не является полной противоположностью восходящему, но в первом приближении может рассматриваться как таковое. При нисходящем подходе программа собирается и тестируется «сверху вниз». Изолированно тестируется только головной модуль. После того как тестирование этого модуля завершено, с ним соединяются (например, редактором связей) один за другим модули, непосредственно вызываемые им, и тестируется получен-

ная комбинация. Процесс повторяется до тех пор, пока не будут собраны и проверены все модули.

При этом подходе немедленно возникают два вопроса: 1. «Что делать, когда тестируемый модуль вызывает модуль более низкого уровня (которого в данный момент еще не существует)?» и 2. «Как подаются тестовые данные?»

Ответ на первый вопрос состоит в том, что для имитации функций недостающих модулей программируются модули-«заглушки», которые моделируют функции отсутствующих модулей.

Интересен и второй вопрос: в какой форме готовятся тестовые данные и как они передаются программе? Если бы головной модуль содержал все нужные операции ввода и вывода, ответ был бы прост: тесты пишутся в виде обычных для пользователей внешних данных и передаются программе через выделенные ей устройства ввода. Так, однако, случается редко. В хорошо спроектированной программе физические операции ввода-вывода выполняются на нижних уровнях структуры, поскольку физический ввод-вывод — абстракция довольно низкого уровня. Поэтому для того, чтобы решить проблему экономически эффективно, модули добавляются не в строго нисходящей последовательности (все модули одного горизонтального уровня, затем модули следующего уровня), а таким образом, чтобы *обеспечить функционирование операций физического ввода-вывода как можно быстрее.* Когда эта цель достигнута, нисходящее тестирование получает значительное преимущество: все дальнейшие тесты готовятся в той же форме, которая рассчитана на пользователя.

Нисходящий метод имеет как достоинства, так и недостатки по сравнению с восходящим. Самое значительное достоинство — то, что этот метод совмещает тестирование модуля, тестирование сопряжений и частично тестирование внешних функций. С этим же связано другое его достоинство: когда модули ввода-вывода уже подключены, тесты можно готовить в удобном виде. Нисходящий подход выгоден также в том случае, когда есть сомнения относительно осуществимости программы в целом или когда в проекте программы могут оказаться серьезные дефекты.

Преимуществом нисходящего подхода очень часто считают отсутствие^ необходимости в драйверах; вместо драйверов вам просто следует написать «заглушки».

Нисходящий метод тестирования имеет, к сожалению, некоторые недостатки. Основным из них является то, что модуль ред-

ко тестируется досконально сразу после его подключения. Дело в том, что основательное тестирование некоторых модулей может потребовать крайне изощренных заглушек. Программист часто решает не тратить массу времени на их программирование, а вместо этого пишет простые заглушки и проверяет лишь часть условий в модуле. Он, конечно, собирается вернуться и закончить тестирование рассматриваемого модуля позже, когда уберет заглушки. Такой план тестирования — определенно не лучшее решение, поскольку об отложенных условиях часто забывают.

Второй тонкий недостаток нисходящего подхода состоит в том, что он может породить веру в возможность начать программирование и тестирование верхнего уровня программы до того, как вся программа будет полностью спроектирована. Эта идея на первый взгляд кажется экономичной, но обычно дело обстоит совсем наоборот. Большинство опытных проектировщиков признает, что проектирование программы — процесс итеративный. Редко первый проект оказывается совершенным. Нормальный стиль проектирования структуры программы предполагает по окончании проектирования нижних уровней вернуться назад и подправить верхний уровень, внеся в него некоторые усовершенствования или исправляя ошибки, либо иногда даже выбросить проект и начать все сначала, потому что разработчик внезапно увидел лучший подход. Если же головная часть программы уже запрограммирована и оттестирована, то возникает серьезное сопротивление любым улучшениям ее структуры. В конечном итоге за счет таких улучшений обычно можно сэкономить больше, чем те несколько дней или недель, которые рассчитывает выиграть проектировщик, приступая к программированию слишком рано.

Метод «большого скачка»

Вероятно, самый распространенный подход к интеграции модулей —- метод «большого скачка». В соответствии с этим методом каждый модуль тестируется автономно. По окончании тестирования модулей они интегрируются в систему все сразу.

Метод «большого скачка» по сравнению с другими подходами имеет много недостатков и мало достоинств Заглушки и драйверы необходимы для каждого модуля. Модули не интегрируются до самого последнего момента, а это означает, что в течение

долгого времени серьезные ошибки в сопряжениях могут остаться необнаруженными.

Если программа мала (как. например, программа загрузчика) и хорошо спроектирована, метод «большого скачка» может оказаться приемлемым. Однако для крупных программ метод «большого скачка» обычно губителен.

Метод сандвича

Тестирование методом сандвича представляет собой компромисс между восходящим и нисходящим подходами. Здесь делается попытка воспользоваться достоинствами обоих методов, избежав их недостатков.

При использовании этого метода одновременно начинают восходящее и нисходящее тестирование, собирая программу как снизу, так и сверху и встречаясь в конце концов где-то в середине. Точка встречи зависит от конкретной тестируемой программы и должна быть заранее определена при изучении ее структуры. Например, если разработчик может представить свою систему в виде уровня прикладных модулей, затем уровня модулей обработки запросов, затем уровня примитивных функций, то он может решить применять нисходящий метод на уровне прикладных модулей (программируя заглушки вместо модулей обработки запросов), а на остальных уровнях применить восходящий метод.

Метод сандвича сохраняет такое достоинство нисходящего и восходящего подходов, как начало интеграции системы на самом раннем этапе. Поскольку вершина программы вступает в строй рано, мы, как в нисходящем методе, уже на раннем этапе получаем работающий каркас программы. Так как нижние уровни программы создаются восходящим методом, снимаются проблемы нисходящего метода, которые были связаны с невозможностью тестировать некоторые условия в глубине программы.

Модифицированный метод сандвича

При тестировании методом сандвича возникает та же проблема, что и при нисходящем подходе, хотя здесь она стоит не так остро. Проблема эта в том, что невозможно досконально тестировать отдельные модули. Восходящий этап тестирования по

методу сандвича решает эту проблему для модулей нижних уровней, но она может по-прежнему оставаться открытой для нижней половины верхней части программы. В модифицированном методе сандвича нижние уровни также тестируются строго снизу вверх. А модули верхних уровней сначала тестируются изолированно, а затем собираются нисходящим методом. Таким образом, модифицированный метод сандвича также представляет собой компромисс между восходящим и нисходящим подходами [50].

5.6. Комплексное тестирование

Комплексное тестирование, вероятно, самая непонятная форма тестирования. Во всяком случае комплексное тестирование не является тестированием всех функций полностью собранной системы; тестирование такого типа называется *тестированием внешних функций.* Комплексное тестирование — процесс поисков несоответствия системы ее исходным целям. Элементами, участвующими в комплексном тестировании, служат сама система, описание целей продукта и вся документация, которая будет поставляться вместе с системой. Внешние спецификации, которые были ключевым элементом тестирования внешних функций, играют лишь незначительную роль в комплексном тестировании.

Часть аргументов в пользу этого должна быть уже очевидной: измеримые цели необходимы, чтобы определить правила для процессов проектирования. Остальные соображения должны проясниться сейчас. Если цели сформулированы, например, в виде требования, чтобы система была достаточно быстрой, вполне надежной и,чтобы в разумных пределах обеспечивалась безопасность, тогда нет способа определить при тестировании, в какой степени система достигает своих целей.

Если вы не сформулировали цели вашего продукта или если эти цели неизмеримы, вы не можете выполнить комплексное тестирование

Комплексное тестирование может быть процессом и контроля, и испытаний. Процессом испытаний оно является тогда, когда выполняется в реальной среде пользователя или в обстановке, которая специально создана так, чтобы напоминать среду пользователя. Однако такая роскошь часто недоступна по ряду причин,

и в подобных случаях комплексное тестирование системы является процессом контроля (т.е. выполняется в имитируемой, или тестовой, среде). Например, в случае бортовой вычислительной системы космического корабля или системы противоракетной защиты вопрос о реальной среде (запуск настоящего космического корабля или выстрел настоящей ракетой) обычно не стоит. Кроме того, как мы увидим дальше, некоторые типы комплексных тестов не осуществимы в реальной обстановке по экономическим соображениям, и лучше всего выполнять их в моделируемой среде.

Проектирование комплексного теста

Комплексное тестирование — наиболее творческий из всех обсуждавшихся до сих пор видов тестирования. Разработка хороших комплексных тестов требует часто даже больше изобретательности, чем само проектирование системы. Здесь нет простых рекомендаций типа тестирования всех ветвей или построения функциональных диаграмм. Однако следующие 15 пунктов дают некоторое представление о том, какие виды тестов могут понадобиться (рис. 5.3).

1. *Тестирование стрессов.* Распространенный недостаток больших систем состоит в том, что они функционируют как будто бы нормально при слабой или умеренной нагрузке, но выходят из строя при большой нагрузке и в стрессовых ситуациях реальной среды. Тестирование стрессов представляет собой попытки подвергнуть систему крайнему «давлению», например, попытку одновременно подключить к системе разделения времени 100 терминалов, насытить банковскую систему мощным потоком входных сообщений или систему управления — процессами аварийных сигналов от всех ее процессов.

Одна из причин, по которой тестирование стрессов опускают (кроме очевидных логических проблем, рассматриваемых в следующем разделе), состоит в том, что персонал, занимающийся тестированием, хотя и признает потенциальную пользу таких тестов, считает, что столь жесткие стрессовые ситуации никогда не возникнут в реальной среде. Это предположение редко оправдывается. Например, бывают случаи, когда все пользователи системы разделения времени пытаются подключиться в одно и то же время (например, когда произошел отказ системы на 1-2 ми-

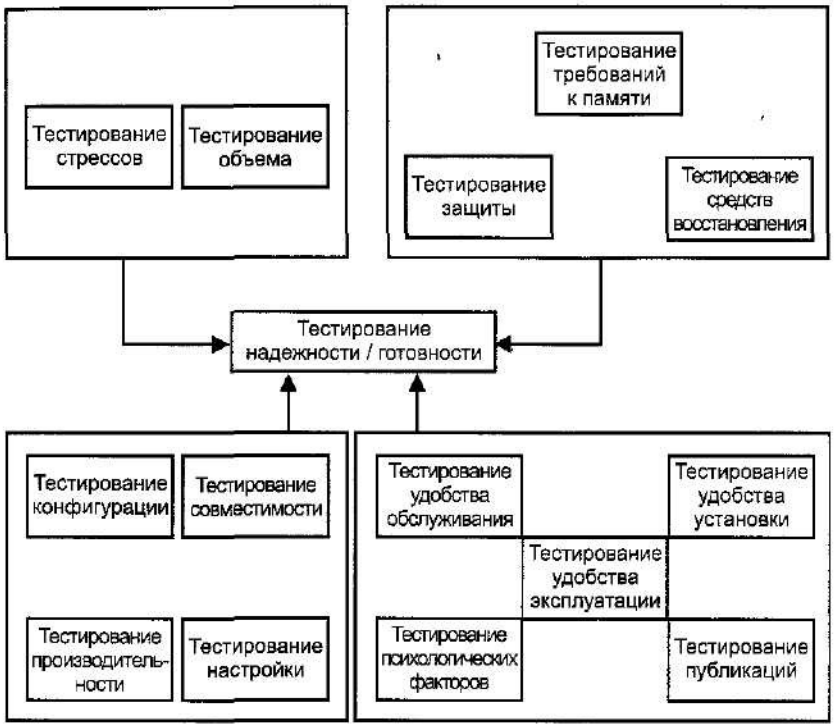


Рис. 5.3. Схема проектирования комплексного теста

нуты и система только что восстановлена). У банковских систем, обслуживающих терминалы покупателей, бывают пиковые нагрузки в первые часы работы магазинов и в час обеденного перерыва.

2. Тестирование объема. В то время как при тестировании стрессов делается попытка подвергнуть систему серьезным нагрузкам в короткий интервал времени, тестирование объема представляет собой попытку предъявить системе большие объемы данных в течение более длительного времени. На вход компилятора следует подать до нелепости громадную программу. Программа обработки текстов — огромный документ. Очередь заданий операционной системы следует заполнить до предела. Цель тестирования объема — показать, что система или программа не может обрабатывать данные в количествах, указанных в их спецификациях.

3. *Тестирование конфигурации.* Многие системы, например операционные системы или системы управления файлами, обес печивают работу различных конфигураций аппаратуры и про граммного обеспечения. Число таких конфигураций часто слиш ком велико, чтобы можно было проверить все варианты. Однако следует тестировать по крайней мере максимальную и минималь ную конфигурации. Система должна быть проверена со всяким аппаратным устройством, которое она обслуживает, или со вся кой программой, с которой она должна взаимодействовать. Если сама программная система допускает несколько конфигураций (т.е. покупатель может выбрать только определенные части или варианты системы), должна быть тестирована каждая из них.

4. *Тестирование совместимости,* В большинстве своем разра батываемые системы не являются совершенно новыми; они пред ставляют собой улучшение прежних версий или замену устарев ших систем. В таких случаях на систему, вероятно, накладывает ся дополнительное требование совместимости, в соответствии с которым взаимодействие пользователя с прежней версией долж но полностью сохраниться и в новой системе. Например, воз можно, потребуется, чтобы в новом выпуске операционной сис темы язык управления заданиями, язык общения с терминалом и скомпилированные прикладные программы, использовавшиеся раньше, могли применяться без изменений. Такие требования совместимости следует тестировать. Как периодически подчер кивалось при разговоре обо всех других формах тестирования, цель при тестировании совместимости должна состоять в том, чтобы показать наличие несовместимости.

5. *Тестирование защиты.* Так как внимание к вопросам сохра нения секретности в обществе возрастает, к большинству систем предъявляются определенные требования по обеспечению заши ты от несанкционированного доступа. Например, операционная система должна устранить всякую возможность для программы пользователя увидеть данные или программу другого пользова теля. Административная информационная система не должна позволять подчиненному получить сведения о зарплате тех, кто стоит выше его по служебной лестнице. Цель тестирования за шиты — нарушить секретность в системе. Один из методов — нанять профессиональную группу «взломщиков», т. е. людей с опытом разрушения средств обеспечения защиты в системах. Для тестирования защиты важно построить такие тесты, которые

нарушат программные средства защиты, например, механизм защиты памяти операционной системы или механизмы защиты данных системы управления базой данных. Одним из путей разработки подобных тестов является изучение известных проблем защиты в этих системах и генерация тестов, которые позволяют проверить, как решаются аналогичные проблемы в тестируемой системе.

6. *Тестирование требований к памяти.* При проектировании многих систем ставятся цели, определяющие объем основной и вторичной памяти, которую системе разрешено использовать в различных условиях. С помощью специальных тестов нужно по пытаться показать, что система этих целей не достигает.

7. *Тестирование производительности.* При разработке многих программ ставится задача — обеспечить их производительность, или эффективность. Определяются такие характеристики, как вре мя отклика и уровень пропускной способности при определен ной нагрузке и конфигурации оборудования. Проверка системы в этих случаях сводится к демонстрации того, что данная программа не удовлетворяет поставленным целям. Поэтому не обходимо разработать тесты, с помощью которых можно попы таться показать, что система не обладает требуемой производи тельностью.

8. *Тестирование настройки.* К сожалению, процедуры настрой ки многих систем сложны. Тестирование процесса настройки си стемы очень важно, поскольку одна из наиболее обескураживаю щих ситуаций, с которыми сталкивается покупатель, заключает ся в том, что он оказывается не в состоянии даже настроить новую систему.

9. *Тестирование надежности/готовности.* Конечно, назначе ние всех видов тестирования — повысить надежность тестируе мой программы, но если в исходном документе, отражающем цели программы, есть особые указания, касающиеся надежности, мож но разработать специальные тесты для ее проверки. Ключевой момент в комплексном тестировании заключается в попытке до казать, что система не удовлетворяет исходным требованиям к надежности (среднее время между отказами, количество ошибок, способность к обнаружению, исправлению ошибок и/или устой чивость к ошибкам и т. д.). Тестирование надежности крайне сложно, и все же следует постараться тестировать как можно боль ше этих требований. Например, в систему можно намеренно вне-

сти ошибки (как аппаратные, так и программные), чтобы тестировать средства обнаружения, исправления и обеспечения устойчивости. Другие требования к надежности тестировать почти невозможно.

10. *Тестирование средств восстановления.* Важная составная часть требований к операционным системам, системам управления базами данных и системам передачи данных — обеспечение способности к восстановлению, например восстановлению утраченных данных в базе данных или восстановлению после отказа в телекоммуникационной линии. Во многих проектах совершенно забывают о тестировании соответствующих средств. Лучше всего попытаться показать, что эти средства работают неправильно, при комплексном тестировании системы. Можно намеренно ввести в операционную систему программные ошибки, чтобы проверить, восстановится ли она после их устранения. Неисправности аппаратуры, например ошибки устройств ввода-вывода и контроля на четность ячеек памяти, можно промоделировать. Ошибки в данных (помехи в линиях связи и неправильные значения указателей в базе данных) можно намеренно создать или промоделировать для анализа реакции на них системы.

11 *Тестирование удобства обслуживания.* Либо в требованиях к продукту, либо в требованиях к проекту должны быть перечислены задачи, определяющие удобство обслуживания (сопровождения) системы. Все сервисные средства системы нужно проверять при комплексном тестировании. Все документы, описывающие внутреннюю логику, следует проанализировать глазами обслуживающего персонала, чтобы понять, как быстро и точно можно указать причину ошибки, если известны только некоторые се симптомы. Все средства, обеспечивающие сопровождение и поставляемые вместе с системой, также должны быть проверены.

12. *Тестирование публикаций.* Проверка точности всей документации для пользователя является важной частью комплексного тестирования. Все комплексные тесты следует строить только на основе 'документации для пользователя. Ее проверяют при определении правильности представления предшествующих тестов системы. Пользовательская документация должна быть и предметом инспекции при проверке ее на точность и ясность. Любые примеры, приведенные в документации, следует оформить как тест и подать на вход программы.

13. *Тестирование психологических факторов.* Хотя во время тестирования системы следует проверить и психологические факторы, эта сторона не так важна, как другие, потому что обычно при тестировании уже слишком поздно исправлять серьезные просчеты в таких вопросах. Однако мелкие недостатки могут быть обнаружены и устранены при тестировании системы. Например, может оказаться, что ответы или сообщения системы плохо сформулированы или ввод команды пользователя требует постоянных переключений верхнего и нижнего регистров.

14. *Тестирование удобства установки.* Процедуры установки (настройки) некоторых типов систем программного обеспечения весьма сложны. Тестирование подобных процедур является частью процесса тестирования системы.

15. *Тестирование удобства эксплуатации.* Не менее важным видом тестирования системы является попытка выявления психологических (пользовательских) проблем, или проблем удобства эксплуатации. Анализ психологических факторов является, к сожалению, до сих пор весьма субъективным, так как при производстве вычислительной техники уделялось недостаточное внимание изучению и определению психологических аспектов программных систем. Большинство систем обработки данных либо является компонентами более крупных систем, предполагающих деятельность человека, либо сами регламентируют такую деятельность во время своей работы. Нужно проверить, что вся эта деятельность (например, поведение оператора или пользователя за терминалом) удовлетворяет определенным условиям.

Не все из перечисленных 15 пунктов применимы к тестированию всякой системы (например, когда тестируется отдельная прикладная программа), но тем не менее это перечень вопросов, которые разумно иметь в виду.

Основное правило при комплексном тестировании — все годится. Пишите разрушительные тесты, проверяйте все функциональные границы системы, пишите тесты, представляющие ошибки пользователя (например, оператора системы) Нужно, чтобы тесговик думал так же, как пользователь или покупатель, а это предполагает доскональное понимание того, для чего система будет применяться. Поэтому возникает вопрос: «Кто же должен выполнять комплексное тестирование и, в частности, кто должен проектировать гесты?» Во всяком случае этого не должны делать

программисты или организации, ответственные за разработку системы. Группа тестирования системы должна быть независимой организацией и должна включать профессиональных специалистов по комплексному тестированию систем; несколько пользователей, для которых система разрабатывалась; основных аналитиков и проектировщиков системы и, возможно, одного или двух психологов. Очень важно включить самих проектировщиков системы. Это не противоречит аксиоме, согласно которой невозможно тестировать свою собственную систему, поскольку система прошла через много рук после того, как ее описали архитектор или проектировщик. В действительности проектировщик и не пытается тестировать собственную систему: он ищет расхождения между окончательной версией и тем, что он первоначально, возможно год или два назад, имел в виду. Для комплексного тестирования желательно также иметь информацию о рынке или пользователях, уточняющую предположения о конфигурации и характере применения системы.

Комплексное тестирование системы — такая особая и такая важная работа, что в будущем возможно появление компаний, специализирующихся в основном на комплексном тестировании систем, разработанных другими.

Как уже упоминалось, компонентами комплексного теста являются исходные цели, документация, публикации для пользователей и сама система. Все комплексные тесты должны быть подготовлены на основе публикаций для пользователя (а не внешних спецификаций). Ко внешним спецификациям обращаться следует только для того, чтобы разбираться в противоречиях между системой и публикациями о ней.

По своей природе комплексные тесты никогда не сводятся к проверке отдельных функций системы. Они часто пишутся в форме сценариев, представляющих ряд последовательных действий пользователя. Например, один комплексный тест может представлять подключение терминала к системе, выдачу последовательно 10—20 команд и затем отключение от системы. Вследствие их особой сложности тесты системы состоят из нескольких компонентов: сценария, входных данных и ожидаемых выходных данных. В сценарии точно указываются действия, которые должны быть совершены во время выполнения теста.

Выполнение комплексного теста

Комплексное тестирование находится у самой левой границы спектра (см. рис. 5.2). При этом вся система рассматривается как «черный яшик»; в частности, несущественно, какие участки программы затрагивает комплексный тест.

Один из методов, позволяющих вовлечь в тестирование пользователей, — *опытная эксплуатация.* Для проведения опытной эксплуатации с одной или несколькими организациями пользователей заключаются контракты на установку у них созданной системы. Это часто выгодно обеим сторонам: организация-разработчик оповещается об ошибках в программном обеспечении, которые она не заметила сама, а организация-пользователь получает возможность изучить систему и экспериментировать с ней до того, как она станет доступной официально.

Второй полезный метод — использовать систему в организации-изготовителе для внутренних нужд. Это можно сделать часто, но далеко не всегда (например, компании по разработке программного обеспечения негде использовать систему управления процессом очистки нефти). Так, производитель ЭВМ может установить новую операционную систему для опытной эксплуатации во всех своих внутренних вычислительных центрах, прежде чем начинать поставлять ее пользователям, а компания по разработке программного обеспечения может использовать свои собственные компиляторы, программы обработки текстов, начисления зарплаты и т. д.

Воспользуйтесь собственным продуктом, прежде чем передавать его другим.

При комплексном тестировании часто начинают с простых тестов, приберегая более сложные тесты к концу. Так делать не нужно, потому что комплексное тестирование приходится на самый конец цикла разработки, так что на отладку и исправление найденных ошибок остается мало времени. Поскольку сложные тесты часто обнаруживают более сложные для исправления ошибки, измените последовательность: начните с самых трудных тестов, а затем переходите к более простым [49].

5.7. ГОСТ Р ИСО/МЭК 12119-2000

Рассмотрим требования, которые предъявляет ГОСТ Р ИСО/МЭК 12119-2000 к тестированию пакетов программ.

ГОСТ Р ИСО/МЭК 12119-2000 содержит указания, которые определяют порядок тестирования продукта на соответствие его требованиям к качеству. Эти указания охватывают как тестирование для характеристик, присущих аналогичным продуктам, так и тестирование для характеристик, указанных в описании продукта. Указания охватывают тестирование путем проверки документов и тестирование программ и данных по принципу «черного ящика».

ГОСТ описывает только функциональное тестирование (тестирование по принципу «черного ящика»), а структурное тестирование не охватывается, потому что для его проведения необходимо наличие исходного кода. В нем рассматривают только тестирование продукта в необходимых для него системах. Эргономическую оценку на рабочем пространстве вычислительной системы в настоящем стандарте тоже не рассматривают.

Роботы по тестированию

Описание продукта, документация пользователя, программы и любые данные, поставляемые как части пакета программ, должны быть протестированы на выполнение ими формулировок и требований.

Программы должны быть протестированы во всех вычислительных системах, указанных в описании продукта. При наличии нескольких вариантов программы должен быть протестирован каждый из них. Любая из функций, которые в соответствии с описанием продукта и документацией пользователя одинаковы в ряде вариантов, может быть протестирована в одном из вариантов. Программы и данные должны быть протестированы с использованием контрольных примеров, разработанных на основе описания продукта и документации пользователя. Другие материалы (например, исходные программы) не проверяют, за исключением случаев, когда это необходимо при тестировании формулировок из описания продукта или документации пользователя.

Контрольные примеры должны быть методологически и систематически проработаны.

Если в документации пользователя приведены примеры, то они должны быть использованы в качестве контрольных, но проводимое тестирование не должно быть ограничено только этими примерами.

Могут быть использованы контрольные примеры, предоставляемые поставщиком программного пакета, но проводимое тестирование не должно быть ограничено только этими примерами.

Установка (инсталляция). Если в соответствии с описанием продукта установка пакета может быть выполнена пользователем, должна быть проверена возможность инсталляции программ и протестирована возможность успешной установки пакета согласно описанию, приведенному в руководстве по установке.

Л^обым способом должно быть обеспечено, чтобы техническая и программная среда, в которой установлены программы, соответствовала формулировкам из описания продукта в части рассматриваемой вычислительной системы.

Выполнение программы. Контрольные примеры должны охватывать все функции, приведенные в описании продукта и документации пользователя, а также учитывать комбинации функций, характерные для рабочей задачи.

Программы должны быть протестированы по всем граничным значения'м (в соответствии с описанием продукта и документацией пользователя) в необходимой системе, для которой заданы эти значения.

При тестировании должны быть использованы исходные данные и последовательности команд, которые в документации пользователя явно не рекомендуются или объявляются запрещенными.

Протоколы тестирования

Протоколы по каждому тесту должны содержать информацию, достаточную для повторения теста. Данная информация должна включать:

- план тестирования или технические требования (спецификацию) к тестированию, содержащие контрольные примеры (для каж дого контрольного примера указаны его цели);
- все результаты, связанные с контрольными примерами, вклю чая все ошибки, выявленные при выполнении теста;
- штат персонала, вовлеченного в тестирование.

Отчет о тестировании

В отчете о тестировании должны быгь суммированы цели и результаты тестирования (описанные в протоколах тестирова-

ния для каждого теста). Отчет о тестировании должен иметь следующую структуру.

1. Обозначение продукта.

2. Вычислительные системы, использованные при тестирова нии (технические средства, программные средства и их конфигу рация).

3. Использованные документы (включая их обозначения).

4. Результаты тестирования описания продукта, документа ции пользователя, программ и данных.

5. Перечень несоответствий требованиям.

6. Перечень несоответствий рекомендациям либо перечень не учтенных в продукте рекомендаций, либо формулировка того, что продукт не был протестирован на соответствие рекомендациям.

7. Дата окончания тестирования.

Дополнительное тестирование

Когда продукт, который уже был протестирован, тестируется повторно (с учетом результатов предыдущего тестирования), тогда выполняются следующие требования:

- все измененные части документов, функций и данных должны быть протестированы как новый продукт;
- все неизмененные части, на которые могут влиять измененные части или изменения в необходимой системе (в соответствии с опытной оценкой тестировщика), должны быть протестирова ны как новый продукт;
- все другие части должны быть по крайней мере выборочно про тестированы.

5.8. Требования к средствам обеспечения тестирования

Сложность программных средств обычно адекватна сложности поведения объектов внешней среды, в которой функционирует соответствующее ПС. В процессе испытаний специалисты должны стремиться охватить тестированием функционирование ПС во всей доступной области варьирования исходных данных и режимов применения. При этом номенклатура и диапазоны варь-

ирования тестов ограничены либо допустимой длительностью подготовки и исполнения тестов, либо вычислительными ресурсами, доступными для использования с целью контроля и тестирования в режиме нормальной эксплуатации. Это отражается на объеме применяемых тестов и на глубине тестирования. Однако при эксплуатации ПС в реальной внешней среде могут создаваться отдельные ситуации, угрожающие надежности и не проверенные при испытаниях или сертификации. Для регистрации таких ситуаций и снижения возможности их катастрофических последствий частично используются те же средства, что и при специализированных испытаниях.

В стандарте ШЕЕ 1209-1992 сформулированы *общие требования к функциям средств автоматизации тестирования,* входящим в CASE-средства, которые должны обеспечивать:

- определение тестов — реализацию процесса тестирования пользователем: ввод тестовых наборов, генерацию тестовых на боров:
- генерацию тестовых данных, ввод ожидаемых, эталонных ре зультатов, генерацию ожидаемых результатов;
- выполнение участка тестируемой программы между конт рольными точками, для которого CASE-средство может пере хватить операторский ввод (клавиатуры, мыши и т.д.) и для которого вводимые данные могут быть отредактированы и включены в последующие тестовые наборы;
- управление тестами и участком программы, для которого CASE-
  средство может автоматически выполнять тестовые наборы;
- регрессионное тестирование с возвратом от более сложных тестов к простым, возможность перезапускать предыдущие тесты и возможность модифицировать предыдущие тесты, что бы учесть различия в системе и/или среде (например, дату и время);
- анализ тестовых результатов — возможность CASE-средства автоматически анализировать тестовые результаты: сравнение ожидаемых и реальных результатов, сравнение файлов, статис тический анализ результатов;
- анализ покрытия тестами исходного кода для обнаружения опе раторов (элементов текста программы, выражающих законченные действия), которые были/не были выполнены, процедур, которые были/не были вызваны, и переменных, к которым были/не были обращения;

- анализ производительности программы, когда она выполняет ся: загрузку центрального процессора, загрузку памяти, обра щения к специфицированным элементам данных и/или сегмен там кода, временные характеристики;
- верификацию условий или исключительных ситуаций во время выполнения теста;
- моделирование среды — поддержку процесса тестирования с помощью модели, например, возможность CASE-средства ав томатически генерировать входы моделируемых систем на ос нове полученных системных выходов.

Перечисленные требования определяют необходимость разработки соответствующих проблемно-ориентированных интегрированных систем, способных достаточно полно заменить испытания программ с реальными объектами внешней среды. При этом высокая стоимость и риск испытаний с реальными объектами почти всегда оправдывают значительные затраты на такие интегрированные системы, если предстоят испытания *критических ПС* с высокими требованиями к надежности функционирования программ и их длительным жизненным циклом с множеством развивающихся версий. Технология разработки средств имитации внешней среды в значительной степени близка к технологии разработки ПС, для отладки и испытаний которых они предназначены. Наиболее высокие требования к имитации, естественно, предъявляют испытания критических ПС, которые функционируют в реальном времени. *Требования к средствам обеспечения испытаний на надежность* таких ПС сводятся к следующим:

- все данные от реальных объектов и имитаторов внешней среды должны поступать на испытываемое ПС в соответствии с есте ственным ходом процессов в этих объектах реального вре мени;
- диапазоны изменения исходных данных в имитаторах должны обеспечивать перекрытие всех характеристик современных ре альных объектов внешней среды, а также предусматривать воз можность их расширения с учетом предполагаемого развития ПС и прогресса в соответствующих областях применения ком плекса программ;
- необходимо совмещать данные от реальных объектов внешней среды и от имитаторов, заменяющих некоторые из них, кото рые нерационально или невозможно применять при испыта ниях в натуральном виде;

- необходимо обеспечить регистрацию, контроль и обобщение характеристик генерируемых тестовых данных, эталонных данных и всех видов искажений и аномалий,-поступающих на испытываемое ПС в любой момент времени и на любом заданном шаге обработки информации;
- при формировании тестовых данных от ряда объектов должны оперативно учитываться воздействия результатов функциони рования испытываемого ПС по ранее поступавшим данным от тех же объектов с учетом обратных информационных и управ ляющих связей;
- для всех тестовых данных должны быть подготовлены эталон ные реакции ПС, с которыми следует сравнивать результаты, получаемые в процессе испытаний;
- необходимо обеспечить измерение и обобщение показателей качества и надежности ПС по результатам проведения сеансов испытаний с определенными целевыми задачами;
- следует обеспечить максимально возможную повторяемость се ансов испытаний и генерируемых тестов после обнаружения и устранения дефектов в функционировании ПС.

Документирование процессов тестирования программ целесообразно проводить непосредственно при выполнении каждой из соответствующих *работ с определенными целями.* Должны быть описаны и документированы: рекомендуемый состав исходных данных, решаемые задачи и целесообразные методы, содержание результатов и отчетных документов. Таким образом, должен быть упорядочен и конкретизирован весь технологический процесс тестирования и документирования объектов и частных результатов тестирования программных компонентов и ПС в целом. В плане следует детализировать и документировать процесс тестирования на каждой фазе ЖЦ ПС:

- задачи проведения тестирования;
- методы и критерии оценок качества;
- входные и результирующие данные;
- графики решения частных задач тестирования и необходимые для них ресурсы;
- оценки риска и распределение ответственности между специа листами по компонентам и видам проверок.

Приведенные выше требования и рекомендации ориентированы на создание надежных программ, их тестирование и испытания в основном до передачи в регулярную эксплуатацию. Пос-

Цщ,

ле приемки заказчиком или приобретения пользователями в процессе функционирования ПС должно продолжаться их тестирование и оценка текущего качества и надежности. Для этого в *составе комплекса программ необходимы средства, обеспечивающие:*

- генерацию тестовых наборов или хранения тестов для контро ля работоспособности, сохранности и целостности ПС при функционировании в реальном времени;
- оперативный контроль и обнаружение дефектов исполнения программ и обработки данных при использовании ПС по пря мому назначению;
- реализацию процедур анализа выявленных дефектов и опера тивное восстановление вычислительного процесса, программ и данных (рестарта) после обнаружения аномалий функциони рования ПС;
- накопление и хранение данных о выявленных дефектах, сбоях и отказах в процессе исполнения программ и обработки данных.

Для размещения таких средств обеспечения надежности и безопасности функционирования ПС в реализующей ЭВМ необходимы ресурсы внешней и оперативной памяти этой ЭВМ. Кроме того, для функционирования средств оперативного тестирования и обеспечения надежности необходима временная избыточность — производительность ЭВМ. Эти виды избыточности всегда ограничены и различаются для различных проектов. Поэтому средства для решения перечисленных задач трудно унифицировать, и они обычно ориентированы на применение в конкретном комплексе программ.

Средства генерации тестов и имитации внешней среды предназначены для оперативной подготовки исходных данных при проверке различных режимов функционирования в процессе применения ПС и при диагностике дефектов. Минимальный состав средств имитации должен передаваться пользователям для контроля использования рабочих версий ПС в реальном времени и входит в комплект поставки каждой пользовательской версии. Для более глубоких испытаний функционирования версий и локализации ошибок следует создавать комплекс средств имитации внешней среды высшего уровня на технологической ЭВМ, которые используются специалистами по испытаниям и сертификации. Часть этих средств имитации может использоваться как средства нижнего уровня (пользовательские) на реализующей ЭВМ Для диагностики и обеспечения полного повторения ситуаций,

при которых пользователем обнаружены дефекты функционирования.

Средства упорядочения и каталогизации тестовых наборов должны обеспечивать возможность многократного использования тестов в процессе эксплуатации ПС. Для эффективного использования тестов необходима система управления базой данных для их накопления и хранения с тщательно продуманной идентификацией и каталогизацией тестов. Система каталогизации должна обеспечивать достаточно простой и надежный поиск тестов среди имеющихся, а также достоверное выявление тестов, которые понадобились, но отсутствуют среди сохраняемых. Быстрый рост числа и суммарной сложности тестовых наборов в ряде случаев приводит к необходимости их селекции по степени важности и удобства повторной генерации. Средства тестирования тождественности и сохранности копий программ используются для обеспечения полной идентичности подлинников, дубликатов и копий каждой пользовательской версии ПС. Точным эталоном служат предварительно подготовленные контрольные суммы версии в целом и ее компонентов различного размера.

Средства встроенного контроля процесса исполнения программ должны периодически контролировать промежуточные и результирующие данные или включаться по запросу при обнаружении сомнительных результатов, так как непрерывный контроль требует значительных ресурсов памяти и производительности ЭВМ. Эти средства должны позволять получать диагностическую информацию о состоянии переменных в процессе решения конкретных задач, о маршрутах исполнения программ, в которых нарушаются некоторые заданные условия. Для эксплуатации создаются методики и инструкции, которые должны позволять пользователям достаточно квалифицированно осуществлять диагностику состояния ПС и результатов их функционирования. Унификация средств у пользователей и коллективов сопровождения облегчает селекцию и локализацию ошибок, а также идентификацию исходных данных, при которых они обнаружены.

Виды тестирования сложных программных средств. Формализация процесса тестирования на этапе испытаний наиболее трудна, и оценки полноты тестирования осуществляются преимущественно по степени выполнения функций и по характеристикам надежности функционирования ПС. Значительную помощь в повышении надежности сложных, критических ПС могут ока-

зать систематизация видов тестирования и упорядоченное их проведение при испытаниях. Эти виды тестирования ориентированы на *дифференцированное* выявление определенных классов дефектов. Для каждого вида тестирования целесообразно разрабатывать методику его выполнения с указанием контролируемых параметров, ожидаемых и эталонных результатов. Кроме того, при заключительных испытаниях или сертификации должно проводиться *интегральное* тестирование при максимально широком варьировании тестов в условиях, соответствующих нормальной эксплуатации. Рациональную последовательность испытаний сложных ПС в реальном времени можно представить следующими видами тестирования.

Тестирование полноты решения функциональных задач при типовых исходных данных предназначено для обнаружения дефектов функционирования в нормальных условиях, определенных техническим заданием на базовую версию ПС. Первичным эталоном являются цели и задачи создания ПС. В соответствии с этими задачами создаются подробное формализованное техническое задание и спецификация требований на комплекс программ, которые являются основными эталонами при создании данного вида тестов. Для систем реального времени тесты содержат в основном динамические и стохастические данные. Эти данные имитируются моделями реальных объектов внешней среды. Результаты тестирования обрабатываются и сравниваются с эталонами преимущественно автоматически. Некоторая часть тестов может содержать детерминированные исходные данные, для анализа которых часто применяются различные системы графического отображения. Особое внимание целесообразно обращать на варианты тестов, позволивших обнаружить ошибки. Для этих условий следует проводить дополнительное тестирование.

Тестирование функционирования программ в критических ситуациях по условиям и логике решения задач (стрессовое тестирование) проводится при испытаниях исполнения программ в нештатных ситуациях, которые редко реализуются, но важны для надежности функционирования системы обработки информации. Для разработки таких тестов создаются сценарии критических сочетаний значений исходных данных и условий решения задач, при которых необходимо проверить функционирование программ и можно ожидать искажения результатов и отказы. Такие стрессовые, нештатные сочетания подготавливаются вручную или

предусматривается их реализация в составе данных стохастических тестов и реальном времени. Особая важность проверки в критических ситуациях определяется опасностью проявления ошибок при функционировании ПС в реальных условиях. Поэтому при тестировании активно применяются имитаторы внешней среды, автоматически подготавливающие исходные данные, и средства контроля, реагирующие на аномальные результаты исполнения тестируемых программ, отражающиеся на надежности.

Тестирование для измерения достигнутых значений надежности базовых версий ПС предназначено для определения основных показателей .надежности при реальном функционировании программ. В процессе тестирования при типовых и критических условиях определяются значения наработки на отказ, длительности восстановления, коэффициента готовности и других показателей. Для сложных систем реального времени организуются многочасовые прогоны ПС при стохастических исходных данных, при которых регистрируются искажения результатов и выделяются нарушения работоспособности программ. При таком тестировании особое значение имеет соотношение типовых и критических условий функционирования и исходных данных. Это соотношение должно устанавливаться в соответствии с техническим заданием на ПС и формализоваться в методике тестирования по согласованию между разработчиком и заказчиком. Для ПС с высокими показателями надежности могут применяться форсированные методы тестирования, при которых искусственно повышается интенсивность искажения исходных данных, вводятся частичные отказы и повышенные уровни сбоев в аппаратуре. Значения надежности при форсированных испытаниях затем должны корректно пересчитываться на нормальные условия эксплуатации. Имитация исходных данных и регистрация отказов могут производиться автоматически, при этом особенно важно обеспечить регистрацию условий нарушения работоспособности.

Тестирование корректности использования ресурсов памяти и производительности вычислительной системы служит для оценки надежности исполнения программ при перегрузках памяти и производительности. Тестирование проводится в основном в стохастическом режиме по подготовленным сценариям, создающим перегрузки в реальном времени одного из ресурсов системы. Проверке подлежит изменение качества и надежности функциониро-

вания ПС вследствие пропусков в обработке сообщений, возрастания длительности ожидания перед их обработкой или растягивания периодов решения задач. Имитация тестов проводится преимущественно автоматически по сценариям, создающим критические условия для определенной проверки. В результате тестирования устанавливаются реальные характеристики ПС на выбранной вычислительной системе по пропускной способности решения всего комплекса задач, а также по допустимой интенсивности решения отдельных типов задач и обработки различных сообщений. При кратковременных перегрузках памяти или производительности должны обеспечиваться защита от отказов и восстановление нормального режима при последующем снижении загрузки.

Тестирование параллельного исполнения программ используется для обнаружения снижений надежности, обусловленных несогласованным использованием исходных и промежуточных данных. а также устройств вычислительной системы при параллельном функционировании программ Тестирование проводится преимущественно стохастически с основной задачей: осуществить проверку при различных сочетаниях исполнения частей программы одновременно функционирующими процессорами. Особенно важно обнаружить все тупиковые ситуации, при которых параллельные программы обращаются к одним и тем же ресурсам вычислительной системы и не могут продолжить вычислений до их освобождения. Малая вероятность образования таких ситуаций может приводить к необходимости генерации большого объема стохастических тестов. Тестирование параллельно исполняемых программ обычно требует большого количества исходных данных, содержащих как случайные, так и детерминированные составляющие. Такие данные подготавливаются обычно автоматически по сценариям наиболее критических сочетаний данных.

Тестирование эффективности защиты от искажений исходных данных служит для выявления дефектов и ошибок в программах, проявляющихся при ложных или искаженных данных. Тестирование проводится при относительно небольших искажениях исходных данных, соответствующих нормированному возрастанию в них ошибок, а также при случайном полном искажении данных. Целесообразно измерять вероятности появления искажений и другие их характеристики. При тестировании выявляются ситуации нарушения работоспособности ПС и величины

снижения надежности функционирования в зависимости от интенсивности искажений. Искажения данных производятся в основном стохастически, однако в некоторых случаях могут быть необходимы детерминированные и коррелированные искажения различных данных.

Тестирование для оценки эффективности защиты от сбоев аппаратуры и невыделенных дефектов и ошибок программ и данных служит для проверки качества средств программного контроля и оперативного восстановления (рестарта) при различных непреднамеренных искажениях функционирования ПС. Стохастическое тестирование средств рестарта проводится в процессе определения показателей надежности функционирования программ. При этом оцениваются вероятность обнаружения отказовой ситуации и средняя длительность восстановления. Специализированные тесты оценки эффективности защиты должны позволять определить вероятность выявления каждого вида сбоев и соответствующую ему длительность восстановления нормального функционирования. Для этого разрабатываются сценарии имитации аппаратных сбоев, искажений исходных данных и программных ошибок, вызывающих срабатывание средств программного контроля и оперативного восстановления. Таким образом, обнаруживаются ошибки программ контроля или программ восстановления, а также определяются их динамические характеристики. Сложность данного вида тестирования обусловлена трудностью регламентированного ввода сбоев вычислительных систем и сложностью имитации проявления ошибок в разработанных программах. Кроме того, вследствие редких событий отказовых ситуаций для статистических оценок необходимо искусственное повышение интенсивности их возникновения.

Тестирование удобства эксплуатации и взаимодействия человека с ПС предназначено для обнаружения трудно формализуемых ошибок отображения и использования исходных и результирующих данных. При тестировании оцениваются объем, удобство представления и контроля исходных данных, вводимых непосредственно человеком-пользователем, а также отображаемых результирующих данных, удобство их анализа и надежность использования. Кроме того, проверяются динамические характеристики ввода и отображения данных в реальном времени. В сложных автоматизированных системах управления, в которых основные данные поступают по каналам связи, большое значе-

ние имеет тестирование принятия решений человеком в динамике функционирования системы, особенно в критических ситуациях. Тестирование позволяет выявить ошибки распределения автоматизируемых функций между человеком и ЭВМ. а также оценить возможность надежного решения задач обслуживающим персоналом системы. Часть проверок, связанная с оценкой удобства использования документации, может выполняться без вычислительной системы, путем сравнения целей и действий человека с содержанием пользовательской документации. При оценке психологического удобства эксплуатации большое значение может иметь выбор представительной группы операторов-пользователей. Их квалификация, критичность и доброжелательность могут сильно изменять результаты испытаний.

Тестирование удобства и качества подготовки пользовательских версий ПС служит для выявления ошибок методов и средств настройки базовых версий ПС к конкретным условиям применения. Многие ПС перед использованием адаптируются к операционной среде или к конкретным условиям, при которых должны решаться задачи. Для этого могут автоматизирование подготавливаться данные, характеризующие эти условия. Тестирование преследует цель проверки и обнаружения ошибок средств настройки, а также надежности функционирования адаптированных к разным условиям версий ПС. Для проверки средств адаптации создаются специальные тесты, охватывающие наиболее типовые режимы использования ПС пользователями. Тестирование адаптированных версий может проводиться на базе тестов испытаний на соответствие техническому заданию, доработанных по специальной методике для проверки адаптации.

Тестирование работы базовых версий ПС при их переносе и конфигурациях оборудования используется для обнаружения ошибок, проявляющихся при изменении состава или характеристик компонентов вычислительной системы или внешней среды. Серийно выпускаемые и широко применяемые программы могут функционировать на вычислительных системах, различающихся составом оборудования или подключаемых внешних абонентов и их характеристиками. Число возможных конфигураций оборудования может быть слишком велико, чтобы их все протестировать при подготовке базовой версии ПС. Поэтому важное значение имеют методика и средства подготовки ПС к переносу на различные конфигурации оборудования. В состав базовой версии ПС вводятся средства, по-

зволяющие адаптировать ее к таким изменениям оборудования. Тесты должны обеспечивать проверку этих средств автоматизированной адаптации во всех допустимых режимах комплектации оборудования, а также испытания надежности адаптированных версий. Для этого разрабатывается методика подготовки тестов проверки адаптированной версии пользователей, которая используется настройщиками версий. Проверка пригодности к реконфигурации внешней среды и сопровождению включает тестирование настройки базовых версий на условия конкретного применения пользователей и анализ удобства модифицирования версий ПС.

Оценка методов тестирования по показателю «эффективность /стоимость». Для выбора и применения методов тестирования и испытаний важно анализировать не только их трудоемкость («стоимость»), но и достигаемую эффективность. В качестве показателей эффективности методов и средств автоматизации тестирования и отладки сложных программных средств реального времени можно использовать:

- интенсивность (вероятность) обнаружения и устранения ошибок за единицу времени тестирования;
- достигаемую надежность (или корректность) функционирования программного средства или его компонентов за счет рационального применения данного метода.

Эти два подхода значительно различаются характером функциональной зависимости соответствующего показателя эффективности каждого метода от трудоемкости («стоимости») его применения, поэтому целесообразно рассмотреть их оба. Простейшие методы тестирования эффективны в некоторых пределах и для обнаружения только некоторых классов ошибок. На начальных этапах разработки, когда в программе много простейших ошибок, наиболее полезны ручные методы тестирования, которые обеспечивают высокую интенсивность выявления грубых ошибок этими методами при относительно небольших затратах. По мере использования каждого метода, сокращения числа и повышения сложности выявления ошибок его эффективность падает, что уменьшает целесообразность его применения. В то же время применение сложных и трудоемких методов на ранних этапах разработки нерентабельно, так как ряд типов ошибок может быть выявлен более простыми и дешевыми методами.

Таким образом, для каждого метода и вида тестирования в зависимости от объекта, этапа или календарного времени суще-

ствует зона максимальной эффективности его использования. В других областях эффективными оказываются другие методы. В то же время в процессе отладки уменьшается число ошибок в ПС и соответственно снижается вероятность их обнаружения даже самыми эффективными для этого методами.

В качестве *«стоимости»* применения методов тестирования рассмотрены совокупные затраты на их эксплуатацию и на ис-, пользование средств автоматизации, обеспечивающих соответствующую интенсивность обнаружения ошибок. В соответствии с зоной активного использования для каждого метода имеется зона наиболее интенсивных затрат. Максимумы значений затрат для каждого метода более или менее соответствуют максимумам интенсивности применения и выявления ошибок каждым методом. Усложнение методов и средств генерации тестов и контроля результатов тестирования по мере перехода от ручных к более сложным автоматизированным методам приводит к возрастанию максимума затрат на эти методы. Такой рост максимумов затрат обусловлен'возрастанием i рудоемкости выявления каждой ошибки и снижением вероятности их проявления под воздействием тестов.

В процессе разработки программ изменяются активность применения различных методов тестирования и относительные затраты на использование каждого. По мере разработки ПС происходит постепенный переход к более сложным методам тестирования и соответственно возрастают затраты на тестирование этими методами. На завершающих стадиях динамической комплексной отладки и испытаний ПС эти сложные методы становятся доминирующими. При стохастическом тестировании и тестировании в реальном времени благодаря расширению областей варьирования переменных и увеличению числа используемых тестовых значений возрастает обнаруживающая способность тестов на единицу затрат. Однако далее по мере выявления и устранения определенных видов ошибок интенсивность обнаружения дефектов также падает, что приводит к целесообразности снижения затрат сначала на стохастическое, а затем на тестирование в реальном времени.

Характеристики интенсивности обнаружения ошибок и затрат на применение каждого метода позволяют построить гипотетическую зависимость показателя «эффективность/стоимость» для каждого метода. На начальных этапах разработки

этот показатель имеет наибольшие значения для простейших методов и достаточно быстро падает по мере возрастания трудоемкости их использования. В некоторой зоне значений времени разработки ПС показатели для соседних методов пересекаются, и доминирующим по соотношению эффективность/стоимость становится следующий по сложности метод.

При наличии значительного количества ошибок в программе наработка на отказ, используемая как показатель надежности на начальных этапах, чрезвычайно низка. Интенсивное устранение ошибок на этих этапах простейшими методами слабо влияет на рост надежности программ. Однако их применение необходимо, так как на начальных этапах более сложные методы еще менее эффективны. Наработка на отказ начинает заметно возрастать при применении стохастического тестирования и тестирования в реальном времени после того, как предшествующие методы выявления ошибок исчерпали свои возможности. Такая особенность изменения показателя наработки на отказ отражает особое значение тестирования в реальном времени для соответствующего класса ПС. Для других классов ПС также могут быть выявлены методы, позволяющие достигнуть наивысших показателей качества на завершающих этапах отладки, когда основная часть ошибок выявлена более простыми методами.

Приведенный качественный анализ эффективности/стоимости методов тестирования показывает пути возможной оптимизации затрат на достижение заданной надежности программ. Однако для выполнения такой оптимизации необходимы конкретные характеристики эффективности каждого метода тестирования и достигаемой корректности или надежности разных классов программ в зависимости от затрат на использование этого метода.

5.9. Организация и этапы тестирования при испытаниях надежности сложных программных средств

Цели и этапы испытаний надежности комплексов программ. Основные этапы тестирования и испытаний комплекса программ и его компонентов представлены на рис. 5.4.

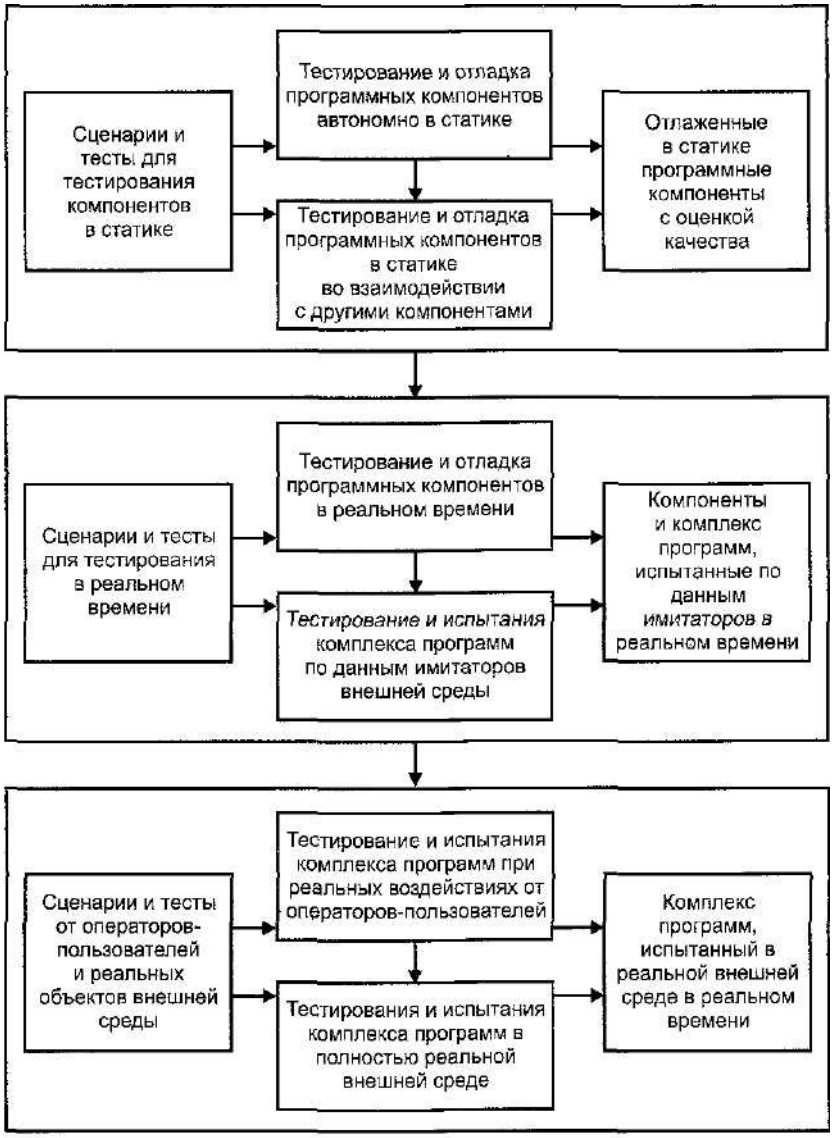


Рис. 5.4. Схема этапов тестирования и испытаний сложных комплексов программ

Для каждого этапа на рисунке представлены основные исходные данные и результаты тестирования и испытаний. При тестировании, отладке и испытаниях *корректности компонентов комплексов программ выделены следующие этапы.*

- комплексирование модулей и отладка автономных групп про грамм в статике без взаимодействия с другими компонентами и, возможно, без подключения к операционной системе реаль ного времени;
- тестирование и отладка групп программ в статике с учетом вза имодействия с некоторыми другими важнейшими компонента ми и с базой данных;
- тестирование и отладка отдельных программных компонентов в реальном времени во взаимодействии с другими функциональ ными компонентами и с основными компонентами операцион ной системы и базы данных.

Сложность тестирования компонентов на этих этапах в значительной степени обусловлена несинхронным процессом их разработки и отладки. Первично спланированная логика сопряжения между собой отдельных компонентов и подключения их к операционной системе не всегда выполняется из-за задержек в автономной отладке некоторых из них. Целесообразная последовательность отладки определенных компонентов может нарушаться неготовностью к сопряжению с ними других взаимодействующих программ В результате к комплексному тестированию и испытаниям ПС могут быть готовы не все необходимые компоненты, и их приходится начинать с некоторой не всегда самой важной и целесообразной совокупности групп программ.

Для обеспечения имитации объектов внешней среды и других взаимодействующих групп программ на этих этапах используются частные генераторы соответствующих тестов., Эти генераторы тестов целесообразно разрабатывать и оформлять как отдельные модули или группы программ, функционирующие на той же технологической ЭВМ и в той же операционной среде, что и отлаживаемые компоненты. Совместно с ними также реализуются и функционируют частные специализированные программы для обработки отдельных результатов отладки соответствующих групп программ, что также требует некоторых ресурсов ЭВМ При этом не всегда удается полностью реализовать реальный масштаб времени для отдельных функциональных компонентов и приходится применять стартстопныи режим тестирования или

растягивать циклы решения функциональных задач и имитировать псевдореальный масштаб.

После комплексирования основных функциональных компонентов начинаются тестирование и испытания ПС в целом. Для них наиболее характерны следующие *стадии комплексного тестирования и испытаний ПС в реальном времени:*

- по данным моделирующего стенда или генераторов тестов, ими тирующих отдельные объекты внешней среды;
- с имитаторами отдельных объектов внешней среды и с реаль ными воздействиями от операторов-пользователей;
- в полностью адекватной реальной или имитированной внеш ней среде и с реальными воздействиями от операторов-пользо вателей.

На всех стадиях отладки, кроме операций непосредственной проверки функционирования программ, можно выделить еще две важные группы работ. *Первая группа* — это работы по методическому обеспечению тестирования и по созданию средств автоматизированной генерации тестов. *Вторая группа* работ должна обеспечивать возможность обработки результатов тестирования и оценки достигнутых показателей качества функционирования программ.

Средства генерации тестов и обработки результатов отладки можно разделить на три вида (см. рис, 5.4). Одни и те же средства автоматизации тестирования в статике обычно обеспечивают отладку групп программ как автономно, так и во взаимодействии с другими компонентами. Средства, имитирующие внешнюю среду в реальном времени, чаще всего ориентированы на отладку как функциональных компонентов, так и ПС в целом. Еще один вид генераторов тестов в той или иной степени использует реальные объекты внешней среды. Первоначально такими объектами являются имитирующие стенды с участием реального функционирования операторов-пользователей. Затем источниками тестов могут быть полные комплексы реальной аппаратуры внешних объектов или их аппаратные аналоги.

Каждая из выделенных стадий тестирования может рассматриваться как обеспечивающая создание определенного промежуточного продукта — функциональной группы программ или программного средства с некоторыми ограниченными характеристиками качества. Эти характеристики выделяются и детализируются на основе первичного технического задания и специ-

фикации требований на ПС. В процессе проектирования ПС они уточняются и конкретизируются в спецификациях требований на группы программ и их компоненты. В результате создается совокупность эталонов, имеющих последовательно расширяющиеся номенклатуру и наборы значений показателей качества, которым должны соответствовать отлаживаемые и испытываемые компоненты на каждой стадии тестирования.

Подобная совокупность эталонных показателей качества промежуточных продуктов обеспечивает возможность управления процессом тестирования с целью достижения необходимого качества конечного продукта — ПС при минимальных или разумных затратах. Для этого каждая стадия тестирования должна иметь фиксированный и документированный результат с определенными эталонами и достигнутыми характеристиками программ. Подобный систематизированный контроль тестирования гарантирует высокое качество ПС реального времени, к которым предъявляются обычно высокие требования по надежности Кроме того, компоненты, прошедшие все стадии тестирования, с большой уверенностью могут применяться как повторно используемые компоненты в последующих версиях ПС.

Организация завершающих испытаний комплексов программ. Испытания главного конструктора, которые зачастую совмещаются с завершением комплексной отладки, должны оформляться документально и являются основанием для предъявления ПС заказчику на завершающиеся совместные испытания. Любые испытания ограничены допустимым объемом проверок и длительностью работы комиссии, поэтому не могут гарантировать абсолютную проверку изделия. Для повышения достоверности определения и улучшения характеристик ПС после испытаний главного конструктора программы целесообразно передавать некоторым пользователям на *опытную эксплуатацию в типовых условиях.* Это позволяет более глубоко оценить эксплуатационные характеристики созданного комплекса и устранить некоторые дефекты и ошибки. Опытная эксплуатация проводится разработчиками с участием испытателей и некоторых пользователей, назначаемых заказчиком. Результаты и показатели надежности опытной эксплуатации после испытаний главного конструктора могут учитываться при проведении совместных испытаний для их сокращения.

Совместные приемо-сдаточные испытания проводятся комиссией заказчика, в которой участвуют главный конструктор раз-

работки и некоторые ведущие разработчики, аттестованные сертификационной лабораторией. *Комиссия при испытании руководствуется следующими документами:*

- утвержденным заказчиком и согласованным с разработчиком техническим заданием и спецификациями на ПС;
- действующими государственными и ведомственными стандартами на проектирование и испытания программ и на техническую документацию, а также согласованными с заказчиком стандартами «де-факто»;
- программой испытаний по всем требованиям технического задания;
- методиками испытаний по каждому разделу требований технического задания;
- комплектом эксплуатационной документации на комплекс программ.

Программа испытаний является планом проведения серии экспериментов и разрабатывается с позиции минимизации объема тестирования в процессе проведения испытаний для проверки выполнения требований технического задания и соответствия предъявленной документации. Программа испытаний, методики их проведения и оценки результатов, разработанные совместно заказчиком и разработчиком, должны быть согласованы и утверждены. Они должны содержать уточнения и детализацию требований технического задания для данного ПС, а также гарантировать корректную проверку всех заданных характеристик, в том числе надежности. *Программа испытаний должна содержать следующие четко сформулированные разделы:*

- объект испытаний, его назначение и перечень основных документов, определивших его разработку;
- цель испытаний с указанием всех требований технического задания, подлежащих проверке, и ограничений на проведение испытаний;
- собственно программу испытаний, содержащую проверку комплектности спроектированного ПС в соответствии с техническим заданием, и план тестирования для проверки по всем разделам технического задания и дополнительным требованиям, формализованным отдельными решениями разработчиков и заказчика;
- методики испытаний, однозначно определяющие все понятия проверяемых характеристик, условия и сценарии тестирования, средства, используемые для испытаний;

- методики обработки и оценки результатов тестирования по каждому разделу программы испытаний.

Большой объем разнородных данных, получаемых при испытаниях крупномасштабных ПС, и разнообразие возможных способов их обработки, интерпретации и оценки приводят к тому, что важнейшими факторами становятся *методики обработки и оценки результатов, а также протоколы проверки по пунктам программы испытаний.* В соответствии с методиками испытаний средства автоматизации должны обеспечивать всю полноту проверок характеристик по каждому разделу методик Результаты испытаний фиксируются в протоколах, которые обычно содержат следующие разделы:

- назначение тестирования и раздел требований технического задания, по которому проводились испытания;
- указания методик, в соответствии с которыми проводились ис пытания, обработка и оценка результатов;
- условия и сценарии проведения тестирования и характеристи ки исходных данных:;
- обобщенные результаты испытаний с оценкой их на соответ ствие требованиям технического задания и другим руководя щим документам, а также технической документации;
- выводы о результатах испытаний и соответствии созданного ПС определенному разделу требований технического задания.

Протоколы по всей программе испытаний *обобщаются в акте,* в результате чего делается заключение о соответствии системы требованиям заказчика и завершении работы с положительным или отрицательным итогом. При полном выполнении всех требований технического задания заказчик обязан принять систему, и работа считается завершенной.

Наиболее полным и разносторонним испытаниям должна подвергаться первая базовая версия ПС. При *испытаниях очередных модернизированных версий ПС* возможны значительные сокращения объемов тестирования повторно используемых компонентов. Однако комплексные и завершающие испытания каждой новой версии ПС, как правило, проводятся в полном объеме, гарантирующем проверку выполнения всех требований измененного технического задания. Для обеспечения выявления дефектов в процессе эксплуатации серийных образцов в каждом из них должен быть предусмотрен некоторый минимум средств проверки функционирования и обнаружения искажений результатов. Этот

минимум средств должен позволять фиксировать условия неправильной работы программ и характер проявления дефектов. Последующее исправление ошибок должно проводиться специалистами, осуществляющими сопровождение.

При завершающих испытаниях основное внимание, кроме проверок функциональной пригодности, должно сосредоточиваться на подготовке стрессовых тестов, тестировании в режимах предельного использования ресурсов, надежности функционирования ПС. Задача испытателей и заказчика при проведении совместных испытаний состоит в выделении условий и области изменения переменных, которые недостаточно проверены разработчиком и важны для последующего надежного функционирования программ. При этом разработчик контролирует, чтобы планируемые сценарии и тесты не выходили из областей, заданных техническим заданием и спецификацией требований. Испытания за пределами технического задания могут квалифицироваться как его расширение или могут исключаться по требованию разработчика.

До начала испытаний подлежат проверке и паспортизации средства, обеспечивающие получение эталонных данных, средства имитации тестов от внешних объектов, средства фиксирования и обработки результатов тестирования. При испытаниях важную роль играют оценка и обеспечение близких значений методической и статистической достоверности результатов испытаний. *Методическая достоверность приемо-сдаточных испытаний ПС определяется следующими факторами:*

- полнотой программы испытаний и корректностью методик тестирования по охвату возможных условий и сценариев функционирования программ и областей изменения исходных данных;
- достоверностью и точностью эталонных значений, с которыми сравниваются результаты тестирования испытываемой программы или которые служат опорными при расчете параметров, зафиксированных в техническом задании;
- адекватностью и точностью моделей, используемых для имитации тестов от внешней среды и подыгрыша их реакции на управляющие воздействия;
- точностью и корректностью регистрации и обработки результатов тестирования, а также сравнения полученных данных с требованиями технического задания.

Представленная выше организация испытаний сложных ПС ориентирована на наличие конкретного заказчика комплекса программ и ограниченное число пользователей, контролируемых заказчиком. Несколько иначе организуются испытания коммерческих пакетов прикладных программ, создаваемых по инициативе разработчиков для широкого круга пользователей при отсутствии конкретного заказчика. Для таких коммерческих прикладных программ принято проводить испытания в два последовательных этапа — *Альфа- и Бета-тестирование.* Испытания проводятся на соответствие критериям, формализованным руководителем проекта. Они заключаются в нормальной и форсированной (стрессовой) опытной эксплуатации конечными пользователями оформленного программного продукта в соответствии с сопроводительной документацией и различаются количеством участвующих пользователей.

При Альфа-тестировании привлекаются конечные пользователи, работающие в той же компании, но не участвовавшие непосредственно в разработке комплекса программ. Для Бета-тестирования привлекаются добровольные пользователи (потенциальные покупатели), которым бесплатно передается версия ПС для опытной эксплуатации. При этом особое значение имеет выделение компетентных, тщательных и доброжелательных пользователей, способных своими рекомендациями улучшить качество испытываемых программ. Их деятельность стимулируется бесплатным и ранним получением и освоением нового программного продукта и собственной оценкой его качества. Эти пользователи обязуются сообщать разработчикам сведения о всех выявленных дефектах и ошибках, а также вносить изменения в программы и данные или заменять версии по указаниям разработчиков. Только после успешной эксплуатации и Бета-тестирования ограниченным контингентом пользователей, руководителем проекта или фирмы разработчиков принимается решение о передаче ПС в продажу для широкого круга пользователей Обобщение результатов Бета-тестирования может использоваться как часть или основа сертификационных испытаний.

При Альфа- и Бета-испытаниях принято разделять прогрессивное и регрессивное тестирование Под *прогрессивным* понимается тестирование новых программных компонентов, для выявления дефектов и ошибок в исходных текстах программ и спецификациях. *Регрессивное* тестирование предназначено для контроля

качества и корректности изменений в программах и данных после проведения корректировок. Необходимость и широта регрессивного тестирования определяются тем, что значительная доля изменений после Альфа- и Бета-тестирования, в свою очередь, содержит ошибки. Объем тестов и длительность обоих этапов тестирования определяются руководителями проекта в зависимости от сложности комплекса программ и интенсивности потока изменений.

5.10. Методика тестирования при испытаниях надежности сложных программных средств

Целью методики являются организация и регламентирование тестирования и испытаний сложных прикладных программ и их компонентов для достижения соответствия характеристик создаваемых ПС техническим требованиям и спецификациям, согласованным между заказчиком и разработчиком. Методика ориентирована на тестирование крупномасштабных ПС, функционирующих в реальном времени и состоящих из многих программных компонентов. Этапы тестирования и испытаний надежности комплекса программ представлены на рис. 5.4

В данной методике представлены четыре завершающих этапа тестирования в реальном времени, направленные на достижение высокого качества и надежности комплекса программ, состоящего из нескольких функциональных компонентов. Методика предполагает достаточную оснащенность процессов тестирования средствами автоматизации, а также высокую квалификацию специалистов в области современной программной инженерии, интеграции функциональных программных компонентов, проверки и обеспечения качества сложных информационных систем.

Тестирование и отладка программных компонентов в реальном времени

Разработка средств имитации внешней среды в реальном времени. Эти средства реализуются либо на специальной моделирующей ЭВМ, либо иными методами, обеспечивающими сохранение реального масштаба времени при тестировании. Их, может

быть, целесообразно унифицировать для всех последующих стадий испытаний без реальных объектов. Задавая режимы генерации тестов, можно обеспечить широкий спектр условий проверки функционирования различных компонентов и ПС в целом.

Разработка средств обработки результатов тестирования, испытаний и оценки качества и надежности функциональной компоненты в реальном времени. Эти средства значительно отличаются от используемых при тестировании компонентов в статике. Их применение должно обеспечивать необходимую информацию о процессе и надежности функционирования каждой компоненты и не нарушать при этом реальный масштаб времени. Результаты оценки качества компонент следует представлять в достаточно обобщенной и наглядной форме. Эти средства должны обеспечивать возможность определения практически всех показателей качества заданных техническими требованиями на компоненту и на ПС в целом.

Подготовка тестов для тестирования функциональной компоненты в реальном времени. Эти тесты должны генерироваться либо заранее для определенных моментов времени, либо непосредственно перед их выдачей на обработку. Они должны определять развитие процесса обработки информации тестируемой группой программ и отражать естественные процессы, протекающие во внешней среде, Сценарии тестирования должны содержать диаграммы изменения реального времени, которым соответствуют определенные содержания тестов в имитируемой внешней среде.

Завершение тестирования надежности функциональной компоненты в реальном времени. Это последние работы, на которых каждая компонента представлена более или менее автономно. Интеграция компонент во взаимодействии с другими компонентами должна обеспечить постепенное наращивание их до полного состава в версии ПС при функционировании в реальном времени. На этом этапе подключаются программы ввода-вывода и взаимодействия с внешней средой, визуализации и организации ■ всего вычислительного процесса в реальном'времени.

Обработка результатов тестирования и оценка качества и надежности функциональной компоненты в реальном времени. На этом этапе качество каждой компоненты должно определяться в полном ее окружении с другими группами программ при нормальном рабочем функционировании. Выделение и измерение характеристик надежности каждой компоненты на этом этапе

могут быть проще, чем на последующих вследствие неполного состава функционирующих групп программ в ПС. Достигнутые характеристики качества и надежности должны гарантировать возможность использования каждой компоненты повторно в различных версиях ПС. Испытанные компоненты, удовлетворяющие критериям качества, следует передавать от их разработчиков на комплексирование ПС специалистам, ответственным за тестирование и надежность ПС в целом.

Тестирование и испытания комплекса программ по данным имитаторов внешней среды

Комплексирование всех функциональных компонент в полной версии ПС. На этой стадии следует сформировать полную совокупность компонент в соответствии с требованиями, предъявленными к ПС. и начать тестирование и испытания ПС в полном составе. При этом сокращается участие в тестировании разработчиков соответствующих компонент, и они передаются вместе с комплектом откорректированной документации специалистам, ответственным за тестирование ПС в целом.

Подготовка тестов для испытаний ПС в реальном времени по данным имитаторов. Эти тесты в значительной степени повторяют использовавшиеся ранее и формируются теми же имитаторами. Основное их отличие состоит в расширении условий взаимодействия всех функциональных компонент в соответствии с требованиями технического задания на ПС с целью проверки выполнения всех предъявляемых требований, в том числе по надежности функционирования. В то же время некоторые достаточно глубоко проведенные проверки компонент могут пропускаться при формировании тестов и сценариев тестирования ПС в целом. При тестировании в некоторых системах тесты могут генерировать вручную специалисты на экранах дисплеев Б соответствии со сценариями тестирования и заданными временными диаграммами.

Завершение тестирования и испытаний ПС е реальном времени по данным имитаторов. На этой стадии тестирования основная цель состоит в полной проверке соответствия созданного ПС показателям качества и основным требованиями технического задания, в том числе по надежности функционирования. Для оценки наработки на отказ должен проводиться завершающий мно-

гочасовой прогон ПС при полном решении всех функциональных задач. При этом следует выделять и оценивать критические сценарии и параметры форсированного тестирования, при которых имитируемые тесты и эталоны могут отличаться от реальных характеристик объектов управления или источников информации. Такие параметры фиксируются для дополнительных проверок надежности на последующих этапах.

Обработка результатов испытаний и оценка достигнутого качества и надежности функционирования ПС в реальном времени по данным имитаторов. В тех случаях, когда ПС функционирует без участия пользователей-операторов и невозможно или нерентабельно его тестирование во взаимодействии с реальными объектами, на данной стадии испытания практически завершаются. После этого следует проводить окончательную корректировку документации и ПС предъявлять на приемосдаточные испытания. В иных случаях тестирование и испытания версии ПС проходят еще два этапа.

Тестирование и испытания надежности комплекса программ при воздействиях операторов-пользователей

Разработка тренажера и полунатурного имитатора внешней среды для испытаний ПС с участием операторов-пользователей, В ряде автоматизированных систем обработки информации при решении основных задач непосредственно функционирует один оператор или группа операторов-пользователей. Их динамические характеристики, уровень подготовки к работе с создаваемой версией ПС и реальные ошибки могут значительно влиять на процесс и результаты тестирования и достигаемые показатели надежности ПС. Поэтому необходимо создание средств, с использованием которых должна достигаться необходимая квалификация пользователей и обеспечиваться возможность испытаний ПС с учетом их реакции и реальных характеристик.

Подготовка тестов, имитирующих внешнюю среду, и сценариев воздействий от операторов-пользователей на функционирование ПС в реальном времени. Основная особенность этих сценариев и тестов состоит в необходимости оперативного учета реакции пользователей на данные, обрабатываемые испытываемым ПС Реакция и реальные характеристики пользователей могут оказывать существенное влияние на процесс тестирования и на дости-

гаемые показатели качества и надежности версии ПС. Следует учитывать, что временная диаграмма сценариев тестирования может выполняться операторами-пользователями с ограниченной точностью порядка 1-—3 секунд и значительно влиять на достоверность результатов тестирования. При совместном использовании тестов, генерируемых вручную, операторами и автоматизированно — программными имитаторами необходимо обеспечивать временную синхронизацию обобщения и ввода этих тестов.

Завершение испытаний версии ПС в реальном времени, по данным имитатора внешней среды, при наличии воздействий от операторов-пользователей. На этой стадии тестирование и испытания надежности ПС в значительной степени усложняются за счет неполной определенности и сложности фиксирования реальных реакций пользователей. При обнаружении аномалий и отказов функционирования программ, вследствие возможных различий реакций пользователей, не каждый сценарий и тест можно повторить абсолютно точно. Это усложняет диагностику и локализацию некоторых видов сложных дефектов в программах. Сценарии и тесты должны обеспечить полную проверку надежности комплекса программ при наличии воздействий от операторов и от имитаторов в реальном времени.

Тестирование комплексов программ, ориентированное на выявление определенных типов дефектов, при экстремальных ситуациях. Предыдущие этапы тестирования и отладки направлены в основном на выявление и устранение дефектов функционирования программ в типовых, штатных условиях применения. Попутно создаются некоторые критические и экстремальные ситуации в сценариях и тестах. Однако для обеспечения высокого качества и надежности функционирования комплекса программ при любых исходных данных и характеристиках внешней среды (в пределах заданных технических требований) необходимо провести упорядоченное, систематическое тестирование по сценариям и тестам, ориентированным на проверку ПС при нештатных, экстремальных и редко встречающихся условиях, которые нерационально или опасно создавать при тестировании в реальной внешней среде.

Обработка результатов испытаний в реальном времени и оценка качества и надежности функционирования ПС при взаимодействии с операторами-пользователями и в экстремальных ситуациях. Характеристики и подготовка пользователей, а также полно-

та проверок при нештатных и предельных ситуациях могут значительно влиять на показатели качества и надежности ПС. Гарантии качества ПС при проведении этого этапа испытаний предполагают определенный уровень подготовки пользователей и специалистов по тестированию сложных комплексов программ. Поэтому степень соответствия ПС техническому заданию должна определяться после необходимой подготовки и аттестации как пользователей, так и коллективов, проводящих тестирование и обработку результатов испытаний.

Испытания комплекса программ в реальной внешней среде

Разработка методики безопасного сопряжения комплекса программ с реальной внешней средой. Реальные объекты внешней среды под воздействием ошибочных данных от испытываемого ПС могут переходить в критические состояния, угрожающие значительным ущербом, разрушением объектов или порчей выпускаемой продукции. Поэтому методика сопряжения должна предусматривать последовательное подключение данных от ПС к объектам внешней среды, гарантирующее безопасность их функционирования даже при наличии дефектов и ошибок в тестируемом программном средстве.

Комплектование испытываемого программного средства с реальной внешней средой. Целесообразно ранжировать функциональные задачи и компоненты ПС по степени безопасности их воздействия на объекты внешней среды. Функциональные компоненты, потенциально опасные, при проявлении дефектов и отказов в программах и данных следует комплексировать с внешней средой в последнюю очередь. Техническое сопряжение ЭВМ с объектами внешней среды подготавливает возможность подключения к ним реальных воздействий от функционирующего ПС. При комплексировании следует проверить правильность воздействий данных от ПС на объекты внешней среды и информации от них, поступающей на ПС. Для уменьшения риска аварийных ситуаций целесообразно первоначально проверять поступление внешней информации на комплекс программ, затем выдачу воздействий на объекты внешней среды и, наконец, полное замыкание контура обработки информации и управления объектами внешней среды от комплекса программ.

Подготовка и аттестация измерителей параметров данных от объектов внешней среды, поступающих на комплекс программ. Для объективной оценки качества и надежности функционирования ПС в реальной среде необходимо достаточно точное измерение основных ее характеристик в процессе тестирования программ. Тем самым должно обеспечиваться оперативное определение тестовых значений, поступающих из внешней среды, и воздействий данных от ПС на ее объекты. Эти тесты, состояния внешней среды и реакции комплекса программ при измерениях и регистрации показателей качества должны быть синхронизированы единым временем.

Завершение испытаний комплекса программ в реальном вречени и в реальной внешней среде. Длительные испытания надежности ПС с реальными внешними объектами могут отличаться недопустимо высокой стоимостью и недостаточной повторяемостью. Кроме того, эта заключительная стадия испытаний ПС может иметь ограничения, обусловленные опасностью или невозможностью создавать критические ситуации, способные вызвать разрушения объектов внешней среды или опасные искажения накопленной и обработанной информации. Такие ситуации целесообразно проверять во взаимодействии с имитаторами внешней среды. Поэтому испытания на этой стадии часто следует сводить к сравнению результатов функционирования программ с данными, полученными вне критических ситуаций на предшествующих стадиях испытаний, и с требованиями технического задания. Уточнение результатов тестирования и характеристик надежности ПС необходимо вследствие возможной неполной адекватности имитаторов реальным объектам внешней среды. Кроме того, функционирующая версия ПС может изменять характеристики реальной внешней среды, что не всегда удается полностью предусмотреть в имитаторах.

Обработка результатов испытаний и оценка достигнутого качества и надежности комп чекса программ в реальном времени и в реальной внешней среде Завершающая оценка качества и надежности ПС целесообразна с привлечением результатов, полученных на предшествующих этапах. При этом следует сопоставлять все данные проведенных проверок и дополнять их результатами тестирования в критических ситуациях при использовании имитаторов и тренажеров. Обобщение всех результатов испытаний должно позволить установить степень соответствия програм-

много средства техническому заданию и документации и пригодность его для предъявления заказчику на завершающие приемо-сдаточные испытания.

Корректировка технологической и эксплуатационной документации на версию программного средства по результатам всех стадий испытаний для предъявления заказчику В ходе всех этапов комплексной отладки и испытаний разработчиков производится естественная корректировка отдельных документов. Однако некоторые документы на компоненты и на ПС в целом могут отставать по содержанию изменений от реального состояния программ. Это относится преимущественно к текстовым документам, содержащим описания программ, функций и алгоритмов решения задач. Поэтому должны быть проведены работы, в результате которых обеспечено полное взаимно однозначное соответствие между программами и сопровождающими их документами, необходимыми как для пользователей, так и для специалистов, осуществляющих сопровождение версий ПС или использующих испытанные компоненты в иных областях применения.

Оформление акта завершения испытаний разработчиков и предъявление версии комплекса программ заказчику для проведения приемо-сдаточных испытаний или сертификации. Обобщенные характеристики достигнутого качества и надежности ПС, отлаженные программы и комплект документации позволяют фактически и формально соответствующим актом завершить комплексную отладку и предварительные испытания ПС. При этом не всегда удается выполнить все требования заказчика, и некоторые характеристики комплекса программ могут отличаться от требовавшихся. По таким данным следует подготовить обоснование причин отклонения от технического задания и спецификаций и, возможно, провести дополнительные проверки комплекса программ или корректировки исходных требований. После согласования корректировок и акта комплекс программ должен быть передан заказчику для приемо-сдаточных испытаний или сертификации по специальной программе.

Состав документов, поддерживающих интеграцию программных компонентов, тестирование и комплексные испытания версий программных средств:

- план и методика комплексирования и сборки программных и информационных компонентов версии ПС;

описание средств автоматизации тестирования программ, обработки результатов отладки и поддержки конфигурационного управления изменениями в версии ПС; план и методика комплексной отладки в имитированной внешней среде;
результаты тестирования и полные характеристики комплекса программ в имитированной внешней среде; план и методика интеграции версии ПС с аппаратными средствами в реальной операционной и внешней среде; план и методика комплексной отладки и квалификационного тестирования версии ПС в реальной операционной и внешней среде;
тесты, сценарии и генераторы тестовых данных, использованные для отладки программных и информационных компонентов и версии ПС в целом;
результаты квалификационного тестирования, функциональные характеристики ПС в реальной внешней среде; полные характеристики достигнутого качества и надежности функционирования, а также степени покрытия тестами спецификации требований к ПС;
откорректированные тексты программ, описания данных и полные спецификации требований на программные компоненты и ПС в целом после полного завершения тестирования и испытаний главного конструктора;
отчет о результатах завершения комплексной отладки, подтверждении заданного качества и надежности и готовности ПС к предъявлению заказчику на приемо-сдаточные испытания;
программа приемо-сдаточных испытаний комплекса программ по всем требованиям технического задания; комплект методик испытаний и обработки результатов по всем разделам программы испытаний;
план методики и средства автоматизации обучения заказчика и пользователей применению испытанной версии ПС;
комплект эксплуатационной документации, описание ПС и руководство пользователя в соответствии с условиями контракта;
технические условия на версию ПС, базу данных и эксплуатационную документацию для тиражирования и серийного производства;

- руководство по установке, генерации пользовательской версии ПС и загрузке базы данных в соответствии с условиями и характеристиками внешней среды;
- отчет о технико-экономических показателях завершенного проекта версии ПС, выполнении планов и использованных ресурсах;
- акт о завершении комплексной отладки и испытаний главного конструктора и готовности к предъявлению для приемо-сдаточных или сертификационных испытаний версии ПС [49].

5.11. Тестирование программного обеспечения

На современном этапе развития информационных технологий программное обеспечение характеризуется большой степенью сложности. Особенностью программного обеспечения, разрабатываемого для сферы экономики, является то, что оно постоянно изменяется. Связано это прежде всего с изменениями, происходящими в предметной области (введение новых услуг, бизнес-процессов, изменение законодательства).

Сегодня можно говорить (нужно говорить) о программном обеспечении как о промышленном продукте, соответственно о создании ПО — как о производстве. Существует множество стандартов поддержки жизненного цикла программного обеспечения. Разработано множество стандартов и методик поддержки стадий ЖЦ ПО, например стандарты ISO 9000 и ISO 9001, разработанные Международной организацией по стандартизации (ISO).

На сегодняшний день автоматизировано большинство этапов разработки программного обеспечения, в том числе и этап тестирования. Однако в отечественной практике тестированию программных средств отведена незаслуженно маленькая роль. Причинами могут быть отсутствие денег на приобретение дорогостоящих CASE-средств, поддерживающих все этапы разработки ПО, в том числе тестирование, или нежелание держать такую штатную единицу, как специалист по тестированию. Обычно приобретают средства автоматизации проектирования (создание ER-моделей, информационных моделей и пр.) и программирования (автоматическое создание БД на основе ER-модели, создание

интерфейса на основе ER-модели, визуальное программирование). С тестированием дела обстоят сложнее.

Тестирование занимает важное место в жизненном цикле программного обеспечения, это трудоемкий и дорогостоящий процесс. В организационной структуре современной фирмы — разработчика ПО должен быть отдел по тестированию программного обеспечения или специалист по тестированию программного обеспечения, так как одним из принципов тестирования является избежание тестирования автором, а тестирование программного средства напрямую связано с его качеством.

Отечественные производители коммерческого программного обеспечения только сейчас серьезно задумались о качестве своей продукции и, как следствие, о тестировании.

В качестве объективных причин, почему это происходит именно сейчас и почему этого не было раньше, можно выделить следующие.

1. Сформировалась нормативно-правовая база для осуществ ления деятельности по разработке программного обеспечения, база в области авторского права и смежных прав, а также защи ты прав потребителей.

2. Законодательно закрепленные принципы работают и ак тивно используются в правоприминительной практике. Особен но это относится к защите авторских прав.

Исполнительная власть начала пресекать попытки распространения нелицензионного программного обеспечения. Есть прецеденты процессов, по которым возмещаются потери от нарушения авторских прав.

3. Возросли требования заказчика к качеству программного обеспечения. Данный момент связан с правовой базой (законы о защите прав потребителей), жесткой конкуренцией на рынке.

4. Накоплен большой опыт создания программных средств, российские компании выходят на другие рынки (в том числе на мировой рынок), что влечет необходимость выполнения новых норм по качеству программных средств.

По этим причинам процессы обеспечения и контроля качества, одним из которых является тестирование, приобретают в настоящий момент большое значение и актуальность. О современном тестировании программных средств и пойдет речь в этом разделе. В нем будут рассмотрены аспекты, связанные с теорией тестирования, методологией тестирования, организацией тестирования, автоматизацией тестирования.

Цель тестирования

Целью тестирования является обнаружение максимального количества ошибок, а не всех ошибок в программе. Обнаружение всех ошибок невозможно. Полное и абсолютное тестирование выглядит скорее мечтой, чем реальностью. Вот простой пример: еще в 1979 г. Майерс (Myers) описал некоторый простой алгоритм. В нем был всего один цикл и несколько операторов условного перехода. В большинстве языков программирования для кодирования такого алгоритма потребуется не более 20 строк кода. Но такая программа имеет более 100 триллионов путей выполнения! Самому быстрому тестировщику для полного тестирования потребовался бы как минимум миллион лет. Аналогичная программа из 100 строк имеет 10 триллионов путей выполнения, если на проверку выполнения одного пути тратить 1 секунду, то на полное ее завершение не хватит времени существования всей нашей вселенной, время жизни которой меньше $4 \blacksquare 10^{17}$ секунд!

Таким образом, целью тестирования является не тотальное обнаружение всех ошибок (это принципиально невозможно), а выявление наибольшего количества наиболее критичных ошибок. Если исправление их задерживается, то пользователи программного продукта должны быть предупреждены о наличии такого рода ошибок и рекомендуемых путях обхода.

Если процесс тестирования становится бесконечным, а полное тестирование невозможно, то чем же определяется принятие решения о выпуске в свет исследуемой версии программного продукта? Основными критериями завершенности тестирования является отсутствие критичных ошибок, каждая из которых может сделать абсолютно невозможной реализацию декларированной в системе прикладной функциональности (решение принимается по результатам функционального тестирования). Кроме того, при принятии решения учитывается общее количество зарегистрированных, но неисправленных ошибок. Компания-разработчик обычно заранее выбирает по каждому программному продукту общее количество ошибок (лимит), с которым уже нельзя выпускать программный продукт.

Количественная оценка завершенности процесса тестирования и готовности программного продукта для эксплуатации может быть получена при помощи моделей надежности програм-

много обеспечения. Самый простой способ представления информации для принятия решения — графический: по одной оси откладывается время от начала процесса тестирования, по другой — количество обнаруженных ошибок в программном средстве. По графику (знаку производной) определяется необходимость продолжения тестирования. Существует множество методов, которые помогают принять решение в выпуске программного обеспечения, однако самое веское слово остается за специалистом, осуществляющим тестирование программного обеспечения, так как на основе количества и характера найденных проблем он может судить о том, удовлетворит данный продукт потребности и ожидания пользователя или нет.

Тестирование и качество

Как уже говорилось в предыдущей главе, тестирование программного обеспечения имеет тесную связь с качеством программного обеспечения.

Современные идеологи проблем качества разделяют понятие «качество» на внешнее (external) и внутреннее (internal). Внешнее качество программного обеспечения — его способность удовлетворить потребность конечного пользователя. Именно на это и направлен процесс тестирования программного обеспечения — обнаружение ошибок и несоответствий, т.е. в процессе тестирования выявляются те моменты (ошибки, неправильная реализация или отсутствие функциональных возможностей), которые не удовлетворили бы конечного пользователя. Тестирование программного обеспечения обеспечивает контроль качества продукта, поставляемого конечным пользователям.

Внутреннее качество программного обеспечения связано с удобством его производства для тех, кто его производит, с его технологичностью, стандартизованностью, безопасностью. Вопросы внутреннего качества в большей степени связаны с реализацией процессов жизненного цикла программного обеспечения, с процессами управления разработкой программного обеспечения. Улучшая документированность тестов, их более простую адаптируемость от версии к версии программного обеспечения, специалист по тестированию улучшает внутреннее качество программного обеспечения.

Виды тестирования

В этой главе было перечислено большое количество видов тестирования программных средств. Наиболее важные для практики перечислены ниже.

1. *Функциональное* — тестирование возможностей системы, ее реакция на те или иные ситуации. Обычно результат тестиро вания (реакция системы) сравнивается с постановкой задачи, при несоответствии фиксируется ошибка.
2. *Регрессионное* — проверка полноты реализуемых функций си стемы по сравнению с предыдущей версией программного про дукта.
3. *Нагрузочное* — тестирование работы системы на пиковую на грузку, при этом делается вывод о производительности систе мы. Например, выясняется среднее время ввода одного доку мента (если программное обеспечение предназначено для хра нения и обработки документов). Условием для нагрузочного тестирования является выполнение испытаний на одной и той же конфигурации системы. Если тестируется производитель ность на 2-х разных СУБД, то конфигурация системы должна быть идентичной (тот же сервер, те же рабочие станции), в испытаниях меняются лишь СУБД. На основе нагрузочного тестирования выдвигаются требования к аппаратной части и программной части системы (операционная система, СУБД).
4. **Контроль после исправления** (обратная связь). Этот вид тес тирования подразумевает под собой проверку уже исправлен ных ошибок.
5. **Стрессовое тестирование** — проверка реакции системы на вне
   штатные ситуации. Примером может служить проверка систе мы на восстановление работоспособности после отключения питания на сервере базы данных.
6. **Адаптационное тестирование** — проверка корректности пере
   вода программного обеспечения на другой национальный язык.

Место тестирования в процессе разработки ПО

Рассмотрим организационную структуру типичной компании — разработчика программного обеспечения и более подробно подразделение, которое занимается тестированием программно-

го обеспечения (отдел тестирования, группа тестирования). Выделим задачи, которые решает отдел тестирования, его внутреннюю структуру, взаимодействие с другими отделами.

Вообще структура фирмы — разработчика программного обеспечения отражает этапы жизненного цикла программного средства. То или иное подразделение обеспечивает выполнение работ по одному или нескольким этапам жизненного цикла программного обеспечения.

Аналитический отдел. В задачи аналитического отдела входят;

- определение концепций **и** функционального направления раз вития программного продукта;
- проведение предпроектного обследования;
- определение функциональных возможностей системы;
- определение (совместно с разработчиками) технических требо ваний к системе;
- описание бизнес-процессов предметной области в терминах, понятных разработчикам (постановки задач и спецификации на разработку);
- написание постановок задач и спецификаций на доработку про граммного средства при изменении законодательства, требо ваний клиентов, расширении функциональных возможностей продукта;
- контроль процесса реализации новых возможностей в про граммных продуктах компании.

Отдел документации. Часто данный отдел не выделяется в обособленную структуру, он может входить, например, в состав аналитического отдела.

В задачи отдела входят написание технической документации для конечного пользователя, отслеживание изменений в программном средстве и актуализация в документации.

Отдел разработки. Это ключевой отдел для фирмы. Если без остальных отделов зачастую можно обойтись, то без отдела разработки никак нельзя. В его задачи входят:

- определение (совместно с аналитиками) технических требова ний к системе;
- реализация базовых функций программного средства;
- расширение перечня функций программного средства (реали зация доработок);
- исправление найденных ошибок;

- адаптация программного продукта для функционирования в других условиях (переход на новую СУБД, новый язык программирования и пр.);
- оптимизация программного продукта (увеличение быстродействия, надежности и пр.).

Отдел технической поддержки (горячая линия). Осуществляет консультации пользователей по вопросам, связанным с установкой и эксплуатацией программного средства по различным каналам связи (телефон, почта, электронная почта).

Отдел тестирования. В задачи отдела входят:

- комплексный контроль качества;
- подготовка тестовой документации (планы тестирования и пр.);
- обнаружение и локализация ошибок в функционировании программных продуктов;
- фиксирование и отслеживание ошибок в функционировании программных средств,
- проверка соответствия документации программного продукта стандартам и реально реализованным функциям,
- участие в разработке и внедрении системы качества;
- автоматизация тестирования;
- оценка производительности разрабатываемых программных средств на различных программно-аппаратных платформах и их специфических конфигурациях.

В некоторых компаниях на отдел тестирования возлагаются сборка и выпуск программного обеспечения (в некоторых компаниях этим занимается отдел разработки).

Все отделы компании взаимодействуют между собой, данные взаимодействия упорядочены между собой и представляют производственные технологические процессы Технологические процессы, как правило, регламентированы внутренними документами или внутрикорпоративными стандартами, в совокупности представляют собой технологический цикл производства программного средства.

Типичных технологических цепочек внутри компании — разработчика программного обеспечения большое множество. В качестве примера рассмотрим схему взаимодействия отдела тестирования программного обеспечения с другими отделами при обнаружении ошибки во время функционирования программного обеспечения у пользователя (рис. 5.5).

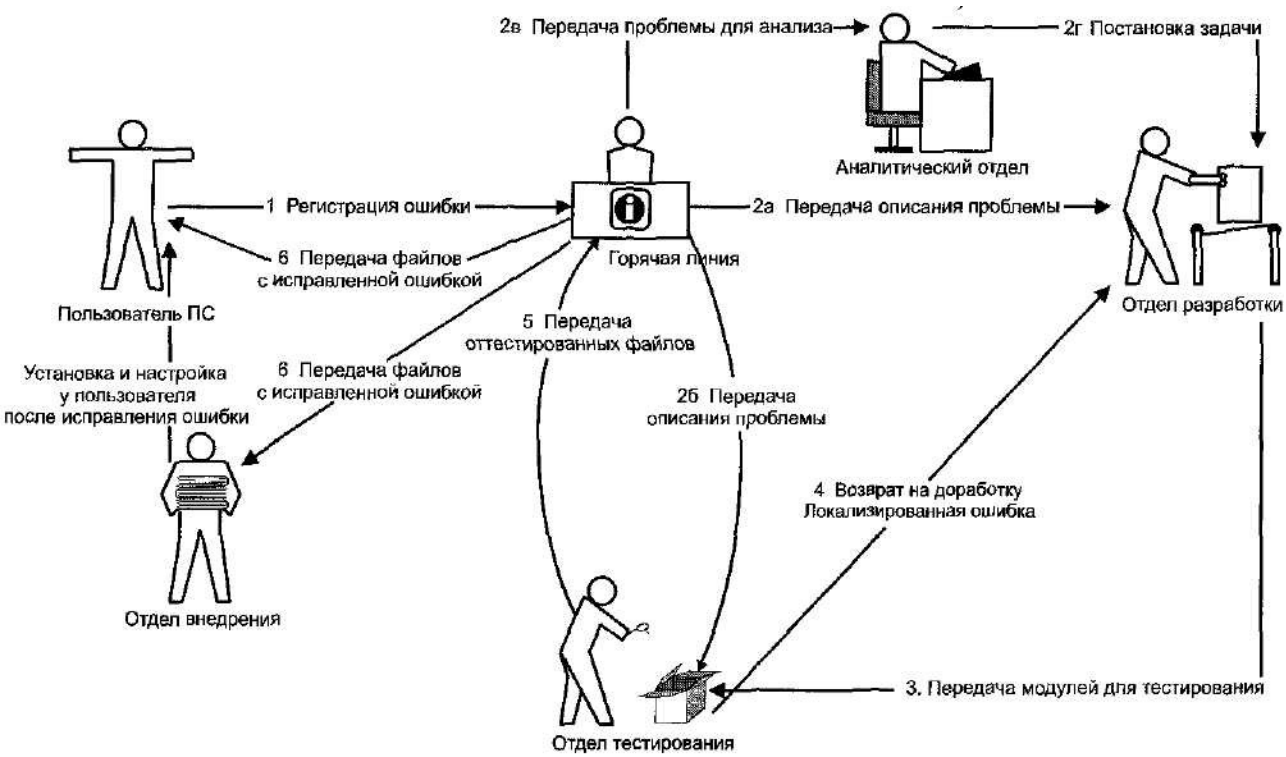


Рис. 5.5. Схема технологического процесса исправления ошибки ПС при обнаружении ее пользователем

Особую роль при разработке программного обеспечения играет контроль качества программ, поставляемых пользователям. Здесь особую роль играет отдел тестирования и/или подразделение, осуществляющее приемку готового продукта. В задачи данных подразделений входит промежуточный и конечный контроль качества программных средств. Если качество программного продукта при приемке не соответствует необходимому уровню, то подразделение, ответственное за контроль качества, может забраковать его, т.е. наложить вето на выход программного продукта. Некачественное программное обеспечение к пользователю не попадет.

1. По поддерживаемым каналам связи (обычно различные виды электронной почты, телефон) пользователь при обнаруже нии ошибки в программном средстве обращается в службу тех нической поддержки. Специалист пытается воспроизвести про блему, и если она не является ошибкой в силу неправильных дей ствий пользователя или недостаточных знаний пользователя, то специалист службы технической поддержки дает пользователю консультацию или ссылку на документацию пользователя.

2. Если специалист службы технической поддержки определил возникшую проблему как ошибку, то он подробно описывает ее и отправляет описание ошибки либо в отдел разработки для ис правления (2а), либо в отдел тестирования для локализации, если ошибку, возникшую у пользователя, воспроизвести не удалось (26). Если проявление ошибки связано с ситуацией, которая тре бует изменения постановки задачи и анализа, описание пробле мы передается в аналитический отдел ответственному аналитику (2в). После анализа и написания постановки задачи на доработ ку аналитик передает описание проблемы вместе с постановкой в отдел разработки (2г).

3. Разработчик исправляет ошибку, после чего передает опи сание проблемы, модули и постановку задачи (если проблема про шла через аналитический отдел) в отдел тестирования (3). Тестер воспроизводит ошибку по описанию, сверяет полученный резуль тат с постановкой задачи. Проверяет все возможные места в про граммном средстве, где ошибка могла иметь свое проявление. В случае, если ошибка исправлена, тестер передает оттестиро ванные модули в отдел технической поддержки (5). Если ошибка не была исправлена или исправлена не полностью, тестер воз вращает задачу с описанием проблем, которые не были исправ-

лены, в отдел разработки (4). Цикл разработчик — тестер (3—4 на рис. 5.5) может повторяться несколько раз, пока ошибка не будет полностью исправлена.

4. Отдел технической поддержи принимает оттестированные модули, описание проблемы, комментарии тестера, далее по со кращенной программе удостоверяется, что ошибка исправлена (осуществляет приемочное тестирование), и передает файлы од ним из способов пользователю (по электронной почте, через ftp-сервер, на магнитном или ином носителе) (6а). В случае, если пользователь не в состоянии самостоятельно установить файлы, в которых исправлена ошибка, или этого не позволяет програм мное средство, или с пользователем оговорен иной способ пере дачи файлов в технологический процесс, то подключается отдел внедрения (6). В зависимости от внутренней технологии оттести рованные файлы могут попадать как из отдела технической под держки (6), так и непосредственно от тестера. Далее специалист отдела внедрения проводит установку файлов, настройку систе мы и конвертацию данных, если исправление ошибки повлекло изменение хранимых данных программного средства.

5. Описанный выше технологический процесс не является дог мой и может быть изменен в зависимости от сложившихся обсто ятельств. Один принцип должен выполняться всегда для обеспе чения качества программного продукта — программный продукт не должен попадать к пользователю неоттестированным.

Специалист отдела тестирования - квалификационные требования

Рассмотрим требования к навыкам и умениям, которыми должен обладать специалист по тестированию программного обеспечения. Такого специалиста обычно называют инженером по тестированию ПО, тестером, тестировщиком. Уникальность специалиста по тестированию обусловлена его работой и состоит в том, что сфера его деятельности находится на стыке нескольких дисциплин. Выделим требования, предъявляемые к знаниям и навыкам специалиста по тестированию программных средств.

1. Тестер должен очень хорошо знать предметную область, для которой пишется программное обеспечение, будь то торговля, банковское дело или бухгалтерский учет. Он должен знать все

аспекты и тонкости предметной области — документооборот, процессы.

2. Тестер должен иметь знания в области проектирования и программирования программного обеспечения, чтобы уметь чи тать постановки задачи, ER-модели и уметь предположить, чем вызвана ошибка (сбой операционной системы, отсутствие файла таблицы БД и пр.).

3. Тестер должен уметь ориентироваться в широком спектре программного обеспечения — текстовых редакторах. СУБД, элек тронных таблицах, операционных системах, архиваторах, ути литах сравнения, языках программирования, средствах разработ ки, отладчиках и др.

4. Тестер должен уметь составлять техническую документацию, чтобы изложить четко и логично ситуацию, в которой возникает ошибка, и описагь. на что она похожа, а также для составления собственных методик тестирования.

5. Иметь представление о принципах, методах и стратегиях тестирования, а также о моделях надежности программного обес печения.

6. Знать и уметь программировать на одном или нескольких языках программирования (для написания автоматических тес тов, а также для поиска ошибок в коде).

Инструментарий специалиста по тестированию

Текстовый процессор. Используется как средство описания проблем, создания тестовой документации, просмотра постановок задач и спецификаций.

Плановый процессор. Используется для создания планов тестирования, распределения людских и машинных ресурсов по задачам тестирования. Активно используется руководителями отдела тестирования.

Электронная таблица. В повседневной работе используется для проверки правильности вычислений, операциями над матрицами, может выступать как средство для фиксации проблем.

Система отслеживания и документирования проблем. В качестве примеров приведем продукты Diasoft Office4x4 (компании

Diasoft4x4), SoDa (компании Rational). TeamTrack (компании TeamShare).

Утилиты сравнения файлов, просмотра файлов.

Конверторы файлов.

Утилиты для создания копий экрана. Используются для анализа ошибок.

Языки программирования. Используются для просмотра исходного текста программы, для написания автоматизированных тестов.

Специализированные средства. К ним относятся средства записи/эмуляции клавиатурного ввода, отладчики.

Запись клавиатурного ввода используется для фиксирования действий пользователя в программе, которая привела к ошибке.

Устройства видеозаписи. К ним относятся специализированные платы для записи видеоизображения с экрана, видеомагнитофон и видеокамера. Предназначены для фиксирования действий пользователя в программе, которые привели **к** сложной ошибке. Использовать видеотехнику целесообразно при воспроизведении сложных ошибок.

Передовые технологии в тестировании (автоматизация тестирования)

Автоматизация тестирования применяется широко при таких видах тестирования, как регрессионное тестирование, приемочное тестирование.

Принципы создания автоматического теста:

- Время по поддержке и сложность теста должны быть меньше, чем сложность кода, которую он тестирует.
- Время по адаптации — минимальное.
- Минимальная зависимость от национального языка интерфей са программного продукта.
- Необходимо анализировать значащие элементы экрана (элемен ты, содержащие информацию).
- Тесты должны быть независимы от разрешающей способности экрана.

- Если программный продукт реализован на нескольких языках программирования, не ориентироваться на особенности одно го языка.
- Не имеет смысла использовать автоматизированные тесты для новых функциональных возможностей. Здесь появляется наи большее количество ошибок, код непостоянен, что влечет боль шие затраты по адаптации теста.

Что касается инструментария, применяемого при контроле качества, лучшие результаты дает использование автоматических тестов с применением специальных промышленных средств автоматизации тестирования. Высокую эффективность имеют также специальные тестовые процедуры, написанные на языке программирования, на котором написано само ПС. Преимущества использования автоматических тестов перед тестированием очевидны:

- они беспристрастны;
- позволяют выполнять проверку необходимое количество раз;
- не устают и не ошибаются;
- могут протестировать гораздо больше за меньшее количество времени;
- не требуют дополнительной оплаты при работе по ночам и выходным;
- более четко отвечают на вопрос, что протестировано и с каким результатом.

Однако создание и поддержка банка тестов — более сложная задача и требует высокой квалификации сотрудников отдела тестирования. За все надо платить, но качество конечного продукта того стоит. Тестирование — это дорогостоящий и трудоемкий процесс, поэтому зарубежными компаниями ведутся разработки в области автоматизации тестирования. Попытки применения автоматизации тестирования связаны с тем, что в принципе невозможно полностью протестировать программный продукт, соответственно специализированные пакеты приближают «покрытие» тестами программы к 100%. На рынке специальных сред для тестирования программного обеспечения можно отметить разработки ведущих в этой области фирм: Rational (Visual Test, Rational Robot, Team Test и др.), Mercury Interactive (WinRunner), Segue Software (QA Partner).

Надо сказать, что такое ПО весьма специфично и имеет достаточно высокую цену — порядка нескольких десятков тысяч долларов.

Пакеты тестирования можно разделить по поддерживаемой стратегии тестирования на пакеты, поддерживающие стратегию «белого ящика» и на поддерживающие стратегию «черного ящика».

Пакеты, реализующие стратегию «белого ящика», позволяют:

- записывать, а потом воспроизводить последовательность пользовательского ввода (нажатие клавиатуры, движения «мышью»);
- распознавать объекты и их свойства .(окна Windows, текст в окне и пр.);
- запоминать копию экрана;
- сравнивать состояние программы относительно предыдущего тестового прогона;
- производить математические вычисления на основе данных из тестируемой программы;
- замерять выполнение одной и той же последовательности дей ствий в различных условиях;
- эмулировать выполнение программы несколькими пользовате лями одновременно:
- записывать подробный протокол выполнения автоматическо го теста;
- другие функции.

Пакеты, реализующие стратегию «черного ящика», позволяют:

- отслеживать выполнение того или иного фрагмента кода про граммы;
- подсчитывать количество выполнения того или иного фрагмента кода программы;
- вычислять время выполнения участка кода (важно при пере смотрах кода и его оптимизации);
- подсчитывать общее «покрытие» программы;
- автоматически контролировать значение переменных и выда вать ошибку или предупреждение, если значения не совпадают с теми, которые ожидаются;
- на основе данных, полученных от пакета автоматизации тести рования, возможно выполнять расчеты о надежности программ ного обеспечения;
- получать различные статистические данные о программе.

Средства автоматизации тестирования не предполагают отсутствие инженера по тестированию, а требуют от него новых

знаний. Программа автоматизации тестов не выполнит всю работу по тестированию сама. Для нее нужны специальные инструкции — сценарии тестов, написанных на специально разработанном языке. Таким образом, автоматизация заключается в избавлении инженера по тестированию от рутинной работы, теперь тестер занимается разработкой тестов и программированием тестов на языке системы автоматизации тестирования [64, 66].

ВОПРОСЫ ДЛЯ САМОПРОВЕРКИ

1. Дайте определение понятию тестирования.
2. Что такое тестирование «белого ящика»?
3. Что такое тестирование «черного ящика»?
4. В чем на ваш взгляд заключается «философия» тестирования?
5. Перечислите основные инструментальные средства тестировщика.
6. Расскажите про метод сандвича.
7. В чем заключается метод большого скачка?
8. Каково место отдела тестирования в компании —• разработчике
   программного обеспечения?
9. Как узнать о необходимости завершения тестирования?
10. Можно ли на практике обнаружить все ошибки в программном средстве, если можно, то как это сделать?
11. Опишите место и роль тестирования в процессе разработки про граммного обеспечения.
12. Перечислите основные аксиомы (принципы) тестирования.
13. Что представляет собой тестирование психологических факторов?
14. Какие из передовых технологий тестирования вам запомнились?

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

Стандарты

1. ГОСТ 19.001-77 ЕСПД. Общие положения.
2. ГОСТ 19.005-85 ЕСПД. Р-схемы алгоритмов и программ. Обозначения условные графические и правила выполнения.
3. ГОСТ 19.101-77 ЕСПД. Виды программ и программных документов.
4. ГОСТ 19.102-77 ЕСПД. Стадии разработки.
5. ГОСТ 19.103-77 ЕСПД. Обозначение программ и программных документов.
6. ГОСТ 19.104-78 ЕСПД. Основные надписи.
7. ГОСТ 19.105-78 ЕСПД. Общие требования к программным документам.
8. ГОСТ 19.106-78 ЕСПД. Требования к программным документам, выполненным печатным способом.
9. ГОСТ 19.201-78 ЕСПД. Техническое задание. Требования к содержанию и оформлению.
10. ГОСТ 19.202-78 ЕСПД. Спецификация. Требования к содержанию и оформлению.
11. ГОСТ 19.301-79 ЕСПД. Порядок и методика испытаний.
12. ГОСТ 19.401-78 ЕСПД. Текст программы. Требования к содержанию и оформлению.
13. ГОСТ 19.402-78 ЕСПД. Описание программы.
14. ГОСТ 19.403-79 ЕСПД. Ведомость держателей подлинников.
15. ГОСТ 19.404-79 ЕСПД. Пояснительная записка. Требования к содержанию и оформлению.
16. ГОСТ 19.501-78 ЕСПД. Формуляр. Требования к содержанию и оформлению.
17. ГОСТ 19.502-78 ЕСПД. Описание применения. Требования к содержанию и оформлению.

18. ГОСТ 19.503-79 ЕСПД. Руководство системного программиста.
Требования к содержанию и оформлению.
19. ГОСТ 19.504-79 ЕСПД. Руководство программиста. Требования
к содержанию и оформлению.
20. ГОСТ 19.505-79 ЕСПД. Руководство оператора. Требования к
содержанию и оформлению.
21. ГОСТ 19.506-79 ЕСПД. Описание языка. Требования к содержа
нию и оформлению.
22. ГОСТ 19.507-79 ЕСПД. Ведомость эксплуатационных докумен
тов.
23. ГОСТ 19.508-79 ЕСПД. Руководство по техническому обслужи
ванию. Требования к содержанию и оформлению.
24. ГОСТ 19.601-78 ЕСПД. Общие правила дублирования, учета и
хранения.
25. ГОСТ 19.602-78 ЕСПД. Правила дублирования, учета и хране
ния программных документов, выполненных печатным образом.
26. ГОСТ 19.603-78 ЕСПД. Общие правила внесения изменений.
27. ГОСТ 19.604-78 ЕСПД. Правила внесения изменений в программ
ные документы, выполняемые печатным способом. ■
28. ГОСТ 19.701-90 ЕСПД. Схемы алгоритмов, программ, данных и
систем. Условные обозначения и правила выполнения.
29. ГОСТ 19781-90. Обеспечение систем обработки информации про
граммное. Термины и определения.
30. ГОСТ 34.601-90. Информационная технология. Комплекс стан
дартов на автоматизированные системы. Автоматизированные си
стемы. Стадии создания.
31. ГОСТ 34.602-89. Информационная технология. Комплекс стан
дартов на автоматизированные системы. Техническое задание на
создание автоматизированной системы.
32. ГОСТ 34.603-92. Информационная технология. Виды испытаний
автоматизированных систем.
33. MIL-STD-498. Разработка и документирование программного
обеспечения.
34. ISO 9126:1991. Информационная технология. Оценка программ
ного продукта. Характеристики качества и руководство по их при
менению.

35. IEEE 1074-1995. Процессы жизненного цикла для развития про
граммного обеспечения.
36. ANSI/IEEE 829-1983. Документация при тестировании программ.
37. ANSI/IEEE 1008-1986. Тестирование программных модулей и
компонентов ПС.
38. ANSI/IEEE 983-1986. Руководство по планированию обеспече
ния качества программных средств.
39. ГОСТ Р ИСО/МЭК 9294-93. Информационная технология. Руко
водство по управлению документированием программного обес
печения.
40. ГОСТ Р ИСО/МЭК 9126-93. Информационная технология. Оцен
ка программной продукции. Характеристики качества и руковод
ство по их применению.
41. ГОСТ Р ИСО/МЭК 9127-94. Системы обработки информации.
Документация пользователя и информация на упаковке для потре
бительских программных пакетов.
42. ГОСТ Р ИСО/МЭК 8631-94. Информационная технология. Про
граммные конструктивы и условные обозначения для их представ
ления.
43. ГОСТ Р ИСО/МЭК 12119:1994. Информационная технология.
Пакеты программных средств. Требования к качеству и испы
тания.
44. ГОСТ Р ИСО/МЭК 12207. Процессы жизненного цикла програм
мных средств.

Книги

45. **Вендров** А.М. Проектирование программного обеспечения эконо
мических информационных систем: Учебник. — М.: Финансы и
статистика. 2000. — 352 с: ил.
46. Гласе Р. Руководство по надежному программированию / Пер. с
англ. Ю.П. Кондранина, В.М. Рабиновича; Под ред. В.М, Раби
новича. Предисл. В.В. Липаева. — М.: Финансы и статистика,
1982.—256 с: ил.
47. Конституция Российской Федерации. —М.: Ассоциация авторов
и издателей «ТАНДЕМ». Изд-во ЭКМОС, 1999. — 48 с.

48. Крылова Г.Д. Основы стандартизации, сертификации, метроло гии: Учебник для вузов. — 2-е изд., перераб. и доп. — М.: ЮНИ-
ТИ-ДАНА, 2001. — 711 с.

49. Липаев В.В. Надежность программных средств. — М.: СИНТЕГ,
1998. — 232 с. — (Серия «Информатизация России на пороге XXI
века»).

50. **Майерс** Г. Искусство тестирования программ / Пер. с англ.; Под
ред. Б. А. Позина. — М.: Финансы и статистика, 1982. —176 с: ил.

51. **Майерс Г.** Надежность программного обеспечения. — М.: Мир,
1980.

52. **Першиков В.И., Савинков В.М.** Толковый словарь по информа тике. — 2-е изд., доп. — М.: Финансы и статистика, 1995. — 544 с.

53. **Саймон** А.Р. Стратегические технологии баз данных: менеджмент на 2000 год / Пер. с англ.; Под ред. и с предисл. М.Р. Когаловско-
го. — М.: Финансы и статистика, 1999. — 479 с: ил.

54. **Тейер** Т. **и др.** Надежность программного обеспечения / Т. Тейер,
М. Липов, Э. Нельсон /Пер. с англ. — М.: Мир, 1981. — 323 с: ил.

5 5. Тестирование программного обеспечения / Сэм Канер, Джек Фолк, Енг Кек Нгуен / Пер. с англ. — Киев: Изд-во «ДиаСофт», 2000. — 544 с.

56. Экономика, разработка и использование программного обеспече ния ЭВМ: Учебник / В.А. Благодатских, М.А. Енгибарян, Е.В. Ко
валевская и др. — М.: Финансы и статистика, 1995. — 288 с: ил.

Статьи

57. **Батурин Ю.** Ошибка компьютера // Новое время. — 1987. — № 9.

58. **Васютович В., Самотохин С, Никифоров** Г. Регламентация жиз ненного цикла программных средств // Директор ИС. — 2000. —
№ 07-08.

59. **Васютович В., Самотохин С.** Стандартизация в области докумен тирования программных средств // Computerworld. — 6 июля 1999. — №25(186).

60. **Зиндер Е.З.** Соотнесение и использование стандартов организации жизненных циклов систем // СУБД. — 1997. — № 3.

61. **Карпов В.Э.** Об оформлении программной документации, http://
uiits.miem.edu.ru/karpov.

62. Липаев В.В. Стандарты, регламентирующие жизненный цикл слож
ных систем программных комплексов,
http://kis.pcweek.ru/kis/win/
techno/stdpo.htm.

63. **Малышева** Л. Разработка внуртикорпоративных стандартов
//
СУБД. — 2001.—- №9.

64. **Поскакалов** К.Ф. Тестирование программного обеспечения как средство обеспечения качества // «DiasoftlNFO» — 2000. — сен
тябрь.

65. **Codd E.F.** «A Relational Model of Data for Large Shared Data Banks», Communications of the ACM (June 1970). Русский перевод:
Кодд Э.Ф. Реляционная модель данных для больших совместно используемых банков данных // СУБД. — 1995. — № 1.

66. «Visual Test 4.0 White Paper» By Thomas Arnold. 1996. — December.

ПРЕДМЕТНЫЙ УКАЗАТЕЛЬ

Аттестация 75
Аудит 77

Базовый стандарт 57
Белый ящик 204
Буч Гради (Booch Grady) 22

Ведомость держателей подлинников 101 Верификация 74
Внутрифирменный стандарт 32
Вспомогательные процессы 71

Госстандарт РФ 26
ГОСТ 34 80
ГОСТР 113
ГОСТ 28195-89 61, 136
Государственный комитет РФ по стандартизации 26

Дефект 141, 142
Длительность восстановления 140
Доказательство 201
Документ технических условий 12
Документирование 71

ЕСПД 60, 99

Жизненный цикл разработки программного средства (ЖЦ ПС) 7, 56

Интеграция системы 68
ИСО/МЭК 12207 16, 60
Испытание 102

Каскадная модель 90 Качество программного обеспечения 194
Квалификационное тестирование 68
Квалификационное требование 68
Кодирование 68 Комплекс 101
Компонент 101 Контроль 73
Коэффициент готовности 140

Методы и средства контроля 133
Мобильность 124
Модель
— Джелинского - Моранды 163
— жизненного цикла 90
— LaPadula 162
— простая интуитивная 169
— Коркорэна 170
— Лилова 168
— Миллса 167
— Муса 165
— Нельсона 171
— переходных вероятностей 167
— Шика - Волвертона 165
— сложности 172
— Шумана 159
Модификация ПО 71

Надежность 129, 134
Национальная стандартизация 10

Обеспечение качества 73
Обязательная сертификация 128

Описание программы 101
Описание языка 112 Опытная эксплуатация 228 Основные процессы 63 Отказ 138
Отладка 185 Оценка 76
Ошибка 142

Перенос ПО в другую среду 71
Переносимость 136
Применимость 136 Понятность 120 Поставщик 21 Пояснительная записка 102 Практичность 118
Предварительный стандарт 11
Приемка ПО 69
Программа и методика испытаний 102
Программное обеспечение 15
Программное средство 7 Профили стандартов 57 Процесс обучения 80
— поставки 65
— приобретения 63
— разработки 65
— создания инфраструктуры 78
— сопровождения 69
— управления 78
— усовершенствования 79
— эксплуатации 69

Рамбо Джеймс (Rumbaugh James) 22 Регламент 13
Решение проблем 77
Руководство оператора 111
— программиста 111
— системного программиста 110

Сбой 138 Свод правил 12
Сертификация 174, 175
Систематическое тестирование 178
Сложность 182 Снятие ПО с эксплуатации 71 Совместная оценка 76 Сопровождаемость 124
Спецификация 46, 101
Спиральная модель 93 Стандарт 11 Стандартизация 9 СУБД 18

Текст программы 101
Тестирование 201
— внешних функций 202
—- защиты 223
— комплексное 202
— конфигурации 223
— модуля 212
— надежности 224
— настройки 203
— объема 222
— приемлемости 203
— производительности 224
— психологических фактрров 226
— публикаций 225
— совместимости 223
— сопряжений 202
— средств восстановления 225
— стрессов 221
— требований к памяти 224
— удобства обслуживания 225

Управление конфигурацией 72
Уровень стандартизации 9
Установка ПО 68 Устойчивость 139

Функциональная пригодность 134, 136

Чамберлин Дональд Д 20
Черный ящик 203

Эксплуатационное тестирование 69
Эксплуатационные документы 102
Эффективность 124

Якобсон Ивар (Jacobson Ivar) 22
Bp Win 37
CASE 86
DOD-STD-2167A 58
ER Win 43
Ethernet 18
GUI 16
IEC (МЭК) 24
IEEE 1074-1995 85
IEEE 1209 189
ISO (ИСО) 23 ISO 8402 1994 195 ISO 9000 195 ISO 9001 262 ISO 9126 134 КОЛЕС 12207 16
JTC1 (Joint Technical Committee 1) 20
MIL-STD-498 59 NIST 30 OLE 15 ORACLE 20 OSI18 POSIX 18 Rational Rose 43 SQL (Structured Query Language) 18
TQM (Total Quality Management) 195
UML (Unified Modeling Language) 22

Учебное издание

Благодатских Виктор Алексеевич
Волнин Владимир Александрович
Поскакалов Кирилл Феликсович

СТАНДАРТИЗАЦИЯ РАЗРАБОТКИ ПРОГРАММНЫХ СРЕДСТВ

Заведующая редакцией *Л А Табакова*
Ведущий редактор *Л Д Григорьева*
Младший редактор *НА Федорова*
Художественный редактор *Ю И Артюхов*
Технический редактор *В Ю Фотиева*
Корректоры *Т М Кочпакова Г В Хлощева*
Обложка художника *Н М Биксентеева*

ИБ № 4554

Подписано в печать 25 11 2004 Формат 60x88'/i_6 Гарнитура «Тайме» Печать офсетная
Услпл 17,64 Уч-издл 16,33
Тираж 3000 экз Заказ 3057 «С» 260

Издательство «Финансы и статистика»
101000, Москва, ул Покровка, 7
Телефон (095) 925-35-02, факс (095) 925-09-57
E-mail mail@fmstat ru http //www fmstat ru

ОАО «Типография «НОВОСТИ»
105005, Москва, ул Ф Энгельса 46